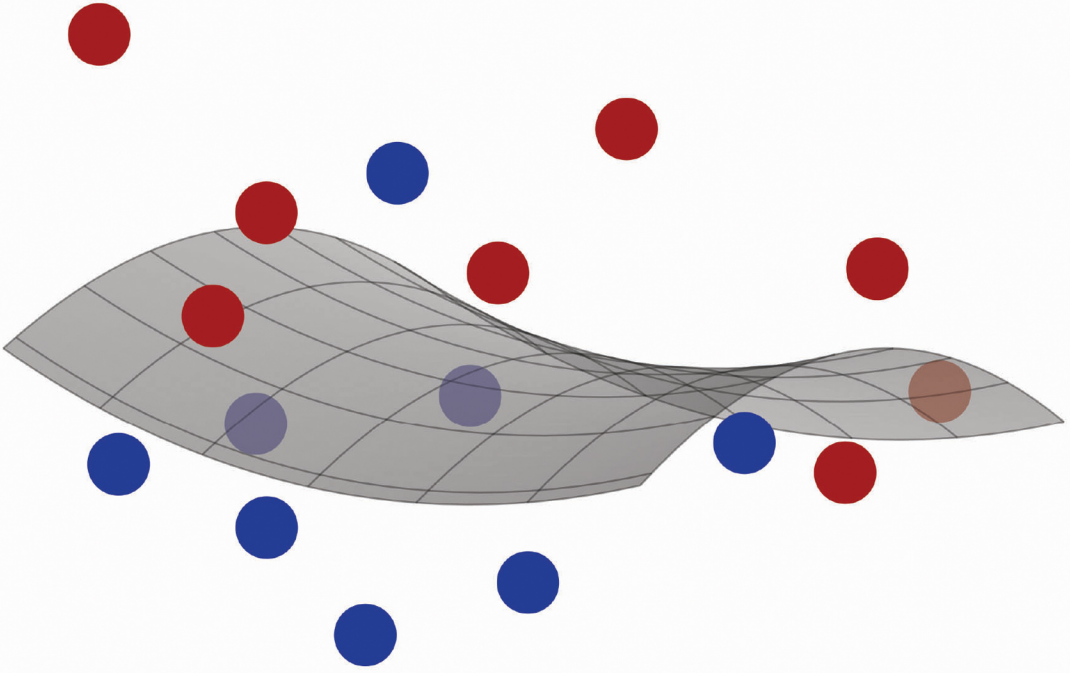
Foundations of Machine Learning

second edition



Mehryar Mohri,
Afshin Rostamizadeh,
and Ameet Talwalkar

Foundations of Machine Learning

second edition

Adaptive Computation and Machine Learning

Francis Bach, Editor

A complete list of books published in The Adaptive Computations and Machine Learning series appears at the back of this book.

Foundations of Machine Learning

second edition

Mehryar Mohri

Afshin Rostamizadeh

Ameet Talwalkar

The MIT Press
Cambridge, Massachusetts
London, England

© 2018 Massachusetts Institute of Technology

All rights reserved. No part of this book may be reproduced in any form by any electronic or mechanical means (including photocopying, recording, or information storage and retrieval) without permission in writing from the publisher.

This book was set in LaTeX by the authors. Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data

Names: Mohri, Mehryar, author. | Rostamizadeh, Afshin, author. | Talwalkar, Ameet, author.
Title: Foundations of machine learning / Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar.
Description: Second edition. | Cambridge, MA : The MIT Press, [2018] | Series: Adaptive computation and machine learning series | Includes bibliographical references and index.
Identifiers: LCCN 2018022812 | ISBN 9780262039406 (hardcover : alk. paper)
Subjects: LCSH: Machine learning. | Computer algorithms.
Classification: LCC Q325.5 .M64 2018 | DDC 006.3/1--dc23 LC record available at https://lccn.loc.gov/2018022812

10 9 8 7 6 5 4 3 2 1

Contents

Preface xiii

1 Introduction 1

1.1 What is machine learning? 1
1.2 What kind of problems can be tackled using machine learning? 2
1.3 Some standard learning tasks 3
1.4 Learning stages 4
1.5 Learning scenarios 6
1.6 Generalization 7

2 The PAC Learning Framework 9

2.1 The PAC learning model 9
2.2 Guarantees for finite hypothesis sets — consistent case 15
2.3 Guarantees for finite hypothesis sets — inconsistent case 19
2.4 Generalities 21
2.4.1 Deterministic versus stochastic scenarios 21
2.4.2 Bayes error and noise 22
2.5 Chapter notes 23
2.6 Exercises 23

3 Rademacher Complexity and VC-Dimension 29

3.1 Rademacher complexity 30
3.2 Growth function 34
3.3 VC-dimension 36
3.4 Lower bounds 43
3.5 Chapter notes 48
3.6 Exercises 50

4 Model Selection 61

4.1 Estimation and approximation errors 61
4.2 Empirical risk minimization (ERM) 62
4.3 Structural risk minimization (SRM) 64

4.4 Cross-validation 68
4.5 n-Fold cross-validation 71
4.6 Regularization-based algorithms 72
4.7 Convex surrogate losses 73
4.8 Chapter notes 77
4.9 Exercises 78

5 Support Vector Machines 79

5.1 Linear classification 79
5.2 Separable case 80
5.2.1 Primal optimization problem 81
5.2.2 Support vectors 83
5.2.3 Dual optimization problem 83
5.2.4 Leave-one-out analysis 85
5.3 Non-separable case 87
5.3.1 Primal optimization problem 88
5.3.2 Support vectors 89
5.3.3 Dual optimization problem 90
5.4 Margin theory 91
5.5 Chapter notes 100
5.6 Exercises 100

6 Kernel Methods 105

6.1 Introduction 105
6.2 Positive definite symmetric kernels 108
6.2.1 Definitions 108
6.2.2 Reproducing kernel Hilbert space 110
6.2.3 Properties 112
6.3 Kernel-based algorithms 116
6.3.1 SVMs with PDS kernels 116
6.3.2 Representer theorem 117
6.3.3 Learning guarantees 117
6.4 Negative definite symmetric kernels 119
6.5 Sequence kernels 121
6.5.1 Weighted transducers 122
6.5.2 Rational kernels 126
6.6 Approximate kernel feature maps 130
6.7 Chapter notes 135
6.8 Exercises 137

7 Boosting 145

7.1 Introduction 145
7.2 AdaBoost 146
7.2.1 Bound on the empirical error 149
7.2.2 Relationship with coordinate descent 150
7.2.3 Practical use 154

7.3 Theoretical results 154
7.3.1 VC-dimension-based analysis 154
7.3.2 L_1-geometric margin 155
7.3.3 Margin-based analysis 157
7.3.4 Margin maximization 161
7.3.5 Game-theoretic interpretation 162
7.4 L_1-regularization 165
7.5 Discussion 167
7.6 Chapter notes 168
7.7 Exercises 170

8 On-Line Learning 177

8.1 Introduction 178
8.2 Prediction with expert advice 178
8.2.1 Mistake bounds and Halving algorithm 179
8.2.2 Weighted majority algorithm 181
8.2.3 Randomized weighted majority algorithm 183
8.2.4 Exponential weighted average algorithm 186
8.3 Linear classification 190
8.3.1 Perceptron algorithm 190
8.3.2 Winnow algorithm 198
8.4 On-line to batch conversion 201
8.5 Game-theoretic connection 204
8.6 Chapter notes 205
8.7 Exercises 206

9 Multi-Class Classification 213

9.1 Multi-class classification problem 213
9.2 Generalization bounds 215
9.3 Uncombined multi-class algorithms 221
9.3.1 Multi-class SVMs 221
9.3.2 Multi-class boosting algorithms 222
9.3.3 Decision trees 224
9.4 Aggregated multi-class algorithms 228
9.4.1 One-versus-all 229
9.4.2 One-versus-one 229
9.4.3 Error-correcting output codes 231
9.5 Structured prediction algorithms 233
9.6 Chapter notes 235
9.7 Exercises 237

10 Ranking 239

10.1 The problem of ranking 240
10.2 Generalization bound 241
10.3 Ranking with SVMs 243

10.4 RankBoost 244
10.4.1 Bound on the empirical error 246
10.4.2 Relationship with coordinate descent 248
10.4.3 Margin bound for ensemble methods in ranking 250
10.5 Bipartite ranking 251
10.5.1 Boosting in bipartite ranking 252
10.5.2 Area under the ROC curve 255
10.6 Preference-based setting 257
10.6.1 Second-stage ranking problem 257
10.6.2 Deterministic algorithm 259
10.6.3 Randomized algorithm 260
10.6.4 Extension to other loss functions 262
10.7 Other ranking criteria 262
10.8 Chapter notes 263
10.9 Exercises 264

11 Regression 267

11.1 The problem of regression 267
11.2 Generalization bounds 268
11.2.1 Finite hypothesis sets 268
11.2.2 Rademacher complexity bounds 269
11.2.3 Pseudo-dimension bounds 271
11.3 Regression algorithms 275
11.3.1 Linear regression 275
11.3.2 Kernel ridge regression 276
11.3.3 Support vector regression 281
11.3.4 Lasso 285
11.3.5 Group norm regression algorithms 289
11.3.6 On-line regression algorithms 289
11.4 Chapter notes 290
11.5 Exercises 292

12 Maximum Entropy Models 295

12.1 Density estimation problem 295
12.1.1 Maximum Likelihood (ML) solution 296
12.1.2 Maximum a Posteriori (MAP) solution 297
12.2 Density estimation problem augmented with features 297
12.3 Maxent principle 298
12.4 Maxent models 299
12.5 Dual problem 299
12.6 Generalization bound 303
12.7 Coordinate descent algorithm 304
12.8 Extensions 306
12.9 L_2-regularization 308

12.10 Chapter notes 312
12.11 Exercises 313

13 Conditional Maximum Entropy Models 315

13.1 Learning problem 315
13.2 Conditional Maxent principle 316
13.3 Conditional Maxent models 316
13.4 Dual problem 317
13.5 Properties 319
13.5.1 Optimization problem 320
13.5.2 Feature vectors 320
13.5.3 Prediction 321
13.6 Generalization bounds 321
13.7 Logistic regression 325
13.7.1 Optimization problem 325
13.7.2 Logistic model 325
13.8 L_2-regularization 326
13.9 Proof of the duality theorem 328
13.10 Chapter notes 330
13.11 Exercises 331

14 Algorithmic Stability 333

14.1 Definitions 333
14.2 Stability-based generalization guarantee 334
14.3 Stability of kernel-based regularization algorithms 336
14.3.1 Application to regression algorithms: SVR and KRR 339
14.3.2 Application to classification algorithms: SVMs 341
14.3.3 Discussion 342
14.4 Chapter notes 342
14.5 Exercises 343

15 Dimensionality Reduction 347

15.1 Principal component analysis 348
15.2 Kernel principal component analysis (KPCA) 349
15.3 KPCA and manifold learning 351
15.3.1 Isomap 351
15.3.2 Laplacian eigenmaps 352
15.3.3 Locally linear embedding (LLE) 353
15.4 Johnson-Lindenstrauss lemma 354
15.5 Chapter notes 356
15.6 Exercises 356

16 Learning Automata and Languages 359

16.1 Introduction 359

16.2 Finite automata 360
16.3 Efficient exact learning 361
16.3.1 Passive learning 362
16.3.2 Learning with queries 363
16.3.3 Learning automata with queries 364
16.4 Identification in the limit 369
16.4.1 Learning reversible automata 370
16.5 Chapter notes 375
16.6 Exercises 376

17 Reinforcement Learning 379
17.1 Learning scenario 379
17.2 Markov decision process model 380
17.3 Policy 381
17.3.1 Definition 381
17.3.2 Policy value 382
17.3.3 Optimal policies 382
17.3.4 Policy evaluation 385
17.4 Planning algorithms 387
17.4.1 Value iteration 387
17.4.2 Policy iteration 390
17.4.3 Linear programming 392
17.5 Learning algorithms 393
17.5.1 Stochastic approximation 394
17.5.2 TD(0) algorithm 397
17.5.3 Q-learning algorithm 398
17.5.4 SARSA 402
17.5.5 TD(λ) algorithm 402
17.5.6 Large state space 403
17.6 Chapter notes 405

Conclusion 407

A Linear Algebra Review 409
A.1 Vectors and norms 409
A.1.1 Norms 409
A.1.2 Dual norms 410
A.1.3 Relationship between norms 411
A.2 Matrices 411
A.2.1 Matrix norms 411
A.2.2 Singular value decomposition 412
A.2.3 Symmetric positive semidefinite (SPSD) matrices 412

B Convex Optimization **415**

B.1 Differentiation and unconstrained optimization 415
B.2 Convexity 415
B.3 Constrained optimization 419
B.4 Fenchel duality 422
B.4.1 Subgradients 422
B.4.2 Core 423
B.4.3 Conjugate functions 423
B.5 Chapter notes 426
B.6 Exercises 427

C Probability Review **429**

C.1 Probability 429
C.2 Random variables 429
C.3 Conditional probability and independence 431
C.4 Expectation and Markov's inequality 431
C.5 Variance and Chebyshev's inequality 432
C.6 Moment-generating functions 434
C.7 Exercises 435

D Concentration Inequalities **437**

D.1 Hoeffding's inequality 437
D.2 Sanov's theorem 438
D.3 Multiplicative Chernoff bounds 439
D.4 Binomial distribution tails: Upper bounds 440
D.5 Binomial distribution tails: Lower bound 440
D.6 Azuma's inequality 441
D.7 McDiarmid's inequality 442
D.8 Normal distribution tails: Lower bound 443
D.9 Khintchine-Kahane inequality 443
D.10 Maximal inequality 444
D.11 Chapter notes 445
D.12 Exercises 445

E Notions of Information Theory **449**

E.1 Entropy 449
E.2 Relative entropy 450
E.3 Mutual information 453
E.4 Bregman divergences 453
E.5 Chapter notes 456
E.6 Exercises 457

F Notation **459**

Bibliography **461**

Index **475**

Preface

This book is a general introduction to machine learning that can serve as a reference book for researchers and a textbook for students. It covers fundamental modern topics in machine learning while providing the theoretical basis and conceptual tools needed for the discussion and justification of algorithms. It also describes several key aspects of the application of these algorithms.

We have aimed to present the most novel theoretical tools and concepts while giving concise proofs, even for relatively advanced results. In general, whenever possible, we have chosen to favor succinctness. Nevertheless, we discuss some crucial complex topics arising in machine learning and highlight several open research questions. Certain topics often merged with others or treated with insufficient attention are discussed separately here and with more emphasis: for example, a different chapter is reserved for multi-class classification, ranking, and regression.

Although we cover a very wide variety of important topics in machine learning, we have chosen to omit a few important ones, including graphical models and neural networks, both for the sake of brevity and because of the current lack of solid theoretical guarantees for some methods.

The book is intended for students and researchers in machine learning, statistics and other related areas. It can be used as a textbook for both graduate and advanced undergraduate classes in machine learning or as a reference text for a research seminar. The first three or four chapters of the book lay the theoretical foundation for the subsequent material. Other chapters are mostly self-contained, with the exception of chapter 6 which introduces some concepts that are extensively used in later ones and chapter 13, which is closely related to chapter 12. Each chapter concludes with a series of exercises, with full solutions presented separately.

The reader is assumed to be familiar with basic concepts in linear algebra, probability, and analysis of algorithms. However, to further help, we have included an extensive appendix presenting a concise review of linear algebra, an introduction to convex optimization, a brief probability review, a collection of concentration

inequalities useful to the analyses and discussions in this book, and a short introduction to information theory.

Our goal has been to give a unified presentation of multiple topics and areas, as opposed to a more specialized presentation adopted by some books which favor a particular viewpoint, such as for example a Bayesian view, or a particular topic, such as for example kernel methods. The theoretical foundation of this book and its deliberate emphasis on proofs and analysis make it also very distinct from many other presentations.

In this second edition, we have updated the entire book. The changes include a different writing style in most chapters, new figures and illustrations, many simplifications, some additions to existing chapters, in particular chapter 6 and chapter 17, and several new chapters. We have added a full chapter on model selection (chapter 4), which is an important topic that was only briefly discussed in the previous edition. We have also added a new chapter on Maximum Entropy models (chapter 12) and a new chapter on Conditional Maximum Entropy models (chapter 13) which are both essential topics in machine learning. We have also significantly changed the appendix. In particular, we have added a full section on Fenchel duality to appendix B on convex optimization, made a number of changes and additions to appendix D dealing with concentration inequalities, added appendix E on information theory, and updated most of the material. Additionally, we have included a number of new exercises and their solutions for existing and new chapters.

Most of the material presented here takes its origins in a machine learning graduate course (*Foundations of Machine Learning*) taught by the first author at the Courant Institute of Mathematical Sciences in New York University over the last fourteen years. This book has considerably benefited from the comments and suggestions from students in these classes, along with those of many friends, colleagues and researchers to whom we are deeply indebted.

We are particularly grateful to Corinna Cortes and Yishay Mansour who made a number of key suggestions for the design and organization of the material presented in the first edition, with detailed comments that we have fully taken into account and that have greatly improved the presentation. We are also grateful to Yishay Mansour for using a preliminary version of the first edition of the book for teaching, and for reporting his feedback to us.

We also thank for discussions, suggested improvement, and contributions of many kinds the following colleagues and friends from academic and corporate research laboratories: Jacob Abernethy, Cyril Allauzen, Kareem Amin, Stephen Boyd, Aldo Corbisiero, Giulia DeSalvo, Claudio Gentile, Spencer Greenberg, Lisa Hellerstein, Sanjiv Kumar, Vitaly Kuznetsov, Ryan McDonald, Andrès Muñoz Medina, Tyler Neylon, Peter Norvig, Fernando Pereira, Maria Pershina, Borja de Balle Pigem,

Ashish Rastogi, Michael Riley, Dmitry Storcheus, Ananda Theertha Suresh, Umar Syed, Csaba Szepesvári, Toshiyuki Tanaka, Eugene Weinstein, Jason Weston, Scott Yang, and Ningshan Zhang.

Finally, we thank the MIT Press publication team for their help and support in the development of this text.

1 Introduction

This chapter presents a preliminary introduction to machine learning, including an overview of some key learning tasks and applications, basic definitions and terminology, and the discussion of some general scenarios.

1.1 What is machine learning?

Machine learning can be broadly defined as computational methods using experience to improve performance or to make accurate predictions. Here, *experience* refers to the past information available to the learner, which typically takes the form of electronic data collected and made available for analysis. This data could be in the form of digitized human-labeled training sets, or other types of information obtained via interaction with the environment. In all cases, its quality and size are crucial to the success of the predictions made by the learner.

An example of a learning problem is how to use a finite sample of randomly selected documents, each labeled with a topic, to accurately predict the topic of unseen documents. Clearly, the larger is the sample, the easier is the task. But the difficulty of the task also depends on the quality of the labels assigned to the documents in the sample, since the labels may not be all correct, and on the number of possible topics.

Machine learning consists of designing efficient and accurate prediction *algorithms*. As in other areas of computer science, some critical measures of the quality of these algorithms are their time and space complexity. But, in machine learning, we will need additionally a notion of *sample complexity* to evaluate the sample size required for the algorithm to learn a family of concepts. More generally, theoretical learning guarantees for an algorithm depend on the complexity of the concept classes considered and the size of the training sample.

Since the success of a learning algorithm depends on the data used, machine learning is inherently related to data analysis and statistics. More generally, learning

techniques are data-driven methods combining fundamental concepts in computer science with ideas from statistics, probability and optimization.

1.2 What kind of problems can be tackled using machine learning?

Predicting the label of a document, also known as document classification, is by no means the only learning task. Machine learning admits a very broad set of practical applications, which include the following:

- Text or document classification. This includes problems such as assigning a topic to a text or a document, or determining automatically if the content of a web page is inappropriate or too explicit; it also includes spam detection.
- Natural language processing (NLP). Most tasks in this field, including part-of-speech tagging, named-entity recognition, context-free parsing, or dependency parsing, are cast as learning problems. In these problems, predictions admit some structure. For example, in part-of-speech tagging, the prediction for a sentence is a sequence of part-of-speech tags labeling each word. In context-free parsing the prediction is a tree. These are instances of richer learning problems known as *structured prediction problems*.
- Speech processing applications. This includes speech recognition, speech synthesis, speaker verification, speaker identification, as well as sub-problems such as language modeling and acoustic modeling.
- Computer vision applications. This includes object recognition, object identification, face detection, Optical character recognition (OCR), content-based image retrieval, or pose estimation.
- Computational biology applications. This includes protein function prediction, identification of key sites, or the analysis of gene and protein networks.
- Many other problems such as fraud detection for credit card, telephone or insurance companies, network intrusion, learning to play games such as chess, backgammon, or Go, unassisted control of vehicles such as robots or cars, medical diagnosis, the design of recommendation systems, search engines, or information extraction systems, are tackled using machine learning techniques.

This list is by no means comprehensive. Most prediction problems found in practice can be cast as learning problems and the practical application area of machine learning keeps expanding. The algorithms and techniques discussed in this book can be used to derive solutions for all of these problems, though we will not discuss in detail these applications.

1.3 Some standard learning tasks

The following are some standard machine learning tasks that have been extensively studied:

- *Classification*: this is the problem of assigning a category to each item. For example, document classification consists of assigning a category such as *politics*, *business*, *sports*, or *weather* to each document, while image classification consists of assigning to each image a category such as *car*, *train*, or *plane*. The number of categories in such tasks is often less than a few hundreds, but it can be much larger in some difficult tasks and even unbounded as in OCR, text classification, or speech recognition.
- *Regression*: this is the problem of predicting a real value for each item. Examples of regression include prediction of stock values or that of variations of economic variables. In regression, the penalty for an incorrect prediction depends on the magnitude of the difference between the true and predicted values, in contrast with the classification problem, where there is typically no notion of closeness between various categories.
- *Ranking*: this is the problem of learning to order items according to some criterion. Web search, e.g., returning web pages relevant to a search query, is the canonical ranking example. Many other similar ranking problems arise in the context of the design of information extraction or natural language processing systems.
- *Clustering*: this is the problem of partitioning a set of items into homogeneous subsets. Clustering is often used to analyze very large data sets. For example, in the context of social network analysis, clustering algorithms attempt to identify natural *communities* within large groups of people.
- *Dimensionality reduction* or *manifold learning*: this problem consists of transforming an initial representation of items into a lower-dimensional representation while preserving some properties of the initial representation. A common example involves preprocessing digital images in computer vision tasks.

The main practical objectives of machine learning consist of generating accurate predictions for unseen items and of designing efficient and robust algorithms to produce these predictions, even for large-scale problems. To do so, a number of algorithmic and theoretical questions arise. Some fundamental questions include: Which concept families can actually be learned, and under what conditions? How well can these concepts be learned computationally?

1.4 Learning stages

Here, we will use the canonical problem of spam detection as a running example to illustrate some basic definitions and describe the use and evaluation of machine learning algorithms in practice, including their different stages.

Spam detection is the problem of learning to automatically classify email messages as either SPAM or non-SPAM. The following is a list of definitions and terminology commonly used in machine learning:

- *Examples*: Items or instances of data used for learning or evaluation. In our spam problem, these examples correspond to the collection of email messages we will use for learning and testing.
- *Features*: The set of attributes, often represented as a vector, associated to an example. In the case of email messages, some relevant features may include the length of the message, the name of the sender, various characteristics of the header, the presence of certain keywords in the body of the message, and so on.
- *Labels*: Values or categories assigned to examples. In classification problems, examples are assigned specific categories, for instance, the SPAM and non-SPAM categories in our binary classification problem. In regression, items are assigned real-valued labels.
- *Hyperparameters*: Free parameters that are not determined by the learning algorithm, but rather specified as inputs to the learning algorithm.
- *Training sample*: Examples used to train a learning algorithm. In our spam problem, the training sample consists of a set of email examples along with their associated labels. The training sample varies for different learning scenarios, as described in section 1.5.
- *Validation sample*: Examples used to tune the parameters of a learning algorithm when working with labeled data. The validation sample is used to select appropriate values for the learning algorithm's free parameters (hyperparameters).
- *Test sample*: Examples used to evaluate the performance of a learning algorithm. The test sample is separate from the training and validation data and is not made available in the learning stage. In the spam problem, the test sample consists of a collection of email examples for which the learning algorithm must predict labels based on features. These predictions are then compared with the labels of the test sample to measure the performance of the algorithm.
- *Loss function*: A function that measures the difference, or loss, between a predicted label and a true label. Denoting the set of all labels as $\mathcal{Y}$ and the set of possible predictions as $\mathcal{Y}'$, a loss function L is a mapping $L\colon \mathcal{Y} \times \mathcal{Y}' \to \mathbb{R}_+$. In most cases, $\mathcal{Y}' = \mathcal{Y}$ and the loss function is bounded, but these conditions do not always hold. Common examples of loss functions include the zero-one (or

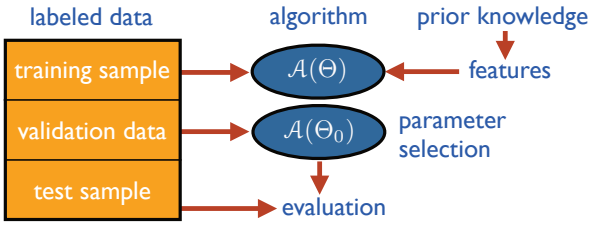


Figure 1.1
Illustration of the typical stages of a learning process.

misclassification) loss defined over $\{-1,+1\} \times \{-1,+1\}$ by $L(y,y') = 1_{y' \neq y}$ and the squared loss defined over $\mathcal{I} \times \mathcal{I}$ by $L(y,y') = (y'-y)^2$, where $\mathcal{I} \subseteq \mathbb{R}$ is typically a bounded interval.

- *Hypothesis set*: A set of functions mapping features (feature vectors) to the set of labels $\mathcal{Y}$. In our example, these may be a set of functions mapping email features to $\mathcal{Y} = \{\text{SPAM}, \text{non-SPAM}\}$. More generally, hypotheses may be functions mapping features to a different set $\mathcal{Y}'$. They could be linear functions mapping email feature vectors to real numbers interpreted as *scores* ($\mathcal{Y}' = \mathbb{R}$), with higher score values more indicative of SPAM than lower ones.

We now define the learning stages of our spam problem (see figure 1.1). We start with a given collection of labeled examples. We first randomly partition the data into a training sample, a validation sample, and a test sample. The size of each of these samples depends on a number of different considerations. For example, the amount of data reserved for validation depends on the number of hyperparameters of the algorithm, which are represented here by the vector Θ. Also, when the labeled sample is relatively small, the amount of training data is often chosen to be larger than that of the test data since the learning performance directly depends on the training sample.

Next, we associate relevant features to the examples. This is a critical step in the design of machine learning solutions. Useful features can effectively guide the learning algorithm, while poor or uninformative ones can be misleading. Although it is critical, to a large extent, the choice of the features is left to the user. This choice reflects the user's *prior knowledge* about the learning task which in practice can have a dramatic effect on the performance results.

Now, we use the features selected to train our learning algorithm $\mathcal{A}$ by tuning the values of its free parameters Θ (also called *hyperparameters*). For each value of these parameters, the algorithm selects a different hypothesis out of the hypothesis set. We choose the one resulting in the best performance on the validation sample (Θ_0). Finally, using that hypothesis, we predict the labels of the examples in the test sample. The performance of the algorithm is evaluated by using the loss

function associated to the task, e.g., the zero-one loss in our spam detection task, to compare the predicted and true labels. Thus, the performance of an algorithm is of course evaluated based on its test error and not its error on the training sample.

1.5 Learning scenarios

We next briefly describe some common machine learning scenarios. These scenarios differ in the types of training data available to the learner, the order and method by which training data is received and the test data used to evaluate the learning algorithm.

- *Supervised learning*: The learner receives a set of labeled examples as training data and makes predictions for all unseen points. This is the most common scenario associated with classification, regression, and ranking problems. The spam detection problem discussed in the previous section is an instance of supervised learning.
- *Unsupervised learning*: The learner exclusively receives unlabeled training data, and makes predictions for all unseen points. Since in general no labeled example is available in that setting, it can be difficult to quantitatively evaluate the performance of a learner. Clustering and dimensionality reduction are example of unsupervised learning problems.
- *Semi-supervised learning*: The learner receives a training sample consisting of both labeled and unlabeled data, and makes predictions for all unseen points. Semi-supervised learning is common in settings where unlabeled data is easily accessible but labels are expensive to obtain. Various types of problems arising in applications, including classification, regression, or ranking tasks, can be framed as instances of semi-supervised learning. The hope is that the distribution of unlabeled data accessible to the learner can help him achieve a better performance than in the supervised setting. The analysis of the conditions under which this can indeed be realized is the topic of much modern theoretical and applied machine learning research.
- *Transductive inference*: As in the semi-supervised scenario, the learner receives a labeled training sample along with a set of unlabeled test points. However, the objective of transductive inference is to predict labels only for these particular test points. Transductive inference appears to be an easier task and matches the scenario encountered in a variety of modern applications. However, as in the semi-supervised setting, the assumptions under which a better performance can be achieved in this setting are research questions that have not been fully resolved.

- *On-line learning*: In contrast with the previous scenarios, the online scenario involves multiple rounds where training and testing phases are intermixed. At each round, the learner receives an unlabeled training point, makes a prediction, receives the true label, and incurs a loss. The objective in the on-line setting is to minimize the cumulative loss over all rounds or to minimize the *regret*, that is the difference of the cumulative loss incurred and that of the best expert in hindsight. Unlike the previous settings just discussed, no distributional assumption is made in on-line learning. In fact, instances and their labels may be chosen adversarially within this scenario.
- *Reinforcement learning*: The training and testing phases are also intermixed in reinforcement learning. To collect information, the learner actively interacts with the environment and in some cases affects the environment, and receives an immediate reward for each action. The object of the learner is to maximize his reward over a course of actions and iterations with the environment. However, no long-term reward feedback is provided by the environment, and the learner is faced with the *exploration versus exploitation* dilemma, since he must choose between exploring unknown actions to gain more information versus exploiting the information already collected.
- *Active learning*: The learner adaptively or interactively collects training examples, typically by querying an oracle to request labels for new points. The goal in active learning is to achieve a performance comparable to the standard supervised learning scenario (or *passive learning* scenario), but with fewer labeled examples. Active learning is often used in applications where labels are expensive to obtain, for example computational biology applications.

In practice, many other intermediate and somewhat more complex learning scenarios may be encountered.

1.6 Generalization

Machine learning is fundamentally about *generalization*. As an example, the standard supervised learning scenario consists of using a finite sample of labeled examples to make accurate predictions about unseen examples. The problem is typically formulated as that of selecting a function out of a *hypothesis set*, that is a subset of the family of all functions. The function selected is subsequently used to label all instances, including unseen examples.

How should a hypothesis set be chosen? With a rich or *complex* hypothesis set, the learner may choose a function or predictor that is *consistent* with the training sample, that is one that commits no error on the training sample. With a less complex family, incurring some errors on the training sample may be unavoidable. But,

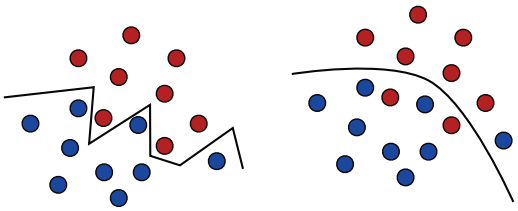


Figure 1.2
The zig-zag line on the left panel is consistent over the blue and red training sample, but it is a complex separation surface that is not likely to generalize well to unseen data. In contrast, the decision surface on the right panel is simpler and might generalize better in spite of its misclassification of a few points of the training sample.

which will lead to a better generalization? How should we define the complexity of a hypothesis set?

Figure 1.2 illustrates these two types of solution: one is a zig-zag line that perfectly separates the two populations of blue and red points and that is chosen from a complex family; the other one is a smoother line chosen from a simpler family that only imperfectly discriminates between the two sets. We will see that, in general, the best predictor on the training sample may not be the best overall. A predictor chosen from a very complex family can essentially memorize the data, but generalization is distinct from the memorization of the training labels.

We will see that the trade-off between the sample size and complexity plays a critical role in generalization. When the sample size is relatively small, choosing from a too complex a family may lead to poor generalization, which is also known as *overfitting*. On the other hand, with a too simple a family it may not be possible to achieve a sufficient accuracy, which is known as *underfitting*.

In the next chapters, we will analyze more in detail the problem of generalization and will seek to derive theoretical guarantees for learning. This will depend on different notions of complexity that we will thoroughly discuss.

2 The PAC Learning Framework

Several fundamental questions arise when designing and analyzing algorithms that learn from examples: What can be learned efficiently? What is inherently hard to learn? How many examples are needed to learn successfully? Is there a general model of learning? In this chapter, we begin to formalize and address these questions by introducing the *Probably Approximately Correct* (PAC) learning framework. The PAC framework helps define the class of learnable concepts in terms of the number of sample points needed to achieve an approximate solution, *sample complexity*, and the time and space complexity of the learning algorithm, which depends on the cost of the computational representation of the concepts.

We first describe the PAC framework and illustrate it, then present some general learning guarantees within this framework when the hypothesis set used is finite, both for the *consistent* case where the hypothesis set used contains the concept to learn and for the opposite *inconsistent* case.

2.1 The PAC learning model

We first introduce several definitions and the notation needed to present the PAC model, which will also be used throughout much of this book.

We denote by $\mathcal{X}$ the set of all possible *examples* or *instances*. $\mathcal{X}$ is also sometimes referred to as the *input space*. The set of all possible *labels* or *target values* is denoted by $\mathcal{Y}$. For the purpose of this introductory chapter, we will limit ourselves to the case where $\mathcal{Y}$ is reduced to two labels, $\mathcal{Y} = \{0, 1\}$, which corresponds to the so-called *binary classification*. Later chapters will extend these results to more general settings.

A *concept* $c\colon \mathcal{X} \to \mathcal{Y}$ is a mapping from $\mathcal{X}$ to $\mathcal{Y}$. Since $\mathcal{Y} = \{0, 1\}$, we can identify c with the subset of $\mathcal{X}$ over which it takes the value 1. Thus, in the following, we equivalently refer to a concept to learn as a mapping from $\mathcal{X}$ to $\{0, 1\}$, or as a subset of $\mathcal{X}$. As an example, a concept may be the set of points inside a triangle

or the indicator function of these points. In such cases, we will say in short that the concept to learn is a triangle. A *concept class* is a set of concepts we may wish to learn and is denoted by $\mathcal{C}$. This could, for example, be the set of all triangles in the plane.

We assume that examples are independently and identically distributed (i.i.d.) according to some fixed but unknown distribution $\mathcal{D}$. The learning problem is then formulated as follows. The learner considers a fixed set of possible concepts $\mathcal{H}$, called a *hypothesis set*, which might not necessarily coincide with $\mathcal{C}$. It receives a sample $S = (x_1, \ldots, x_m)$ drawn i.i.d. according to $\mathcal{D}$ as well as the labels $(c(x_1), \ldots, c(x_m))$, which are based on a specific target concept $c \in \mathcal{C}$ to learn. The task is then to use the labeled sample S to select a hypothesis $h_S \in \mathcal{H}$ that has a small *generalization error* with respect to the concept c. The generalization error of a hypothesis $h \in \mathcal{H}$, also referred to as the *risk* or *true error* (or simply *error*) of h is denoted by $R(h)$ and defined as follows.[1]

Definition 2.1 (Generalization error) *Given a hypothesis $h \in \mathcal{H}$, a target concept $c \in \mathcal{C}$, and an underlying distribution $\mathcal{D}$, the* generalization error *or* risk *of h is defined by*

$$R(h) = \mathop{\mathbb{P}}_{x \sim \mathcal{D}}[h(x) \neq c(x)] = \mathop{\mathbb{E}}_{x \sim \mathcal{D}}\left[1_{h(x) \neq c(x)}\right], \tag{2.1}$$

where 1_ω is the indicator function of the event ω.[2]

The generalization error of a hypothesis is not directly accessible to the learner since both the distribution $\mathcal{D}$ and the target concept c are unknown. However, the learner can measure the *empirical error* of a hypothesis on the labeled sample S.

Definition 2.2 (Empirical error) *Given a hypothesis $h \in \mathcal{H}$, a target concept $c \in \mathcal{C}$, and a sample $S = (x_1, \ldots, x_m)$, the* empirical error *or* empirical risk *of h is defined by*

$$\widehat{R}_S(h) = \frac{1}{m} \sum_{i=1}^{m} 1_{h(x_i) \neq c(x_i)}. \tag{2.2}$$

Thus, the empirical error of $h \in \mathcal{H}$ is its average error over the sample S, while the generalization error is its expected error based on the distribution $\mathcal{D}$. We will see in this chapter and the following chapters a number of guarantees relating these two quantities with high probability, under some general assumptions. We can already note that for a fixed $h \in \mathcal{H}$, the expectation of the empirical error based on an i.i.d.

[1] The choice of R instead of E to denote an error avoids possible confusions with the notation for expectations and is further justified by the fact that the term *risk* is also used in machine learning and statistics to refer to an error.

[2] For this and other related definitions, the family of functions $\mathcal{H}$ and the target concept c must be measurable. The function classes we consider in this book all have this property.

sample S is equal to the generalization error:

$$\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[\widehat{R}_S(h)] = R(h). \tag{2.3}$$

Indeed, by the linearity of the expectation and the fact that the sample is drawn i.i.d., we can write

$$\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[\widehat{R}_S(h)] = \frac{1}{m}\sum_{i=1}^{m}\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[1_{h(x_i)\neq c(x_i)}] = \frac{1}{m}\sum_{i=1}^{m}\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[1_{h(x)\neq c(x)}],$$

for any x in sample S. Thus,

$$\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[\widehat{R}_S(h)] = \mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[1_{h(x)\neq c(x)}] = \mathop{\mathbb{E}}_{x\sim\mathcal{D}}[1_{h(x)\neq c(x)}] = R(h).$$

The following introduces the *Probably Approximately Correct* (PAC) learning framework. Let n be a number such that the computational cost of representing any element $x \in \mathcal{X}$ is at most $O(n)$ and denote by size(c) the maximal cost of the computational representation of $c \in \mathcal{C}$. For example, x may be a vector in $\mathbb{R}^n$, for which the cost of an array-based representation would be in $O(n)$. In addition, let h_S denote the hypothesis returned by algorithm $\mathcal{A}$ after receiving a labeled sample S. To keep notation simple, the dependency of h_S on $\mathcal{A}$ is not explicitly indicated.

Definition 2.3 (PAC-learning) *A concept class $\mathcal{C}$ is said to be* PAC-learnable *if there exists an algorithm $\mathcal{A}$ and a polynomial function poly$(\cdot,\cdot,\cdot,\cdot)$ such that for any $\epsilon > 0$ and $\delta > 0$, for all distributions $\mathcal{D}$ on $\mathcal{X}$ and for any target concept $c \in \mathcal{C}$, the following holds for any sample size $m \geq$ poly$(1/\epsilon, 1/\delta, n,$ size$(c))$:*

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}[R(h_S) \leq \epsilon] \geq 1-\delta. \tag{2.4}$$

If $\mathcal{A}$ further runs in poly$(1/\epsilon, 1/\delta, n,$ size$(c))$, then $\mathcal{C}$ is said to be efficiently PAC-learnable. *When such an algorithm $\mathcal{A}$ exists, it is called a* PAC-learning algorithm *for $\mathcal{C}$.*

A concept class $\mathcal{C}$ is thus PAC-learnable if the hypothesis returned by the algorithm after observing a number of points polynomial in $1/\epsilon$ and $1/\delta$ is *approximately correct* (error at most ϵ) with high *probability* (at least $1-\delta$), which justifies the PAC terminology. The parameter $\delta > 0$ is used to define the *confidence* $1-\delta$ and $\epsilon > 0$ the *accuracy* $1-\epsilon$. Note that if the running time of the algorithm is polynomial in $1/\epsilon$ and $1/\delta$, then the sample size m must also be polynomial if the full sample is received by the algorithm.

Several key points of the PAC definition are worth emphasizing. First, the PAC framework is a *distribution-free model*: no particular assumption is made about the distribution $\mathcal{D}$ from which examples are drawn. Second, the training sample and the test examples used to define the error are drawn according to the same distribution $\mathcal{D}$. This is a natural and necessary assumption for generalization to

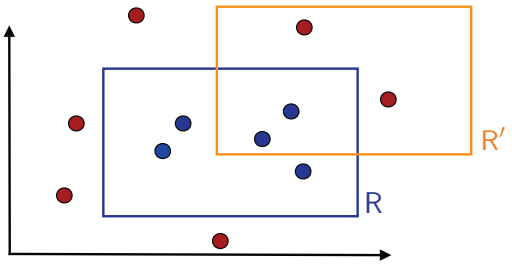


Figure 2.1
Target concept R and possible hypothesis R′. Circles represent training instances. A blue circle is a point labeled with 1, since it falls within the rectangle R. Others are red and labeled with 0.

be possible in general. It can be relaxed to include favorable *domain adaptation* problems. Finally, the PAC framework deals with the question of learnability for a concept class $\mathcal{C}$ and not a particular concept. Note that the concept class $\mathcal{C}$ is known to the algorithm, but of course the target concept $c \in \mathcal{C}$ is unknown.

In many cases, in particular when the computational representation of the concepts is not explicitly discussed or is straightforward, we may omit the polynomial dependency on n and size(c) in the PAC definition and focus only on the sample complexity.

We now illustrate PAC-learning with a specific learning problem.

Example 2.4 (Learning axis-aligned rectangles) Consider the case where the set of instances are points in the plane, $\mathcal{X} = \mathbb{R}^2$, and the concept class $\mathcal{C}$ is the set of all axis-aligned rectangles lying in $\mathbb{R}^2$. Thus, each concept c is the set of points inside a particular axis-aligned rectangle. The learning problem consists of determining with small error a target axis-aligned rectangle using the labeled training sample. We will show that the concept class of axis-aligned rectangles is PAC-learnable.

Figure 2.1 illustrates the problem. R represents a target axis-aligned rectangle and R′ a hypothesis. As can be seen from the figure, the error regions of R′ are formed by the area within the rectangle R but outside the rectangle R′ and the area within R′ but outside the rectangle R. The first area corresponds to *false negatives*, that is, points that are labeled as 0 or *negatively* by R′, which are in fact *positive* or labeled with 1. The second area corresponds to *false positives*, that is, points labeled positively by R′ which are in fact negatively labeled.

To show that the concept class is PAC-learnable, we describe a simple PAC-learning algorithm $\mathcal{A}$. Given a labeled sample S, the algorithm consists of returning the tightest axis-aligned rectangle $\mathsf{R}' = \mathsf{R}_S$ containing the points labeled with 1. Figure 2.2 illustrates the hypothesis returned by the algorithm. By definition, R_S does not produce any false positives, since its points must be included in the target concept R. Thus, the error region of R_S is included in R.

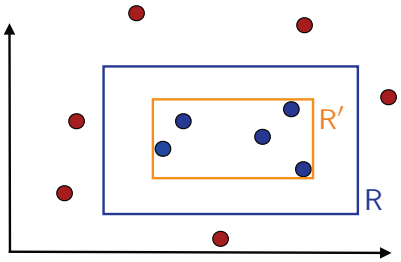


Figure 2.2
Illustration of the hypothesis $\mathsf{R}' = \mathsf{R}_S$ returned by the algorithm.

Let $\mathsf{R} \in \mathcal{C}$ be a target concept. Fix $\epsilon > 0$. Let $\mathbb{P}[\mathsf{R}]$ denote the probability mass of the region defined by R, that is the probability that a point randomly drawn according to $\mathcal{D}$ falls within R. Since errors made by our algorithm can be due only to points falling inside R, we can assume that $\mathbb{P}[\mathsf{R}] > \epsilon$; otherwise, the error of R_S is less than or equal to ϵ regardless of the training sample S received.

Now, since $\mathbb{P}[\mathsf{R}] > \epsilon$, we can define four rectangular regions r_1, r_2, r_3, and r_4 along the sides of R, each with probability at least $\epsilon/4$. These regions can be constructed by starting with the full rectangle R and then decreasing the size by moving one side as much as possible while keeping a distribution mass of at least $\epsilon/4$. Figure 2.3 illustrates the definition of these regions.

Let l, r, b, and t be the four real values defining R: $\mathsf{R} = [l, r] \times [b, t]$. Then, for example, the left rectangle r_4 is defined by $r_4 = [l, s_4] \times [b, t]$, with $s_4 = \inf\{s\colon \mathbb{P}[[l, s] \times [b, t]] \geq \epsilon/4\}$. It is not hard to see that the probability of the region $\overline{r}_4 = [l, s_4[\times[b, t]$ obtained from r_4 by excluding the rightmost side is at most $\epsilon/4$. r_1, r_2, r_3 and $\overline{r}_1$, $\overline{r}_2$, $\overline{r}_3$ are defined in a similar way.

Observe that if R_S meets all of these four regions r_i, $i \in [4]$, then, because it is a rectangle, it will have one side in each of these regions (geometric argument). Its error area, which is the part of R that it does not cover, is thus included in the union of the regions $\overline{r}_i$, $i \in [4]$, and cannot have probability mass more than ϵ. By contraposition, if $R(\mathsf{R}_S) > \epsilon$, then R_S must miss at least one of the regions r_i, $i \in [4]$. As a result, we can write

$$\begin{aligned}
\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}[R(\mathsf{R}_S) > \epsilon] &\leq \mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}[\cup_{i=1}^4 \{\mathsf{R}_S \cap r_i = \emptyset\}] && (2.5)\\
&\leq \sum_{i=1}^4 \mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}[\{\mathsf{R}_S \cap r_i = \emptyset\}] && \text{(by the union bound)}\\
&\leq 4(1-\epsilon/4)^m && \text{(since } \mathbb{P}[r_i] \geq \epsilon/4)\\
&\leq 4\exp(-m\epsilon/4),
\end{aligned}$$

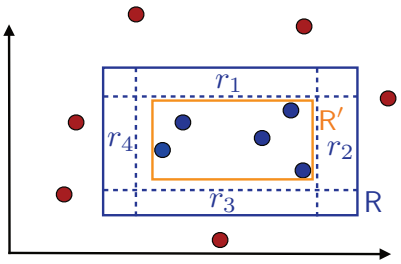


Figure 2.3
Illustration of the regions $r_1, \ldots, r_4$.

where for the last step we used the general inequality $1 - x \leq e^{-x}$ valid for all $x \in \mathbb{R}$. For any $\delta > 0$, to ensure that $\mathbb{P}_{S \sim \mathcal{D}^m}[R(\mathsf{R_S}) > \epsilon] \leq \delta$, we can impose

$$4 \exp(-\epsilon m/4) \leq \delta \Leftrightarrow m \geq \frac{4}{\epsilon} \log \frac{4}{\delta}. \tag{2.6}$$

Thus, for any $\epsilon > 0$ and $\delta > 0$, if the sample size m is greater than $\frac{4}{\epsilon} \log \frac{4}{\delta}$, then $\mathbb{P}_{S \sim \mathcal{D}^m}[R(\mathsf{R_S}) > \epsilon] \leq \delta$. Furthermore, the computational cost of the representation of points in $\mathbb{R}^2$ and axis-aligned rectangles, which can be defined by their four corners, is constant. This proves that the concept class of axis-aligned rectangles is PAC-learnable and that the sample complexity of PAC-learning axis-aligned rectangles is in $O(\frac{1}{\epsilon} \log \frac{1}{\delta})$.

An equivalent way to present sample complexity results like (2.6), which we will often see throughout this book, is to give a *generalization bound*. A generalization bound states that with probability at least $1 - \delta$, $R(\mathsf{R_S})$ is upper bounded by some quantity that depends on the sample size m and δ. To obtain this, it suffices to set δ to be equal to the upper bound derived in (2.5), that is $\delta = 4 \exp(-m\epsilon/4)$ and solve for ϵ. This yields that with probability at least $1 - \delta$, the error of the algorithm is bounded as follows:

$$R(\mathsf{R_S}) \leq \frac{4}{m} \log \frac{4}{\delta}. \tag{2.7}$$

Other PAC-learning algorithms could be considered for this example. One alternative is to return the largest axis-aligned rectangle not containing the negative points, for example. The proof of PAC-learning just presented for the tightest axis-aligned rectangle can be easily adapted to the analysis of other such algorithms.

Note that the hypothesis set $\mathcal{H}$ we considered in this example coincided with the concept class $\mathcal{C}$ and that its cardinality was infinite. Nevertheless, the problem admitted a simple proof of PAC-learning. We may then ask if a similar proof can readily apply to other similar concept classes. This is not as straightforward because the specific geometric argument used in the proof is key. It is non-trivial to extend the proof to other concept classes such as that of non-concentric circles

(see exercise 2.4). Thus, we need a more general proof technique and more general results. The next two sections provide us with such tools in the case of a finite hypothesis set.

2.2 Guarantees for finite hypothesis sets — consistent case

In the example of axis-aligned rectangles that we examined, the hypothesis h_S returned by the algorithm was always *consistent*, that is, it admitted no error on the training sample S. In this section, we present a general sample complexity bound, or equivalently, a generalization bound, for consistent hypotheses, in the case where the cardinality $|\mathcal{H}|$ of the hypothesis set is finite. Since we consider consistent hypotheses, we will assume that the target concept c is in $\mathcal{H}$.

Theorem 2.5 (Learning bound — finite $\mathcal{H}$, consistent case) *Let $\mathcal{H}$ be a finite set of functions mapping from $\mathcal{X}$ to $\mathcal{Y}$. Let $\mathcal{A}$ be an algorithm that for any target concept $c \in \mathcal{H}$ and i.i.d. sample S returns a consistent hypothesis h_S: $\widehat{R}_S(h_S) = 0$. Then, for any $\epsilon, \delta > 0$, the inequality $\mathbb{P}_{S\sim\mathcal{D}^m}[R(h_S) \leq \epsilon] \geq 1 - \delta$ holds if*

$$m \geq \frac{1}{\epsilon}\Big(\log|\mathcal{H}| + \log\frac{1}{\delta}\Big). \tag{2.8}$$

This sample complexity result admits the following equivalent statement as a generalization bound: for any $\epsilon, \delta > 0$, with probability at least $1 - \delta$,

$$R(h_S) \leq \frac{1}{m}\Big(\log|\mathcal{H}| + \log\frac{1}{\delta}\Big). \tag{2.9}$$

Proof: Fix $\epsilon > 0$. We do not know which consistent hypothesis $h_S \in \mathcal{H}$ is selected by the algorithm $\mathcal{A}$. This hypothesis further depends on the training sample S. Therefore, we need to give a *uniform convergence bound*, that is, a bound that holds for the set of all consistent hypotheses, which a fortiori includes h_S. Thus, we will bound the probability that some $h \in \mathcal{H}$ would be consistent and have error more than ϵ. For any $\epsilon > 0$, define $\mathcal{H}_\epsilon$ by $\mathcal{H}_\epsilon = \{h \in \mathcal{H} \colon R(h) > \epsilon\}$. The probability that a hypothesis h in $\mathcal{H}_\epsilon$ is consistent on a training sample S drawn i.i.d., that is, that it would have no error on any point in S, can be bounded as follows:

$$\mathbb{P}[\widehat{R}_S(h) = 0] \leq (1-\epsilon)^m.$$

Thus, by the union bound, the following holds:

$$\begin{aligned}
\mathbb{P}\Big[\exists h \in \mathcal{H}_\epsilon \colon \widehat{R}_S(h) = 0\Big] &= \mathbb{P}\Big[\widehat{R}_S(h_1) = 0 \vee \cdots \vee \widehat{R}_S(h_{|\mathcal{H}_\epsilon|}) = 0\Big] \\
&\leq \sum_{h\in\mathcal{H}_\epsilon} \mathbb{P}\Big[\widehat{R}_S(h) = 0\Big] \qquad \text{(union bound)} \\
&\leq \sum_{h\in\mathcal{H}_\epsilon} (1-\epsilon)^m \leq |\mathcal{H}|(1-\epsilon)^m \leq |\mathcal{H}|e^{-m\epsilon}.
\end{aligned}$$

Setting the right-hand side to be equal to δ and solving for ϵ concludes the proof. □

The theorem shows that when the hypothesis set $\mathcal{H}$ is finite, a consistent algorithm $\mathcal{A}$ is a PAC-learning algorithm, since the sample complexity given by (2.8) is dominated by a polynomial in $1/\epsilon$ and $1/\delta$. As shown by (2.9), the generalization error of consistent hypotheses is upper bounded by a term that decreases as a function of the sample size m. This is a general fact: as expected, learning algorithms benefit from larger labeled training samples. The decrease rate of $O(1/m)$ guaranteed by this theorem, however, is particularly favorable.

The price to pay for coming up with a consistent algorithm is the use of a larger hypothesis set $\mathcal{H}$ containing target concepts. Of course, the upper bound (2.9) increases with $|\mathcal{H}|$. However, that dependency is only logarithmic. Note that the term $\log|\mathcal{H}|$, or the related term $\log_2|\mathcal{H}|$ from which it differs by a constant factor, can be interpreted as the number of bits needed to represent $\mathcal{H}$. Thus, the generalization guarantee of the theorem is controlled by the ratio of this number of bits, $\log_2|\mathcal{H}|$, and the sample size m.

We now use theorem 2.5 to analyze PAC-learning with various concept classes.

Example 2.6 (Conjunction of Boolean literals) Consider learning the concept class $\mathcal{C}_n$ of conjunctions of at most n Boolean literals $x_1, \ldots, x_n$. A Boolean literal is either a variable x_i, $i \in [n]$, or its negation $\overline{x}_i$. For $n = 4$, an example is the conjunction: $x_1 \wedge \overline{x}_2 \wedge x_4$, where $\overline{x}_2$ denotes the negation of the Boolean literal x_2. $(1, 0, 0, 1)$ is a positive example for this concept while $(1, 0, 0, 0)$ is a negative example.

Observe that for $n = 4$, a positive example $(1, 0, 1, 0)$ implies that the target concept cannot contain the literals $\overline{x}_1$ and $\overline{x}_3$ and that it cannot contain the literals x_2 and x_4. In contrast, a negative example is not as informative since it is not known which of its n bits are incorrect. A simple algorithm for finding a consistent hypothesis is thus based on positive examples and consists of the following: for each positive example $(b_1, \ldots, b_n)$ and $i \in [n]$, if $b_i = 1$ then $\overline{x}_i$ is ruled out as a possible literal in the concept class and if $b_i = 0$ then x_i is ruled out. The conjunction of all the literals not ruled out is thus a hypothesis consistent with the target. Figure 2.4 shows an example training sample as well as a consistent hypothesis for the case $n = 6$.

We have $|\mathcal{H}| = |\mathcal{C}_n| = 3^n$, since each literal can be included positively, with negation, or not included. Plugging this into the sample complexity bound for consistent hypotheses yields the following sample complexity bound for any $\epsilon > 0$ and $\delta > 0$:

$$m \geq \frac{1}{\epsilon}\Big((\log 3)n + \log\frac{1}{\delta}\Big). \tag{2.10}$$

Thus, the class of conjunctions of at most n Boolean literals is PAC-learnable. Note that the computational complexity is also polynomial, since the training cost per example is in $O(n)$. For $\delta = 0.02$, $\epsilon = 0.1$, and $n = 10$, the bound becomes

0	1	1	0	1	1	+
0	1	1	1	1	1	+
0	0	1	1	0	1	-
0	1	1	1	1	1	+
1	0	0	1	1	0	-
0	1	0	0	1	1	+
0	1	?	?	1	1	

Figure 2.4
Each of the first six rows of the table represents a training example with its label, + or −, indicated in the last column. The last row contains 0 (respectively 1) in column $i \in [6]$ if the ith entry is 0 (respectively 1) for all the positive examples. It contains "?" if both 0 and 1 appear as an ith entry for some positive example. Thus, for this training sample, the hypothesis returned by the consistent algorithm described in the text is $\overline{x}_1 \wedge x_2 \wedge x_5 \wedge x_6$.

$m \geq 149$. Thus, for a labeled sample of at least 149 examples, the bound guarantees 90% accuracy with a confidence of at least 98%.

Example 2.7 (Universal concept class) Consider the set $\mathcal{X} = \{0,1\}^n$ of all Boolean vectors with n components, and let $\mathcal{U}_n$ be the concept class formed by all subsets of $\mathcal{X}$. Is this concept class PAC-learnable? To guarantee a consistent hypothesis the hypothesis class must include the concept class, thus $|\mathcal{H}| \geq |\mathcal{U}_n| = 2^{(2^n)}$. Theorem 2.5 gives the following sample complexity bound:

$$m \geq \frac{1}{\epsilon}\left((\log 2)2^n + \log\frac{1}{\delta}\right). \tag{2.11}$$

Here, the number of training samples required is exponential in n, which is the cost of the representation of a point in $\mathcal{X}$. Thus, PAC-learning is not guaranteed by the theorem. In fact, it is not hard to show that this universal concept class is not PAC-learnable.

Example 2.8 (k-term DNF formulae) A disjunctive normal form (DNF) formula is a formula written as the disjunction of several terms, each term being a conjunction of Boolean literals. A k-term DNF is a DNF formula defined by the disjunction of k terms, each term being a conjunction of at most n Boolean literals. Thus, for $k = 2$ and $n = 3$, an example of a k-term DNF is $(x_1 \wedge \overline{x}_2 \wedge x_3) \vee (\overline{x}_1 \wedge x_3)$.

Is the class $\mathcal{C}$ of k-term DNF formulae PAC-learnable? The cardinality of the class is 3^{nk}, since each term is a conjunction of at most n variables and there are 3^n such conjunctions, as seen previously. The hypothesis set $\mathcal{H}$ must contain $\mathcal{C}$ for

consistency to be possible, thus $|\mathcal{H}| \geq 3^{nk}$. Theorem 2.5 gives the following sample complexity bound:

$$m \geq \frac{1}{\epsilon}\Big((\log 3)nk + \log\frac{1}{\delta}\Big), \tag{2.12}$$

which is polynomial. However, it can be shown by a reduction from the graph 3-coloring problem that the problem of learning k-term DNF, even for $k = 3$, is not efficiently PAC-learnable, unless RP, the complexity class of problems that admit a randomized polynomial-time decision solution, coincides with NP(RP = NP), which is commonly conjectured not to be the case. Thus, while the sample size needed for learning k-term DNF formulae is only polynomial, efficient PAC-learning of this class is not possible if RP $\neq$ NP.

Example 2.9 (k-CNF formulae) A conjunctive normal form (CNF) formula is a conjunction of disjunctions. A k-CNF formula is an expression of the form $T_1 \wedge \ldots \wedge T_j$ with arbitrary length $j \in \mathbb{N}$ and with each term T_i being a disjunction of at most k Boolean attributes.

The problem of learning k-CNF formulae can be reduced to that of learning conjunctions of Boolean literals, which, as seen previously, is a PAC-learnable concept class. This can be done at the cost of introducing $(2n)^k$ new variables $Y_{u_1,\ldots,u_k}$ using the following bijection:

$$(u_1, \ldots, u_k) \to Y_{u_1,\ldots,u_k}, \tag{2.13}$$

where $u_1, \ldots, u_k$ are Boolean literals over the original variables $x_1, \ldots, x_n$. The value of $Y_{u_1,\ldots,u_k}$ is determined by $Y_{u_1,\ldots,u_k} = u_1 \vee \cdots \vee u_k$. Using this mapping, the original training sample can be transformed into one defined in terms of the new variables and any k-CNF formula over the original variables can be written as a conjunction over the variables $Y_{u_1,\ldots,u_k}$. This reduction to PAC-learning of conjunctions of Boolean literals can affect the original distribution of examples, but this is not an issue since in the PAC framework no assumption is made about the distribution. Thus, using this transformation, the PAC-learnability of conjunctions of Boolean literals implies that of k-CNF formulae.

This is a surprising result, however, since any k-term DNF formula can be written as a k-CNF formula. Indeed, using associativity, a k-term DNF $T_1 \vee \cdots \vee T_k$ with $T_i = u_{i,1} \wedge \cdots \wedge u_{i,n_i}$ for $i \in [k]$ can be rewritten as a k-CNF formula via

$$\bigvee_{i=1}^{k} u_{i,1} \wedge \cdots \wedge u_{i,n_i} = \bigwedge_{j_1 \in [n_1], \ldots, j_k \in [n_k]} u_{1,j_1} \vee \cdots \vee u_{k,j_k},$$

To illustrate this rewriting in a specific case, observe, for example, that

$$(u_1 \wedge u_2 \wedge u_3) \vee (v_1 \wedge v_2 \wedge v_3) = \bigwedge_{i,j=1}^{3} (u_i \vee v_j).$$

But, as we previously saw, k-term DNF formulae are not efficiently PAC-learnable if RP $\neq$ NP! What can explain this apparent inconsistency? The issue is that converting into a k-term DNF a k-CNF formula we have learned (which is equivalent to a k-term DNF) is in general intractable if RP $\neq$ NP.

This example reveals some key aspects of PAC-learning, which include the cost of the representation of a concept and the choice of the hypothesis set. For a fixed concept class, learning can be intractable or not depending on the choice of the representation.

2.3 Guarantees for finite hypothesis sets — inconsistent case

In the most general case, there may be no hypothesis in $\mathcal{H}$ consistent with the labeled training sample. This, in fact, is the typical case in practice, where the learning problems may be somewhat difficult or the concept classes more complex than the hypothesis set used by the learning algorithm. However, inconsistent hypotheses with a small number of errors on the training sample can be useful and, as we shall see, can benefit from favorable guarantees under some assumptions. This section presents learning guarantees precisely for this inconsistent case and finite hypothesis sets.

To derive learning guarantees in this more general setting, we will use Hoeffding's inequality (theorem D.2) or the following corollary, which relates the generalization error and empirical error of a single hypothesis.

Corollary 2.10 *Fix $\epsilon > 0$. Then, for any hypothesis $h\colon X \to \{0,1\}$, the following inequalities hold:*

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}\left[\widehat{R}_S(h) - R(h) \geq \epsilon\right] \leq \exp(-2m\epsilon^2) \tag{2.14}$$

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}\left[\widehat{R}_S(h) - R(h) \leq -\epsilon\right] \leq \exp(-2m\epsilon^2). \tag{2.15}$$

By the union bound, this implies the following two-sided inequality:

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}\left[\left|\widehat{R}_S(h) - R(h)\right| \geq \epsilon\right] \leq 2\exp(-2m\epsilon^2). \tag{2.16}$$

Proof: The result follows immediately from theorem D.2. □

Setting the right-hand side of (2.16) to be equal to δ and solving for ϵ yields immediately the following bound for a single hypothesis.

Corollary 2.11 (Generalization bound — single hypothesis) *Fix a hypothesis $h\colon \mathcal{X} \to \{0,1\}$. Then, for any $\delta > 0$, the following inequality holds with probability at least $1-\delta$:*

$$R(h) \leq \widehat{R}_S(h) + \sqrt{\frac{\log\frac{2}{\delta}}{2m}}. \tag{2.17}$$

The following example illustrates this corollary in a simple case.

Example 2.12 (Tossing a coin) Imagine tossing a biased coin that lands heads with probability p, and let our hypothesis be the one that always guesses tails. Then the true error rate is $R(h) = p$ and the empirical error rate $\widehat{R}_S(h) = \widehat{p}$, where $\widehat{p}$ is the empirical probability of heads based on the training sample drawn i.i.d. Thus, corollary 2.11 guarantees with probability at least $1 - \delta$ that

$$|p - \widehat{p}| \leq \sqrt{\frac{\log \frac{2}{\delta}}{2m}}. \tag{2.18}$$

Therefore, if we choose $\delta = 0.02$ and use a sample of size 500, with probability at least 98%, the following approximation quality is guaranteed for $\widehat{p}$:

$$|p - \widehat{p}| \leq \sqrt{\frac{\log(10)}{1000}} \approx 0.048. \tag{2.19}$$

Can we readily apply corollary 2.11 to bound the generalization error of the hypothesis h_S returned by a learning algorithm when training on a sample S? No, since h_S is not a fixed hypothesis, but a random variable depending on the training sample S drawn. Note also that unlike the case of a fixed hypothesis for which the expectation of the empirical error is the generalization error (equation (2.3)), the generalization error $R(h_S)$ is a random variable and in general distinct from the expectation $\mathbb{E}[\widehat{R}_S(h_S)]$, which is a constant.

Thus, as in the proof for the consistent case, we need to derive a uniform convergence bound, that is a bound that holds with high probability for all hypotheses $h \in \mathcal{H}$.

Theorem 2.13 (Learning bound — finite $\mathcal{H}$, inconsistent case) *Let $\mathcal{H}$ be a finite hypothesis set. Then, for any $\delta > 0$, with probability at least $1-\delta$, the following inequality holds:*

$$\forall h \in \mathcal{H}, \quad R(h) \leq \widehat{R}_S(h) + \sqrt{\frac{\log |\mathcal{H}| + \log \frac{2}{\delta}}{2m}}. \tag{2.20}$$

Proof: Let $h_1, \ldots, h_{|\mathcal{H}|}$ be the elements of $\mathcal{H}$. Using the union bound and applying corollary 2.11 to each hypothesis yield:

$$\begin{aligned}
&\mathbb{P}\left[\exists h \in \mathcal{H} \middle| \widehat{R}_S(h) - R(h)\middle| > \epsilon\right] \\
&\qquad = \mathbb{P}\left[\left(\middle|\widehat{R}_S(h_1) - R(h_1)\middle| > \epsilon\right) \vee \ldots \vee \left(\middle|\widehat{R}_S(h_{|\mathcal{H}|}) - R(h_{|\mathcal{H}|})\middle| > \epsilon\right)\right] \\
&\qquad \leq \sum_{h \in \mathcal{H}} \mathbb{P}\left[\middle|\widehat{R}_S(h) - R(h)\middle| > \epsilon\right] \\
&\qquad \leq 2|\mathcal{H}| \exp(-2m\epsilon^2).
\end{aligned}$$

Setting the right-hand side to be equal to δ completes the proof. □

Thus, for a finite hypothesis set $\mathcal{H}$,

$$R(h) \leq \widehat{R}_S(h) + O\left(\sqrt{\frac{\log_2 |\mathcal{H}|}{m}}\right).$$

As already pointed out, $\log_2 |\mathcal{H}|$ can be interpreted as the number of bits needed to represent $\mathcal{H}$. Several other remarks similar to those made on the generalization bound in the consistent case can be made here: a larger sample size m guarantees better generalization, and the bound increases with $|\mathcal{H}|$, but only logarithmically. But, here, the bound is a less favorable function of $\frac{\log_2 |\mathcal{H}|}{m}$; it varies as the square root of this term. This is not a minor price to pay: for a fixed $|\mathcal{H}|$, to attain the same guarantee as in the consistent case, a quadratically larger labeled sample is needed.

Note that the bound suggests seeking a trade-off between reducing the empirical error versus controlling the size of the hypothesis set: a larger hypothesis set is penalized by the second term but could help reduce the empirical error, that is the first term. But, for a similar empirical error, it suggests using a smaller hypothesis set. This can be viewed as an instance of the so-called *Occam's Razor principle* named after the theologian William of Occam: *Plurality should not be posited without necessity*, also rephrased as, *the simplest explanation is best*. In this context, it could be expressed as follows: All other things being equal, a simpler (smaller) hypothesis set is better.

2.4 Generalities

In this section we will discuss some general aspects of the learning scenario, which, for simplicity, we left out of the discussion of the earlier sections.

2.4.1 Deterministic versus stochastic scenarios

In the most general scenario of supervised learning, the distribution $\mathcal{D}$ is defined over $\mathcal{X} \times \mathcal{Y}$, and the training data is a labeled sample S drawn i.i.d. according to $\mathcal{D}$:

$$S = ((x_1, y_1), \ldots, (x_m, y_m)).$$

The learning problem is to find a hypothesis $h \in \mathcal{H}$ with small generalization error

$$R(h) = \mathop{\mathbb{P}}_{(x,y)\sim\mathcal{D}}[h(x) \neq y] = \mathop{\mathbb{E}}_{(x,y)\sim\mathcal{D}}[1_{h(x)\neq y}].$$

This more general scenario is referred to as the *stochastic scenario*. Within this setting, the output label is a probabilistic function of the input. The stochastic scenario captures many real-world problems where the label of an input point is not unique. For example, if we seek to predict gender based on input pairs formed by the height and weight of a person, then the label will typically not be unique.

For most pairs, both male and female are possible genders. For each fixed pair, there would be a probability distribution of the label being male.

The natural extension of the PAC-learning framework to this setting is known as the *agnostic PAC-learning*.

Definition 2.14 (Agnostic PAC-learning) *Let $\mathcal{H}$ be a hypothesis set. $\mathcal{A}$ is an* agnostic PAC-learning *algorithm if there exists a polynomial function $poly(\cdot,\cdot,\cdot,\cdot)$ such that for any $\epsilon > 0$ and $\delta > 0$, for all distributions $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, the following holds for any sample size $m \geq poly(1/\epsilon, 1/\delta, n, size(c))$:*

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}[R(h_S) - \min_{h\in\mathcal{H}} R(h) \leq \epsilon] \geq 1 - \delta. \tag{2.21}$$

If $\mathcal{A}$ further runs in $poly(1/\epsilon, 1/\delta, n)$, then it is said to be an efficient agnostic PAC-learning algorithm.

When the label of a point can be uniquely determined by some measurable function $f\colon \mathcal{X} \to \mathcal{Y}$ (with probability one), then the scenario is said to be *deterministic*. In that case, it suffices to consider a distribution $\mathcal{D}$ over the input space. The training sample is obtained by drawing $(x_1, \ldots, x_m)$ according to $\mathcal{D}$ and the labels are obtained via f: $y_i = f(x_i)$ for all $i \in [m]$. Many learning problems can be formulated within this deterministic scenario.

In the previous sections, as well as in most of the material presented in this book, we have restricted our presentation to the deterministic scenario in the interest of simplicity. However, for all of this material, the extension to the stochastic scenario should be straightforward for the reader.

2.4.2 Bayes error and noise

In the deterministic case, by definition, there exists a target function f with no generalization error: $R(h) = 0$. In the stochastic case, there is a minimal non-zero error for any hypothesis.

Definition 2.15 (Bayes error) *Given a distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, the* Bayes error R^* *is defined as the infimum of the errors achieved by measurable functions $h\colon \mathcal{X} \to \mathcal{Y}$:*

$$R^* = \inf_{\substack{h \\ h \text{ measurable}}} R(h). \tag{2.22}$$

A hypothesis h with $R(h) = R^*$ is called a Bayes hypothesis *or* Bayes classifier.

By definition, in the deterministic case, we have $R^* = 0$, but, in the stochastic case, $R^* \neq 0$. Clearly, the Bayes classifier h_{Bayes} can be defined in terms of the conditional probabilities as:

$$\forall x \in \mathcal{X}, \quad h_{\text{Bayes}}(x) = \operatorname*{argmax}_{y\in\{0,1\}} \mathbb{P}[y|x]. \tag{2.23}$$

The average error made by h_{Bayes} on $x \in \mathcal{X}$ is thus $\min\{\mathbb{P}[0|x], \mathbb{P}[1|x]\}$, and this is the minimum possible error. This leads to the following definition of *noise*.

Definition 2.16 (Noise) *Given a distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, the* noise *at point $x \in \mathcal{X}$ is defined by*

$$noise(x) = \min\{\mathbb{P}[1|x], \mathbb{P}[0|x]\}. \tag{2.24}$$

The average noise *or the* noise *associated to $\mathcal{D}$ is $\mathbb{E}[noise(x)]$.*

Thus, the average noise is precisely the Bayes error: noise $= \mathbb{E}[\text{noise}(x)] = R^*$. The noise is a characteristic of the learning task indicative of its level of difficulty. A point $x \in \mathcal{X}$, for which noise(x) is close to $1/2$, is sometimes referred to as *noisy* and is of course a challenge for accurate prediction.

2.5 Chapter notes

The PAC learning framework was introduced by Valiant [1984]. The book of Kearns and Vazirani [1994] is an excellent reference dealing with most aspects of PAC-learning and several other foundational questions in machine learning. Our example of learning axis-aligned rectangles, also discussed in that reference, is originally due to Blumer et al. [1989].

The PAC learning framework is a computational framework since it takes into account the cost of the computational representations and the time complexity of the learning algorithm. If we omit the computational aspects, it is similar to the learning framework considered earlier by Vapnik and Chervonenkis [see Vapnik, 2000]. The definition of noise presented in this chapter can be generalized to arbitrary loss functions (see exercise 2.14).

Occam's razor principle is invoked in a variety of contexts, such as in linguistics to justify the superiority of a set of rules or syntax. The Kolmogorov complexity can be viewed as the corresponding framework in information theory. In the context of the learning guarantees presented in this chapter, the principle suggests selecting the most parsimonious explanation (the hypothesis set with the smallest cardinality). We will see in the next sections other applications of this principle with different notions of simplicity or complexity.

2.6 Exercises

2.1 Two-oracle variant of the PAC model. Assume that positive and negative examples are now drawn from two separate distributions $\mathcal{D}_+$ and $\mathcal{D}_-$. For an accuracy $(1-\epsilon)$, the learning algorithm must find a hypothesis h such that:

$$\mathop{\mathbb{P}}_{x \sim \mathcal{D}_+}[h(x) = 0] \leq \epsilon \text{ and } \mathop{\mathbb{P}}_{x \sim \mathcal{D}_-}[h(x) = 1] \leq \epsilon\,. \tag{2.25}$$

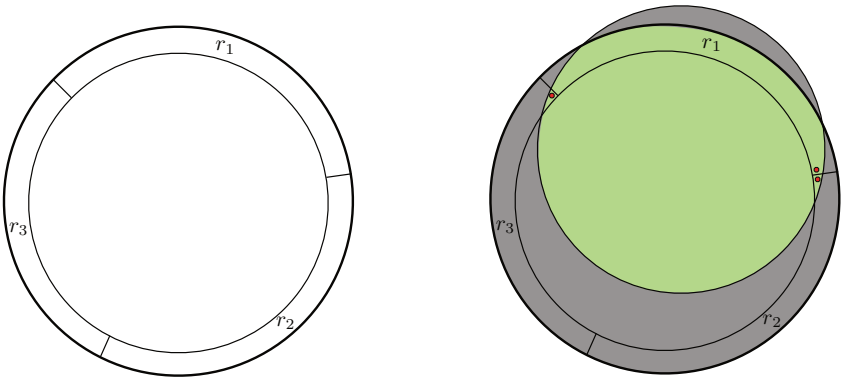


Figure 2.5
(a) Gertrude's regions r_1, r_2, r_3. (b) Hint for solution.

Thus, the hypothesis must have a small error on both distributions. Let $\mathcal{C}$ be any concept class and $\mathcal{H}$ be any hypothesis space. Let h_0 and h_1 represent the identically 0 and identically 1 functions, respectively. Prove that $\mathcal{C}$ is efficiently PAC-learnable using $\mathcal{H}$ in the standard (one-oracle) PAC model if and only if it is efficiently PAC-learnable using $\mathcal{H} \cup \{h_0, h_1\}$ in this two-oracle PAC model.

2.2 PAC learning of hyper-rectangles. An axis-aligned hyper-rectangle in $\mathbb{R}^n$ is a set of the form $[a_1, b_1] \times \ldots \times [a_n, b_n]$. Show that axis-aligned hyper-rectangles are PAC-learnable by extending the proof given in Example 2.4 for the case $n = 2$.

2.3 Concentric circles. Let $\mathcal{X} = \mathbb{R}^2$ and consider the set of concepts of the form $c = \{(x, y) \colon x^2 + y^2 \leq r^2\}$ for some real number r. Show that this class can be (ϵ, δ)-PAC-learned from training data of size $m \geq (1/\epsilon)\log(1/\delta)$.

2.4 Non-concentric circles. Let $\mathcal{X} = \mathbb{R}^2$ and consider the set of concepts of the form $c = \{x \in \mathbb{R}^2 \colon ||x - x_0|| \leq r\}$ for some point $x_0 \in \mathbb{R}^2$ and real number r. Gertrude, an aspiring machine learning researcher, attempts to show that this class of concepts may be (ϵ, δ)-PAC-learned with sample complexity $m \geq (3/\epsilon)\log(3/\delta)$, but she is having trouble with her proof. Her idea is that the learning algorithm would select the smallest circle consistent with the training data. She has drawn three regions r_1, r_2, r_3 around the edge of concept c, with each region having probability $\epsilon/3$ (see figure 2.5(a)). She wants to argue that if the generalization error is greater than or equal to ϵ, then one of these regions must have been missed by the training data, and hence this event will occur with probability at most δ. Can you tell Gertrude if her approach works? (*Hint*: You may wish to use figure 2.5(b) in your solution).

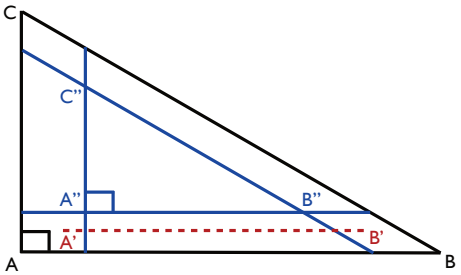


Figure 2.6
Axis-aligned right triangles.

2.5 Triangles. Let $\mathcal{X} = \mathbb{R}^2$ with orthonormal basis $(\mathbf{e}_1, \mathbf{e}_2)$, and consider the set of concepts defined by the area inside a right triangle ABC with two sides parallel to the axes, with $\overrightarrow{AB}/\|\overrightarrow{AB}\| = \mathbf{e}_1$ and $\overrightarrow{AC}/\|\overrightarrow{AC}\| = \mathbf{e}_2$, and $\|\overrightarrow{AB}\|/\|\overrightarrow{AC}\| = \alpha$ for some positive real $\alpha \in \mathbb{R}_+$. Show, using similar methods to those used in the chapter for the axis-aligned rectangles, that this class can be (ϵ, δ)-PAC-learned from training data of size $m \geq (3/\epsilon)\log(3/\delta)$. (*Hint*: You may consider using figure 2.6 in your solution).

2.6 Learning in the presence of noise — rectangles. In example 2.4, we showed that the concept class of axis-aligned rectangles is PAC-learnable. Consider now the case where the training points received by the learner are subject to the following noise: points negatively labeled are unaffected by noise but the label of a positive training point is randomly flipped to negative with probability $\eta \in (0, \frac{1}{2})$. The exact value of the noise rate η is not known to the learner but an upper bound η' is supplied to him with $\eta \leq \eta' < 1/2$. Show that the algorithm returning the tightest rectangle containing positive points can still PAC-learn axis-aligned rectangles in the presence of this noise. To do so, you can proceed using the following steps:

(a) Using the same notation as in example 2.4, assume that $\mathbb{P}[\mathsf{R}] > \epsilon$. Suppose that $R(\mathsf{R}') > \epsilon$. Give an upper bound on the probability that R' misses a region r_j, $j \in [4]$ in terms of ϵ and η'?

(b) Use that to give an upper bound on $\mathbb{P}[R(\mathsf{R}') > \epsilon]$ in terms of ϵ and η' and conclude by giving a sample complexity bound.

2.7 Learning in the presence of noise — general case. In this question, we will seek a result that is more general than in the previous question. We consider a finite hypothesis set $\mathcal{H}$, assume that the target concept is in $\mathcal{H}$, and adopt the following noise model: the label of a training point received by the learner is

randomly changed with probability $\eta \in (0, \frac{1}{2})$. The exact value of the noise rate η is not known to the learner but an upper bound η' is supplied to him with $\eta \leq \eta' < 1/2$.

(a) For any $h \in \mathcal{H}$, let $d(h)$ denote the probability that the label of a training point received by the learner disagrees with the one given by h. Let h^* be the target hypothesis, show that $d(h^*) = \eta$.

(b) More generally, show that for any $h \in \mathcal{H}$, $d(h) = \eta + (1 - 2\eta)\, R(h)$, where $R(h)$ denotes the generalization error of h.

(c) Fix $\epsilon > 0$ for this and all the following questions. Use the previous questions to show that if $R(h) > \epsilon$, then $d(h) - d(h^*) \geq \epsilon'$, where $\epsilon' = \epsilon(1 - 2\eta')$.

(d) For any hypothesis $h \in \mathcal{H}$ and sample S of size m, let $\widehat{d}(h)$ denote the fraction of the points in S whose labels disagree with those given by h. We will consider the algorithm L which, after receiving S, returns the hypothesis h_S with the smallest number of disagreements (thus $\widehat{d}(h_S)$ is minimal). To show PAC-learning for L, we will show that for any h, if $R(h) > \epsilon$, then with high probability $\widehat{d}(h) \geq \widehat{d}(h^*)$. First, show that for any $\delta > 0$, with probability at least $1 - \delta/2$, for $m \geq \frac{2}{\epsilon'^2} \log \frac{2}{\delta}$, the following holds:

$$\widehat{d}(h^*) - d(h^*) \leq \epsilon'/2$$

(e) Second, show that for any $\delta > 0$, with probability at least $1 - \delta/2$, for $m \geq \frac{2}{\epsilon'^2}(\log |\mathcal{H}| + \log \frac{2}{\delta})$, the following holds for all $h \in \mathcal{H}$:

$$d(h) - \widehat{d}(h) \leq \epsilon'/2$$

(f) Finally, show that for any $\delta > 0$, with probability at least $1 - \delta$, for $m \geq \frac{2}{\epsilon^2(1-2\eta')^2}(\log |\mathcal{H}| + \log \frac{2}{\delta})$, the following holds for all $h \in \mathcal{H}$ with $R(h) > \epsilon$:

$$\widehat{d}(h) - \widehat{d}(h^*) \geq 0.$$

(*Hint*: use $\widehat{d}(h) - \widehat{d}(h^*) = [\widehat{d}(h) - d(h)] + [d(h) - d(h^*)] + [d(h^*) - \widehat{d}(h^*)]$ and use previous questions to lower bound each of these three terms).

2.8 Learning intervals. Give a PAC-learning algorithm for the concept class $\mathcal{C}$ formed by closed intervals $[a, b]$ with $a, b \in \mathbb{R}$.

2.9 Learning union of intervals. Give a PAC-learning algorithm for the concept class $\mathcal{C}_2$ formed by unions of two closed intervals, that is $[a, b] \cup [c, d]$, with $a, b, c, d \in \mathbb{R}$. Extend your result to derive a PAC-learning algorithm for the concept class $\mathcal{C}_p$ formed by unions of $p \geq 1$ closed intervals, thus $[a_1, b_1] \cup \cdots \cup [a_p, b_p]$, with $a_k, b_k \in \mathbb{R}$ for $k \in [p]$. What are the time and sample complexities of your algorithm as a function of p?

2.10 Consistent hypotheses. In this chapter, we showed that for a finite hypothesis set $\mathcal{H}$, a consistent learning algorithm $\mathcal{A}$ is a PAC-learning algorithm. Here, we consider a converse question. Let $\mathcal{Z}$ be a finite set of m labeled points. Suppose that you are given a PAC-learning algorithm $\mathcal{A}$. Show that you can use $\mathcal{A}$ and a finite training sample S to find in polynomial time a hypothesis $h \in \mathcal{H}$ that is consistent with $\mathcal{Z}$, with high probability. (*Hint*: you can select an appropriate distribution $\mathcal{D}$ over $\mathcal{Z}$ and give a condition on $R(h)$ for h to be consistent.)

2.11 Senate laws. For important questions, President Mouth relies on expert advice. He selects an appropriate advisor from a collection of $\mathcal{H} = 2{,}800$ experts.

(a) Assume that laws are proposed in a random fashion independently and identically according to some distribution $\mathcal{D}$ determined by an unknown group of senators. Assume that President Mouth can find and select an expert senator out of $\mathcal{H}$ who has consistently voted with the majority for the last $m = 200$ laws. Give a bound on the probability that such a senator incorrectly predicts the global vote for a future law. What is the value of the bound with 95% confidence?

(b) Assume now that President Mouth can find and select an expert senator out of $\mathcal{H}$ who has consistently voted with the majority for all but $m' = 20$ of the last $m = 200$ laws. What is the value of the new bound?

2.12 Bayesian bound. Let $\mathcal{H}$ be a countable hypothesis set of functions mapping $\mathcal{X}$ to $\{0, 1\}$ and let p be a probability measure over $\mathcal{H}$. This probability measure represents the *prior probability* over the hypothesis class, i.e. the probability that a particular hypothesis is selected by the learning algorithm. Use Hoeffding's inequality to show that for any $\delta > 0$, with probability at least $1 - \delta$, the following inequality holds:

$$\forall h \in \mathcal{H}, R(h) \leq \widehat{R}_S(h) + \sqrt{\frac{\log \frac{1}{p(h)} + \log \frac{1}{\delta}}{2m}}. \tag{2.26}$$

Compare this result with the bound given in the inconsistent case for finite hypothesis sets (*Hint*: you could use $\delta' = p(h)\delta$ as confidence parameter in Hoeffding's inequality).

2.13 Learning with an unknown parameter. In example 2.9, we showed that the concept class of k-CNF is PAC-learnable. Note, however, that the learning algorithm is given k as input. Is PAC-learning possible even when k is not provided? More generally, consider a family of concept classes $\{\mathcal{C}_s\}_s$ where $\mathcal{C}_s$ is the set of concepts in $\mathcal{C}$ with size at most s. Suppose we have a PAC-learning algorithm $\mathcal{A}$ that can be used for learning any concept class $\mathcal{C}_s$ when s is given.

Can we convert $\mathcal{A}$ into a PAC-learning algorithm $\mathcal{B}$ that does not require the knowledge of s? This is the main objective of this problem.

To do this, we first introduce a method for testing a hypothesis h, with high probability. Fix $\epsilon > 0$, $\delta > 0$, and $i \geq 1$ and define the sample size n by $n = \frac{32}{\epsilon}[i \log 2 + \log \frac{2}{\delta}]$. Suppose we draw an i.i.d. sample S of size n according to some unknown distribution $\mathcal{D}$. We will say that a hypothesis h is *accepted* if it makes at most $3/4\epsilon$ errors on S and that it is *rejected* otherwise. Thus, h is accepted iff $\widehat{R}(h) \leq 3/4\epsilon$.

(a) Assume that $R(h) \geq \epsilon$. Use the (multiplicative) Chernoff bound to show that in that case $\mathbb{P}_{S \sim \mathcal{D}^n}[h \text{ is accepted}] \leq \frac{\delta}{2^{i+1}}$.

(b) Assume that $R(h) \leq \epsilon/2$. Use the (multiplicative) Chernoff bounds to show that in that case $\mathbb{P}_{S \sim \mathcal{D}^n}[h \text{ is rejected}] \leq \frac{\delta}{2^{i+1}}$.

(c) Algorithm $\mathcal{B}$ is defined as follows: we start with $i = 1$ and, at each round $i \geq 1$, we guess the parameter size s to be $\widetilde{s} = \lfloor 2^{(i-1)/\log \frac{2}{\delta}} \rfloor$. We draw a sample S of size n (which depends on i) to test the hypothesis h_i returned by $\mathcal{A}$ when it is trained with a sample of size $S_{\mathcal{A}}(\epsilon/2, 1/2, \widetilde{s})$, that is the sample complexity of $\mathcal{A}$ for a required precision $\epsilon/2$, confidence $1/2$, and size $\widetilde{s}$ (we ignore the size of the representation of each example here). If h_i is accepted, the algorithm stops and returns h_i, otherwise it proceeds to the next iteration. Show that if at iteration i, the estimate $\widetilde{s}$ is larger than or equal to s, then $\mathbb{P}[h_i \text{ is accepted}] \geq 3/8$.

(d) Show that the probability that $\mathcal{B}$ does not halt after $j = \lceil \log \frac{2}{\delta} / \log \frac{8}{5} \rceil$ iterations with $\widetilde{s} \geq s$ is at most $\delta/2$.

(e) Show that for $i \geq \lceil 1 + (\log_2 s) \log \frac{2}{\delta} \rceil$, the inequality $\widetilde{s} \geq s$ holds.

(f) Show that with probability at least $1 - \delta$, algorithm $\mathcal{B}$ halts after at most $j' = \lceil 1 + (\log_2 s) \log \frac{2}{\delta} \rceil + j$ iterations and returns a hypothesis with error at most ϵ.

2.14 In this exercise, we generalize the notion of noise to the case of an arbitrary loss function $L \colon \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_+$.

(a) Justify the following definition of the noise at point $x \in \mathcal{X}$:

$$\text{noise}(x) = \min_{y' \in \mathcal{Y}} \mathop{\mathbb{E}}_{y}[L(y, y')|x].$$

What is the value of noise(x) in a deterministic scenario? Does the definition match the one given in this chapter for binary classification?

(b) Show that the average noise coincides with the Bayes error (minimum loss achieved by a measurable function).

3 Rademacher Complexity and VC-Dimension

The hypothesis sets typically used in machine learning are infinite. But the sample complexity bounds of the previous chapter are uninformative when dealing with infinite hypothesis sets. One could ask whether efficient learning from a finite sample is even possible when the hypothesis set $\mathcal{H}$ is infinite. Our analysis of the family of axis-aligned rectangles (Example 2.4) indicates that this is indeed possible at least in some cases, since we proved that that infinite concept class was PAC-learnable. Our goal in this chapter will be to generalize that result and derive general learning guarantees for infinite hypothesis sets.

A general idea for doing so consists of reducing the infinite case to the analysis of finite sets of hypotheses and then proceed as in the previous chapter. There are different techniques for that reduction, each relying on a different notion of complexity for the family of hypotheses. The first complexity notion we will use is that of *Rademacher complexity*. This will help us derive learning guarantees using relatively simple proofs based on McDiarmid's inequality, while obtaining high-quality bounds, including data-dependent ones, which we will frequently make use of in future chapters. However, the computation of the empirical Rademacher complexity is NP-hard for some hypothesis sets. Thus, we subsequently introduce two other purely combinatorial notions, the *growth function* and the *VC-dimension*. We first relate the Rademacher complexity to the growth function and then bound the growth function in terms of the VC-dimension. The VC-dimension is often easier to bound or estimate. We will review a series of examples showing how to compute or bound it, then relate the growth function and the VC-dimensions. This leads to generalization bounds based on the VC-dimension. Finally, we present lower bounds based on the VC-dimension for two different settings: The *realizable* setting, where there is at least one hypothesis in the hypothesis set under consideration that achieves zero expected error, as well as the *non-realizable* setting, where no hypothesis in the set achieves zero expected error.

3.1 Rademacher complexity

We will continue to use $\mathcal{H}$ to denote a hypothesis set as in the previous chapters. Many of the results of this section are general and hold for an arbitrary loss function $L\colon \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$. In what follows, $\mathcal{G}$ will generally be interpreted as *the family of loss functions associated to* $\mathcal{H}$ mapping from $\mathcal{Z} = \mathcal{X} \times \mathcal{Y}$ to $\mathbb{R}$:

$$\mathcal{G} = \{g\colon (x, y) \mapsto L(h(x), y)\colon h \in \mathcal{H}\}.$$

However, the definitions are given in the general case of a family of functions $\mathcal{G}$ mapping from an arbitrary input space $\mathcal{Z}$ to $\mathbb{R}$.

The Rademacher complexity captures the richness of a family of functions by measuring the degree to which a hypothesis set can fit random noise. The following states the formal definitions of the empirical and average Rademacher complexity.

Definition 3.1 (Empirical Rademacher complexity) *Let $\mathcal{G}$ be a family of functions mapping from $\mathcal{Z}$ to $[a, b]$ and $S = (z_1, \ldots, z_m)$ a fixed sample of size m with elements in $\mathcal{Z}$. Then, the* empirical Rademacher complexity *of $\mathcal{G}$ with respect to the sample S is defined as:*

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) = \mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\sup_{g \in \mathcal{G}} \frac{1}{m} \sum_{i=1}^{m} \sigma_i g(z_i)\right], \tag{3.1}$$

where $\boldsymbol{\sigma} = (\sigma_1, \ldots, \sigma_m)^\top$, with σ_is independent uniform random variables taking values in $\{-1, +1\}$.[3] *The random variables σ_i are called* Rademacher variables.

Let $\mathbf{g}_S$ denote the vector of values taken by function g over the sample S: $\mathbf{g}_S = (g(z_1), \ldots, g(z_m))^\top$. Then, the empirical Rademacher complexity can be rewritten as

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) = \mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\sup_{g \in \mathcal{G}} \frac{\boldsymbol{\sigma} \cdot \mathbf{g}_S}{m}\right].$$

The inner product $\boldsymbol{\sigma} \cdot \mathbf{g}_S$ measures the correlation of $\mathbf{g}_S$ with the vector of random noise $\boldsymbol{\sigma}$. The supremum $\sup_{g \in \mathcal{G}} \frac{\boldsymbol{\sigma} \cdot \mathbf{g}_S}{m}$ is a measure of how well the function class $\mathcal{G}$ correlates with $\boldsymbol{\sigma}$ over the sample S. Thus, the empirical Rademacher complexity measures on average how well the function class $\mathcal{G}$ correlates with random noise on S. This describes the richness of the family $\mathcal{G}$: richer or more complex families $\mathcal{G}$ can generate more vectors $\mathbf{g}_S$ and thus better correlate with random noise, on average.

[3] We assume implicitly that the supremum over the family $\mathcal{G}$ in this definition is measurable and in general will adopt the same assumption throughout this book for other suprema over a class of functions. This assumption does not hold for arbitrary function classes but it is valid for the hypotheses sets typically considered in practice in machine learning, and the instances discussed in this book.

Definition 3.2 (Rademacher complexity) *Let $\mathcal{D}$ denote the distribution according to which samples are drawn. For any integer $m \geq 1$, the* Rademacher complexity *of $\mathcal{G}$ is the expectation of the empirical Rademacher complexity over all samples of size m drawn according to $\mathcal{D}$:*

$$\mathfrak{R}_m(\mathcal{G}) = \mathop{\mathbb{E}}_{S \sim \mathcal{D}^m}[\widehat{\mathfrak{R}}_S(\mathcal{G})]. \tag{3.2}$$

We are now ready to present our first generalization bounds based on Rademacher complexity.

Theorem 3.3 *Let $\mathcal{G}$ be a family of functions mapping from $\mathcal{Z}$ to $[0,1]$. Then, for any $\delta > 0$, with probability at least $1-\delta$ over the draw of an i.i.d. sample S of size m, each of the following holds for all $g \in \mathcal{G}$:*

$$\mathbb{E}[g(z)] \leq \frac{1}{m}\sum_{i=1}^{m} g(z_i) + 2\mathfrak{R}_m(\mathcal{G}) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}} \tag{3.3}$$

$$\textit{and} \quad \mathbb{E}[g(z)] \leq \frac{1}{m}\sum_{i=1}^{m} g(z_i) + 2\widehat{\mathfrak{R}}_S(\mathcal{G}) + 3\sqrt{\frac{\log \frac{2}{\delta}}{2m}}\,. \tag{3.4}$$

Proof: For any sample $S = (z_1, \ldots, z_m)$ and any $g \in \mathcal{G}$, we denote by $\widehat{\mathbb{E}}_S[g]$ the empirical average of g over S: $\widehat{\mathbb{E}}_S[g] = \frac{1}{m}\sum_{i=1}^{m} g(z_i)$. The proof consists of applying McDiarmid's inequality to function Φ defined for any sample S by

$$\Phi(S) = \sup_{g \in \mathcal{G}} \Big(\mathbb{E}[g] - \widehat{\mathbb{E}}_S[g] \Big). \tag{3.5}$$

Let S and S' be two samples differing by exactly one point, say z_m in S and z'_m in S'. Then, since the difference of suprema does not exceed the supremum of the difference, we have

$$\Phi(S') - \Phi(S) \leq \sup_{g \in \mathcal{G}} \Big(\widehat{\mathbb{E}}_S[g] - \widehat{\mathbb{E}}_{S'}[g] \Big) = \sup_{g \in \mathcal{G}} \frac{g(z_m) - g(z'_m)}{m} \leq \frac{1}{m}. \tag{3.6}$$

Similarly, we can obtain $\Phi(S) - \Phi(S') \leq 1/m$, thus $|\Phi(S) - \Phi(S')| \leq 1/m$. Then, by McDiarmid's inequality, for any $\delta > 0$, with probability at least $1 - \delta/2$, the following holds:

$$\Phi(S) \leq \mathop{\mathbb{E}}_{S}[\Phi(S)] + \sqrt{\frac{\log \frac{2}{\delta}}{2m}}. \tag{3.7}$$

We next bound the expectation of the right-hand side as follows:

$$\begin{aligned}
\mathop{\mathbb{E}}_{S}[\Phi(S)] &= \mathop{\mathbb{E}}_{S}\left[\sup_{g\in\mathcal{G}}\left(\mathbb{E}[g] - \widehat{\mathbb{E}}_S(g)\right)\right] \\
&= \mathop{\mathbb{E}}_{S}\left[\sup_{g\in\mathcal{G}}\mathop{\mathbb{E}}_{S'}\left[\widehat{\mathbb{E}}_{S'}(g) - \widehat{\mathbb{E}}_S(g)\right]\right] && (3.8)\\
&\leq \mathop{\mathbb{E}}_{S,S'}\left[\sup_{g\in\mathcal{G}}\left(\widehat{\mathbb{E}}_{S'}(g) - \widehat{\mathbb{E}}_S(g)\right)\right] && (3.9)\\
&= \mathop{\mathbb{E}}_{S,S'}\left[\sup_{g\in\mathcal{G}}\frac{1}{m}\sum_{i=1}^{m}(g(z'_i) - g(z_i))\right] && (3.10)\\
&= \mathop{\mathbb{E}}_{\boldsymbol{\sigma},S,S'}\left[\sup_{g\in\mathcal{G}}\frac{1}{m}\sum_{i=1}^{m}\sigma_i(g(z'_i) - g(z_i))\right] && (3.11)\\
&\leq \mathop{\mathbb{E}}_{\boldsymbol{\sigma},S'}\left[\sup_{g\in\mathcal{G}}\frac{1}{m}\sum_{i=1}^{m}\sigma_i g(z'_i)\right] + \mathop{\mathbb{E}}_{\boldsymbol{\sigma},S}\left[\sup_{g\in\mathcal{G}}\frac{1}{m}\sum_{i=1}^{m}-\sigma_i g(z_i)\right] && (3.12)\\
&= 2\mathop{\mathbb{E}}_{\boldsymbol{\sigma},S}\left[\sup_{g\in\mathcal{G}}\frac{1}{m}\sum_{i=1}^{m}\sigma_i g(z_i)\right] = 2\mathfrak{R}_m(\mathcal{G}). && (3.13)
\end{aligned}$$

Equation (3.8) uses the fact that points in S' are sampled in an i.i.d. fashion and thus $\mathbb{E}[g] = \mathbb{E}_{S'}[\widehat{\mathbb{E}}_{S'}(g)]$, as in (2.3). Inequality 3.9 holds due to the sub-additivity of the supremum function.

In equation (3.11), we introduce Rademacher variables σ_i, which are uniformly distributed independent random variables taking values in $\{-1, +1\}$ as in definition 3.2. This does not change the expectation appearing in (3.10): when $\sigma_i = 1$, the associated summand remains unchanged; when $\sigma_i = -1$, the associated summand flips signs, which is equivalent to swapping z_i and z'_i between S and S'. Since we are taking the expectation over all possible S and S', this swap does not affect the overall expectation; we are simply changing the order of the summands within the expectation.

Equation (3.12) holds by the sub-additivity of the supremum function, that is the inequality $\sup(U + V) \leq \sup(U) + \sup(V)$. Finally, (3.13) stems from the definition of Rademacher complexity and the fact that the variables σ_i and $-\sigma_i$ are distributed in the same way.

The reduction to $\mathfrak{R}_m(\mathcal{G})$ in equation (3.13) yields the bound in equation (3.3), using δ instead of $\delta/2$. To derive a bound in terms of $\widehat{\mathfrak{R}}_S(\mathcal{G})$, we observe that, by definition 3.1, changing one point in S changes $\widehat{\mathfrak{R}}_S(\mathcal{G})$ by at most $1/m$. Then, using again McDiarmid's inequality, with probability $1 - \delta/2$ the following holds:

$$\mathfrak{R}_m(\mathcal{G}) \leq \widehat{\mathfrak{R}}_S(\mathcal{G}) + \sqrt{\frac{\log\frac{2}{\delta}}{2m}}. \tag{3.14}$$

Finally, we use the union bound to combine inequalities 3.7 and 3.14, which yields with probability at least $1-\delta$:

$$\Phi(S) \leq 2\widehat{\mathfrak{R}}_S(\mathcal{G}) + 3\sqrt{\frac{\log \frac{2}{\delta}}{2m}}, \tag{3.15}$$

which matches (3.4). □

The following result relates the empirical Rademacher complexities of a hypothesis set $\mathcal{H}$ and to the family of loss functions $\mathcal{G}$ associated to $\mathcal{H}$ in the case of binary loss (zero-one loss).

Lemma 3.4 *Let $\mathcal{H}$ be a family of functions taking values in $\{-1,+1\}$ and let $\mathcal{G}$ be the family of loss functions associated to $\mathcal{H}$ for the zero-one loss: $\mathcal{G} = \{(x,y) \mapsto 1_{h(x)\neq y} \colon h \in \mathcal{H}\}$. For any sample $S = ((x_1,y_1),\ldots,(x_m,y_m))$ of elements in $\mathcal{X}\times\{-1,+1\}$, let $S_{\mathcal{X}}$ denote its projection over $\mathcal{X}$: $S_{\mathcal{X}} = (x_1,\ldots,x_m)$. Then, the following relation holds between the empirical Rademacher complexities of $\mathcal{G}$ and $\mathcal{H}$:*

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) = \frac{1}{2}\widehat{\mathfrak{R}}_{S_{\mathcal{X}}}(\mathcal{H}). \tag{3.16}$$

Proof: For any sample $S = ((x_1,y_1),\ldots,(x_m,y_m))$ of elements in $\mathcal{X}\times\{-1,+1\}$, by definition, the empirical Rademacher complexity of $\mathcal{G}$ can be written as:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\mathcal{G}) &= \mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in\mathcal{H}} \frac{1}{m}\sum_{i=1}^{m} \sigma_i 1_{h(x_i)\neq y_i}\Big] \\
&= \mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in\mathcal{H}} \frac{1}{m}\sum_{i=1}^{m} \sigma_i \tfrac{1-y_i h(x_i)}{2}\Big] \\
&= \frac{1}{2}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in\mathcal{H}} \frac{1}{m}\sum_{i=1}^{m} -\sigma_i y_i h(x_i)\Big] \\
&= \frac{1}{2}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in\mathcal{H}} \frac{1}{m}\sum_{i=1}^{m} \sigma_i h(x_i)\Big] = \frac{1}{2}\widehat{\mathfrak{R}}_{S_{\mathcal{X}}}(\mathcal{H}),
\end{aligned}$$

where we used the fact that $1_{h(x_i)\neq y_i} = (1-y_i h(x_i))/2$ and the fact that for a fixed $y_i \in \{-1,+1\}$, σ_i and $-y_i\sigma_i$ are distributed in the same way. □

Note that the lemma implies, by taking expectations, that for any $m \geq 1$, $\mathfrak{R}_m(\mathcal{G}) = \frac{1}{2}\mathfrak{R}_m(\mathcal{H})$. These connections between the empirical and average Rademacher complexities can be used to derive generalization bounds for binary classification in terms of the Rademacher complexity of the hypothesis set $\mathcal{H}$.

Theorem 3.5 (Rademacher complexity bounds – binary classification) *Let $\mathcal{H}$ be a family of functions taking values in $\{-1,+1\}$ and let $\mathcal{D}$ be the distribution over the input space $\mathcal{X}$. Then, for any $\delta > 0$, with probability at least $1-\delta$ over a sample S of*

size m drawn according to $\mathcal{D}$, each of the following holds for any $h \in \mathcal{H}$:

$$R(h) \leq \widehat{R}_S(h) + \mathfrak{R}_m(\mathcal{H}) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}} \tag{3.17}$$

$$\textit{and} \quad R(h) \leq \widehat{R}_S(h) + \widehat{\mathfrak{R}}_S(\mathcal{H}) + 3\sqrt{\frac{\log \frac{2}{\delta}}{2m}}. \tag{3.18}$$

Proof: The result follows immediately by theorem 3.3 and lemma 3.4. □

The theorem provides two generalization bounds for binary classification based on the Rademacher complexity. Note that the second bound, (3.18), is data-dependent: the empirical Rademacher complexity $\widehat{\mathfrak{R}}_S(\mathcal{H})$ is a function of the specific sample S drawn. Thus, this bound could be particularly informative if we could compute $\widehat{\mathfrak{R}}_S(\mathcal{H})$. But, how can we compute the empirical Rademacher complexity? Using again the fact that σ_i and $-\sigma_i$ are distributed in the same way, we can write

$$\widehat{\mathfrak{R}}_S(\mathcal{H}) = \mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h\in\mathcal{H}} \frac{1}{m}\sum_{i=1}^{m} -\sigma_i h(x_i)\Big] = -\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\inf_{h\in\mathcal{H}} \frac{1}{m}\sum_{i=1}^{m} \sigma_i h(x_i)\Big].$$

Now, for a fixed value of $\boldsymbol{\sigma}$, computing $\inf_{h\in\mathcal{H}} \frac{1}{m}\sum_{i=1}^{m} \sigma_i h(x_i)$ is equivalent to an *empirical risk minimization* problem, which is known to be computationally hard for some hypothesis sets. Thus, in some cases, computing $\widehat{\mathfrak{R}}_S(\mathcal{H})$ could be computationally hard. In the next sections, we will relate the Rademacher complexity to combinatorial measures that are easier to compute and also of independent interest for their usefulness in the analysis of learning in many contexts.

3.2 Growth function

Here we will show how the Rademacher complexity can be bounded in terms of the *growth function*.

Definition 3.6 (Growth function) *The* growth function $\Pi_{\mathcal{H}}\colon \mathbb{N} \to \mathbb{N}$ *for a hypothesis set $\mathcal{H}$ is defined by:*

$$\forall m \in \mathbb{N},\ \Pi_{\mathcal{H}}(m) = \max_{\{x_1,\ldots,x_m\}\subseteq X} \Big|\big\{\big(h(x_1),\ldots,h(x_m)\big)\colon h \in \mathcal{H}\big\}\Big|. \tag{3.19}$$

In other words, $\Pi_{\mathcal{H}}(m)$ is the maximum number of distinct ways in which m points can be classified using hypotheses in $\mathcal{H}$. Each one of these distinct classifications is called a *dichotomy* and, thus, the growth function counts the number of dichotomies that are realized by the hypothesis. This provides another measure of the richness of the hypothesis set $\mathcal{H}$. However, unlike the Rademacher complexity, this measure does not depend on the distribution, it is purely combinatorial.

To relate the Rademacher complexity to the growth function, we will use Massart's lemma.

Theorem 3.7 (Massart's lemma) *Let $\mathcal{A} \subseteq \mathbb{R}^m$ be a finite set, with $r = \max_{\mathbf{x}\in\mathcal{A}} \|\mathbf{x}\|_2$, then the following holds:*

$$\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\frac{1}{m}\sup_{\mathbf{x}\in\mathcal{A}}\sum_{i=1}^{m}\sigma_i x_i\right] \leq \frac{r\sqrt{2\log|\mathcal{A}|}}{m}\,, \tag{3.20}$$

where σ_is are independent uniform random variables taking values in $\{-1,+1\}$ and $x_1,\ldots,x_m$ are the components of vector $\mathbf{x}$.

Proof: The result follows immediately from the bound on the expectation of a maximum given by Corollary D.11 since the random variables $\sigma_i x_i$ are independent and each $\sigma_i x_i$ takes values in $[-|x_i|,|x_i|]$ with $\sqrt{\sum_{i=1}^m x_i^2} \leq r^2$. □

Using this result, we can now bound the Rademacher complexity in terms of the growth function.

Corollary 3.8 *Let $\mathcal{G}$ be a family of functions taking values in $\{-1,+1\}$. Then the following holds:*

$$\mathfrak{R}_m(\mathcal{G}) \leq \sqrt{\frac{2\log\Pi_{\mathcal{G}}(m)}{m}}. \tag{3.21}$$

Proof: For a fixed sample $S = (x_1,\ldots,x_m)$, we denote by $\mathcal{G}_{|S}$ the set of vectors of function values $(g(x_1),\ldots,g(x_m))^\top$ where g is in $\mathcal{G}$. Since $g \in \mathcal{G}$ takes values in $\{-1,+1\}$, the norm of these vectors is bounded by $\sqrt{m}$. We can then apply Massart's lemma as follows:

$$\mathfrak{R}_m(\mathcal{G}) = \mathop{\mathbb{E}}_{S}\left[\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\sup_{u\in\mathcal{G}_{|S}}\frac{1}{m}\sum_{i=1}^{m}\sigma_i u_i\right]\right] \leq \mathop{\mathbb{E}}_{S}\left[\frac{\sqrt{m}\sqrt{2\log|\mathcal{G}_{|S}|}}{m}\right].$$

By definition, $|\mathcal{G}_{|S}|$ is bounded by the growth function, thus,

$$\mathfrak{R}_m(\mathcal{G}) \leq \mathop{\mathbb{E}}_{S}\left[\frac{\sqrt{m}\sqrt{2\log\Pi_{\mathcal{G}}(m)}}{m}\right] = \sqrt{\frac{2\log\Pi_{\mathcal{G}}(m)}{m}},$$

which concludes the proof. □

Combining the generalization bound (3.17) of theorem 3.5 with corollary 3.8 yields immediately the following generalization bound in terms of the growth function.

Corollary 3.9 (Growth function generalization bound) *Let $\mathcal{H}$ be a family of functions taking values in $\{-1,+1\}$. Then, for any $\delta > 0$, with probability at least $1-\delta$, for any $h \in \mathcal{H}$,*

$$R(h) \leq \widehat{R}_S(h) + \sqrt{\frac{2\log\Pi_{\mathcal{H}}(m)}{m}} + \sqrt{\frac{\log\frac{1}{\delta}}{2m}}. \tag{3.22}$$

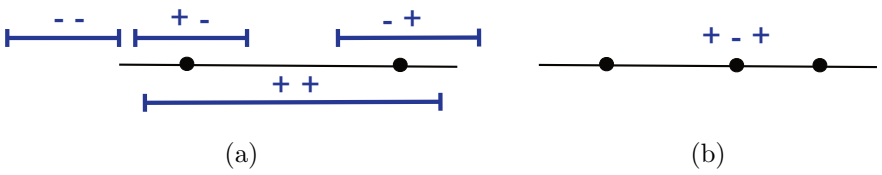


Figure 3.1
VC-dimension of intervals on the real line. (a) Any two points can be shattered. (b) No sample of three points can be shattered as the $(+,-,+)$ labeling cannot be realized.

Growth function bounds can be also derived directly (without using Rademacher complexity bounds first). The resulting bound is then the following:

$$\mathbb{P}\left[\left|R(h)-\widehat{R}_S(h)\right|>\epsilon\right] \leq 4\Pi_{\mathcal{H}}(2m)\exp\left(-\frac{m\epsilon^2}{8}\right), \tag{3.23}$$

which only differs from (3.22) by constants.

The computation of the growth function may not be always convenient since, by definition, it requires computing $\Pi_{\mathcal{H}}(m)$ for all $m \geq 1$. The next section introduces an alternative measure of the complexity of a hypothesis set $\mathcal{H}$ that is based instead on a single scalar, which will turn out to be in fact deeply related to the behavior of the growth function.

3.3 VC-dimension

Here, we introduce the notion of *VC-dimension* (Vapnik-Chervonenkis dimension). The VC-dimension is also a purely combinatorial notion but it is often easier to compute than the growth function (or the Rademacher Complexity). As we shall see, the VC-dimension is a key quantity in learning and is directly related to the growth function.

To define the VC-dimension of a hypothesis set $\mathcal{H}$, we first introduce the concept of *shattering*. Recall from the previous section, that given a hypothesis set $\mathcal{H}$, a dichotomy of a set S is one of the possible ways of labeling the points of S using a hypothesis in $\mathcal{H}$. A set S of $m \geq 1$ points is said to be shattered by a hypothesis set $\mathcal{H}$ when $\mathcal{H}$ realizes all possible dichotomies of S, that is when $\Pi_{\mathcal{H}}(m) = 2^m$.

Definition 3.10 (VC-dimension) *The VC-dimension of a hypothesis set $\mathcal{H}$ is the size of the largest set that can be shattered by $\mathcal{H}$:*

$$\mathrm{VCdim}(\mathcal{H}) = \max\{m\colon \Pi_{\mathcal{H}}(m) = 2^m\}. \tag{3.24}$$

Note that, by definition, if $\mathrm{VCdim}(\mathcal{H}) = d$, there exists a set of size d that can be shattered. However, this does not imply that all sets of size d or less are shattered and, in fact, this is typically not the case.

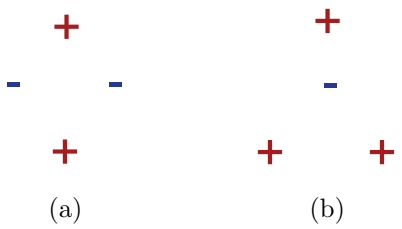


Figure 3.2
Unrealizable dichotomies for four points using hyperplanes in $\mathbb{R}^2$. (a) All four points lie on the convex hull. (b) Three points lie on the convex hull while the remaining point is interior.

To further illustrate this notion, we will examine a series of examples of hypothesis sets and will determine the VC-dimension in each case. To compute the VC-dimension we will typically show a lower bound for its value and then a matching upper bound. To give a lower bound d for $\text{VCdim}(\mathcal{H})$, it suffices to show that a set S of cardinality d can be shattered by $\mathcal{H}$. To give an upper bound, we need to prove that no set S of cardinality $d+1$ can be shattered by $\mathcal{H}$, which is typically more difficult.

Example 3.11 (Intervals on the real line) Our first example involves the hypothesis class of intervals on the real line. It is clear that the VC-dimension is at least two, since all four dichotomies $(+,+),(-,-),(+,-),(-,+)$ can be realized, as illustrated in figure 3.1(a). In contrast, by the definition of intervals, no set of three points can be shattered since the $(+,-,+)$ labeling cannot be realized. Hence, $\text{VCdim}(\text{intervals in } \mathbb{R}) = 2$.

Example 3.12 (Hyperplanes) Consider the set of hyperplanes in $\mathbb{R}^2$. We first observe that any three non-collinear points in $\mathbb{R}^2$ can be shattered. To obtain the first three dichotomies, we choose a hyperplane that has two points on one side and the third point on the opposite side. To obtain the fourth dichotomy we have all three points on the same side of the hyperplane. The remaining four dichotomies are realized by simply switching signs. Next, we show that four points cannot be shattered by considering two cases: (i) the four points lie on the convex hull defined by the four points, and (ii) three of the four points lie on the convex hull and the remaining point is internal. In the first case, a positive labeling for one diagonal pair and a negative labeling for the other diagonal pair cannot be realized, as illustrated in figure 3.2(a). In the second case, a labeling which is positive for the points on the convex hull and negative for the interior point cannot be realized, as illustrated in figure 3.2(b). Hence, $\text{VCdim}(\text{hyperplanes in } \mathbb{R}^2) = 3$.

More generally in $\mathbb{R}^d$, we derive a lower bound by starting with a set of $d+1$ points in $\mathbb{R}^d$, setting $\mathbf{x}_0$ to be the origin and defining $\mathbf{x}_i$, for $i \in \{1,\ldots,d\}$, as the point whose ith coordinate is 1 and all others are 0. Let $y_0, y_1, \ldots, y_d \in \{-1,+1\}$ be an

arbitrary set of labels for $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_d$. Let $\mathbf{w}$ be the vector whose ith coordinate is y_i. Then the classifier defined by the hyperplane of equation $\mathbf{w} \cdot \mathbf{x} + \frac{y_0}{2} = 0$ shatters $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_d$ since for any $i \in \{0, \ldots, d\}$,

$$\operatorname{sgn}\left(\mathbf{w} \cdot \mathbf{x}_i + \frac{y_0}{2}\right) = \operatorname{sgn}\left(y_i + \frac{y_0}{2}\right) = y_i. \tag{3.25}$$

To obtain an upper bound, it suffices to show that no set of $d+2$ points can be shattered by halfspaces. To prove this, we will use the following general theorem.

Theorem 3.13 (Radon's theorem) Any set $\mathcal{X}$ of $d+2$ points in $\mathbb{R}^d$ can be partitioned into two subsets $\mathcal{X}_1$ and $\mathcal{X}_2$ such that the convex hulls of $\mathcal{X}_1$ and $\mathcal{X}_2$ intersect.

Proof: Let $\mathcal{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_{d+2}\} \subset \mathbb{R}^d$. The following is a system of $d+1$ linear equations in $\alpha_1, \ldots, \alpha_{d+2}$:

$$\sum_{i=1}^{d+2} \alpha_i \mathbf{x}_i = 0 \qquad \text{and} \qquad \sum_{i=1}^{d+2} \alpha_i = 0, \tag{3.26}$$

since the first equality leads to d equations, one for each component. The number of unknowns, $d+2$, is larger than the number of equations, $d+1$, therefore the system admits a non-zero solution $\beta_1, \ldots, \beta_{d+2}$. Since $\sum_{i=1}^{d+2} \beta_i = 0$, both $\mathcal{I}_1 = \{i \in [d+2] \colon \beta_i > 0\}$ and $\mathcal{I}_2 = \{i \in [d+2] \colon \beta_i \leq 0\}$ are non-empty sets and $\mathcal{X}_1 = \{\mathbf{x}_i \colon i \in \mathcal{I}_1\}$ and $\mathcal{X}_2 = \{\mathbf{x}_i \colon i \in \mathcal{I}_2\}$ form a partition of $\mathcal{X}$. By the last equation of (3.26), $\sum_{i \in \mathcal{I}_1} \beta_i = -\sum_{i \in \mathcal{I}_2} \beta_i$. Let $\beta = \sum_{i \in \mathcal{I}_1} \beta_i$. Then, the first part of (3.26) implies

$$\sum_{i \in \mathcal{I}_1} \frac{\beta_i}{\beta} \mathbf{x}_i = \sum_{i \in \mathcal{I}_2} \frac{-\beta_i}{\beta} \mathbf{x}_i,$$

with $\sum_{i \in \mathcal{I}_1} \frac{\beta_i}{\beta} = \sum_{i \in \mathcal{I}_2} \frac{-\beta_i}{\beta} = 1$, and $\frac{\beta_i}{\beta} \geq 0$ for $i \in \mathcal{I}_1$ and $\frac{-\beta_i}{\beta} \geq 0$ for $i \in \mathcal{I}_2$. By definition of the convex hulls (B.6), this implies that $\sum_{i \in \mathcal{I}_1} \frac{\beta_i}{\beta} \mathbf{x}_i$ belongs both to the convex hull of $\mathcal{X}_1$ and to that of $\mathcal{X}_2$. □

Now, let $\mathcal{X}$ be a set of $d+2$ points. By Radon's theorem, it can be partitioned into two sets $\mathcal{X}_1$ and $\mathcal{X}_2$ such that their convex hulls intersect. Observe that when two sets of points $\mathcal{X}_1$ and $\mathcal{X}_2$ are separated by a hyperplane, their convex hulls are also separated by that hyperplane. Thus, $\mathcal{X}_1$ and $\mathcal{X}_2$ cannot be separated by a hyperplane and $\mathcal{X}$ is not shattered. Combining our lower and upper bounds, we have proven that $\operatorname{VCdim}(\text{hyperplanes in } \mathbb{R}^d) = d+1$.

Example 3.14 (Axis-aligned Rectangles) We first show that the VC-dimension is at least four, by considering four points in a diamond pattern. Then, it is clear that all 16 dichotomies can be realized, some of which are illustrated in figure 3.3(a). In contrast, for any set of five distinct points, if we construct the minimal axis-aligned rectangle containing these points, one of the five points is in the interior of

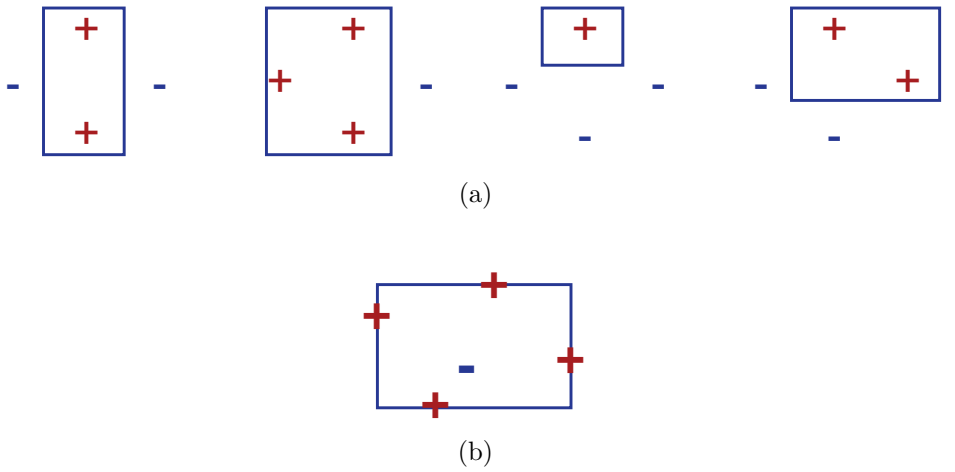


Figure 3.3
VC-dimension of axis-aligned rectangles. (a) Examples of realizable dichotomies for four points in a diamond pattern. (b) No sample of five points can be realized if the interior point and the remaining points have opposite labels.

this rectangle. Imagine that we assign a negative label to this interior point and a positive label to each of the remaining four points, as illustrated in figure 3.3(b). There is no axis-aligned rectangle that can realize this labeling. Hence, no set of five distinct points can be shattered and VCdim(axis-aligned rectangles) = 4.

Example 3.15 (Convex Polygons) We focus on the class of convex d-gons in the plane. To get a lower bound, we show that any set of $2d+1$ points can be shattered. To do this, we select $2d+1$ points that lie on a circle, and for a particular labeling, if there are more negative than positive labels, then the points with the positive labels are used as the polygon's vertices, as in figure 3.4(a). Otherwise, the tangents of the negative points serve as the edges of the polygon, as shown in (3.4)(b). To derive an upper bound, it can be shown that choosing points on the circle maximizes the number of possible dichotomies, and thus VCdim(convex d-gons) $= 2d + 1$. Note also that VCdim(convex polygons) $= +\infty$.

Example 3.16 (Sine Functions) The previous examples could suggest that the VC-dimension of $\mathcal{H}$ coincides with the number of free parameters defining $\mathcal{H}$. For example, the number of parameters defining hyperplanes matches their VC-dimension. However, this does not hold in general. Several of the exercises in this chapter illustrate this fact. The following provides a striking example from this point of view. Consider the following family of sine functions: $\{t \mapsto \sin(\omega t) \colon \omega \in \mathbb{R}\}$. One instance of this function class is shown in figure 3.5. These sine functions can be

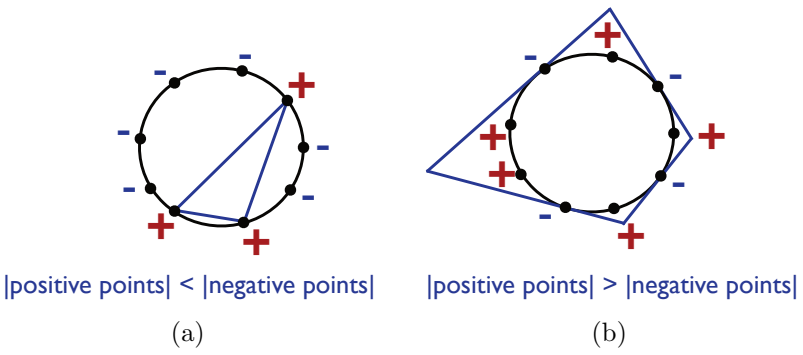


Figure 3.4
Convex d-gons in the plane can shatter $2d+1$ points. (a) d-gon construction when there are more negative labels. (b) d-gon construction when there are more positive labels.

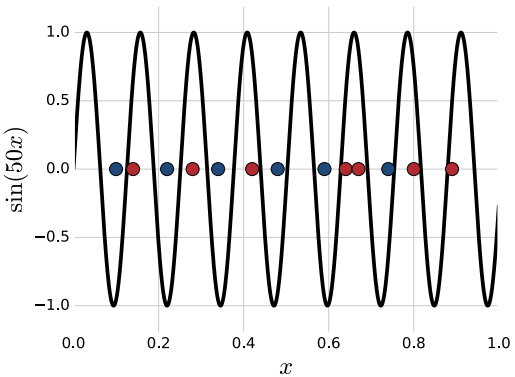


Figure 3.5
An example of a sine function (with $\omega = 50$) used for classification.

used to classify the points on the real line: a point is labeled positively if it is above the curve, negatively otherwise. Although this family of sine functions is defined via a single parameter, ω, it can be shown that VCdim(sine functions) $= +\infty$ (exercise 3.20).

The VC-dimension of many other hypothesis sets can be determined or upper-bounded in a similar way (see this chapter's exercises). In particular, the VC-dimension of any vector space of dimension $r < \infty$ can be shown to be at most r (exercise 3.19). The next result, known as *Sauer's lemma*, clarifies the connection between the notions of growth function and VC-dimension.

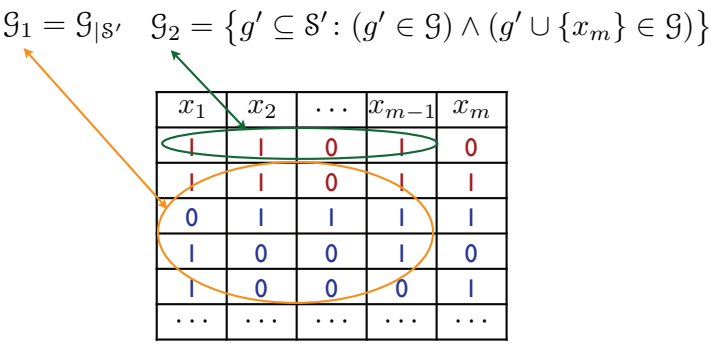


Figure 3.6
Illustration of how $\mathcal{G}_1$ and $\mathcal{G}_2$ are constructed in the proof of Sauer's lemma.

Theorem 3.17 (Sauer's lemma) *Let $\mathcal{H}$ be a hypothesis set with $\mathrm{VCdim}(\mathcal{H}) = d$. Then, for all $m \in \mathbb{N}$, the following inequality holds:*

$$\Pi_{\mathcal{H}}(m) \leq \sum_{i=0}^{d} \binom{m}{i}. \tag{3.27}$$

Proof: The proof is by induction on $m + d$. The statement clearly holds for $m = 1$ and $d = 0$ or $d = 1$. Now, assume that it holds for $(m-1, d-1)$ and $(m-1, d)$. Fix a set $\mathcal{S} = \{x_1, \ldots, x_m\}$ with $\Pi_{\mathcal{H}}(m)$ dichotomies and let $\mathcal{G} = \mathcal{H}_{|\mathcal{S}}$ be the set of concepts $\mathcal{H}$ induced by restriction to $\mathcal{S}$.

Now consider the following families over $\mathcal{S}' = \{x_1, \ldots, x_{m-1}\}$. We define $\mathcal{G}_1 = \mathcal{G}_{|\mathcal{S}'}$ as the set of concepts $\mathcal{H}$ induced by restriction to S'. Next, by identifying each concept as the set of points (in $\mathcal{S}'$ or $\mathcal{S}$) for which it is non-zero, we can define $_2$ as

$$\mathcal{G}_2 = \{g' \subseteq \mathcal{S}' : (g' \in \mathcal{G}) \wedge (g' \cup \{x_m\} \in \mathcal{G})\}.$$

Since $g' \subseteq \mathcal{S}'$, $g' \in \mathcal{G}$ means that without adding x_m it is a concept of $\mathcal{G}$. Further, the constraint $g' \cup \{x_m\} \in \mathcal{G}$ means that adding x_m to g' also makes it a concept of $\mathcal{G}$. The construction of $\mathcal{G}_1$ and $\mathcal{G}_2$ is illustrated pictorially in figure 3.6. Given our definitions of $\mathcal{G}_1$ and $\mathcal{G}_2$, observe that $|\mathcal{G}_1| + |\mathcal{G}_2| = |\mathcal{G}|$.

Since $\mathrm{VCdim}(\mathcal{G}_1) \leq \mathrm{VCdim}(\mathcal{G}) \leq d$, then by definition of the growth function and using the induction hypothesis,

$$|\mathcal{G}_1| \leq \Pi_{\mathcal{G}_1}(m-1) \leq \sum_{i=0}^{d} \binom{m-1}{i}.$$

Further, by definition of $\mathcal{G}_2$, if a set $\mathcal{Z} \subseteq \mathcal{S}'$ is shattered by $\mathcal{G}_2$, then the set $\mathcal{Z} \cup \{x_m\}$ is shattered by $\mathcal{G}$. Hence,

$$\mathrm{VCdim}(\mathcal{G}_2) \leq \mathrm{VCdim}(\mathcal{G}) - 1 = d - 1,$$

and by definition of the growth function and using the induction hypothesis,

$$|\mathcal{G}_2| \leq \Pi_{\mathcal{G}_2}(m-1) \leq \sum_{i=0}^{d-1} \binom{m-1}{i}.$$

Thus,

$$|\mathcal{G}| = |\mathcal{G}_1| + |\mathcal{G}_2| \leq \sum_{i=0}^{d} \tbinom{m-1}{i} + \sum_{i=0}^{d-1} \tbinom{m-1}{i} = \sum_{i=0}^{d} \tbinom{m-1}{i} + \tbinom{m-1}{i-1} = \sum_{i=0}^{d} \tbinom{m}{i},$$

which completes the inductive proof. □

The significance of Sauer's lemma can be seen by corollary 3.18, which remarkably shows that growth function only exhibits two types of behavior: either $\text{VCdim}(\mathcal{H}) = d < +\infty$, in which case $\Pi_{\mathcal{H}}(m) = O(m^d)$, or $\text{VCdim}(\mathcal{H}) = +\infty$, in which case $\Pi_{\mathcal{H}}(m) = 2^m$.

Corollary 3.18 *Let $\mathcal{H}$ be a hypothesis set with $\text{VCdim}(\mathcal{H}) = d$. Then for all $m \geq d$,*

$$\Pi_{\mathcal{H}}(m) \leq \left(\frac{em}{d}\right)^d = O(m^d). \tag{3.28}$$

Proof: The proof begins by using Sauer's lemma. The first inequality multiplies each summand by a factor that is greater than or equal to one since $m \geq d$, while the second inequality adds non-negative summands to the summation.

$$\begin{aligned}
\Pi_{\mathcal{H}}(m) &\leq \sum_{i=0}^{d} \binom{m}{i} \\
&\leq \sum_{i=0}^{d} \binom{m}{i} \left(\frac{m}{d}\right)^{d-i} \\
&\leq \sum_{i=0}^{m} \binom{m}{i} \left(\frac{m}{d}\right)^{d-i} \\
&= \left(\frac{m}{d}\right)^d \sum_{i=0}^{m} \binom{m}{i} \left(\frac{d}{m}\right)^i \\
&= \left(\frac{m}{d}\right)^d \left(1 + \frac{d}{m}\right)^m \leq \left(\frac{m}{d}\right)^d e^d.
\end{aligned}$$

After simplifying the expression using the binomial theorem, the final inequality follows using the general inequality $(1-x) \leq e^{-x}$. □

The explicit relationship just formulated between VC-dimension and the growth function combined with corollary 3.9 leads immediately to the following generalization bounds based on the VC-dimension.

Corollary 3.19 (VC-dimension generalization bounds) *Let $\mathcal{H}$ be a family of functions taking values in $\{-1, +1\}$ with VC-dimension d. Then, for any $\delta > 0$, with proba-*

bility at least $1-\delta$*, the following holds for all* $h \in \mathcal{H}$*:*

$$R(h) \leq \widehat{R}_S(h) + \sqrt{\frac{2d \log \frac{em}{d}}{m}} + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}. \tag{3.29}$$

Thus, the form of this generalization bound is

$$R(h) \leq \widehat{R}_S(h) + O\left(\sqrt{\frac{\log(m/d)}{(m/d)}}\right), \tag{3.30}$$

which emphasizes the importance of the ratio m/d for generalization. The theorem provides another instance of Occam's razor principle where simplicity is measured in terms of smaller VC-dimension.

VC-dimension bounds can be derived directly without using an intermediate Rademacher complexity bound, as for (3.23): combining Sauer's lemma with (3.23) leads to the following high-probability bound

$$R(h) \leq \widehat{R}_S(h) + \sqrt{\frac{8d \log \frac{2em}{d} + 8 \log \frac{4}{\delta}}{m}},$$

which has the general form of (3.30). The log factor plays only a minor role in these bounds. A finer analysis can be used in fact to eliminate that factor.

3.4 Lower bounds

In the previous section, we presented several upper bounds on the generalization error. In contrast, this section provides lower bounds on the generalization error of any learning algorithm in terms of the VC-dimension of the hypothesis set used.

These lower bounds are shown by finding for any algorithm a 'bad' distribution. Since the learning algorithm is arbitrary, it will be difficult to specify that particular distribution. Instead, it suffices to prove its existence non-constructively. At a high level, the proof technique used to achieve this is the *probabilistic method* of Paul Erdös. In the context of the following proofs, first a lower bound is given on the expected error over the parameters defining the distributions. From that, the lower bound is shown to hold for at least one set of parameters, that is one distribution.

Theorem 3.20 (Lower bound, realizable case) *Let* $\mathcal{H}$ *be a hypothesis set with VC-dimension* $d > 1$*. Then, for any* $m \geq 1$ *and any learning algorithm* $\mathcal{A}$*, there exist a distribution* $\mathcal{D}$ *over* $\mathcal{X}$ *and a target function* $f \in \mathcal{H}$ *such that*

$$\mathop{\mathbb{P}}_{S \sim \mathcal{D}^m}\left[R_{\mathcal{D}}(h_S, f) > \frac{d-1}{32m}\right] \geq 1/100. \tag{3.31}$$

Proof: Let $\overline{\mathcal{X}} = \{x_0, x_1, \ldots, x_{d-1}\} \subseteq \mathcal{X}$ be a set that is shattered by $\mathcal{H}$. For any $\epsilon > 0$, we choose $\mathcal{D}$ such that its support is reduced to $\overline{\mathcal{X}}$ and so that one point (x_0)

has very high probability $(1-8\epsilon)$, with the rest of the probability mass distributed uniformly among the other points:

$$\mathop{\mathbb{P}}_{\mathcal{D}}[x_0] = 1 - 8\epsilon \quad \text{and} \quad \forall i \in [d-1], \mathop{\mathbb{P}}_{\mathcal{D}}[x_i] = \frac{8\epsilon}{d-1}. \tag{3.32}$$

With this definition, most samples would contain x_0 and, since $\mathcal{X}$ is shattered, $\mathcal{A}$ can essentially do no better than tossing a coin when determining the label of a point x_i not falling in the training set.

We assume without loss of generality that $\mathcal{A}$ makes no error on x_0. For a sample S, we let $\overline{S}$ denote the set of its elements falling in $\{x_1, \ldots, x_{d-1}\}$, and let $\mathcal{S}$ be the set of samples S of size m such that $|\overline{S}| \leq (d-1)/2$. Now, fix a sample $S \in \mathcal{S}$, and consider the uniform distribution $\mathcal{U}$ over all labelings $f\colon \overline{\mathcal{X}} \to \{0,1\}$, which are all in $\mathcal{H}$ since the set is shattered. Then, the following lower bound holds:

$$\begin{aligned}
\mathop{\mathbb{E}}_{f\sim\mathcal{U}}[R_{\mathcal{D}}(h_S, f)] &= \sum_f \sum_{x\in\overline{\mathcal{X}}} 1_{h_S(x)\neq f(x)} \,\mathbb{P}[x]\,\mathbb{P}[f] \\
&\geq \sum_f \sum_{x\notin\overline{S}} 1_{h_S(x)\neq f(x)} \,\mathbb{P}[x]\,\mathbb{P}[f] \\
&= \sum_{x\notin\overline{S}} \Big(\sum_f 1_{h_S(x)\neq f(x)} \,\mathbb{P}[f]\Big)\,\mathbb{P}[x] \\
&= \frac{1}{2}\sum_{x\notin\overline{S}} \mathbb{P}[x] \geq \frac{1}{2}\frac{d-1}{2}\frac{8\epsilon}{d-1} = 2\epsilon.
\end{aligned} \tag{3.33}$$

The first lower bound holds because we remove non-negative terms from the summation when we only consider $x \notin \overline{S}$ instead of all x in $\overline{\mathcal{X}}$. After rearranging terms, the subsequent equality holds since we are taking an expectation over $f \in \mathcal{H}$ with uniform weight on each f and $\mathcal{H}$ shatters $\overline{\mathcal{X}}$. The final lower bound holds due to the definitions of $\mathcal{D}$ and $\overline{S}$, the latter which implies that $|\overline{\mathcal{X}} - \overline{S}| \geq (d-1)/2$.

Since (3.33) holds for all $S \in \mathcal{S}$, it also holds in expectation over all $S \in \mathcal{S}$: $\mathbb{E}_{S\in\mathcal{S}}\left[\mathbb{E}_{f\sim\mathcal{U}}[R_{\mathcal{D}}(h_S, f)]\right] \geq 2\epsilon$. By Fubini's theorem, the expectations can be permuted, thus,

$$\mathop{\mathbb{E}}_{f\sim\mathcal{U}}\Big[\mathop{\mathbb{E}}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f)]\Big] \geq 2\epsilon. \tag{3.34}$$

This implies that $\mathbb{E}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f_0)] \geq 2\epsilon$ for at least one labeling $f_0 \in \mathcal{H}$. Decomposing this expectation into two parts and using $R_{\mathcal{D}}(h_S, f_0) \leq \mathbb{P}_{\mathcal{D}}[\overline{\mathcal{X}} - \{x_0\}]$, we obtain:

$$\begin{aligned}
\mathop{\mathbb{E}}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f_0)] &= \sum_{S:R_{\mathcal{D}}(h_S,f_0)\geq\epsilon} R_{\mathcal{D}}(h_S, f_0)\,\mathbb{P}[R_{\mathcal{D}}(h_S, f_0)] + \sum_{S:R_{\mathcal{D}}(h_S,f_0)<\epsilon} R_{\mathcal{D}}(h_S, f_0)\,\mathbb{P}[R_{\mathcal{D}}(h_S, f_0)] \\
&\leq \mathop{\mathbb{P}}_{\mathcal{D}}[\overline{\mathcal{X}} - \{x_0\}] \mathop{\mathbb{P}}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f_0) \geq \epsilon] + \epsilon \mathop{\mathbb{P}}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f_0) < \epsilon] \\
&\leq 8\epsilon \mathop{\mathbb{P}}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f_0) \geq \epsilon] + \epsilon\big(1 - \mathop{\mathbb{P}}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f_0) \geq \epsilon]\big).
\end{aligned}$$

Collecting terms in $\mathbb{P}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f_0) \geq \epsilon]$ yields

$$\mathop{\mathbb{P}}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f_0) \geq \epsilon] \geq \frac{1}{7\epsilon}(2\epsilon - \epsilon) = \frac{1}{7}. \tag{3.35}$$

Thus, the probability over all samples S (not necessarily in $\mathcal{S}$) can be lower bounded as

$$\mathop{\mathbb{P}}_{S}[R_{\mathcal{D}}(h_S, f_0) \geq \epsilon] \geq \mathop{\mathbb{P}}_{S\in\mathcal{S}}[R_{\mathcal{D}}(h_S, f_0) \geq \epsilon]\,\mathbb{P}[\mathcal{S}] \geq \frac{1}{7}\,\mathbb{P}[\mathcal{S}]. \tag{3.36}$$

This leads us to find a lower bound for $\mathbb{P}[\mathcal{S}]$. By the multiplicative Chernoff bound (Theorem D.4), for any $\gamma > 0$, the probability that more than $(d-1)/2$ points are drawn in a sample of size m verifies:

$$1 - \mathbb{P}[\mathcal{S}] = \mathbb{P}[S_m \geq 8\epsilon m(1+\gamma)] \leq e^{-8\epsilon m\frac{\gamma^2}{3}}. \tag{3.37}$$

Therefore, for $\epsilon = (d-1)/(32m)$ and $\gamma = 1$,

$$\mathbb{P}[S_m \geq \tfrac{d-1}{2}] \leq e^{-(d-1)/12} \leq e^{-1/12} \leq 1 - 7\delta, \tag{3.38}$$

for $\delta \leq .01$. Thus $\mathbb{P}[\mathcal{S}] \geq 7\delta$ and $\mathbb{P}_S[R_{\mathcal{D}}(h_S, f_0) \geq \epsilon] \geq \delta$. □

The theorem shows that for any algorithm $\mathcal{A}$, there exists a 'bad' distribution over $\mathcal{X}$ and a target function f for which the error of the hypothesis returned by $\mathcal{A}$ is a constant times $\frac{d}{m}$ with some constant probability. This further demonstrates the key role played by the VC-dimension in learning. The result implies in particular that PAC-learning in the realizable case is not possible when the VC-dimension is infinite.

Note that the proof shows a stronger result than the statement of the theorem: the distribution $\mathcal{D}$ is selected independently of the algorithm $\mathcal{A}$. We now present a theorem giving a lower bound in the non-realizable case. The following two lemmas will be needed for the proof.

Lemma 3.21 *Let α be a uniformly distributed random variable taking values in $\{\alpha_-, \alpha_+\}$, where $\alpha_- = \frac{1}{2} - \frac{\epsilon}{2}$ and $\alpha_+ = \frac{1}{2} + \frac{\epsilon}{2}$, and let S be a sample of $m \geq 1$ random variables $X_1, \ldots, X_m$ taking values in $\{0,1\}$ and drawn i.i.d. according to the distribution $\mathcal{D}_\alpha$ defined by $\mathbb{P}_{\mathcal{D}_\alpha}[X = 1] = \alpha$. Let h be a function from $\mathcal{X}^m$ to $\{\alpha_-, \alpha_+\}$, then the following holds:*

$$\mathop{\mathbb{E}}_{\alpha}\Big[\mathop{\mathbb{P}}_{S\sim\mathcal{D}_\alpha^m}[h(S) \neq \alpha]\Big] \geq \Phi(2\lceil m/2\rceil, \epsilon), \tag{3.39}$$

where $\Phi(m,\epsilon) = \frac{1}{4}\Big(1 - \sqrt{1 - \exp\big(-\frac{m\epsilon^2}{1-\epsilon^2}\big)}\Big)$ for all m and ϵ.

Proof: The lemma can be interpreted in terms of an experiment with two coins with biases α_- and α_+. It implies that for a discriminant rule $h(S)$ based on a sample S drawn from $\mathcal{D}_{\alpha_-}$ or $\mathcal{D}_{\alpha_+}$, to determine which coin was tossed, the sample size m must be at least $\Omega(1/\epsilon^2)$. The proof is left as an exercise (exercise D.3). □

We will make use of the fact that for any fixed ϵ the function $m \mapsto \Phi(m, x)$ is convex, which is not hard to establish.

Lemma 3.22 *Let Z be a random variable taking values in $[0, 1]$. Then, for any $\gamma \in [0, 1)$,*

$$\mathbb{P}[z > \gamma] \geq \frac{\mathbb{E}[Z] - \gamma}{1 - \gamma} > \mathbb{E}[Z] - \gamma. \tag{3.40}$$

Proof: Since the values taken by Z are in $[0, 1]$,

$$\begin{aligned}
\mathbb{E}[Z] &= \sum_{z \leq \gamma} \mathbb{P}[Z = z]z + \sum_{z > \gamma} \mathbb{P}[Z = z]z \\
&\leq \sum_{z \leq \gamma} \mathbb{P}[Z = z]\gamma + \sum_{z > \gamma} \mathbb{P}[Z = z] \\
&= \gamma \, \mathbb{P}[Z \leq \gamma] + \mathbb{P}[Z > \gamma] \\
&= \gamma(1 - \mathbb{P}[Z > \gamma]) + \mathbb{P}[Z > \gamma] \\
&= (1 - \gamma) \, \mathbb{P}[Z > \gamma] + \gamma,
\end{aligned}$$

which concludes the proof. □

Theorem 3.23 (Lower bound, non-realizable case) *Let $\mathcal{H}$ be a hypothesis set with VC-dimension $d > 1$. Then, for any $m \geq 1$ and any learning algorithm $\mathcal{A}$, there exists a distribution $\mathcal{D}$ over $\mathcal{X} \times \{0, 1\}$ such that:*

$$\mathop{\mathbb{P}}_{S \sim \mathcal{D}^m} \left[R_{\mathcal{D}}(h_S) - \inf_{h \in \mathcal{H}} R_{\mathcal{D}}(h) > \sqrt{\frac{d}{320m}} \right] \geq 1/64. \tag{3.41}$$

Equivalently, for any learning algorithm, the sample complexity verifies

$$m \geq \frac{d}{320\epsilon^2}. \tag{3.42}$$

Proof: Let $\overline{\mathcal{X}} = \{x_1, \ldots, x_d\} \subseteq \mathcal{X}$ be a set shattered by $\mathcal{H}$. For any $\alpha \in [0, 1]$ and any vector $\boldsymbol{\sigma} = (\sigma_1, \ldots, \sigma_d)^\top \in \{-1, +1\}^d$, we define a distribution $\mathcal{D}_{\boldsymbol{\sigma}}$ with support $\overline{\mathcal{X}} \times \{0, 1\}$ as follows:

$$\forall i \in [d], \quad \mathop{\mathbb{P}}_{\mathcal{D}_{\boldsymbol{\sigma}}}[(x_i, 1)] = \frac{1}{d}\Big(\frac{1}{2} + \frac{\sigma_i \alpha}{2}\Big). \tag{3.43}$$

Thus, the label of each point x_i, $i \in [d]$, follows the distribution $\mathbb{P}_{\mathcal{D}_{\boldsymbol{\sigma}}}[\cdot | x_i]$, that of a biased coin where the bias is determined by the sign of σ_i and the magnitude of α. To determine the most likely label of each point x_i, the learning algorithm will therefore need to estimate $\mathbb{P}_{\mathcal{D}_{\boldsymbol{\sigma}}}[1 | x_i]$ with an accuracy better than α. To make this further difficult, α and $\boldsymbol{\sigma}$ will be selected based on the algorithm, requiring, as in lemma 3.21, $\Omega(1/\alpha^2)$ instances of each point x_i in the training sample.

Clearly, the Bayes classifier $h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}$ is defined by $h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}(x_i) = \operatorname{argmax}_{y\in\{0,1\}} \mathbb{P}[y|x_i] = 1_{\sigma_i>0}$ for all $i \in [d]$. $h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}$ is in $\mathcal{H}$ since $\overline{\mathcal{X}}$ is shattered. For all $h \in \mathcal{H}$,

$$R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h) - R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}) = \frac{1}{d}\sum_{x\in\overline{\mathcal{X}}}\Big(\frac{\alpha}{2}+\frac{\alpha}{2}\Big)1_{h(x)\neq h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}(x)} = \frac{\alpha}{d}\sum_{x\in\overline{\mathcal{X}}} 1_{h(x)\neq h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}(x)}. \quad (3.44)$$

Let h_S denote the hypothesis returned by the learning algorithm $\mathcal{A}$ after receiving a labeled sample S drawn according to $\mathcal{D}_{\boldsymbol{\sigma}}$. We will denote by $|S|_x$ the number of occurrences of a point x in S. Let $\mathcal{U}$ denote the uniform distribution over $\{-1,+1\}^d$. Then, in view of (3.44), the following holds:

$$\begin{aligned}
&\mathop{\mathbb{E}}_{\substack{\boldsymbol{\sigma}\sim\mathcal{U}\\ S\sim\mathcal{D}^m_{\boldsymbol{\sigma}}}}\left[\frac{1}{\alpha}\big[R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h_S) - R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h^*_{\mathcal{D}_{\boldsymbol{\sigma}}})\big]\right]\\
&= \frac{1}{d}\sum_{x\in\overline{\mathcal{X}}}\mathop{\mathbb{E}}_{\substack{\boldsymbol{\sigma}\sim\mathcal{U}\\ S\sim\mathcal{D}^m_{\boldsymbol{\sigma}}}}\Big[1_{h_S(x)\neq h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}(x)}\Big]\\
&= \frac{1}{d}\sum_{x\in\overline{\mathcal{X}}}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}\sim\mathcal{U}}\Big[\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m_{\boldsymbol{\sigma}}}\big[h_S(x)\neq h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}(x)\big]\Big]\\
&= \frac{1}{d}\sum_{x\in\overline{\mathcal{X}}}\sum_{n=0}^{m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}\sim\mathcal{U}}\Big[\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m_{\boldsymbol{\sigma}}}\big[h_S(x)\neq h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}(x)\,\big|\,|S|_x=n\big]\,\mathbb{P}[|S|_x=n]\Big]\\
&\geq \frac{1}{d}\sum_{x\in\overline{\mathcal{X}}}\sum_{n=0}^{m}\Phi(n+1,\alpha)\,\mathbb{P}[|S|_x=n] && \text{(lemma 3.21)}\\
&\geq \frac{1}{d}\sum_{x\in\overline{\mathcal{X}}}\Phi(m/d+1,\alpha) && \text{(convexity of } \Phi(\cdot,\alpha) \text{ and Jensen's ineq.)}\\
&= \Phi(m/d+1,\alpha).
\end{aligned}$$

Since the expectation over $\boldsymbol{\sigma}$ is lower-bounded by $\Phi(m/d+1,\alpha)$, there must exist some $\boldsymbol{\sigma} \in \{-1,+1\}^d$ for which

$$\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m_{\boldsymbol{\sigma}}}\left[\frac{1}{\alpha}\big[R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h_S) - R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h^*_{\mathcal{D}_{\boldsymbol{\sigma}}})\big]\right] > \Phi(m/d+1,\alpha). \quad (3.45)$$

Then, by lemma 3.22, for that $\boldsymbol{\sigma}$, for any $\gamma \in [0,1]$,

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m_{\boldsymbol{\sigma}}}\left[\frac{1}{\alpha}\big[R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h_S) - R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h^*_{\mathcal{D}_{\boldsymbol{\sigma}}})\big] > \gamma u\right] > (1-\gamma)u, \quad (3.46)$$

where $u = \Phi(m/d+1,\alpha)$. Selecting δ and ϵ such that $\delta \leq (1-\gamma)u$ and $\epsilon \leq \gamma\alpha u$ gives

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m_{\boldsymbol{\sigma}}}\big[R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h_S) - R_{\mathcal{D}_{\boldsymbol{\sigma}}}(h^*_{\mathcal{D}_{\boldsymbol{\sigma}}}) > \epsilon\big] > \delta. \quad (3.47)$$

To satisfy the inequalities defining ϵ and δ, let $\gamma = 1 - 8\delta$. Then,

$$\delta \leq (1-\gamma)u \iff u \geq \frac{1}{8} \tag{3.48}$$

$$\iff \frac{1}{4}\left(1 - \sqrt{1 - \exp\left(-\frac{(m/d+1)\alpha^2}{1-\alpha^2}\right)}\right) \geq \frac{1}{8} \tag{3.49}$$

$$\iff \frac{(m/d+1)\alpha^2}{1-\alpha^2} \leq \log\frac{4}{3} \tag{3.50}$$

$$\iff \frac{m}{d} \leq \left(\frac{1}{\alpha^2} - 1\right)\log\frac{4}{3} - 1. \tag{3.51}$$

Selecting $\alpha = 8\epsilon/(1-8\delta)$ gives $\epsilon = \gamma\alpha/8$ and the condition

$$\frac{m}{d} \leq \left(\frac{(1-8\delta)^2}{64\epsilon^2} - 1\right)\log\frac{4}{3} - 1. \tag{3.52}$$

Let $f(1/\epsilon^2)$ denote the right-hand side. We are seeking a sufficient condition of the form $m/d \leq \omega/\epsilon^2$. Since $\epsilon \leq 1/64$, to ensure that $\omega/\epsilon^2 \leq f(1/\epsilon^2)$, it suffices to impose $\frac{\omega}{(1/64)^2} = f\big(\frac{1}{(1/64)^2}\big)$. This condition gives

$$\omega = (7/64)^2\log(4/3) - (1/64)^2(\log(4/3)+1) \approx .003127 \geq 1/320 = .003125.$$

Thus, $\epsilon^2 \leq \frac{1}{320(m/d)}$ is sufficient to ensure the inequalities. □

The theorem shows that for any algorithm $\mathcal{A}$, in the non-realizable case, there exists a 'bad' distribution over $\mathcal{X} \times \{0,1\}$ such that the error of the hypothesis returned by $\mathcal{A}$ is a constant times $\sqrt{\frac{d}{m}}$ with some constant probability. The VC-dimension appears as a critical quantity in learning in this general setting as well. In particular, with an infinite VC-dimension, agnostic PAC-learning is not possible.

3.5 Chapter notes

The use of Rademacher complexity for deriving generalization bounds in learning was first advocated by Koltchinskii [2001], Koltchinskii and Panchenko [2000], and Bartlett, Boucheron, and Lugosi [2002a], see also [Koltchinskii and Panchenko, 2002, Bartlett and Mendelson, 2002]. Bartlett, Bousquet, and Mendelson [2002b] introduced the notion of *local Rademacher complexity*, that is the Rademacher complexity restricted to a subset of the hypothesis set limited by a bound on the variance. This can be used to derive better guarantees under some regularity assumptions about the noise.

Theorem 3.7 is due to Massart [2000]. The notion of VC-dimension was introduced by Vapnik and Chervonenkis [1971] and has been since extensively studied [Vapnik, 2006, Vapnik and Chervonenkis, 1974, Blumer et al., 1989, Assouad, 1983, Dudley,

1999]. In addition to the key role it plays in machine learning, the VC-dimension is also widely used in a variety of other areas of computer science and mathematics (e.g., see Shelah [1972], Chazelle [2000]). Theorem 3.17 is known as *Sauer's lemma* in the learning community, however the result was first given by Vapnik and Chervonenkis [1971] (in a somewhat different version) and later independently by Sauer [1972] and Shelah [1972].

In the realizable case, lower bounds for the expected error in terms of the VC-dimension were given by Vapnik and Chervonenkis [1974] and Haussler et al. [1988]. Later, a lower bound for the probability of error such as that of theorem 3.20 was given by Blumer et al. [1989]. Theorem 3.20 and its proof, which improves upon this previous result, are due to Ehrenfeucht, Haussler, Kearns, and Valiant [1988]. Devroye and Lugosi [1995] gave slightly tighter bounds for the same problem with a more complex expression. Theorem 3.23 giving a lower bound in the non-realizable case and the proof presented are due to Anthony and Bartlett [1999]. For other examples of application of the probabilistic method demonstrating its full power, consult the reference book of Alon and Spencer [1992].

There are several other measures of the complexity of a family of functions used in machine learning, including *covering numbers*, *packing numbers*, and some other complexity measures discussed in chapter 11. A covering number $\mathcal{N}_p(\mathcal{G}, \epsilon)$ is the minimal number of L_p balls of radius $\epsilon > 0$ needed to cover a family of loss functions $\mathcal{G}$. A packing number $\mathcal{M}_p(\mathcal{G}, \epsilon)$ is the maximum number of non-overlapping L_p balls of radius ϵ centered in $\mathcal{G}$. The two notions are closely related, in particular it can be shown straightforwardly that $\mathcal{M}_p(\mathcal{G}, 2\epsilon) \leq \mathcal{N}_p(\mathcal{G}, \epsilon) \leq \mathcal{M}_p(\mathcal{G}, \epsilon)$ for $\mathcal{G}$ and $\epsilon > 0$. Each complexity measure naturally induces a different reduction of infinite hypothesis sets to finite ones, thereby resulting in generalization bounds for infinite hypothesis sets. Exercise 3.31 illustrates the use of covering numbers for deriving generalization bounds using a very simple proof. There are also close relationships between these complexity measures: for example, by Dudley's theorem, the empirical Rademacher complexity can be bounded in terms of $\mathcal{N}_2(\mathcal{G}, \epsilon)$ [Dudley, 1967, 1987] and the covering and packing numbers can be bounded in terms of the VC-dimension [Haussler, 1995]. See also [Ledoux and Talagrand, 1991, Alon et al., 1997, Anthony and Bartlett, 1999, Cucker and Smale, 2001, Vidyasagar, 1997] for a number of upper bounds on the covering number in terms of other complexity measures.

3.6 Exercises

3.1 Growth function of intervals in $\mathbb{R}$. Let $\mathcal{H}$ be the set of intervals in $\mathbb{R}$. The VC-dimension of $\mathcal{H}$ is 2. Compute its shattering coefficient $\Pi_{\mathcal{H}}(m)$, $m \geq 0$. Compare your result with the general bound for growth functions.

3.2 Growth function and Rademacher complexity of thresholds in $\mathbb{R}$. Let $\mathcal{H}$ be the family of threshold functions over the real line: $\mathcal{H} = \{x \mapsto 1_{x \leq \theta} \colon \theta \in \mathbb{R}\} \cup \{x \mapsto 1_{x \geq \theta} \colon \theta \in \mathbb{R}\}$. Give an upper bound on the growth function $\Pi_m(\mathcal{H})$. Use that to derive an upper bound on $\mathfrak{R}_m(\mathcal{H})$.

3.3 Growth function of linear combinations. A *linearly separable labeling* of a set $\mathcal{X}$ of vectors in $\mathbb{R}^d$ is a classification of $\mathcal{X}$ into two sets $\mathcal{X}^+$ and $\mathcal{X}^-$ with $\mathcal{X}^+ = \{\mathbf{x} \in \mathcal{X} \colon \mathbf{w} \cdot \mathbf{x} > 0\}$ and $\mathcal{X}^- = \{\mathbf{x} \in \mathcal{X} \colon \mathbf{w} \cdot \mathbf{x} < 0\}$ for some $\mathbf{w} \in \mathbb{R}^d$.

Let $\mathcal{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ be a subset of $\mathbb{R}^d$.

(a) Let $\{\mathcal{X}^+, \mathcal{X}^-\}$ be a dichotomy of $\mathcal{X}$ and let $\mathbf{x}_{m+1} \in \mathbb{R}^d$. Show that $\{\mathcal{X}^+ \cup \{\mathbf{x}_{m+1}\}, \mathcal{X}^-\}$ and $\{\mathcal{X}^+, \mathcal{X}^- \cup \{\mathbf{x}_{m+1}\}\}$ are linearly separable by a hyperplane going through the origin if and only if $\{\mathcal{X}^+, \mathcal{X}^-\}$ is linearly separable by a hyperplane going through the origin and $\mathbf{x}_{m+1}$.

(b) Let $\mathcal{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ be a subset of $\mathbb{R}^d$ such that any k-element subset of $\mathcal{X}$ with $k \leq d$ is linearly independent. Then, show that the number of linearly separable labelings of $\mathcal{X}$ is $C(m, d) = 2 \sum_{k=0}^{d-1} \binom{m-1}{k}$. (*Hint*: prove by induction that $C(m+1, d) = C(m, d) + C(m, d-1)$.

(c) Let $f_1, \ldots, f_p$ be p functions mapping $\mathbb{R}^d$ to $\mathbb{R}$. Define $\mathcal{F}$ as the family of classifiers based on linear combinations of these functions:

$$\mathcal{F} = \left\{ x \mapsto \operatorname{sgn}\left(\sum_{k=1}^{p} a_k f_k(x) \right) \colon a_1, \ldots, a_p \in \mathbb{R} \right\}.$$

Define Ψ by $\Psi(x) = (f_1(x), \ldots, f_p(x))$. Assume that there exists $x_1, \ldots, x_m \in \mathbb{R}^d$ such that every p-subset of $\{\Psi(x_1), \ldots, \Psi(x_m)\}$ is linearly independent. Then, show that

$$\Pi_{\mathcal{F}}(m) = 2 \sum_{i=0}^{p-1} \binom{m-1}{i}.$$

3.4 Lower bound on growth function. Prove that Sauer's lemma (theorem 3.17) is tight, i.e., for any set $\mathcal{X}$ of $m > d$ elements, show that there exists a hypothesis class $\mathcal{H}$ of VC-dimension d such that $\Pi_{\mathcal{H}}(m) = \sum_{i=0}^{d} \binom{m}{i}$.

3.5 Finer Rademacher upper bound. Show that a finer upper bound on the Rademacher complexity of the family $\mathcal{G}$ can be given in terms of $\mathbb{E}_S[\Pi(\mathcal{G}, S)]$, where $\Pi(\mathcal{G}, S)$ is the number of ways to label the points in sample S.

3.6 Singleton hypothesis class. Consider the trivial hypothesis set $\mathcal{H} = \{h_0\}$.

(a) Show that $\mathfrak{R}_m(\mathcal{H}) = 0$ for any $m > 0$.

(b) Use a similar construction to show that Massart's lemma (theorem 3.7) is tight.

3.7 Two function hypothesis class. Let $\mathcal{H}$ be a hypothesis set reduced to two functions: $\mathcal{H} = \{h_{-1}, h_{+1}\}$ and let $S = (x_1, \ldots, x_m) \subseteq \mathcal{X}$ be a sample of size m.

(a) Assume that h_{-1} is the constant function taking value -1 and h_{+1} the constant function taking the value $+1$. What is the VC-dimension d of $\mathcal{H}$? Upper bound the empirical Rademacher complexity $\widehat{\mathfrak{R}}_S(\mathcal{H})$ (*Hint*: express $\widehat{\mathfrak{R}}_S(\mathcal{H})$ in terms of the absolute value of a sum of Rademacher variables and apply Jensen's inequality) and compare your bound with $\sqrt{d/m}$.

(b) Assume that h_{-1} is the constant function taking value -1 and h_{+1} the function taking value -1 everywhere except at x_1 where it takes the value $+1$. What is the VC-dimension d of $\mathcal{H}$? Compute the empirical Rademacher complexity $\widehat{\mathfrak{R}}_S(\mathcal{H})$.

3.8 Rademacher identities. Fix $m \geq 1$. Prove the following identities for any $\alpha \in \mathbb{R}$ and any two hypothesis sets $\mathcal{H}$ and $\mathcal{H}'$ of functions mapping from $\mathcal{X}$ to $\mathbb{R}$:

(a) $\mathfrak{R}_m(\alpha\mathcal{H}) = |\alpha|\mathfrak{R}_m(\mathcal{H})$.

(b) $\mathfrak{R}_m(\mathcal{H} + \mathcal{H}') = \mathfrak{R}_m(\mathcal{H}) + \mathfrak{R}_m(\mathcal{H}')$.

(c) $\mathfrak{R}_m(\{\max(h, h') \colon h \in \mathcal{H}, h' \in \mathcal{H}'\}) \leq \mathfrak{R}_m(\mathcal{H}) + \mathfrak{R}_m(\mathcal{H}')$,
where $\max(h, h')$ denotes the function $x \mapsto \max_{x \in \mathcal{X}}(h(x), h'(x))$ (*Hint*: you could use the identity $\max(a, b) = \frac{1}{2}[a + b + |a - b|]$ valid for all $a, b \in \mathbb{R}$ and Talagrand's contraction lemma (see lemma 5.7)).

3.9 Rademacher complexity of intersection of concepts. Let $\mathcal{H}_1$ and $\mathcal{H}_2$ be two families of functions mapping $\mathcal{X}$ to $\{0, 1\}$ and let $\mathcal{H} = \{h_1 h_2 \colon h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}$. Show that the empirical Rademacher complexity of $\mathcal{H}$ for any sample S of size m can be bounded as follows:

$$\widehat{\mathfrak{R}}_S(H) \leq \widehat{\mathfrak{R}}_S(\mathcal{H}_1) + \widehat{\mathfrak{R}}_S(\mathcal{H}_2).$$

Hint: use the Lipschitz function $x \mapsto \max(0, x-1)$ and Talagrand's contraction lemma.

Use that to bound the Rademacher complexity $\mathfrak{R}_m(\mathcal{U})$ of the family $\mathcal{U}$ of intersections of two concepts c_1 and c_2 with $c_1 \in \mathcal{C}_1$ and $c_2 \in \mathcal{C}_2$ in terms of the Rademacher complexities of $\mathcal{C}_1$ and $\mathcal{C}_2$.

3.10 Rademacher complexity of prediction vector. Let $S = (x_1, \ldots, x_m)$ be a sample of size m and fix $h\colon \mathcal{X} \to \mathbb{R}$.

(a) Denote by $\mathbf{u}$ the vector of predictions of h for S: $\mathbf{u} = \begin{bmatrix} h(x_1) \\ \vdots \\ h(x_m) \end{bmatrix}$. Give an upper bound on the empirical Rademacher complexity $\widehat{\mathfrak{R}}_S(\mathcal{H})$ of $\mathcal{H} = \{h, -h\}$ in terms of $\|\mathbf{u}\|_2$ (*Hint*: express $\widehat{\mathfrak{R}}_S(\mathcal{H})$ in terms of the expectation of an absolute value and apply Jensen's inequality). Suppose that $h(x_i) \in \{0, -1, +1\}$ for all $i \in [m]$. Express the bound on the Rademacher complexity in terms of the sparsity measure $n = |\{i \mid h(x_i) \neq 0\}|$. What is that upper bound for the extreme values of the sparsity measure?

(b) Let $\mathcal{F}$ be a family of functions mapping $\mathcal{X}$ to $\mathbb{R}$. Give an upper bound on the empirical Rademacher complexity of $\mathcal{F} + h = \{f + h\colon f \in \mathcal{F}\}$ and that of $\mathcal{F} \pm h = (\mathcal{F} + h) \cup (\mathcal{F} - h)$ in terms of $\widehat{\mathfrak{R}}_S(\mathcal{F})$ and $\|\mathbf{u}\|_2$.

3.11 Rademacher complexity of regularized neural networks. Let the input space be $\mathcal{X} = \mathbb{R}^{n_1}$. In this problem, we consider the family of regularized neural networks defined by the following set of functions mapping $\mathcal{X}$ to $\mathbb{R}$:

$$\mathcal{H} = \left\{ \mathbf{x} \mapsto \sum_{j=1}^{n_2} w_j \sigma(\mathbf{u}_j \cdot \mathbf{x})\colon \|\mathbf{w}\|_1 \leq \Lambda', \|\mathbf{u}_j\|_2 \leq \Lambda, \forall j \in [n_2] \right\},$$

where σ is an L-Lipschitz function. As an example, σ could be the sigmoid function which is 1-Lipschitz.

(a) Show that $\widehat{\mathfrak{R}}_S(\mathcal{H}) = \frac{\Lambda'}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{u}\|_2 \leq \Lambda} \left| \sum_{i=1}^m \sigma_i \sigma(\mathbf{u} \cdot \mathbf{x}_i) \right| \right]$.

(b) Use the following form of Talagrand's lemma valid for all hypothesis sets $\mathcal{H}$ and L-Lipschitz function Φ:

$$\frac{1}{m} \mathop{\mathbb{E}}_{\boldsymbol{\sigma}} \left[\sup_{h \in \mathcal{H}} \left| \sum_{i=1}^m \sigma_i (\Phi \circ h)(x_i) \right| \right] \leq \frac{L}{m} \mathop{\mathbb{E}}_{\boldsymbol{\sigma}} \left[\sup_{h \in \mathcal{H}} \left| \sum_{i=1}^m \sigma_i h(x_i) \right| \right],$$

to upper bound $\widehat{\mathfrak{R}}_S(\mathcal{H})$ in terms of the empirical Rademacher complexity of $\mathcal{H}'$, where $\mathcal{H}'$ is defined by

$$\mathcal{H}' = \{\mathbf{x} \mapsto s(\mathbf{u} \cdot \mathbf{x}) \colon \|\mathbf{u}\|_2 \leq \Lambda, s \in \{-1, +1\}\}.$$

(c) Use the Cauchy-Schwarz inequality to show that

$$\widehat{\mathfrak{R}}_S(\mathcal{H}') = \frac{\Lambda}{m} \mathop{\mathbb{E}}_{\boldsymbol{\sigma}} \left[\left\| \sum_{i=1}^{m} \sigma_i \mathbf{x}_i \right\|_2 \right].$$

(d) Use the inequality $\mathbb{E}_{\mathbf{v}}[\|\mathbf{v}\|_2] \leq \sqrt{\mathbb{E}_{\mathbf{v}}[\|\mathbf{v}\|_2^2]}$, which holds by Jensen's inequality to upper bound $\widehat{\mathfrak{R}}_S(\mathcal{H}')$.

(e) Assume that for all $\mathbf{x} \in S$, $\|\mathbf{x}\|_2 \leq r$ for some $r > 0$. Use the previous questions to derive an upper bound on the Rademacher complexity of $\mathcal{H}$ in terms of r.

3.12 Rademacher complexity. Professor Jesetoo claims to have found a better bound on the Rademacher complexity of any hypothesis set $\mathcal{H}$ of functions taking values in $\{-1, +1\}$, in terms of its VC-dimension $\mathrm{VCdim}(\mathcal{H})$. His bound is of the form $\mathfrak{R}_m(\mathcal{H}) \leq O\big(\frac{\mathrm{VCdim}(\mathcal{H})}{m}\big)$. Can you show that Professor Jesetoo's claim cannot be correct? (*Hint*: consider a hypothesis set $\mathcal{H}$ reduced to just two simple functions.)

3.13 VC-dimension of union of k intervals. What is the VC-dimension of subsets of the real line formed by the union of k intervals?

3.14 VC-dimension of finite hypothesis sets. Show that the VC-dimension of a finite hypothesis set $\mathcal{H}$ is at most $\log_2 |\mathcal{H}|$.

3.15 VC-dimension of subsets. What is the VC-dimension of the set of subsets I_α of the real line parameterized by a single parameter α: $I_\alpha = [\alpha, \alpha+1] \cup [\alpha+2, +\infty)$?

3.16 VC-dimension of axis-aligned squares and triangles.

(a) What is the VC-dimension of axis-aligned squares in the plane?

(b) Consider right triangles in the plane with the sides adjacent to the right angle both parallel to the axes and with the right angle in the lower left corner. What is the VC-dimension of this family?

3.17 VC-dimension of closed balls in $\mathbb{R}^n$. Show that the VC-dimension of the set of all closed balls in $\mathbb{R}^n$, i.e., sets of the form $\{x \in \mathbb{R}^n \colon \|x - x_0\|^2 \leq r\}$ for some $x_0 \in \mathbb{R}^n$ and $r \geq 0$, is less than or equal to $n + 2$.

3.18 VC-dimension of ellipsoids. What is the VC-dimension of the set of all ellipsoids in $\mathbb{R}^n$?

3.19 VC-dimension of a vector space of real functions. Let F be a finite-dimensional vector space of real functions on $\mathbb{R}^n$, $\dim(F) = r < \infty$. Let $\mathcal{H}$ be the set of hypotheses:

$$\mathcal{H} = \{\{x \colon f(x) \geq 0\} \colon f \in F\}.$$

Show that d, the VC-dimension of $\mathcal{H}$, is finite and that $d \leq r$. (*Hint*: select an arbitrary set of $m = r + 1$ points and consider linear mapping $u \colon F \to \mathbb{R}^m$ defined by: $u(f) = (f(x_1), \ldots, f(x_m))$.)

3.20 VC-dimension of sine functions. Consider the hypothesis family of sine functions (Example 3.16): $\{x \to \sin(\omega x) \colon \omega \in \mathbb{R}\}$.

(a) Show that for any $x \in \mathbb{R}$ the points $x, 2x, 3x$ and $4x$ cannot be shattered by this family of sine functions.

(b) Show that the VC-dimension of the family of sine functions is infinite. (*Hint*: show that $\{2^{-i} \colon i \leq m\}$ can be shattered for any $m > 0$.)

3.21 VC-dimension of union of halfspaces. Provide an upper bound on the VC-dimension of the class of hypotheses described by the unions of k halfspaces.

3.22 VC-dimension of intersection of halfspaces. Consider the class $\mathcal{C}_k$ of convex intersections of k halfspaces. Give lower and upper bound estimates for $\mathrm{VCdim}(\mathcal{C}_k)$.

3.23 VC-dimension of intersection concepts.

(a) Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be two concept classes. Show that for any concept class $\mathcal{C} = \{c_1 \cap c_2 \colon c_1 \in \mathcal{C}_1, c_2 \in \mathcal{C}_2\}$,

$$\Pi_{\mathcal{C}}(m) \leq \Pi_{\mathcal{C}_1}(m)\, \Pi_{\mathcal{C}_2}(m). \tag{3.53}$$

(b) Let $\mathcal{C}$ be a concept class with VC-dimension d and let $\mathcal{C}_s$ be the concept class formed by all intersections of s concepts from $\mathcal{C}$, $s \geq 1$. Show that the VC-dimension of $\mathcal{C}_s$ is bounded by $2ds \log_2(3s)$. (*Hint*: show that $\log_2(3x) < 9x/(2e)$ for any $x \geq 2$.)

3.24 VC-dimension of union of concepts. Let $\mathcal{A}$ and $\mathcal{B}$ be two sets of functions mapping from $\mathcal{X}$ into $\{0,1\}$, and assume that both $\mathcal{A}$ and $\mathcal{B}$ have finite VC-dimension, with $\mathrm{VCdim}(\mathcal{A}) = d_{\mathcal{A}}$ and $\mathrm{VCdim}(\mathcal{B}) = d_{\mathcal{B}}$. Let $\mathcal{C} = \mathcal{A} \cup \mathcal{B}$ be the union of $\mathcal{A}$ and $\mathcal{B}$.

(a) Prove that for all m, $\Pi_{\mathcal{C}}(m) \leq \Pi_{\mathcal{A}}(m) + \Pi_{\mathcal{B}}(m)$.

(b) Use Sauer's lemma to show that for $m \geq d_{\mathcal{A}} + d_{\mathcal{B}} + 2$, $\Pi_{\mathcal{C}}(m) < 2^m$, and give a bound on the VC-dimension of $\mathcal{C}$.

3.25 VC-dimension of symmetric difference of concepts. For two sets $\mathcal{A}$ and $\mathcal{B}$, let $\mathcal{A}\Delta\mathcal{B}$ denote the symmetric difference of $\mathcal{A}$ and $\mathcal{B}$, i.e., $\mathcal{A}\Delta\mathcal{B} = (\mathcal{A}\cup\mathcal{B}) - (\mathcal{A}\cap\mathcal{B})$. Let $\mathcal{H}$ be a non-empty family of subsets of $\mathcal{X}$ with finite VC-dimension. Let $\mathcal{A}$ be an element of $\mathcal{H}$ and define $\mathcal{H}\Delta\mathcal{A} = \{X\Delta\mathcal{A} \colon X \in \mathcal{H}\}$. Show that

$$\mathrm{VCdim}(\mathcal{H}\Delta\mathcal{A}) = \mathrm{VCdim}(\mathcal{H}).$$

3.26 Symmetric functions. A function $h \colon \{0,1\}^n \to \{0,1\}$ is *symmetric* if its value is uniquely determined by the number of 1's in the input. Let $\mathcal{C}$ denote the set of all symmetric functions.

(a) Determine the VC-dimension of $\mathcal{C}$.

(b) Give lower and upper bounds on the sample complexity of any consistent PAC learning algorithm for $\mathcal{C}$.

(c) Note that any hypothesis $h \in \mathcal{C}$ can be represented by a vector $(y_0, y_1, \ldots, y_n) \in \{0,1\}^{n+1}$, where y_i is the value of h on examples having precisely i 1's. Devise a consistent learning algorithm for $\mathcal{C}$ based on this representation.

3.27 VC-dimension of neural networks.

Let $\mathcal{C}$ be a concept class over $\mathbb{R}^r$ with VC-dimension d. A $\mathcal{C}$-neural network with one intermediate layer is a concept defined over $\mathbb{R}^n$ that can be represented by a directed acyclic graph such as that of Figure 3.7, in which the input nodes are those at the bottom and in which each other node is labeled with a concept $c \in \mathcal{C}$.

The output of the neural network for a given input vector $(x_1, \ldots, x_n)$ is obtained as follows. First, each of the n input nodes is labeled with the corresponding value $x_i \in \mathbb{R}$. Next, the value at a node u in the higher layer and labeled with c is obtained by applying c to the values of the input nodes admitting an

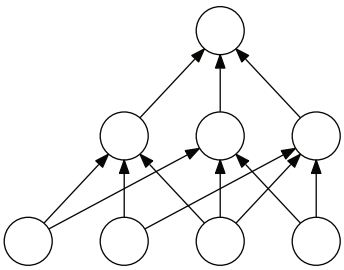


Figure 3.7
A neural network with one intermediate layer.

edge ending in u. Note that since c takes values in $\{0,1\}$, the value at u is in $\{0,1\}$. The value at the top or output node is obtained similarly by applying the corresponding concept to the values of the nodes admitting an edge to the output node.

(a) Let $\mathcal{H}$ denote the set of all neural networks defined as above with $k \geq 2$ internal nodes. Show that the growth function $\Pi_{\mathcal{H}}(m)$ can be upper bounded in terms of the product of the growth functions of the hypothesis sets defined at each intermediate layer.

(b) Use that to upper bound the VC-dimension of the $\mathcal{C}$-neural networks (*Hint*: you can use the implication $m = 2x\log_2(xy) \Rightarrow m > x\log_2(ym)$ valid for $m \geq 1$, and $x, y > 0$ with $xy > 4$).

(c) Let $\mathcal{C}$ be the family of concept classes defined by threshold functions $\mathcal{C} = \{\text{sgn}(\sum_{j=1}^{r} w_j x_j) \colon \mathbf{w} \in \mathbb{R}^r\}$. Give an upper bound on the VC-dimension of $\mathcal{H}$ in terms of k and r.

3.28 VC-dimension of convex combinations. Let $\mathcal{H}$ be a family of functions mapping from an input space $\mathcal{X}$ to $\{-1,+1\}$ and let T be a positive integer. Give an upper bound on the VC-dimension of the family of functions $\mathcal{F}_T$ defined by

$$\mathcal{F} = \left\{ \text{sgn}\left(\sum_{t=1}^{T} \alpha_t h_t \right) : h_t \in \mathcal{H}, \alpha_t \geq 0, \sum_{t=1}^{T} \alpha_t \leq 1 \right\}.$$

(*Hint*: you can use exercise 3.27 and its solution).

3.29 Infinite VC-dimension.

(a) Show that if a concept class $\mathcal{C}$ has infinite VC-dimension, then it is not PAC-learnable.

(b) In the standard PAC-learning scenario, the learning algorithm receives all examples first and then computes its hypothesis. Within that setting, PAC-learning of concept classes with infinite VC-dimension is not possible as seen in the previous question.

Imagine now a different scenario where the learning algorithm can alternate between drawing more examples and computation. The objective of this problem is to prove that PAC-learning can then be possible for some concept classes with infinite VC-dimension.

Consider for example the special case of the concept class $\mathcal{C}$ of all subsets of natural numbers. Professor Vitres has an idea for the first stage of a learning algorithm L PAC-learning $\mathcal{C}$. In the first stage, L draws a sufficient number of points m such that the probability of drawing a point beyond the maximum value M observed be small with high confidence. Can you complete Professor Vitres' idea by describing the second stage of the algorithm so that it PAC-learns $\mathcal{C}$? The description should be augmented with the proof that L can PAC-learn $\mathcal{C}$.

3.30 VC-dimension generalization bound – realizable case. In this exercise we show that the bound given in corollary 3.19 can be improved to $O(\frac{d\log(m/d)}{m})$ in the realizable setting. Assume we are in the realizable scenario, i.e. the target concept is included in our hypothesis class $\mathcal{H}$. We will show that if a hypothesis h is consistent with a sample $S \sim \mathcal{D}^m$ then for any $\epsilon > 0$ such that $m\epsilon \geq 8$

$$\mathbb{P}[R(h) > \epsilon] \leq 2\Big[\frac{2em}{d}\Big]^d 2^{-m\epsilon/2}\,. \tag{3.54}$$

(a) Let $\mathcal{H}_S \subseteq \mathcal{H}$ be the subset of hypotheses consistent with the sample S, let $\widehat{R}_S(h)$ denote the empirical error with respect to the sample S and define S' as another independent sample drawn from $\mathcal{D}^m$. Show that the following inequality holds for any $h_0 \in \mathcal{H}_S$:

$$\mathbb{P}\Big[\sup_{h\in\mathcal{H}_S} |\widehat{R}_S(h) - \widehat{R}_{S'}(h)| > \frac{\epsilon}{2}\Big] \geq \mathbb{P}\left[B(m,\epsilon) > \frac{m\epsilon}{2}\right]\mathbb{P}[R(h_0) > \epsilon]\,,$$

where $B(m,\epsilon)$ is a binomial random variable with parameters (m,ϵ). (*Hint*: prove and use the fact that $\mathbb{P}[\widehat{R}_S(h) \geq \frac{\epsilon}{2}] \geq \mathbb{P}[\widehat{R}_S(h) > \frac{\epsilon}{2} \wedge R(h) > \epsilon]$.)

(b) Prove that $\mathbb{P}\left[B(m,\epsilon) > \frac{m\epsilon}{2}\right] \geq \frac{1}{2}$. Use this inequality along with the result from (a) to show that for any $h_0 \in \mathcal{H}_S$

$$\mathbb{P}\left[R(h_0) > \epsilon\right] \leq 2\,\mathbb{P}\left[\sup_{h\in\mathcal{H}_S} |\widehat{R}_S(h) - \widehat{R}_{S'}(h)| > \frac{\epsilon}{2}\right].$$

(c) Instead of drawing two samples, we can draw one sample T of size $2m$ then uniformly at random split it into S and S'. The right hand side of part (b) can then be rewritten as:

$$\mathbb{P}\left[\sup_{h\in\mathcal{H}_S} |\widehat{R}_S(h) - \widehat{R}_{S'}(h)| > \frac{\epsilon}{2}\right] = \mathop{\mathbb{P}}_{\substack{T\sim\mathcal{D}^{2m}:\\ T\to[S,S']}}\left[\exists h\in\mathcal{H}\colon \widehat{R}_S(h) = 0 \wedge \widehat{R}_{S'}(h) > \frac{\epsilon}{2}\right].$$

Let h_0 be a hypothesis such that $\widehat{R}_T(h_0) > \frac{\epsilon}{2}$ and let $l > \frac{m\epsilon}{2}$ be the total number of errors h_0 makes on T. Show that the probability of all l errors falling into S' is upper bounded by 2^{-l}.

(d) Part (b) implies that for any $h \in \mathcal{H}$

$$\mathop{\mathbb{P}}_{\substack{T\sim\mathcal{D}^{2m}:\\ T\to(S,S')}}\left[\widehat{R}_S(h) = 0 \ \wedge\ \widehat{R}_{S'}(h) > \frac{\epsilon}{2} \;\middle|\; \widehat{R}_T(h_0) > \frac{\epsilon}{2}\right] \leq 2^{-l}.$$

Use this bound to show that for any $h \in \mathcal{H}$

$$\mathop{\mathbb{P}}_{\substack{T\sim\mathcal{D}^{2m}:\\ T\to(S,S')}}\left[\widehat{R}_S(h) = 0 \ \wedge\ \widehat{R}_{S'}(h) > \frac{\epsilon}{2}\right] \leq 2^{-\frac{\epsilon m}{2}}.$$

(e) Complete the proof of inequality (3.54) by using the union bound to upper bound $\mathbb{P}_{\substack{T\sim\mathcal{D}^{2m}:\\ T\to(S,S')}}\left[\exists h \in \mathcal{H}\colon \widehat{R}_S(h) = 0 \ \wedge\ \widehat{R}_{S'}(h) > \frac{\epsilon}{2}\right]$. Show that we can achieve a high probability generalization bound that is of the order $O(\frac{d\log(m/d)}{m})$.

3.31 Generalization bound based on covering numbers. Let $\mathcal{H}$ be a family of functions mapping $\mathcal{X}$ to a subset of real numbers $\mathcal{Y} \subseteq \mathbb{R}$. For any $\epsilon > 0$, the *covering number* $\mathcal{N}(\mathcal{H},\epsilon)$ of $\mathcal{H}$ for the L_∞ norm is the minimal $k \in \mathbb{N}$ such that $\mathcal{H}$ can be covered with k balls of radius ϵ, that is, there exists $\{h_1,\ldots,h_k\} \subseteq \mathcal{H}$ such that, for all $h \in \mathcal{H}$, there exists $i \leq k$ with $\|h - h_i\|_\infty = \max_{x\in\mathcal{X}} |h(x) - h_i(x)| \leq \epsilon$. In particular, when $\mathcal{H}$ is a compact set, a finite covering can be extracted from a covering of $\mathcal{H}$ with balls of radius ϵ and thus $\mathcal{N}(\mathcal{H},\epsilon)$ is finite.

Covering numbers provide a measure of the complexity of a class of functions: the larger the covering number, the richer is the family of functions. The objective of this problem is to illustrate this by proving a learning bound in the case of the squared loss. Let $\mathcal{D}$ denote a distribution over $\mathcal{X}\times\mathcal{Y}$ according to which

labeled examples are drawn. Then, the generalization error of $h \in \mathcal{H}$ for the squared loss is defined by $R(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[(h(x)-y)^2]$ and its empirical error for a labeled sample $S = ((x_1, y_1), \ldots, (x_m, y_m))$ by $\widehat{R}_S(h) = \frac{1}{m}\sum_{i=1}^{m}(h(x_i) - y_i)^2$. We will assume that $\mathcal{H}$ is bounded, that is there exists $M > 0$ such that $|h(x) - y| \leq M$ for all $(x, y) \in \mathcal{X} \times \mathcal{Y}$. The following is the generalization bound proven in this problem:

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}\left[\sup_{h\in\mathcal{H}} |R(h) - \widehat{R}_S(h)| \geq \epsilon\right] \leq \mathcal{N}\Big(\mathcal{H}, \frac{\epsilon}{8M}\Big) 2\exp\left(\frac{-m\epsilon^2}{2M^4}\right). \tag{3.55}$$

The proof is based on the following steps.

(a) Let $L_S = R(h) - \widehat{R}_S(h)$, then show that for all $h_1, h_2 \in \mathcal{H}$ and any labeled sample S, the following inequality holds:

$$|L_S(h_1) - L_S(h_2)| \leq 4M\|h_1 - h_2\|_\infty .$$

(b) Assume that $\mathcal{H}$ can be covered by k subsets $\mathcal{B}_1, \ldots, \mathcal{B}_k$, that is $\mathcal{H} = \mathcal{B}_1 \cup \ldots \cup \mathcal{B}_k$. Then, show that, for any $\epsilon > 0$, the following upper bound holds:

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}\left[\sup_{h\in\mathcal{H}} |L_S(h)| \geq \epsilon\right] \leq \sum_{i=1}^{k} \mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}\left[\sup_{h\in\mathcal{B}_i} |L_S(h)| \geq \epsilon\right].$$

(c) Finally, let $k = \mathcal{N}(\mathcal{H}, \frac{\epsilon}{8M})$ and let $\mathcal{B}_1, \ldots, \mathcal{B}_k$ be balls of radius $\epsilon/(8M)$ centered at $h_1, \ldots, h_k$ covering $\mathcal{H}$. Use part (a) to show that for all $i \in [k]$,

$$\mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}\left[\sup_{h\in\mathcal{B}_i} |L_S(h)| \geq \epsilon\right] \leq \mathop{\mathbb{P}}_{S\sim\mathcal{D}^m}\left[|L_S(h_i)| \geq \frac{\epsilon}{2}\right],$$

and apply Hoeffding's inequality (theorem D.2) to prove (3.55).

4 Model Selection

A key problem in the design of learning algorithms is the choice of the hypothesis set $\mathcal{H}$. This is known as the *model selection* problem. How should the hypothesis set $\mathcal{H}$ be chosen? A rich or complex enough hypothesis set could contain the ideal Bayes classifier. On the other hand, learning with such a complex family becomes a very difficult task. More generally, the choice of $\mathcal{H}$ is subject to a trade-off that can be analyzed in terms of the *estimation* and *approximation errors*.

Our discussion will focus on the particular case of binary classification but much of what is discussed can be straightforwardly extended to different tasks and loss functions.

4.1 Estimation and approximation errors

Let $\mathcal{H}$ be a family of functions mapping $\mathcal{X}$ to $\{-1, +1\}$. The *excess error* of a hypothesis h chosen from $\mathcal{H}$, that is the difference between its error $R(h)$ and the Bayes error R^*, can be decomposed as follows:

$$R(h) - R^* = \underbrace{\Big(R(h) - \inf_{h \in \mathcal{H}} R(h)\Big)}_{\text{estimation}} + \underbrace{\Big(\inf_{h \in \mathcal{H}} R(h) - R^*\Big)}_{\text{approximation}}. \tag{4.1}$$

The first term is called the *estimation error*, the second term the *approximation error*. The estimation error depends on the hypothesis h selected. It measures the error of h with respect to the infimum of the errors achieved by hypotheses in $\mathcal{H}$, or that of the best-in-class hypothesis h^* when that infimum is reached. Note that the definition of agnostic PAC-learning is precisely based on the estimation error.

The approximation error measures how well the Bayes error can be approximated using $\mathcal{H}$. It is a property of the hypothesis set $\mathcal{H}$, a measure of its richness. For a more complex or richer hypothesis $\mathcal{H}$, the approximation error tends to be smaller at the price of a larger estimation error. This is illustrated by Figure 4.1.

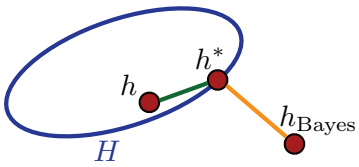


Figure 4.1
Illustration of the estimation error (in green) and approximation error (in orange). Here, it is assumed that there exists a best-in-class hypothesis, that is h^* such that $R(h^*) = \inf_{h \in \mathcal{H}} R(h)$.

Model selection consists of choosing $\mathcal{H}$ with a favorable trade-off between the approximation and estimation errors. Note, however, that the approximation error is not accessible, since in general the underlying distribution $\mathcal{D}$ needed to determine R^* is not known. Even with various noise assumptions, estimating the approximation error is difficult. In contrast, the *estimation error of an algorithm* $\mathcal{A}$, that is, the estimation error of the hypothesis h_S returned after training on a sample S, can sometimes be bounded using generalization bounds as shown in the next section.

4.2 Empirical risk minimization (ERM)

A standard algorithm for which the estimation error can be bounded is *Empirical Risk Minimization* (ERM). ERM seeks to minimize the error on the training sample:[4]

$$h_S^{\text{ERM}} = \operatorname*{argmin}_{h \in \mathcal{H}} \widehat{R}_S(h). \tag{4.2}$$

Proposition 4.1 *For any sample S, the following inequality holds for the hypothesis returned by ERM:*

$$\mathbb{P}\left[R(h_S^{\text{ERM}}) - \inf_{h \in \mathcal{H}} R(h) > \epsilon\right] \leq \mathbb{P}\left[\sup_{h \in \mathcal{H}} |R(h) - \widehat{R}_S(h)| > \frac{\epsilon}{2}\right]. \tag{4.3}$$

Proof: By definition of $\inf_{h \in \mathcal{H}} R(h)$, for any $\epsilon > 0$, there exists h_ϵ such that $R(h_\epsilon) \leq \inf_{h \in \mathcal{H}} R(h) + \epsilon$. Thus, using $\widehat{R}_S(h_S^{\text{ERM}}) \leq \widehat{R}_S(h_\epsilon)$, which holds by the

[4] Note that, if there exists multiple hypotheses with minimal error on the training sample, then ERM returns an arbitrary one.

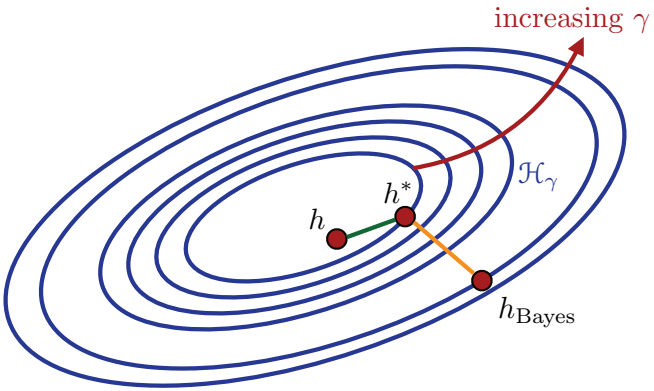


Figure 4.2
Illustration of the decomposition of a rich family $\mathcal{H} = \bigcup_{\gamma \in \Gamma} \mathcal{H}_\gamma$.

definition of the algorithm, we can write

$$\begin{aligned} R(h_S^{\text{ERM}}) - \inf_{h \in \mathcal{H}} R(h) &= R(h_S^{\text{ERM}}) - R(h_\epsilon) + R(h_\epsilon) - \inf_{h \in \mathcal{H}} R(h) \\ &\leq R(h_S^{\text{ERM}}) - R(h_\epsilon) + \epsilon \\ &= R(h_S^{\text{ERM}}) - \widehat{R}_S(h_S^{\text{ERM}}) + \widehat{R}_S(h_S^{\text{ERM}}) - R(h_\epsilon) + \epsilon \\ &\leq R(h_S^{\text{ERM}}) - \widehat{R}_S(h_S^{\text{ERM}}) + \widehat{R}_S(h_\epsilon) - R(h_\epsilon) + \epsilon \\ &\leq 2 \sup_{h \in \mathcal{H}} |R(h) - \widehat{R}_S(h)| + \epsilon. \end{aligned}$$

Since the inequality holds for all $\epsilon > 0$, it implies the following:

$$R(h_S^{\text{ERM}}) - \inf_{h \in \mathcal{H}} R(h) \leq 2 \sup_{h \in \mathcal{H}} |R(h) - \widehat{R}_S(h)|,$$

which concludes the proof. □

The right-hand side of (4.3) can be upper-bounded using the generalization bounds presented in the previous chapter in terms of the Rademacher complexity, the growth function, or the VC-dimension of $\mathcal{H}$. In particular, it can be bounded by $2e^{-2m[\epsilon - \mathfrak{R}_m(\mathcal{H})]^2}$. Thus, when $\mathcal{H}$ admits a favorable Rademacher complexity, for example a finite VC-dimension, for a sufficiently large sample, with high probability, the estimation error is guaranteed to be small. Nevertheless, the performance of ERM is typically very poor. This is because the algorithm disregards the complexity of the hypothesis set $\mathcal{H}$: in practice, either $\mathcal{H}$ is not complex enough, in which case the approximation error can be very large, or $\mathcal{H}$ is very rich, in which case the bound on the estimation error becomes very loose. Additionally, in many cases, determining the ERM solution is computationally intractable. For example, finding

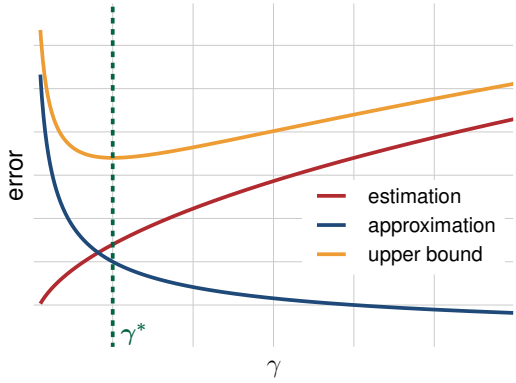


Figure 4.3
Choice of γ^* with the most favorable trade-off between estimation and approximation errors.

a linear hypothesis with the smallest error on the training sample is NP-hard, as a function of the dimension of the space.

4.3 Structural risk minimization (SRM)

In the previous section, we showed that the estimation error can be sometimes bounded or estimated. But, since the approximation error cannot be estimated, how should we choose $\mathcal{H}$? One way to proceed is to choose a very complex family $\mathcal{H}$ with no approximation error or a very small one. $\mathcal{H}$ may be too rich for generalization bounds to hold for $\mathcal{H}$, but suppose we can decompose $\mathcal{H}$ as a union of increasingly complex hypothesis sets $\mathcal{H}_\gamma$, that is $\mathcal{H} = \bigcup_{\gamma \in \Gamma} \mathcal{H}_\gamma$, with the complexity of $\mathcal{H}_\gamma$ increasing with γ, for some set Γ. Figure 4.2 illustrates this decomposition. The problem then consists of selecting the parameter $\gamma^* \in \Gamma$ and thus the hypothesis set $\mathcal{H}_{\gamma^*}$ with the most favorable trade-off between estimation and approximation errors. Since these quantities are not known, instead, as illustrated by Figure 4.3, a uniform upper bound on their sum, the excess error (also called excess risk), can be used.

This is precisely the idea behind the *Structural Risk Minimization* (SRM) method. For SRM, $\mathcal{H}$ is assumed to be decomposable into a countable set, thus, we will write its decomposition as $\mathcal{H} = \bigcup_{k \geq 1} \mathcal{H}_k$. Additionally, the hypothesis sets $\mathcal{H}_k$ are assumed to be nested: $\mathcal{H}_k \subset \mathcal{H}_{k+1}$ for all $k \geq 1$. However, many of the results presented in this section also hold for non-nested hypothesis sets. Thus, we will not make use of that assumption, unless explicitly specified. SRM consists of choosing the index $k^* \geq 1$ and the ERM hypothesis h in $\mathcal{H}_{k^*}$ that minimize an upper bound on the excess error.

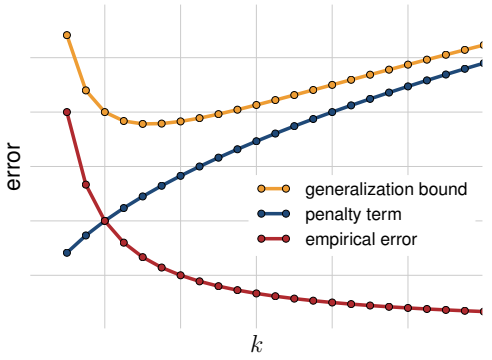


Figure 4.4
Illustration of structural risk minimization. The plots of three errors are shown as a function of the index k. Clearly, as k, or equivalently the complexity the hypothesis set $\mathcal{H}_k$, increases, the training error decreases, while the penalty term increases. SRM selects the hypothesis minimizing a bound on the generalization error, which is a sum of the empirical error and the penalty term.

As we shall see, the following learning bound holds for all $h \in \mathcal{H}$: for any $\delta > 0$, with probability at least $1 - \delta$ over the draw of a sample S of size m from $\mathcal{D}^m$, for all $h \in \mathcal{H}_k$ and $k \geq 1$,

$$R(h) \leq \widehat{R}_S(h) + \mathfrak{R}_m(\mathcal{H}_{k(h)}) + \sqrt{\frac{\log k}{m}} + \sqrt{\frac{\log \frac{2}{\delta}}{2m}}.$$

Thus, to minimize the resulting bound on the excess error $(R(h) - R^*)$, the index k and the hypothesis $h \in \mathcal{H}_k$ should be chosen to minimize the following objective function:

$$F_k(h) = \widehat{R}_S(h) + \mathfrak{R}_m(\mathcal{H}_k) + \sqrt{\frac{\log k}{m}}.$$

This is precisely the definition of the SRM solution h_S^{SRM}:

$$h_S^{\mathrm{SRM}} = \operatorname*{argmin}_{k \geq 1, h \in \mathcal{H}_k} F_k(h) = \operatorname*{argmin}_{k \geq 1, h \in \mathcal{H}_k} \widehat{R}_S(h) + R_m(\mathcal{H}_k) + \sqrt{\frac{\log k}{m}}. \tag{4.4}$$

Thus, SRM identifies an optimal index k^* and therefore hypothesis set $\mathcal{H}_{k^*}$, and returns the ERM solution based on that hypothesis set. Figure 4.4 further illustrates the selection of the index k^* and hypothesis set $\mathcal{H}_{k^*}$ by SRM by minimizing an upper bound on the sum of the training error and the penalty term $R_m(\mathcal{H}_k) + \sqrt{\log k/m}$. The following theorem shows that the SRM solution benefits from a strong learning guarantee. For any $h \in \mathcal{H}$, we will denote by $\mathcal{H}_{k(h)}$ the least complex hypothesis set among the $\mathcal{H}_k$s that contain h.

Theorem 4.2 (SRM Learning guarantee) *For any $\delta > 0$, with probability at least $1 - \delta$ over the draw of an i.i.d. sample S of size m from $\mathcal{D}^m$, the generalization error of the hypothesis h_S^{SRM} returned by the SRM method is bounded as follows:*

$$R(h_S^{\mathrm{SRM}}) \leq \inf_{h \in \mathcal{H}} \left(R(h) + 2\mathfrak{R}_m(\mathcal{H}_{k(h)}) + \sqrt{\frac{\log k(h)}{m}} \right) + \sqrt{\frac{2 \log \frac{3}{\delta}}{m}}.$$

Proof: Observe first that, by the union bound, the following general inequality holds:

$$\begin{aligned}
&\mathbb{P}\left[\sup_{h \in \mathcal{H}} R(h) - F_{k(h)}(h) > \epsilon \right] \\
&\qquad = \mathbb{P}\left[\sup_{k \geq 1} \sup_{h \in \mathcal{H}_k} R(h) - F_k(h) > \epsilon \right] \\
&\qquad \leq \sum_{k=1}^{\infty} \mathbb{P}\left[\sup_{h \in \mathcal{H}_k} R(h) - F_k(h) > \epsilon \right] \\
&\qquad = \sum_{k=1}^{\infty} \mathbb{P}\left[\sup_{h \in \mathcal{H}_k} R(h) - \widehat{R}_S(h) - \mathfrak{R}_m(\mathcal{H}_k) > \epsilon + \sqrt{\frac{\log k}{m}} \right] \\
&\qquad \leq \sum_{k=1}^{\infty} \exp\Big(- 2m \Big[\epsilon + \sqrt{\tfrac{\log k}{m}} \Big]^2 \Big) \\
&\qquad \leq \sum_{k=1}^{\infty} e^{-2m\epsilon^2} e^{-2 \log k} \\
&\qquad = e^{-2m\epsilon^2} \sum_{k=1}^{\infty} \frac{1}{k^2} = \frac{\pi^2}{6} e^{-2m\epsilon^2} \leq 2e^{-2m\epsilon^2}.
\end{aligned} \tag{4.5}$$

Next, for any two random variables X_1 and X_2, if $X_1 + X_2 > \epsilon$, then either X_1 or X_2 must be larger than $\epsilon/2$. In view of that, by the union bound, $\mathbb{P}[X_1 + X_2 > \epsilon] \leq \mathbb{P}[X_1 > \frac{\epsilon}{2}] + \mathbb{P}[X_2 > \frac{\epsilon}{2}]$. Using this inequality, inequality (4.5), and the inequality $F_{k(h_S^{\mathrm{SRM}})}(h_S^{\mathrm{SRM}}) \leq F_{k(h)}(h)$, which holds for all $h \in \mathcal{H}$, by definition of h_S^{SRM}, we

can write, for any $h \in \mathcal{H}$,

$$\begin{aligned}
&\mathbb{P}\left[R(h_S^{\mathrm{SRM}}) - R(h) - 2\mathfrak{R}_m(\mathcal{H}_{k(h)}) - \sqrt{\tfrac{\log k(h)}{m}} > \epsilon\right] \\
&\quad \leq \mathbb{P}\left[R(h_S^{\mathrm{SRM}}) - F_{k(h_S^{\mathrm{SRM}})}(h_S^{\mathrm{SRM}}) > \frac{\epsilon}{2}\right] \\
&\qquad + \mathbb{P}\left[F_{k(h_S^{\mathrm{SRM}})}(h_S^{\mathrm{SRM}}) - R(h) - 2\mathfrak{R}_m(\mathcal{H}_{k(h)}) - \sqrt{\tfrac{\log k(h)}{m}} > \frac{\epsilon}{2}\right] \\
&\quad \leq 2e^{-\frac{m\epsilon^2}{2}} + \mathbb{P}\left[F_{k(h)}(h) - R(h) - 2\mathfrak{R}_m(\mathcal{H}_{k(h)}) - \sqrt{\tfrac{\log k(h)}{m}} > \frac{\epsilon}{2}\right] \\
&\quad = 2e^{-\frac{m\epsilon^2}{2}} + \mathbb{P}\left[\widehat{R}_S(h) - R(h) - \mathfrak{R}_m(\mathcal{H}_{k(h)}) > \frac{\epsilon}{2}\right] \\
&\quad = 2e^{-\frac{m\epsilon^2}{2}} + e^{-\frac{m\epsilon^2}{2}} = 3e^{-\frac{m\epsilon^2}{2}}.
\end{aligned}$$

Setting the right-hand side to be equal to δ completes the proof. □

The learning guarantee just proven for SRM is remarkable. To simplify its discussion, let us assume that there exists h^* such that $R(h^*) = \inf_{h\in\mathcal{H}} R(h)$, that is, that there exists a best-in-class classifier $h^* \in \mathcal{H}$. Then, the theorem implies in particular that, with probability at least $1-\delta$, the following inequality holds for all $h \in \mathcal{H}$:

$$R(h_S^{\mathrm{SRM}}) \leq R(h^*) + 2\mathfrak{R}_m(\mathcal{H}_{k(h^*)}) + \sqrt{\frac{\log k(h^*)}{m}} + \sqrt{\frac{2\log\frac{3}{\delta}}{m}}. \tag{4.6}$$

Observe that, remarkably, this bound is similar to the estimation error bound for $\mathcal{H}_{k(h^*)}$: it differs from it only by the term $\sqrt{\log k(h^*)/m}$. Thus, modulo that term, the guarantee for SRM is as favorable as the one we would have obtained, had an oracle informed us of the index $k(h^*)$ of the best-in-class classifier's hypothesis set.

Furthermore, observe that when $\mathcal{H}$ is rich enough that $R(h^*)$ is close to the Bayes error, the learning bound (4.6) is approximately a bound on the excess error of the SRM solution. Note that, if for some k_0, the empirical error of the ERM solution for $\mathcal{H}_{k_0}$ is zero, which holds in particular if $\mathcal{H}_{k_0}$ contains the Bayes error, then, we have $\min_{h\in\mathcal{H}_k} F_{k_0}(h) \leq \min_{h\in\mathcal{H}_k} F_k(h)$ for all $k > k_0$ and only finitely many indices need to be considered in SRM.

Assume more generally that if $\min_{h\in\mathcal{H}_k} F_k(h) \leq \min_{h\in\mathcal{H}_{k+1}} F_k(h)$ for some k, then indices beyond $k+1$ need not be inspected. This property may hold for example if the empirical error cannot be further improved after some index k. In that case, the minimizing index k^* can be determined via a binary search in the interval $[1, k_{\max}]$, given some maximum value $k_{\max}$. $k_{\max}$ itself can be found by inspecting $\min_{h\in\mathcal{H}_{2^n}} F_k(h)$ for exponentially growing indices 2^n, $n \geq 1$, and setting $k_{\max} = 2^n$ for n such that $\min_{h\in\mathcal{H}_{2^n}} F_k(h) \leq \min_{h\in\mathcal{H}_{2^{n+1}}} F_k(h)$. The number of ERM computations needed to find $k_{\max}$ is in $O(n) = O(\log k_{\max})$ and similarly the

number of ERM computations due to the binary search is in $O(\log k_{\max})$. Thus, if n is the smallest integer such that $k^* < 2^n$, the overall number of ERM computations is in $O(\log k^*)$.

While it benefits from a very favorable guarantee, SRM admits several drawbacks. First, the decomposability of $\mathcal{H}$ into countably many hypothesis sets, each with a converging Rademacher complexity, remains a strong assumption. As an example, the family of all measurable functions cannot be written as a union of countably many hypothesis sets with finite VC-dimension. Thus, the choice of $\mathcal{H}$ or that of the hypothesis sets $\mathcal{H}_k$ is a key component of SRM. Second, and this is the main disadvantage of SRM, the method is typically computationally intractable: for most hypothesis sets, finding the solution of ERM is NP-hard and in general SRM requires determining that solution for a large number of indices k.

4.4 Cross-validation

An alternative method for model selection, *cross-validation*, consists of using some fraction of the training sample as a *validation set* to select a hypothesis set $\mathcal{H}_k$. This is in contrast with the SRM model which relies on a theoretical learning bound assigning a penalty to each hypothesis set. In this section, we analyze the cross-validation method and compare its performance to that of SRM.

As in the previous section, let $(\mathcal{H}_k)_{k\geq 1}$ be a countable sequence of hypothesis sets with increasing complexities. The cross-validation (CV) solution is obtained as follows. Let S be an i.i.d. labeled sample of size m. S is divided into a sample S_1 of size $(1-\alpha)m$ and a sample S_2 of size αm, with $\alpha \in (0,1)$ typically chosen to be relatively small. S_1 is reserved for training, S_2 for validation. For any $k \in \mathbb{N}$, let $h^{\mathrm{ERM}}_{S_1,k}$ denote the solution of ERM run on S_1 using the hypothesis set $\mathcal{H}_k$. The hypothesis h^{CV}_S returned by cross-validation is the ERM solution $h^{\mathrm{ERM}}_{S_1,k}$ with the best performance on S_2:

$$h^{\mathrm{CV}}_S = \operatorname*{argmin}_{h \in \left\{h^{\mathrm{ERM}}_{S_1,k} \colon k \geq 1\right\}} \widehat{R}_{S_2}(h). \tag{4.7}$$

The following general result will help us derive learning guarantees for cross-validation.

Proposition 4.3 *For any $\alpha > 0$ and any sample size $m \geq 1$, the following general inequality holds:*

$$\mathbb{P}\left[\sup_{k\geq 1}\left|R(h^{\mathrm{ERM}}_{S_1,k}) - \widehat{R}_{S_2}(h^{\mathrm{ERM}}_{S_1,k})\right| > \epsilon + \sqrt{\frac{\log k}{\alpha m}}\right] \leq 4e^{-2\alpha m\epsilon^2}.$$

Proof: By the union bound, we can write

$$\begin{aligned}\mathbb{P}\left[\sup_{k\geq 1}\left|R(h_{S_1,k}^{\mathrm{ERM}})-\widehat{R}_{S_2}(h_{S_1,k}^{\mathrm{ERM}})\right|>\epsilon+\sqrt{\frac{\log k}{\alpha m}}\right]&\\ \leq\sum_{k=1}^{\infty}\mathbb{P}\left[\left|R(h_{S_1,k}^{\mathrm{ERM}})-\widehat{R}_{S_2}(h_{S_1,k}^{\mathrm{ERM}})\right|>\epsilon+\sqrt{\frac{\log k}{\alpha m}}\right]&\\ =\sum_{k=1}^{\infty}\mathbb{E}\left[\mathbb{P}\left[\left|R(h_{S_1,k}^{\mathrm{ERM}})-\widehat{R}_{S_2}(h_{S_1,k}^{\mathrm{ERM}})\right|>\epsilon+\sqrt{\frac{\log k}{\alpha m}}\;\middle|\;S_1\right]\right].&\end{aligned} \tag{4.8}$$

The hypothesis $h_{S_1,k}^{\mathrm{ERM}}$ is fixed conditioned on S_1. Furthermore, the sample S_2 is independent from S_1. Therefore, by Hoeffding's inequality, we can bound the conditional probability as follows:

$$\begin{aligned}\mathbb{P}\left[\left|R(h_{S_1,k}^{\mathrm{ERM}})-\widehat{R}_{S_2}(h_{S_1,k}^{\mathrm{ERM}})\right|>\epsilon+\sqrt{\frac{\log k}{\alpha m}}\;\middle|\;S_1\right]&\leq 2e^{-2\alpha m\left(\epsilon+\sqrt{\frac{\log k}{\alpha m}}\right)^2}.\\ &\leq 2e^{-2\alpha m\epsilon^2-2\log k}\\ &=\frac{2}{k^2}e^{-2\alpha m\epsilon^2}.\end{aligned}$$

Plugging in the right-hand side of this bound in (4.8) and summing over k yields

$$\mathbb{P}\left[\sup_{k\geq 1}\left|R(h_{S_1,k}^{\mathrm{ERM}})-\widehat{R}_{S_2}(h_{S_1,k}^{\mathrm{ERM}})\right|>\epsilon+\sqrt{\frac{\log k}{\alpha m}}\right]\leq\frac{\pi^2}{3}e^{-2\alpha m\epsilon^2}<4e^{-2\alpha m\epsilon^2},$$

which completes the proof. □

Let $R(h_{S_1}^{\mathrm{SRM}})$ be the generalization error of the SRM solution using a sample S_1 of size $(1-\alpha)m$) and $R(h_S^{\mathrm{CV}},S)$ the generalization error of the cross-validation solution using a sample S of size m. Then, using Proposition 4.3, the following learning guarantee can be derived which compares the error of the CV method to that of SRM.

Theorem 4.4 (Cross-validation versus SRM) *For any $\delta>0$, with probability at least $1-\delta$, the following holds:*

$$R(h_S^{\mathrm{CV}})-R(h_{S_1}^{\mathrm{SRM}})\leq 2\sqrt{\frac{\log\max(k(h_S^{\mathrm{CV}}),k(h_{S_1}^{\mathrm{SRM}}))}{\alpha m}}+2\sqrt{\frac{\log\frac{4}{\delta}}{2\alpha m}},$$

where, for any h, $k(h)$ denotes the smallest index of a hypothesis set containing h.

Proof: By Proposition 4.3 and Theorem 4.2, using the property of h_S^{CV} as a minimizer, for any $\delta>0$, with probability at least $1-\delta$, the following inequalities

hold:

$$\begin{aligned}
R(h_S^{\mathrm{CV}}) &\leq \widehat{R}_{S_2}(h_S^{\mathrm{CV}}) + \sqrt{\frac{\log(k(h_S^{\mathrm{CV}}))}{\alpha m}} + \sqrt{\frac{\log\frac{4}{\delta}}{2\alpha m}} \\
&\leq \widehat{R}_{S_2}(h_{S_1}^{\mathrm{SRM}}) + \sqrt{\frac{\log(k(h_S^{\mathrm{CV}}))}{\alpha m}} + \sqrt{\frac{\log\frac{4}{\delta}}{2\alpha m}} \\
&\leq R(h_{S_1}^{\mathrm{SRM}}) + \sqrt{\frac{\log(k(h_S^{\mathrm{CV}}))}{\alpha m}} + \sqrt{\frac{\log(k(h_{S_1}^{\mathrm{SRM}}))}{\alpha m}} + 2\sqrt{\frac{\log\frac{4}{\delta}}{2\alpha m}} \\
&\leq R(h_{S_1}^{\mathrm{SRM}}) + 2\sqrt{\frac{\log(\max(k(h_S^{\mathrm{CV}}), k(h_{S_1}^{\mathrm{SRM}}))}{\alpha m}} + 2\sqrt{\frac{\log\frac{4}{\delta}}{2\alpha m}},
\end{aligned}$$

which completes the proof. □

The learning guarantee just proven shows that, with high probability, the generalization error of the CV solution for a sample of size m is close to that of the SRM solution for a sample of size $(1-\alpha)m$. For α relatively small, this suggests a guarantee similar to that of SRM, which, as previously discussed, is very favorable. However, in some unfavorable regimes, an algorithm (here SRM) trained on $(1-\alpha)m$ points may have a significantly worse performance than when trained on m points (avoiding this phase transition issue is one of the main motivations behind the use of the n-fold cross-validation method in practice, see section 4.5). Thus, the bound suggests in fact a trade-off: α should be chosen sufficiently small to avoid the unfavorable regimes just mentioned and yet sufficiently large for the right-hand side of the bound to be small and thus informative.

The learning bound for CV can be made more explicit in some cases in practice. Assume for example that the hypothesis sets $\mathcal{H}_k$ are nested and that the empirical errors of the ERM solutions $h_{S_1,k}^{\mathrm{ERM}}$ are decreasing before reaching zero: for any k, $\widehat{R}_{S_1}(h_{S_1,k+1}^{\mathrm{ERM}}) < \widehat{R}_{S_1}(h_{S_1,k}^{\mathrm{ERM}})$ for all k such that $\widehat{R}_{S_1}(h_{S_1,k}^{\mathrm{ERM}}) > 0$ and $\widehat{R}_{S_1}(h_{S_1,k+1}^{\mathrm{ERM}}) \leq \widehat{R}_{S_1}(h_{S_1,k}^{\mathrm{ERM}})$ otherwise. Observe that $\widehat{R}_{S_1}(h_{S_1,k}^{\mathrm{ERM}}) > 0$ implies at least one error for $h_{S_1,k}^{\mathrm{ERM}}$, therefore $\widehat{R}_{S_1}(h_{S_1,k}^{\mathrm{ERM}}) > \frac{1}{m}$. In view of that, we must then have $\widehat{R}_{S_1}(h_{S_1,n}^{\mathrm{ERM}}) = 0$ for all $n \geq m+1$. Thus, we have $h_{S_1,n}^{\mathrm{ERM}} = h_{S_1,m+1}^{\mathrm{ERM}}$ for all $n \geq m+1$ and we can assume that $k(f_{CV}) \leq m+1$. Since the complexity of $\mathcal{H}_k$ increases with k we also have $k(f_{SRM}) \leq m+1$. In view of that, we obtain the following more explicit learning bound for cross-validation:

$$R(f_{CV}, S) - R(f_{SRM}, S_1) \leq 2\sqrt{\frac{\log(\frac{4}{\delta})}{2\alpha m}} + 2\sqrt{\frac{\log(m+1)}{\alpha m}}.$$

4.5 n-Fold cross-validation

In practice, the amount of labeled data available is often too small to set aside a validation sample since that would leave an insufficient amount of training data. Instead, a widely adopted method known as *n-fold cross-validation* is used to exploit the labeled data both for *model selection* and for training.

Let $\boldsymbol{\theta}$ denote the vector of free parameters of the algorithm. For a fixed value of $\boldsymbol{\theta}$, the method consists of first randomly partitioning a given sample S of m labeled examples into n subsamples, or folds. The ith fold is thus a labeled sample $((x_{i1}, y_{i1}), \ldots, (x_{im_i}, y_{im_i}))$ of size m_i. Then, for any $i \in [n]$, the learning algorithm is trained on all but the ith fold to generate a hypothesis h_i, and the performance of h_i is tested on the ith fold, as illustrated in figure 4.5a. The parameter value $\boldsymbol{\theta}$ is evaluated based on the average error of the hypotheses h_i, which is called the *cross-validation error*. This quantity is denoted by $\widehat{R}_{\mathrm{CV}}(\boldsymbol{\theta})$ and defined by

$$\widehat{R}_{\mathrm{CV}}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \underbrace{\frac{1}{m_i} \sum_{j=1}^{m_i} L(h_i(x_{ij}), y_{ij})}_{\text{error of } h_i \text{ on the } i\text{th fold}} .$$

The folds are generally chosen to have equal size, that is $m_i = m/n$ for all $i \in [n]$. How should n be chosen? The appropriate choice is subject to a trade-off. For a large n, each training sample used in n-fold cross-validation has size $m - m/n = m(1 - 1/n)$ (illustrated by the right vertical red line in figure 4.5b), which is close to m, the size of the full sample, and also implies all training samples are quite similar. At the same time, the ith fold used to measure the error is relatively small and thus the cross-validation error tends to have a small bias but a large variance. In contrast, smaller values of n lead to more diverse training samples but their size (shown by the left vertical red line in figure 4.5b) is significantly less than m. In this regime, the ith fold is relatively large and thus the cross-validation error tends to have a smaller variance but a larger bias.

In applications, n is typically chosen to be 5 or 10. n-fold cross-validation is used as follows in model selection. The full labeled data is first split into a training and a test sample. The training sample of size m is then used to compute the n-fold cross-validation error $\widehat{R}_{\mathrm{CV}}(\boldsymbol{\theta})$ for a small number of possible values of $\boldsymbol{\theta}$. The free parameter $\boldsymbol{\theta}$ is next set to the value $\boldsymbol{\theta}_0$ for which $\widehat{R}_{\mathrm{CV}}(\boldsymbol{\theta})$ is smallest and the algorithm is trained with the parameter setting $\boldsymbol{\theta}_0$ over the full training sample of size m. Its performance is evaluated on the test sample as already described in the previous section.

The special case of n-fold cross-validation where $n = m$ is called *leave-one-out cross-validation*, since at each iteration exactly one instance is left out of the train-

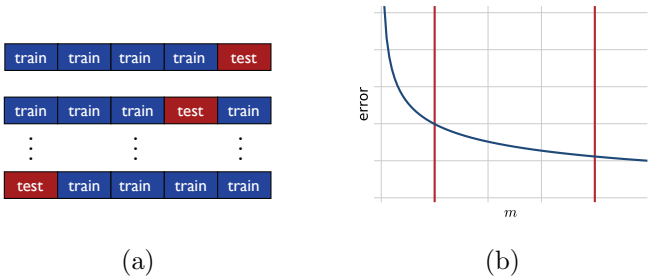


Figure 4.5
n-fold cross-validation. (a) Illustration of the partitioning of the training data into 5 folds. (b) Typical plot of a classifier's prediction error as a function of the size of the training sample m: the error decreases as a function of the number of training points. The red line on the left side marks the region for small values of n, while the red line on the right side marks the region for large values of n.

ing sample. As shown in chapter 5, the average leave-one-out error is an approximately unbiased estimate of the average error of an algorithm and can be used to derive simple guarantees for some algorithms. In general, the leave-one-out error is very costly to compute, since it requires training m times on samples of size $m-1$, but for some algorithms it admits a very efficient computation (see exercise 11.9).

In addition to model selection, n-fold cross-validation is also commonly used for performance evaluation. In that case, for a fixed parameter setting $\boldsymbol{\theta}$, the full labeled sample is divided into n random folds with no distinction between training and test samples. The performance reported is the n-fold cross-validation error on the full sample as well as the standard deviation of the errors measured on each fold.

4.6 Regularization-based algorithms

A broad family of algorithms inspired by the SRM method is that of *regularization-based algorithm*. This consists of selecting a very complex family $\mathcal{H}$ that is an uncountable union of nested hypothesis sets $\mathcal{H}_\gamma$: $\mathcal{H} = \bigcup_{\gamma>0} \mathcal{H}_\gamma$. $\mathcal{H}$ is often chosen to be dense in the space of continuous functions over $\mathcal{X}$. For example, $\mathcal{H}$ may be chosen to be the set of all linear functions in some high-dimensional space and $\mathcal{H}_\gamma$ the subset of those functions whose norm is bounded by γ: $\mathcal{H}_\gamma = \{x \mapsto \mathbf{w} \cdot \boldsymbol{\Phi}(x) \colon \|\mathbf{w}\| \leq \gamma\}$. For some choices of $\boldsymbol{\Phi}$ and the high-dimensional space, it can be shown that $\mathcal{H}$ is indeed dense in the space of continuous functions over $\mathcal{X}$.

Given a labeled sample S, the extension of the SRM method to an uncountable union would then suggest selecting h based on the following optimization problem:

$$\operatorname*{argmin}_{\gamma>0, h\in H_\gamma} \widehat{R}_S(h) + \mathfrak{R}_m(\mathcal{H}_\gamma) + \sqrt{\frac{\log \gamma}{m}},$$

where other penalty terms $\operatorname{pen}(\gamma, m)$ can be chosen in lieu of the specific choice $\operatorname{pen}(\gamma, m) = \mathfrak{R}_m(\mathcal{H}_\gamma) + \sqrt{\frac{\log \gamma}{m}}$. Often, there exists a function $\mathcal{R}\colon \mathcal{H} \to \mathbb{R}$ such that, for any $\gamma > 0$, the constrained optimization problem $\operatorname{argmin}_{\gamma>0, h\in H_\gamma} \widehat{R}_S(h) + \operatorname{pen}(\gamma, m)$ can be equivalently written as the unconstrained optimization problem

$$\operatorname*{argmin}_{h\in\mathcal{H}} \widehat{R}_S(h) + \lambda \mathcal{R}(h),$$

for some $\lambda > 0$. $\mathcal{R}(h)$ is called a *regularization term* and $\lambda > 0$ is treated as a hyperparameter since its optimal value is often not known. For most algorithms, the regularization term $\mathcal{R}(h)$ is chosen to be an increasing function of $\|h\|$ for some choice of the norm $\|\cdot\|$, when $\mathcal{H}$ is the subset of a Hilbert space. The variable λ is often called a *regularization parameter*. Larger values of λ further penalize more complex hypotheses, while, for λ close or equal to zero, the regularization term has no effect and the algorithm coincides with ERM. In practice, λ is typically selected via cross-validation or using n-fold cross-validation.

When the regularization term is chosen to be $\|h\|_p$ for some choice of the norm and $p \geq 1$, then it is a convex function of h, since any norm is convex. However, for the zero-one loss, the first term of the objective function is non-convex, thereby making the optimization problem computationally hard. In practice, most regularization-based algorithms instead use a convex upper bound on the zero-one loss and replace the empirical zero-one term with the empirical value of that convex surrogate. The resulting optimization problem is then convex and therefore admits more efficient solutions than SRM. The next section studies the properties of such convex surrogate losses.

4.7 Convex surrogate losses

The guarantees for the estimation error that we presented in previous sections hold either for ERM or for SRM, which itself is defined in terms of ERM. However, as already mentioned, for many choices of the hypothesis set $\mathcal{H}$, including that of linear functions, solving the ERM optimization problem is NP-hard mainly because the zero-one loss function is not convex. One common method for addressing this problem consists of using a convex surrogate loss function that upper bounds the zero-one loss. This section analyzes learning guarantees for such surrogate losses in terms of the original loss.

The hypotheses we consider are real-valued functions $h\colon \mathcal{X} \to \mathbb{R}$. The sign of h defines a binary classifier $f_h\colon \mathcal{X} \to \{-1, +1\}$ defined for all $x \in \mathcal{X}$ by

$$f_h(x) = \begin{cases} +1 & \text{if } h(x) \geq 0 \\ -1 & \text{if } h(x) < 0. \end{cases}$$

The loss or error of h at point $(x, y) \in \mathcal{X} \times \{-1, +1\}$ is defined as the binary classification error of f_h:

$$1_{f_h(x) \neq y} = 1_{yh(x)<0} + 1_{h(x)=0 \wedge y=-1} \leq 1_{yh(x)\leq 0}.$$

We will denote by $R(h)$ the expected error of h: $R(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\left[1_{f_h(x)\neq y}\right]$. For any $x \in \mathcal{X}$, let $\eta(x)$ denote $\eta(x) = \mathbb{P}[y = +1|x]$ and let $\mathcal{D}_{\mathcal{X}}$ denote the marginal distribution over $\mathcal{X}$. Then, for any h, we can write

$$\begin{aligned} R(h) &= \mathop{\mathbb{E}}_{(x,y)\sim\mathcal{D}}\left[1_{f_h(x)\neq y}\right] \\ &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_{\mathcal{X}}}\left[\eta(x)1_{h(x)<0} + (1-\eta(x))1_{h(x)>0} + (1-\eta(x))1_{h(x)=0}\right] \\ &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_{\mathcal{X}}}\left[\eta(x)1_{h(x)<0} + (1-\eta(x))1_{h(x)\geq 0}\right]. \end{aligned}$$

In view of that, the Bayes classifier can be defined as assigning label $+1$ to x when $\eta(x) \geq \frac{1}{2}$, -1 otherwise. It can therefore be induced by the function h^* defined by

$$h^*(x) = \eta(x) - \tfrac{1}{2}. \tag{4.9}$$

We will refer to $h^*\colon \mathcal{X} \to \mathbb{R}$ as the *Bayes scoring function* and will denote by R^* the error of the Bayes classifier or Bayes scoring function: $R^* = R(h^*)$.

Lemma 4.5 *The excess error of any hypothesis $h\colon \mathcal{X} \to \mathbb{R}$ can be expressed as follows in terms of η and the Bayes scoring function h^*:*

$$R(h) - R^* = 2 \mathop{\mathbb{E}}_{x\sim\mathcal{D}_{\mathcal{X}}}\left[|h^*(x)|\, 1_{h(x)h^*(x)\leq 0}\right].$$

Proof: For any h, we can write

$$\begin{aligned} R(h) &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_{\mathcal{X}}}\left[\eta(x)1_{h(x)<0} + (1-\eta(x))1_{h(x)\geq 0}\right] \\ &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_{\mathcal{X}}}\left[\eta(x)1_{h(x)<0} + (1-\eta(x))(1-1_{h(x)<0})\right] \\ &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_{\mathcal{X}}}\left[[2\eta(x)-1]1_{h(x)<0} + (1-\eta(x))\right] \\ &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_{\mathcal{X}}}\left[2h^*(x)1_{h(x)<0} + (1-\eta(x))\right], \end{aligned}$$

where we used for the last step equation (4.9). In view of that, for any h, the following holds:

$$\begin{aligned} R(h) - R(h^*) &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[2[h^*(x)](1_{h(x)\leq 0} - 1_{h^*(x)\leq 0})\Big] \\ &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[2[h^*(x)]\operatorname{sgn}(h^*(x))1_{(h(x)h^*(x)\leq 0)\wedge((h(x),h^*(x))\neq(0,0))}\Big] \\ &= 2\mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[|h^*(x)|\,1_{h(x)h^*(x)\leq 0}\Big], \end{aligned}$$

which completes the proof, since $R(h^*) = R^*$. □

Let $\Phi\colon \mathbb{R}\to\mathbb{R}$ be a convex and non-decreasing function so that for any $u\in\mathbb{R}$, $1_{u\leq 0}\leq \Phi(-u)$. The Φ-loss of a function $h\colon \mathcal{X}\to\mathbb{R}$ at point $(x,y)\in\mathcal{X}\times\{-1,+1\}$ is defined as $\Phi(-yh(x))$ and its expected loss given by

$$\begin{aligned} \mathcal{L}_\Phi(h) &= \mathop{\mathbb{E}}_{(x,y)\sim\mathcal{D}}\big[\Phi(-yh(x))\big] \\ &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[\eta(x)\Phi(-h(x)) + (1-\eta(x))\Phi(h(x))\Big]. \end{aligned} \tag{4.10}$$

Notice that since $1_{yh(x)\leq 0}\leq\Phi(-yh(x))$, we have $R(h)\leq\mathcal{L}_\Phi(h)$. For any $x\in\mathcal{X}$, let $u\mapsto L_\Phi(x,u)$ be the function defined for all $u\in\mathbb{R}$ by

$$L_\Phi(x,u) = \eta(x)\Phi(-u) + (1-\eta(x))\Phi(u).$$

Then, $\mathcal{L}_\Phi(h) = \mathbb{E}_{x\sim\mathcal{D}_\mathcal{X}}[L_\Phi(x,h(x))]$. Since Φ is convex, $u\mapsto L_\Phi(x,u)$ is convex as a sum of two convex functions. Define $h^*_\Phi\colon\mathcal{X}\to[-\infty,+\infty]$ as the *Bayes solution for the loss function* L_Φ. That is, for any x, $h^*_\Phi(x)$ is a solution of the following convex optimization problem:

$$\begin{aligned} h^*_\Phi(x) &= \operatorname*{argmin}_{u\in[-\infty,+\infty]} L_\Phi(x,u) \\ &= \operatorname*{argmin}_{u\in[-\infty,+\infty]} \eta(x)\Phi(-u) + (1-\eta(x))\Phi(u). \end{aligned}$$

The solution of this optimization is in general not unique. When $\eta(x) = 0$, $h^*_\Phi(x)$ is a minimizer of $u\mapsto\Phi(u)$ and since Φ is non-decreasing, we can choose $h^*_\Phi(x) = -\infty$ in that case. Similarly, when $\eta(x) = 1$, we can choose $h^*_\Phi(x) = +\infty$. When $\eta(x) = \frac{1}{2}$, $L_\Phi(x,u) = \frac{1}{2}[\Phi(-u)+\Phi(u)]$, thus, by convexity, $L_\Phi(x,u)\geq\Phi(-\frac{u}{2}+\frac{u}{2}) = \Phi(0)$. Thus, we can choose $h^*_\Phi(x) = 0$ in that case. For all other values of $\eta(x)$, in case of non-uniqueness, an arbitrary minimizer is chosen in this definition. We will denote by $\mathcal{L}^*_\Phi$ the Φ-loss of h^*_Φ: $\mathcal{L}^*_\Phi = \mathbb{E}_{(x,y)\sim\mathcal{D}}\big[\Phi(-yh^*_\Phi(x))\big]$.

Proposition 4.6 *Let Φ be a convex and non-decreasing function that is differentiable at 0 with $\Phi'(0) > 0$. Then, the minimizer of Φ defines the Bayes classifier: for any $x\in\mathcal{X}$, $h^*_\Phi(x) > 0$ iff $h^*(x) > 0$ and $h^*(x) = 0$ iff $h^*_\Phi(x) = 0$, which implies $\mathcal{L}^*_\Phi = R^*$.*

Proof: Fix $x \in \mathcal{X}$. If $\eta(x) = 0$, then $h^*(x) = -\frac{1}{2}$ and $h^*_\Phi(x) = -\infty$, thus $h^*(x)$ and $h^*_\Phi(x)$ admit the same sign. Similarly, if $\eta(x) = 1$, then $h^*(x) = +\frac{1}{2}$ and $h^*_\Phi(x) = +\infty$, and $h^*(x)$ and $h^*_\Phi(x)$ admit the same sign.

Let u^* denote the minimizer defining $h^*_\Phi(x)$. u^* is a minimizer of $u \mapsto L_\Phi(x, u)$ iff the subdifferential of that function at u^* contains 0, that is, since $\partial L_\Phi(x, u^*) = -\eta(x)\partial\Phi(-u^*) + (1 - \eta(x))\partial\Phi(u^*)$, iff there exist $v_1 \in \partial\Phi(-u^*)$ and $v_2 \in \partial\Phi(u^*)$ such that

$$\eta(x)v_1 = (1 - \eta(x))v_2. \tag{4.11}$$

If $u^* = 0$, by the differentiability of Φ at 0 we have $v_1 = v_2 = \Phi'(0) > 0$ and thus $\eta(x) = \frac{1}{2}$, that is $h^*(x) = 0$. Conversely, If $h^*(x) = 0$, that is $\eta(x) = \frac{1}{2}$, then, by definition, we have $h^*_\Phi(x) = 0$. Thus, $h^*(x) = 0$ iff $h^*_\Phi(x) = 0$ iff $\eta(x) = \frac{1}{2}$.

We can assume now that $\eta(x)$ is not in $\{0, 1, \frac{1}{2}\}$. We first show that for any $u_1, u_2 \in \mathbb{R}$ with $u_1 < u_2$, and any two choices of the subgradients at u_1 and u_2, $v_1 \in \partial\Phi(u_1)$ and $v_2 \in \partial\Phi(u_2)$, we have $v_1 \leq v_2$. By definition of the subgradients at u_1 and u_2, the following inequalities hold:

$$\Phi(u_2) - \Phi(u_1) \geq v_1(u_2 - u_1) \qquad \Phi(u_1) - \Phi(u_2) \geq v_2(u_1 - u_2).$$

Summing up these inequalities yields $v_2(u_2 - u_1) \geq v_1(u_2 - u_1)$ and thus $v_2 \geq v_1$, since $u_1 < u_2$.

Now, if $u^* > 0$, then we have $-u^* < u^*$. By the property shown above, this implies $v_1 \leq v_2$. We cannot have $v_1 = v_2 \neq 0$ since (4.11) would then imply $\eta(x) = \frac{1}{2}$. We also cannot have $v_1 = v_2 = 0$ since by the property shown above, we must have $\Phi'(0) \leq v_2$ and thus $v_2 > 0$. Thus, we must have $v_1 < v_2$ with $v_2 > 0$, which, by (4.11), implies $\eta(x) > 1 - \eta(x)$, that is $h^*(x) > 0$.

Conversely, if $h^*(x) > 0$ then $\eta(x) > 1 - \eta(x)$. We cannot have $v_1 = v_2 = 0$ or $v_1 = v_2 \neq 0$ as already shown. Thus, since $\eta(x) \neq 1$, by (4.11), this implies $v_1 < v_2$. We cannot have $u^* < -u^*$ since, by the property shown above, this would imply $v_2 \leq v_1$. Thus, we must have $-u^* \leq u^*$, that is $u^* \geq 0$, and more specifically $u^* > 0$ since, as already shown above, $u^* = 0$ implies $h^*(x) = 0$. □

Theorem 4.7 *Let Φ be a convex and non-decreasing function. Assume that there exists $s \geq 1$ and $c > 0$ such that the following holds for all $x \in \mathcal{X}$:*

$$|h^*(x)|^s = \left|\eta(x) - \tfrac{1}{2}\right|^s \leq c^s \left[L_\Phi(x, 0) - L_\Phi(x, h^*_\Phi(x))\right].$$

Then, for any hypothesis h, the excess error of h is bounded as follows:

$$R(h) - R^* \leq 2c \left[\mathcal{L}_\Phi(h) - \mathcal{L}^*_\Phi\right]^{\frac{1}{s}}$$

Proof: We will use the following inequality which holds by the convexity of Φ:

$$\begin{aligned}\Phi\big(-2h^*(x)h(x)\big) &= \Phi\big((1-2\eta(x))h(x)\big)\\ &= \Phi\big(\eta(x)(-h(x)) + (1-\eta(x))h(x)\big)\\ &\leq \eta(x)\Phi((-h(x))) + (1-\eta(x))\Phi(h(x)) = L_\Phi(x, h(x)). \quad (4.12)\end{aligned}$$

By Lemma 4.5, Jensen's inequality, and $h^*(x) = \eta(x) - \frac{1}{2}$, we can write

$$\begin{aligned}&R(h) - R(h^*)\\ &= \mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[|2\eta(x)-1|\,1_{h(x)h^*(x)\leq 0}\Big]\\ &\leq \mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[|2\eta(x)-1|^s\,1_{h(x)h^*(x)\leq 0}\Big]^{\frac{1}{s}} && \text{(Jensen's ineq.)}\\ &\leq 2c \mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[\big[\Phi(0) - L_\Phi(x, h^*_\Phi(x))\big]\,1_{h(x)h^*(x)\leq 0}\Big]^{\frac{1}{s}} && \text{(assumption)}\\ &\leq 2c \mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[\big[\Phi\big(-2h^*(x)h(x)\big) - L_\Phi(x, h^*_\Phi(x))\big]\,1_{h(x)h^*(x)\leq 0}\Big]^{\frac{1}{s}} && (\Phi \text{ non-decreasing})\\ &\leq 2c \mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[[L_\Phi(x, h(x)) - L_\Phi(x, h^*_\Phi(x))]\,1_{h(x)h^*(x)\leq 0}\Big]^{\frac{1}{s}} && \text{(convexity ineq. (4.12))}\\ &\leq 2c \mathop{\mathbb{E}}_{x\sim\mathcal{D}_\mathcal{X}}\Big[L_\Phi(x, h(x)) - L_\Phi(x, h^*_\Phi(x))\Big]^{\frac{1}{s}},\end{aligned}$$

which completes the proof, since $\mathbb{E}_{x\sim\mathcal{D}_\mathcal{X}}[L_\Phi(x, h^*_\Phi(x))] = L^*_\Phi$. □

The theorem shows that, when the assumption holds, the excess error of h can be upper bounded in terms of the excess Φ-loss. The assumption of the theorem holds in particular for the following convex loss functions:

- Hinge loss, where $\Phi(u) = \max(0, 1+u)$, with $s = 1$ and $c = \frac{1}{2}$.
- Exponential loss, where $\Phi(u) = \exp(u)$, with $s = 2$ and $c = \frac{1}{\sqrt{2}}$.
- Logistic loss, where $\Phi(u) = \log_2(1+e^u)$, with $s = 2$ and $c = \frac{1}{\sqrt{2}}$.

They also hold for the square loss and the squared Hinge loss (see Exercises 4.2 and 4.3).

4.8 Chapter notes

The structural risk minimization (SRM) technique is due to Vapnik [1998]. The original penalty term used by Vapnik [1998] is based on the VC-dimension of the hypothesis set. The version of SRM with Rademacher complexity-based penalties that we present here leads to finer data-dependent learning guarantees. Penalties based on alternative complexity measures can be used similarly leading to learning bounds in terms of the corresponding complexity measure [Bartlett et al., 2002a].

An alternative model selection theory of *Voted Risk Minimization* (VRM) has been recently developed by Cortes, Mohri, and Syed [2014] and other related publications [Kuznetsov et al., 2014, DeSalvo et al., 2015, Cortes et al., 2015].

Theorem 4.7 is due to Zhang [2003a]. The proof given here is somewhat different and simpler.

4.9 Exercises

4.1 For any hypothesis set $\mathcal{H}$, show that the following inequalities hold:

$$\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}\left[\widehat{R}_S\left(h_S^{\mathrm{ERM}}\right)\right] \leq \inf_{h\in\mathcal{H}} R(h) \leq \mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}\left[R\left(h_S^{\mathrm{ERM}}\right)\right]. \tag{4.13}$$

4.2 Show that for the squared loss, $\Phi(u) = (1+u)^2$, the statement of Theorem 4.7 holds with $s = 2$ and $c = \frac{1}{2}$ and therefore that the excess error can be upper bounded as follows:

$$R(h) - R^* \leq \left[\mathcal{L}_\Phi(h) - \mathcal{L}_\Phi^*\right]^{\frac{1}{2}}.$$

4.3 Show that for the squared Hinge loss, $\Phi(u) = \max(0, 1+u)^2$, the statement of Theorem 4.7 holds with $s = 2$ and $c = \frac{1}{2}$ and therefore that the excess error can be upper bounded as follows:

$$R(h) - R^* \leq \left[\mathcal{L}_\Phi(h) - \mathcal{L}_\Phi^*\right]^{\frac{1}{2}}.$$

4.4 In this problem, the loss of $h\colon \mathcal{X} \to \mathbb{R}$ at point $(x, y) \in \mathcal{X} \times \{-1, +1\}$ is defined to be $1_{yh(x)\leq 0}$.

(a) Define the Bayes classifier and a Bayes scoring function h^* for this loss.

(b) Express the excess error of h in terms of h^* (counterpart of Lemma 4.5, for loss considered here).

(c) Give a counterpart of the result of Theorem 4.7 for this loss.

4.5 Same questions as in Exercise 4.5 with the loss of $h\colon \mathcal{X} \to \mathbb{R}$ at point $(x, y) \in \mathcal{X} \times \{-1, +1\}$ defined instead to be $1_{yh(x)<0}$.

5 Support Vector Machines

This chapter presents one of the most theoretically well motivated and practically most effective classification algorithms in modern machine learning: Support Vector Machines (SVMs). We first introduce the algorithm for separable datasets, then present its general version designed for non-separable datasets, and finally provide a theoretical foundation for SVMs based on the notion of margin. We start with the description of the problem of linear classification.

5.1 Linear classification

Consider an input space $\mathcal{X}$ that is a subset of $\mathbb{R}^N$ with $N \geq 1$, and the output or target space $\mathcal{Y} = \{-1, +1\}$, and let $f\colon \mathcal{X} \to \mathcal{Y}$ be the target function. Given a hypothesis set $\mathcal{H}$ of functions mapping $\mathcal{X}$ to $\mathcal{Y}$, the binary classification task is formulated as follows. The learner receives a training sample S of size m drawn i.i.d. from $\mathcal{X}$ according to some unknown distribution $\mathcal{D}$, $S = ((x_1, y_1), \ldots, (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$, with $y_i = f(x_i)$ for all $i \in [m]$. The problem consists of determining a hypothesis $h \in \mathcal{H}$, a *binary classifier*, with small generalization error:

$$R_{\mathcal{D}}(h) = \mathop{\mathbb{P}}_{x \sim \mathcal{D}}[h(x) \neq f(x)]. \tag{5.1}$$

Different hypothesis sets $\mathcal{H}$ can be selected for this task. In view of the results presented in chapter 3, which formalized Occam's razor principle, hypothesis sets with smaller complexity — e.g., smaller VC-dimension or Rademacher complexity — provide better learning guarantees, everything else being equal. A natural hypothesis set with relatively small complexity is that of *linear classifiers*, or hyperplanes, which can be defined as follows:

$$\mathcal{H} = \{\mathbf{x} \mapsto \operatorname{sign}(\mathbf{w} \cdot \mathbf{x} + b)\colon \mathbf{w} \in \mathbb{R}^N, b \in \mathbb{R}\}. \tag{5.2}$$

The learning problem is then referred to as a *linear classification problem*. The general equation of a hyperplane in $\mathbb{R}^N$ is $\mathbf{w} \cdot \mathbf{x} + b = 0$, where $\mathbf{w} \in \mathbb{R}^N$ is a

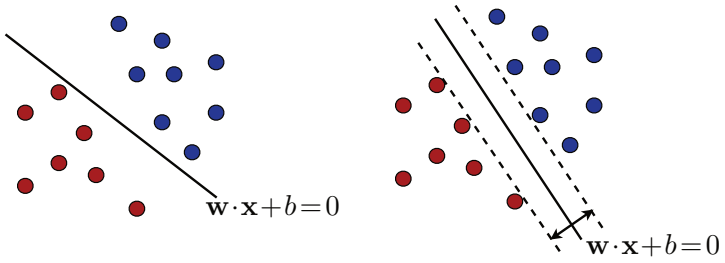


Figure 5.1
Two possible separating hyperplanes. The right-hand side figure shows a hyperplane that maximizes the margin.

non-zero vector normal to the hyperplane and $b \in \mathbb{R}$ a scalar. A hypothesis of the form $\mathbf{x} \mapsto \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$ thus labels positively all points falling on one side of the hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$ and negatively all others.

5.2 Separable case

In this section, we assume that the training sample S can be linearly separated, that is, we assume the existence of a hyperplane that perfectly separates the training sample into two populations of positively and negatively labeled points, as illustrated by the left panel of figure 5.1. This is equivalent to the existence of $(\mathbf{w}, b) \in (\mathbb{R}^N - \{\mathbf{0}\}) \times \mathbb{R}$ such that

$$\forall i \in [m], \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 0. \tag{5.3}$$

But, as can be seen from figure 5.1, there are then infinitely many such separating hyperplanes. Which hyperplane should a learning algorithm select? The definition of the SVM solution is based on the notion of *geometric margin*.

Definition 5.1 (Geometric margin) *The* geometric margin $\rho_h(\mathbf{x})$ *of a linear classifier* $h\colon \mathbf{x} \mapsto \mathbf{w}\cdot\mathbf{x}+b$ *at a point* $\mathbf{x}$ *is its Euclidean distance to the hyperplane* $\mathbf{w}\cdot\mathbf{x}+b = 0$*:*

$$\rho_h(x) = \frac{|\mathbf{w} \cdot \mathbf{x} + b|}{\|\mathbf{w}\|_2}. \tag{5.4}$$

The geometric margin ρ_h *of a linear classifier* h *for a sample* $S = (\mathbf{x}_1, \ldots, \mathbf{x}_m)$ *is the minimum geometric margin over the points in the sample,* $\rho_h = \min_{i\in[m]} \rho_h(x_i)$*, that is the distance of the hyperplane defining* h *to the closest sample points.*

The SVM solution is the separating hyperplane with the maximum geometric margin and is thus known as the *maximum-margin hyperplane*. The right panel of figure 5.1 illustrates the maximum-margin hyperplane returned by the SVM

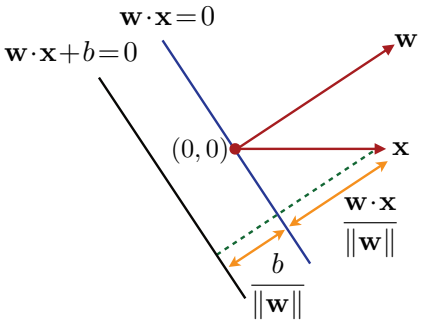


Figure 5.2
An illustration of the geometric margin of a point $\mathbf{x}$ in the case $\mathbf{w}\cdot\mathbf{x} > 0$ and $b > 0$.

algorithm in the separable case. We will present later in this chapter a theory that provides a strong justification for this solution. We can observe already, however, that the SVM solution can also be viewed as the "safest" choice in the following sense: a test point is classified correctly by a separating hyperplane with geometric margin ρ even when it falls within a distance ρ of the training samples sharing the same label; for the SVM solution, ρ is the maximum geometric margin and thus the "safest" value.

5.2.1 Primal optimization problem

We now derive the equations and optimization problem that define the SVM solution. By definition of the geometric margin (see also figure 5.2), the maximum margin ρ of a separating hyperplane is given by

$$\rho = \max_{\mathbf{w},b:\ y_i(\mathbf{w}\cdot\mathbf{x}_i+b)\geq 0} \min_{i\in[m]} \frac{|\mathbf{w}\cdot\mathbf{x}_i+b|}{\|\mathbf{w}\|} = \max_{\mathbf{w},b} \min_{i\in[m]} \frac{y_i\big(\mathbf{w}\cdot\mathbf{x}_i+b\big)}{\|\mathbf{w}\|}. \tag{5.5}$$

The second equality follows from the fact that, since the sample is linearly separable, for the maximizing pair $(\mathbf{w}, b)$, $y_i\big(\mathbf{w}\cdot\mathbf{x}_i+b\big)$ must be non-negative for all $i\in[m]$. Now, observe that the last expression is invariant to multiplication of $(\mathbf{w}, b)$ by a positive scalar. Thus, we can restrict ourselves to pairs $(\mathbf{w}, b)$ scaled such that $\min_{i\in[m]} y_i(\mathbf{w}\cdot\mathbf{x}_i+b) = 1$:

$$\rho = \max_{\substack{\mathbf{w},b:\\ \min_{i\in[m]} y_i(\mathbf{w}\cdot\mathbf{x}_i+b)=1}} \frac{1}{\|\mathbf{w}\|} = \max_{\substack{\mathbf{w},b:\\ \forall i\in[m], y_i(\mathbf{w}\cdot\mathbf{x}_i+b)\geq 1}} \frac{1}{\|\mathbf{w}\|}. \tag{5.6}$$

The second equality results from the fact that for the maximizing pair $(\mathbf{w}, b)$, the minimum of $y_i(\mathbf{w}\cdot\mathbf{x}_i+b)$ is 1.

Figure 5.3 illustrates the solution $(\mathbf{w}, b)$ of the maximization (5.6). In addition to the maximum-margin hyperplane, it also shows the *marginal hyperplanes*, which are

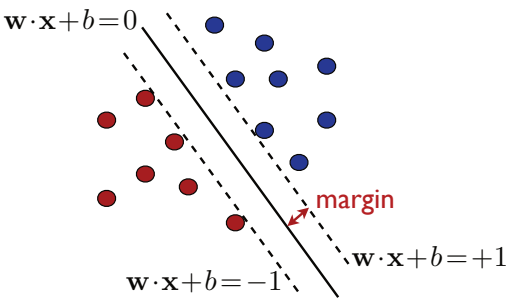


Figure 5.3
Maximum-margin hyperplane solution of (5.6). The marginal hyperplanes are represented by dashed lines on the figure.

the hyperplanes parallel to the separating hyperplane and passing through the closest points on the negative or positive sides. Since they are parallel to the separating hyperplane, they admit the same normal vector $\mathbf{w}$. Furthermore, since $|\mathbf{w}\cdot\mathbf{x}+b| = 1$ for the closest points, the equations of the marginal hyperplanes are $\mathbf{w}\cdot\mathbf{x}+b = \pm 1$.

Since maximizing $1/\|\mathbf{w}\|$ is equivalent to minimizing $\frac{1}{2}\|\mathbf{w}\|^2$, in view of (5.6), the pair $(\mathbf{w}, b)$ returned by SVM in the separable case is the solution of the following convex optimization problem:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \tag{5.7}$$
$$\text{subject to: } y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1, \ \forall i \in [m]\,.$$

The objective function $F\colon \mathbf{w} \mapsto \frac{1}{2}\|\mathbf{w}\|^2$ is infinitely differentiable. Its gradient is $\nabla F(\mathbf{w}) = \mathbf{w}$ and its Hessian is the identity matrix $\nabla^2 F(\mathbf{w}) = \mathbf{I}$, whose eigenvalues are strictly positive. Therefore, $\nabla^2 F(\mathbf{w}) \succ \mathbf{0}$ and F is strictly convex. The constraints are all defined by affine functions $g_i\colon (\mathbf{w}, b) \mapsto 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i + b)$ and are therefore qualified. Thus, in view of the results known for convex optimization (see appendix B for details), the optimization problem of (5.7) admits a unique solution, an important and favorable property that does not hold for all learning algorithms.

Moreover, since the objective function is quadratic and the constraints are affine, the optimization problem of (5.7) is in fact a specific instance of *quadratic programming* (QP), a family of problems extensively studied in optimization. A variety of commercial and open-source solvers are available for solving convex QP problems. Additionally, motivated by the empirical success of SVMs along with its rich theoretical underpinnings, specialized methods have been developed to more efficiently solve this particular convex QP problem, notably the block coordinate descent algorithms with blocks of just two coordinates.

5.2.2 Support vectors

Returning to the optimization problem (5.7), we note that the constraints are affine and thus qualified. The objective function as well as the affine constraints are convex and differentiable. Thus, the requirements of theorem B.30 hold and the KKT conditions apply at the optimum. We shall use these conditions to both analyze the algorithm and demonstrate several of its crucial properties, and subsequently derive the dual optimization problem associated to SVMs in section 5.2.3.

We introduce Lagrange variables $\alpha_i \geq 0$, $i \in [m]$, associated to the m constraints and denote by $\boldsymbol{\alpha}$ the vector $(\alpha_1, \ldots, \alpha_m)^\top$. The Lagrangian can then be defined for all $\mathbf{w} \in \mathbb{R}^N$, $b \in \mathbb{R}$, and $\boldsymbol{\alpha} \in \mathbb{R}^m_+$, by

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{m} \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1]\,. \tag{5.8}$$

The KKT conditions are obtained by setting the gradient of the Lagrangian with respect to the primal variables $\mathbf{w}$ and b to zero and by writing the complementarity conditions:

$$\nabla_{\mathbf{w}} \mathcal{L} = \mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i = 0 \quad \Longrightarrow \quad \mathbf{w} = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i \tag{5.9}$$

$$\nabla_b \mathcal{L} = -\sum_{i=1}^{m} \alpha_i y_i = 0 \quad \Longrightarrow \quad \sum_{i=1}^{m} \alpha_i y_i = 0 \tag{5.10}$$

$$\forall i, \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1] = 0 \quad \Longrightarrow \quad \alpha_i = 0 \vee y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1. \tag{5.11}$$

By equation (5.9), the weight vector $\mathbf{w}$ at the solution of the SVM problem is a linear combination of the training set vectors $\mathbf{x}_1, \ldots, \mathbf{x}_m$. A vector $\mathbf{x}_i$ appears in that expansion iff $\alpha_i \neq 0$. Such vectors are called *support vectors*. By the complementarity conditions (5.11), if $\alpha_i \neq 0$, then $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$. Thus, support vectors lie on the marginal hyperplanes $\mathbf{w} \cdot \mathbf{x}_i + b = \pm 1$.

Support vectors fully define the maximum-margin hyperplane or SVM solution, which justifies the name of the algorithm. By definition, vectors not lying on the marginal hyperplanes do not affect the definition of these hyperplanes — in their absence, the solution to the SVM problem remains unchanged. Note that while the solution $\mathbf{w}$ of the SVM problem is unique, the support vectors are not. In dimension N, $N + 1$ points are sufficient to define a hyperplane. Thus, when more than $N + 1$ points lie on a marginal hyperplane, different choices are possible for the $N + 1$ support vectors.

5.2.3 Dual optimization problem

To derive the dual form of the constrained optimization problem (5.7), we plug into the Lagrangian the definition of $\mathbf{w}$ in terms of the dual variables as expressed in

(5.9) and apply the constraint (5.10). This yields

$$\mathcal{L} = \underbrace{\frac{1}{2}\|\sum_{i=1}^{m}\alpha_i y_i \mathbf{x}_i\|^2 - \sum_{i,j=1}^{m}\alpha_i\alpha_j y_i y_j(\mathbf{x}_i\cdot\mathbf{x}_j)}_{-\frac{1}{2}\sum_{i,j=1}^{m}\alpha_i\alpha_j y_i y_j(\mathbf{x}_i\cdot\mathbf{x}_j)} - \underbrace{\sum_{i=1}^{m}\alpha_i y_i b}_{0} + \sum_{i=1}^{m}\alpha_i\,, \tag{5.12}$$

which simplifies to

$$\mathcal{L} = \sum_{i=1}^{m}\alpha_i - \frac{1}{2}\sum_{i,j=1}^{m}\alpha_i\alpha_j y_i y_j(\mathbf{x}_i\cdot\mathbf{x}_j)\,. \tag{5.13}$$

This leads to the following dual optimization problem for SVMs in the separable case:

$$\begin{aligned}\max_{\boldsymbol{\alpha}}\ & \sum_{i=1}^{m}\alpha_i - \frac{1}{2}\sum_{i,j=1}^{m}\alpha_i\alpha_j y_i y_j(\mathbf{x}_i\cdot\mathbf{x}_j) \\ \text{subject to: } & \alpha_i \geq 0 \wedge \sum_{i=1}^{m}\alpha_i y_i = 0,\ \forall i\in[m]\,.\end{aligned} \tag{5.14}$$

The objective function $G\colon \boldsymbol{\alpha}\mapsto \sum_{i=1}^{m}\alpha_i - \frac{1}{2}\sum_{i,j=1}^{m}\alpha_i\alpha_j y_i y_j(\mathbf{x}_i\cdot\mathbf{x}_j)$ is infinitely differentiable. Its Hessian is given by $\nabla^2 G = -\mathbf{A}$, with $\mathbf{A} = \big(y_i\mathbf{x}_i\cdot y_j\mathbf{x}_j\big)_{ij}$. $\mathbf{A}$ is the Gram matrix associated to the vectors $y_1\mathbf{x}_1,\ldots,y_m\mathbf{x}_m$ and is therefore positive semidefinite (see section A.2.3), which shows that $\nabla^2 G \preceq \mathbf{0}$ and that G is a concave function. Since the constraints are affine and convex, the maximization problem (5.14) is a convex optimization problem. Since G is a quadratic function of $\boldsymbol{\alpha}$, this dual optimization problem is also a QP problem, as in the case of the primal optimization and once again both general-purpose and specialized QP solvers can be used to obtain the solution (see exercise 5.4 for details on the SMO algorithm, which is often used to solve the dual form of the SVM problem in the more general non-separable setting).

Moreover, since the constraints are affine, they are qualified and strong duality holds (see appendix B). Thus, the primal and dual problems are equivalent, i.e., the solution $\boldsymbol{\alpha}$ of the dual problem (5.14) can be used directly to determine the hypothesis returned by SVMs, using equation (5.9):

$$h(\mathbf{x}) = \operatorname{sgn}(\mathbf{w}\cdot\mathbf{x}+b) = \operatorname{sgn}\Big(\sum_{i=1}^{m}\alpha_i y_i(\mathbf{x}_i\cdot\mathbf{x})+b\Big). \tag{5.15}$$

Since support vectors lie on the marginal hyperplanes, for any support vector $\mathbf{x}_i$, $\mathbf{w} \cdot \mathbf{x}_i + b = y_i$, and thus b can be obtained via

$$b = y_i - \sum_{j=1}^{m} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_i) \,. \tag{5.16}$$

The dual optimization problem (5.14) and the expressions (5.15) and (5.16) reveal an important property of SVMs: the hypothesis solution depends only on inner products between vectors and not directly on the vectors themselves. This observation is key and its importance will become clear in Chapter 6 where we introduce kernel methods.

Equation (5.16) can now be used to derive a simple expression of the geometric margin ρ in terms of $\boldsymbol{\alpha}$. Since (5.16) holds for all i with $\alpha_i \neq 0$, multiplying both sides by $\alpha_i y_i$ and taking the sum leads to

$$\sum_{i=1}^{m} \alpha_i y_i b = \sum_{i=1}^{m} \alpha_i y_i^2 - \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \,. \tag{5.17}$$

Using the fact that $y_i^2 = 1$ along with equation (5.9) then yields

$$0 = \sum_{i=1}^{m} \alpha_i - \|\mathbf{w}\|^2. \tag{5.18}$$

Noting that $\alpha_i \geq 0$, we obtain the following expression of the margin ρ in terms of the L_1 norm of $\boldsymbol{\alpha}$:

$$\rho^2 = \frac{1}{\|\mathbf{w}\|_2^2} = \frac{1}{\sum_{i=1}^{m} \alpha_i} = \frac{1}{\|\boldsymbol{\alpha}\|_1}. \tag{5.19}$$

5.2.4 Leave-one-out analysis

We now use the notion of *leave-one-out error* to derive a first learning guarantee for SVMs based on the fraction of support vectors in the training set.

Definition 5.2 (Leave-one-out error) *Let h_S denote the hypothesis returned by a learning algorithm $\mathcal{A}$, when trained on a fixed sample S. Then, the* leave-one-out error *of $\mathcal{A}$ on a sample S of size m is defined by*

$$\widehat{R}_{\mathrm{LOO}}(\mathcal{A}) = \frac{1}{m} \sum_{i=1}^{m} 1_{h_{S-\{x_i\}}(x_i) \neq y_i}.$$

Thus, for each $i \in [m]$, $\mathcal{A}$ is trained on all the points in S except for x_i, i.e., $S - \{x_i\}$, and its error is then computed using x_i. The leave-one-out error is the average of these errors. We will use an important property of the leave-one-out error stated in the following lemma.

Lemma 5.3 *The average leave-one-out error for samples of size $m \geq 2$ is an unbiased estimate of the average generalization error for samples of size $m-1$:*

$$\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[\widehat{R}_{\mathrm{LOO}}(\mathcal{A})] = \mathop{\mathbb{E}}_{S'\sim\mathcal{D}^{m-1}}[R(h_{S'})], \tag{5.20}$$

where $\mathcal{D}$ denotes the distribution according to which points are drawn.

Proof: By the linearity of expectation, we can write

$$\begin{aligned}
\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[\widehat{R}_{\mathrm{LOO}}(\mathcal{A})] &= \frac{1}{m}\sum_{i=1}^{m} \mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[1_{h_{S-\{x_i\}}(x_i)\neq y_i}] \\
&= \mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[1_{h_{S-\{x_1\}}(x_1)\neq y_1}] \\
&= \mathop{\mathbb{E}}_{S'\sim\mathcal{D}^{m-1},x_1\sim\mathcal{D}}[1_{h_{S'}(x_1)\neq y_1}] \\
&= \mathop{\mathbb{E}}_{S'\sim\mathcal{D}^{m-1}}[\mathop{\mathbb{E}}_{x_1\sim\mathcal{D}}[1_{h_{S'}(x_1)\neq y_1}]] \\
&= \mathop{\mathbb{E}}_{S'\sim\mathcal{D}^{m-1}}[R(h_{S'})].
\end{aligned}$$

For the second equality, we used the fact that, since the points of S are drawn in an i.i.d. fashion, the expectation $\mathbb{E}_{S\sim\mathcal{D}^m}[1_{h_{S-\{x_i\}}(x_i)\neq y_i}]$ does not depend on the choice of $i \in [m]$ and is thus equal to $\mathbb{E}_{S\sim\mathcal{D}^m}[1_{h_{S-\{x_1\}}(x_1)\neq y_1}]$. □

In general, computing the leave-one-out error may be costly since it requires training m times on samples of size $m-1$. In some situations however, it is possible to derive the expression of $\widehat{R}_{\mathrm{LOO}}(\mathcal{A})$ much more efficiently (see exercise 11.9).

Theorem 5.4 *Let h_S be the hypothesis returned by SVMs for a sample S, and let $N_{SV}(S)$ be the number of support vectors that define h_S. Then,*

$$\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[R(h_S)] \leq \mathop{\mathbb{E}}_{S\sim\mathcal{D}^{m+1}}\left[\frac{N_{\mathrm{SV}}(S)}{m+1}\right].$$

Proof: Let S be a linearly separable sample of $m+1$. If x is not a support vector for h_S, removing it does not change the SVM solution. Thus, $h_{S-\{x\}} = h_S$ and $h_{S-\{x\}}$ correctly classifies x. By contraposition, if $h_{S-\{x\}}$ misclassifies x, x must be a support vector, which implies

$$\widehat{R}_{\mathrm{LOO}}(\mathrm{SVM}) \leq \frac{N_{\mathrm{SV}}(S)}{m+1}. \tag{5.21}$$

Taking the expectation of both sides and using lemma 5.3 yields the result. □

Theorem 5.4 gives a sparsity argument in favor of SVMs: the average error of the algorithm is upper bounded by the average fraction of support vectors. One may hope that for many distributions seen in practice, a relatively small number of the training points will lie on the marginal hyperplanes. The solution will then be sparse in the sense that a small fraction of the dual variables α_i will be non-

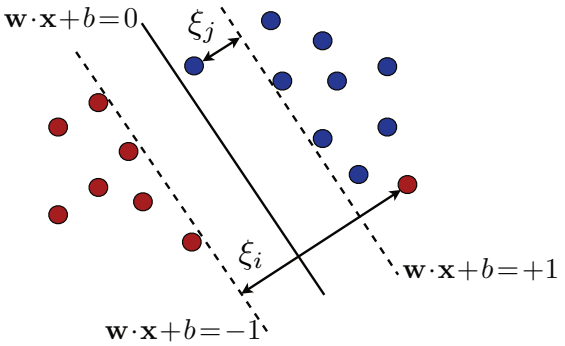


Figure 5.4
A separating hyperplane with point $\mathbf{x}_i$ classified incorrectly and point $\mathbf{x}_j$ correctly classified, but with margin less than 1.

zero. Note, however, that this bound is relatively weak since it applies only to the average generalization error of the algorithm over all samples of size m. It provides no information about the variance of the generalization error. In section 5.4, we present stronger high-probability bounds using a different argument based on the notion of margin.

5.3 Non-separable case

In most practical settings, the training data is not linearly separable, which implies that for any hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$, there exists $\mathbf{x}_i \in S$ such that

$$y_i \left[\mathbf{w} \cdot \mathbf{x}_i + b\right] \ngeq 1 \,. \tag{5.22}$$

Thus, the constraints imposed in the linearly separable case discussed in section 5.2 cannot all hold simultaneously. However, a relaxed version of these constraints can indeed hold, that is, for each $i \in [m]$, there exist $\xi_i \geq 0$ such that

$$y_i \left[\mathbf{w} \cdot \mathbf{x}_i + b\right] \geq 1 - \xi_i \,. \tag{5.23}$$

The variables ξ_i are known as *slack variables* and are commonly used in optimization to define relaxed versions of constraints. Here, a slack variable ξ_i measures the distance by which vector $\mathbf{x}_i$ violates the desired inequality, $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$. Figure 5.4 illustrates the situation. For a hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$, a vector $\mathbf{x}_i$ with $\xi_i > 0$ can be viewed as an *outlier*. Each $\mathbf{x}_i$ must be positioned on the correct side of the appropriate marginal hyperplane to not be considered an outlier. As a consequence, a vector $\mathbf{x}_i$ with $0 < y_i(\mathbf{w} \cdot \mathbf{x}_i + b) < 1$ is correctly classified by the hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$ but is nonetheless considered to be an outlier, that

is, $\xi_i > 0$. If we omit the outliers, the training data is correctly separated by $\mathbf{w} \cdot \mathbf{x} + b = 0$ with a margin $\rho = 1/\|\mathbf{w}\|$ that we refer to as the *soft margin*, as opposed to the *hard margin* in the separable case.

How should we select the hyperplane in the non-separable case? One idea consists of selecting the hyperplane that minimizes the empirical error. But, that solution will not benefit from the large-margin guarantees we will present in section 5.4. Furthermore, the problem of determining a hyperplane with the smallest zero-one loss, that is the smallest number of misclassifications, is NP-hard as a function of the dimension N of the space.

Here, there are two conflicting objectives: on one hand, we wish to limit the total amount of slack due to outliers, which can be measured by $\sum_{i=1}^m \xi_i$, or, more generally by $\sum_{i=1}^m \xi_i^p$ for some $p \geq 1$; on the other hand, we seek a hyperplane with a large margin, though a larger margin can lead to more outliers and thus larger amounts of slack.

5.3.1 Primal optimization problem

This leads to the following general optimization problem defining SVMs in the non-separable case where the parameter $C \geq 0$ determines the trade-off between margin-maximization (or minimization of $\|\mathbf{w}\|^2$) and the minimization of the slack penalty $\sum_{i=1}^m \xi_i^p$:

$$\begin{aligned} \min_{\mathbf{w},b,\boldsymbol{\xi}} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^m \xi_i^p \\ \text{subject to} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \ \wedge \ \xi_i \geq 0, i \in [m]\,, \end{aligned} \tag{5.24}$$

where $\boldsymbol{\xi} = (\xi_1, \ldots, \xi_m)^\top$. The parameter C is typically determined via n-fold cross-validation (see section 4.5).

As in the separable case, (5.24) is a convex optimization problem since the constraints are affine and thus convex and since the objective function is convex for any $p \geq 1$. In particular, $\boldsymbol{\xi} \mapsto \sum_{i=1}^m \xi_i^p = \|\boldsymbol{\xi}\|_p^p$ is convex in view of the convexity of the norm $\|\cdot\|_p$.

There are many possible choices for p leading to more or less aggressive penalizations of the slack terms (see exercise 5.1). The choices $p = 1$ and $p = 2$ lead to the most straightforward solutions and analyses. The loss functions associated with $p = 1$ and $p = 2$ are called the *hinge loss* and the *quadratic hinge loss*, respectively. Figure 5.5 shows the plots of these loss functions as well as that of the standard zero-one loss function. Both hinge losses are convex upper bounds on the zero-one loss, thus making them well suited for optimization. In what follows, the analysis is presented in the case of the hinge loss ($p = 1$), which is the most widely used loss function for SVMs.

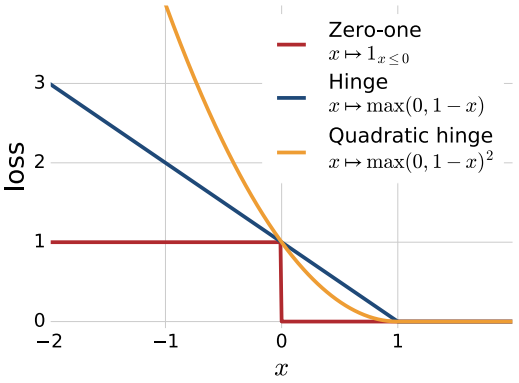


Figure 5.5
Both the hinge loss and the quadratic hinge loss provide convex upper bounds on the binary zero-one loss.

5.3.2 Support vectors

As in the separable case, the constraints are affine and thus qualified. The objective function as well as the affine constraints are convex and differentiable. Thus, the hypotheses of theorem B.30 hold and the KKT conditions apply at the optimum. We use these conditions to both analyze the algorithm and demonstrate several of its crucial properties, and subsequently derive the dual optimization problem associated to SVMs in section 5.3.3.

We introduce Lagrange variables $\alpha_i \geq 0$, $i \in [m]$, associated to the first m constraints and $\beta_i \geq 0$, $i \in [m]$ associated to the non-negativity constraints of the slack variables. We denote by $\boldsymbol{\alpha}$ the vector $(\alpha_1, \ldots, \alpha_m)^\top$ and by $\boldsymbol{\beta}$ the vector $(\beta_1, \ldots, \beta_m)^\top$. The Lagrangian can then be defined for all $\mathbf{w} \in \mathbb{R}^N$, $b \in \mathbb{R}$, and $\boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta} \in \mathbb{R}^m_+$, by

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^m \xi_i - \sum_{i=1}^m \alpha_i[y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1 + \xi_i] - \sum_{i=1}^m \beta_i \xi_i\,. \quad (5.25)$$

The KKT conditions are obtained by setting the gradient of the Lagrangian with respect to the primal variables $\mathbf{w}$, b, and ξ_is to zero and by writing the complemen-

tarity conditions:

$$\nabla_{\mathbf{w}}\mathcal{L} = \mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i = 0 \qquad \Longrightarrow \qquad \mathbf{w} = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i \tag{5.26}$$

$$\nabla_b \mathcal{L} = -\sum_{i=1}^{m} \alpha_i y_i = 0 \qquad \Longrightarrow \qquad \sum_{i=1}^{m} \alpha_i y_i = 0 \tag{5.27}$$

$$\nabla_{\xi_i}\mathcal{L} = C - \alpha_i - \beta_i = 0 \qquad \Longrightarrow \qquad \alpha_i + \beta_i = C \tag{5.28}$$

$$\forall i, \alpha_i[y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1 + \xi_i] = 0 \qquad \Longrightarrow \qquad \alpha_i = 0 \vee y_i(\mathbf{w}\cdot\mathbf{x}_i + b) = 1 - \xi_i \tag{5.29}$$

$$\forall i, \beta_i \xi_i = 0 \qquad \Longrightarrow \qquad \beta_i = 0 \vee \xi_i = 0\,. \tag{5.30}$$

By equation (5.26), as in the separable case, the weight vector $\mathbf{w}$ at the solution of the SVM problem is a linear combination of the training set vectors $\mathbf{x}_1, \ldots, \mathbf{x}_m$. A vector $\mathbf{x}_i$ appears in that expansion iff $\alpha_i \neq 0$. Such vectors are called *support vectors*. Here, there are two types of support vectors. By the complementarity condition (5.29), if $\alpha_i \neq 0$, then $y_i(\mathbf{w}\cdot\mathbf{x}_i+b) = 1-\xi_i$. If $\xi_i = 0$, then $y_i(\mathbf{w}\cdot\mathbf{x}_i+b) = 1$ and $\mathbf{x}_i$ lies on a marginal hyperplane, as in the separable case. Otherwise, $\xi_i \neq 0$ and $\mathbf{x}_i$ is an outlier. In this case, (5.30) implies $\beta_i = 0$ and (5.28) then requires $\alpha_i = C$. Thus, support vectors $\mathbf{x}_i$ are either outliers, in which case $\alpha_i = C$, or vectors lying on the marginal hyperplanes. As in the separable case, note that while the weight vector $\mathbf{w}$ solution is unique, the support vectors are not.

5.3.3 Dual optimization problem

To derive the dual form of the constrained optimization problem (5.24), we plug into the Lagrangian the definition of $\mathbf{w}$ in terms of the dual variables (5.26) and apply the constraint (5.27). This yields

$$\mathcal{L} = \underbrace{\frac{1}{2}\|\sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i\|^2 - \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)}_{-\frac{1}{2}\sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)} - \underbrace{\sum_{i=1}^{m} \alpha_i y_i b}_{0} + \sum_{i=1}^{m} \alpha_i\,. \tag{5.31}$$

Remarkably, we find that the objective function is no different than in the separable case:

$$\mathcal{L} = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)\,. \tag{5.32}$$

However, here, in addition to $\alpha_i \geq 0$, we must impose the constraint on the Lagrange variables $\beta_i \geq 0$. In view of (5.28), this is equivalent to $\alpha_i \leq C$. This leads to the following dual optimization problem for SVMs in the non-separable case, which

only differs from that of the separable case (5.14) by the constraints $\alpha_i \leq C$:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \tag{5.33}$$
$$\text{subject to: } 0 \leq \alpha_i \leq C \wedge \sum_{i=1}^{m} \alpha_i y_i = 0, i \in [m].$$

Thus, our previous comments about the optimization problem (5.14) apply to (5.33) as well. In particular, the objective function is concave and infinitely differentiable and (5.33) is equivalent to a convex QP. The problem is equivalent to the primal problem (5.24).

The solution $\boldsymbol{\alpha}$ of the dual problem (5.33) can be used directly to determine the hypothesis returned by SVMs, using equation (5.26):

$$h(\mathbf{x}) = \operatorname{sgn}(\mathbf{w} \cdot \mathbf{x} + b) = \operatorname{sgn}\Big(\sum_{i=1}^{m} \alpha_i y_i (\mathbf{x}_i \cdot \mathbf{x}) + b\Big). \tag{5.34}$$

Moreover, b can be obtained from any support vector $\mathbf{x}_i$ lying on a marginal hyperplane, that is any vector $\mathbf{x}_i$ with $0 < \alpha_i < C$. For such support vectors, $\mathbf{w} \cdot \mathbf{x}_i + b = y_i$ and thus

$$b = y_i - \sum_{j=1}^{m} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_i)\,. \tag{5.35}$$

As in the separable case, the dual optimization problem (5.33) and the expressions (5.34) and (5.35) show an important property of SVMs: the hypothesis solution depends only on inner products between vectors and not directly on the vectors themselves. This fact can be used to extend SVMs to define non-linear decision boundaries, as we shall see in chapter 6.

5.4 Margin theory

This section presents generalization bounds which provide a strong theoretical justification for the SVM algorithm.

Recall that the VC-dimension of the family of hyperplanes or linear hypotheses in $\mathbb{R}^N$ is $N + 1$. Thus, the application of the VC-dimension bound (3.29) of corollary 3.19 to this hypothesis set yields the following: for any $\delta > 0$, with probability at least $1 - \delta$, for any $h \in \mathcal{H}$,

$$R(h) \leq \widehat{R}_S(h) + \sqrt{\frac{2(N+1)\log \frac{em}{N+1}}{m}} + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}. \tag{5.36}$$

When the dimension of the feature space N is large compared to the sample size m, this bound is uninformative. Remarkably, the learning guarantees presented in this section are independent of the dimension N and thus hold regardless of its value.

The guarantees we will present hold for real-valued functions such as the function $\mathbf{x} \mapsto \mathbf{w} \cdot \mathbf{x} + b$ returned by SVMs, as opposed to classification functions returning $+1$ or -1, such as $\mathbf{x} \mapsto \operatorname{sgn}(\mathbf{w} \cdot \mathbf{x} + b)$. They are based on the notion of *confidence margin*. The confidence margin of a real-valued function h at a point x labeled with y is the quantity $yh(x)$. Thus, when $yh(x) > 0$, h classifies x correctly but we interpret the magnitude of $|h(x)|$ as the *confidence* of the prediction made by h. The notion of confidence margin is distinct from that of geometric margin and does not require a linear separability assumption. But, the two notions are related as follows in the separable case: for $h\colon \mathbf{x} \mapsto \mathbf{w} \cdot \mathbf{x} + b$ with geometric margin ρ_{geom}, the confidence margin at any point $\mathbf{x}$ of the training sample with label y is at least $\rho_{\text{geom}}\|\mathbf{w}\|$, i.e. $|yh(\mathbf{x})| \geq \rho_{\text{geom}}\|\mathbf{w}\|$.

In view of the definition of the confidence margin, for any parameter $\rho > 0$, we will define a *ρ-margin loss function* that, as with the zero-one loss, penalizes h with the cost of 1 when it misclassifies point x ($yh(x) \leq 0$), but also penalizes h (linearly) when it correctly classifies x with confidence less than or equal to ρ ($yh(x) \leq \rho$). The main margin-based generalization bounds of this section are presented in terms of this loss function, which is formally defined as follows.

Definition 5.5 (Margin loss function) *For any $\rho > 0$, the ρ-margin loss is the function $L_\rho\colon \mathbb{R} \times \mathbb{R} \to \mathbb{R}_+$ defined for all $y, y' \in \mathbb{R}$ by $L_\rho(y, y') = \Phi_\rho(yy')$ with,*

$$\Phi_\rho(x) = \min\left(1, \max\left(0, 1 - \frac{x}{\rho}\right)\right) = \begin{cases} 1 & \text{if } x \leq 0 \\ 1 - \frac{x}{\rho} & \text{if } 0 \leq x \leq \rho \\ 0 & \text{if } \rho \leq x. \end{cases}$$

This loss function is illustrated by figure 5.6. The parameter $\rho > 0$ can be interpreted as the confidence margin demanded from a hypothesis h. The empirical margin loss is similarly defined as the margin loss over the training sample.

Definition 5.6 (Empirical margin loss) *Given a sample $S = (x_1, \ldots, x_m)$ and a hypothesis h, the empirical margin loss is defined by*

$$\widehat{R}_{S,\rho}(h) = \frac{1}{m} \sum_{i=1}^{m} \Phi_\rho(y_i h(x_i)) \,. \tag{5.37}$$

Note that, for any $i \in [m]$, $\Phi_\rho(y_i h(x_i)) \leq 1_{y_i h(x_i) \leq \rho}$. Thus, the empirical margin loss can be upper-bounded as follows:

$$\widehat{R}_{S,\rho}(h) \leq \frac{1}{m} \sum_{i=1}^{m} 1_{y_i h(x_i) \leq \rho} \,. \tag{5.38}$$

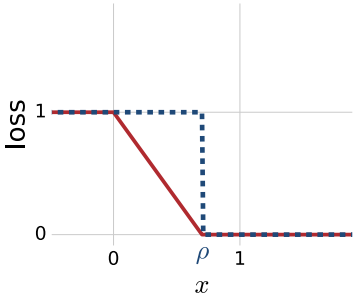


Figure 5.6
The margin loss illustrated in red, defined with respect to margin parameter $\rho = 0.7$.

In all the results that follow, the empirical margin loss can be replaced by this upper bound, which admits a simple interpretation: it is the fraction of the points in the training sample S that have been misclassified or classified with confidence less than ρ. In other words, the upper bound is then the fraction of the points in the training data with margin less than ρ. This corresponds to the loss function indicated by the blue dotted line in figure 5.6.

A key benefit of using a loss function based on Φ_ρ as opposed to the zero-one loss or the loss defined by the blue dotted line of figure 5.6 is that Φ_ρ is $1/\rho$-Lipschitz, since the absolute value of the slope of the function is at most $1/\rho$. The following lemma bounds the empirical Rademacher complexity of a hypothesis set $\mathcal{H}$ after composition with such a Lipschitz function in terms of the empirical Rademacher complexity of $\mathcal{H}$. It will be needed for the proof of the margin-based generalization bound.

Lemma 5.7 (Talagrand's lemma) *Let $\Phi_1, \ldots, \Phi_m$ be l-Lipschitz functions from $\mathbb{R}$ to $\mathbb{R}$ and $\sigma_1, \ldots, \sigma_m$ be Rademacher random variables. Then, for any hypothesis set $\mathcal{H}$ of real-valued functions, the following inequality holds:*

$$\frac{1}{m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i(\Phi_i\circ h)(x_i))\right] \leq \frac{l}{m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i)\right] = l\,\widehat{\mathfrak{R}}_S(\mathcal{H})\,.$$

In particular, if $\Phi_i = \Phi$ for all $i \in [m]$, then the following holds:

$$\widehat{\mathfrak{R}}_S(\Phi\circ\mathcal{H}) \leq l\,\widehat{\mathfrak{R}}_S(\mathcal{H})\,.$$

Proof: First we fix a sample $S = (x_1, \ldots, x_m)$, then, by definition,

$$\frac{1}{m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i(\Phi_m\circ h)(x_i)\right] = \frac{1}{m}\mathop{\mathbb{E}}_{\sigma_1,\ldots,\sigma_{m-1}}\left[\mathop{\mathbb{E}}_{\sigma_m}\left[\sup_{h\in\mathcal{H}}u_{m-1}(h)+\sigma_m(\Phi_m\circ h)(x_m)\right]\right],$$

where $u_{m-1}(h) = \sum_{i=1}^{m-1} \sigma_i(\Phi_i \circ h)(x_i)$. By definition of the supremum, for any $\epsilon > 0$, there exist $h_1, h_2 \in \mathcal{H}$ such that

$$u_{m-1}(h_1) + (\Phi_m \circ h_1)(x_m) \geq (1-\epsilon)\Big[\sup_{h\in\mathcal{H}} u_{m-1}(h) + (\Phi_m \circ h)(x_m)\Big]$$
$$\text{and} \quad u_{m-1}(h_2) - (\Phi_m \circ h_2)(x_m) \geq (1-\epsilon)\Big[\sup_{h\in\mathcal{H}} u_{m-1}(h) - (\Phi_m \circ h)(x_m)\Big].$$

Thus, for any $\epsilon > 0$, by definition of $\mathbb{E}_{\sigma_m}$,

$$\begin{aligned}
&(1-\epsilon)\mathop{\mathbb{E}}_{\sigma_m}\Big[\sup_{h\in\mathcal{H}} u_{m-1}(h) + \sigma_m(\Phi_m \circ h)(x_m)\Big] \\
&= (1-\epsilon)\left[\frac{1}{2}\sup_{h\in\mathcal{H}}\Big[u_{m-1}(h) + (\Phi_m \circ h)(x_m)\Big] + \frac{1}{2}\Big[\sup_{h\in\mathcal{H}} u_{m-1}(h) - (\Phi_m \circ h)(x_m)\Big]\right] \\
&\leq \frac{1}{2}[u_{m-1}(h_1) + (\Phi_m \circ h_1)(x_m)] + \frac{1}{2}[u_{m-1}(h_2) - (\Phi_m \circ h_2)(x_m)].
\end{aligned}$$

Let $s = \operatorname{sgn}(h_1(x_m) - h_2(x_m))$. Then, the previous inequality implies

$$\begin{aligned}
&(1-\epsilon)\mathop{\mathbb{E}}_{\sigma_m}\Big[\sup_{h\in\mathcal{H}} u_{m-1}(h) + \sigma_m(\Phi_m \circ h)(x_m)\Big] \\
&\leq \frac{1}{2}[u_{m-1}(h_1) + u_{m-1}(h_2) + sl(h_1(x_m) - h_2(x_m))] && \text{(Lipschitz property)} \\
&= \frac{1}{2}[u_{m-1}(h_1) + slh_1(x_m)] + \frac{1}{2}[u_{m-1}(h_2) - slh_2(x_m)] && \text{(rearranging)} \\
&\leq \frac{1}{2}\sup_{h\in\mathcal{H}}[u_{m-1}(h) + slh(x_m)] + \frac{1}{2}\sup_{h\in\mathcal{H}}[u_{m-1}(h) - slh(x_m)] && \text{(definition of sup)} \\
&= \mathop{\mathbb{E}}_{\sigma_m}\Big[\sup_{h\in\mathcal{H}} u_{m-1}(h) + \sigma_m lh(x_m)\Big]. && \text{(definition of } \mathop{\mathbb{E}}_{\sigma_m}\text{)}
\end{aligned}$$

Since the inequality holds for all $\epsilon > 0$, we have

$$\mathop{\mathbb{E}}_{\sigma_m}\Big[\sup_{h\in\mathcal{H}} u_{m-1}(h) + \sigma_m(\Phi_m \circ h)(x_m)\Big] \leq \mathop{\mathbb{E}}_{\sigma_m}\Big[\sup_{h\in\mathcal{H}} u_{m-1}(h) + \sigma_m lh(x_m)\Big].$$

Proceeding in the same way for all other σ_i $(i \neq m)$ proves the lemma. □

The following is a general margin-based generalization bound that will be used in the analysis of several algorithms.

Theorem 5.8 (Margin bound for binary classification) *Let $\mathcal{H}$ be a set of real-valued functions. Fix $\rho > 0$, then, for any $\delta > 0$, with probability at least $1-\delta$, each of the following holds for all $h \in \mathcal{H}$:*

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{2}{\rho}\mathfrak{R}_m(\mathcal{H}) + \sqrt{\frac{\log\frac{1}{\delta}}{2m}} \tag{5.39}$$

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{2}{\rho}\widehat{\mathfrak{R}}_S(\mathcal{H}) + 3\sqrt{\frac{\log\frac{2}{\delta}}{2m}}\,. \tag{5.40}$$

Proof: Let $\widetilde{\mathcal{H}} = \{z = (x, y) \mapsto yh(x) \colon h \in \mathcal{H}\}$. Consider the family of functions taking values in $[0, 1]$:

$$\widetilde{\mathcal{H}} = \{\Phi_\rho \circ f \colon f \in \widetilde{\mathcal{H}}\} .$$

By theorem 3.3, with probability at least $1 - \delta$, for all $g \in \widetilde{\mathcal{H}}$,

$$\mathbb{E}[g(z)] \leq \frac{1}{m} \sum_{i=1}^{m} g(z_i) + 2\mathfrak{R}_m(\widetilde{\mathcal{H}}) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}} ,$$

and thus, for all $h \in \mathcal{H}$,

$$\mathbb{E}[\Phi_\rho(yh(x))] \leq \widehat{R}_{S,\rho}(h) + 2\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{\mathcal{H}}\big) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}} .$$

Since $1_{u \leq 0} \leq \Phi_\rho(u)$ for all $u \in \mathbb{R}$, we have $R(h) = \mathbb{E}[1_{yh(x) \leq 0}] \leq \mathbb{E}[\Phi_\rho(yh(x))]$, thus

$$R(h) \leq \widehat{R}_{S,\rho}(h) + 2\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{\mathcal{H}}\big) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}} .$$

Since Φ_ρ is $1/\rho$-Lipschitz, by lemma 5.7, we have $\mathfrak{R}_m\big(\Phi_\rho \circ \widetilde{\mathcal{H}}\big) \leq \frac{1}{\rho}\mathfrak{R}_m(\widetilde{\mathcal{H}})$ and $\mathfrak{R}_m(\widetilde{\mathcal{H}})$ can be rewritten as follows:

$$\mathfrak{R}_m(\widetilde{\mathcal{H}}) = \frac{1}{m} \mathop{\mathbb{E}}_{S,\sigma} \Big[\sup_{h \in \mathcal{H}} \sum_{i=1}^{m} \sigma_i y_i h(x_i) \Big] = \frac{1}{m} \mathop{\mathbb{E}}_{S,\sigma} \Big[\sup_{h \in \mathcal{H}} \sum_{i=1}^{m} \sigma_i h(x_i) \Big] = \mathfrak{R}_m\big(\mathcal{H}\big) .$$

This proves (5.39). The second inequality, (5.40), can be derived in the same way by using the second inequality of theorem 3.3, (3.4), instead of (3.3). □

The generalization bounds of theorem 5.8 suggest a trade-off: a larger value of ρ decreases the complexity term (second term), but tends to increase the empirical margin-loss $\widehat{R}_{S,\rho}(h)$ (first term) by requiring from a hypothesis h a higher confidence margin. Thus, if for a relatively large value of ρ the empirical margin loss of h remains relatively small, then h benefits from a very favorable guarantee on its generalization error. For theorem 5.8, the margin parameter ρ must be selected beforehand. But, the bounds of the theorem can be generalized to hold uniformly for all $\rho \in (0, 1]$ at the cost of a modest additional term $\sqrt{\frac{\log \log_2 \frac{2}{\rho}}{m}}$, as shown in the following theorem (a version of this theorem with better constants can be derived, see exercise 5.2).

Theorem 5.9 *Let $\mathcal{H}$ be a set of real-valued functions. Fix $r > 0$. Then, for any $\delta > 0$, with probability at least $1 - \delta$, each of the following holds for all $h \in \mathcal{H}$ and $\rho \in (0, r]$:*

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{4}{\rho}\mathfrak{R}_m(\mathcal{H}) + \sqrt{\frac{\log \log_2 \frac{2r}{\rho}}{m}} + \sqrt{\frac{\log \frac{2}{\delta}}{2m}} \tag{5.41}$$

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{4}{\rho}\widehat{\mathfrak{R}}_S(\mathcal{H}) + \sqrt{\frac{\log \log_2 \frac{2r}{\rho}}{m}} + 3\sqrt{\frac{\log \frac{4}{\delta}}{2m}}\,. \tag{5.42}$$

Proof: Consider two sequences $(\rho_k)_{k\geq 1}$ and $(\epsilon_k)_{k\geq 1}$, with $\epsilon_k \in (0, 1]$. By theorem 5.8, for any fixed $k \geq 1$,

$$\mathbb{P}\left[\sup_{h\in H} R(h) - \widehat{R}_{S,\rho_k}(h) > \frac{2}{\rho_k}\mathfrak{R}_m(\mathcal{H}) + \epsilon_k\right] \leq \exp(-2m\epsilon_k^2). \tag{5.43}$$

Choose $\epsilon_k = \epsilon + \sqrt{\frac{\log k}{m}}$, then, by the union bound, the following holds:

$$\begin{aligned}
&\mathbb{P}\left[\sup_{\substack{h\in H\\ k\geq 1}} R(h) - \widehat{R}_{S,\rho_k}(h) - \frac{2}{\rho_k}\mathfrak{R}_m(\mathcal{H}) - \epsilon_k > 0\right] \\
&\qquad \leq \sum_{k\geq 1} \exp(-2m\epsilon_k^2) \\
&\qquad = \sum_{k\geq 1} \exp\big[-2m(\epsilon + \sqrt{(\log k)/m})^2\big] \\
&\qquad \leq \sum_{k\geq 1} \exp(-2m\epsilon^2)\exp(-2\log k) \\
&\qquad = \Big(\sum_{k\geq 1} 1/k^2\Big)\exp(-2m\epsilon^2) \\
&\qquad = \frac{\pi^2}{6}\exp(-2m\epsilon^2) \leq 2\exp(-2m\epsilon^2).
\end{aligned}$$

We can choose $\rho_k = r/2^k$. For any $\rho \in (0, r]$, there exists $k \geq 1$ such that $\rho \in (\rho_k, \rho_{k-1}]$, with $\rho_0 = r$. For that k, $\rho \leq \rho_{k-1} = 2\rho_k$, thus $1/\rho_k \leq 2/\rho$ and $\sqrt{\log k} = \sqrt{\log \log_2(r/\rho_k)} \leq \sqrt{\log \log_2(2r/\rho)}$. Furthermore, for any $h \in \mathcal{H}$, $\widehat{R}_{S,\rho_k}(h) \leq \widehat{R}_{S,\rho}(h)$. Thus, the following inequality holds:

$$\mathbb{P}\left[\sup_{\substack{h\in H\\ \rho\in(0,r]}} R(h) - \widehat{R}_{S,\rho}(h) - \frac{4}{\rho}\mathfrak{R}_m(\mathcal{H}) - \sqrt{\frac{\log \log_2(2r/\rho)}{m}} - \epsilon > 0\right] \leq 2\exp(-2m\epsilon^2),$$

which proves the first statement. The second statement can be proven in a similar way. □

The Rademacher complexity of linear hypotheses with bounded weight vector can be bounded as follows.

Theorem 5.10 *Let $S \subseteq \{\mathbf{x}\colon \|\mathbf{x}\| \leq r\}$ be a sample of size m and let $\mathcal{H} = \{\mathbf{x} \mapsto \mathbf{w}\cdot\mathbf{x}\colon \|\mathbf{w}\| \leq \Lambda\}$. Then, the empirical Rademacher complexity of $\mathcal{H}$ can be bounded as follows:*

$$\widehat{\mathfrak{R}}_S(\mathcal{H}) \leq \sqrt{\frac{r^2\Lambda^2}{m}}\,.$$

Proof: The proof follows through a series of inequalities:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\mathcal{H}) &= \frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{\|\mathbf{w}\|\leq\Lambda}\sum_{i=1}^{m}\sigma_i\mathbf{w}\cdot\mathbf{x}_i\right] = \frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{\|\mathbf{w}\|\leq\Lambda}\mathbf{w}\cdot\sum_{i=1}^{m}\sigma_i\mathbf{x}_i\right]\\
&\leq \frac{\Lambda}{m}\mathop{\mathbb{E}}_{\sigma}\left[\left\|\sum_{i=1}^{m}\sigma_i\mathbf{x}_i\right\|\right] \leq \frac{\Lambda}{m}\left[\mathop{\mathbb{E}}_{\sigma}\left[\left\|\sum_{i=1}^{m}\sigma_i\mathbf{x}_i\right\|^2\right]\right]^{\frac{1}{2}}\\
&= \frac{\Lambda}{m}\left[\mathop{\mathbb{E}}_{\sigma}\left[\sum_{i,j=1}^{m}\sigma_i\sigma_j(\mathbf{x}_i\cdot\mathbf{x}_j)\right]\right]^{\frac{1}{2}} \leq \frac{\Lambda}{m}\left[\sum_{i=1}^{m}\|\mathbf{x}_i\|^2\right]^{\frac{1}{2}} \leq \frac{\Lambda\sqrt{mr^2}}{m} = \sqrt{\frac{r^2\Lambda^2}{m}}\,,
\end{aligned}$$

The first inequality makes use of the Cauchy-Schwarz inequality and the bound on $\|\mathbf{w}\|$, the second follows by Jensen's inequality, the third by $\mathbb{E}[\sigma_i\sigma_j] = \mathbb{E}[\sigma_i]\,\mathbb{E}[\sigma_j] = 0$ for $i \neq j$, and the last one by $\|\mathbf{x}_i\| \leq r$. □

Combining theorem 5.10 and theorem 5.8 gives directly the following general margin bound for linear hypotheses with bounded weight vectors, presented in corollary 5.11.

Corollary 5.11 *Let $\mathcal{H} = \{\mathbf{x} \mapsto \mathbf{w}\cdot\mathbf{x}\colon \|\mathbf{w}\| \leq \Lambda\}$ and assume that $\mathcal{X} \subseteq \{\mathbf{x}\colon \|\mathbf{x}\| \leq r\}$. Fix $\rho > 0$, then, for any $\delta > 0$, with probability at least $1-\delta$ over the choice of a sample S of size m, the following holds for any $h \in \mathcal{H}$:*

$$R(h) \leq \widehat{R}_{S,\rho}(h) + 2\sqrt{\frac{r^2\Lambda^2/\rho^2}{m}} + \sqrt{\frac{\log\frac{1}{\delta}}{2m}}\,. \tag{5.44}$$

As with theorem 5.8, the bound of this corollary can be generalized to hold uniformly for all $\rho \in (0,1]$ at the cost of an additional term $\sqrt{\frac{\log\log_2\frac{2}{\rho}}{m}}$ by combining theorems 5.10 and 5.9.

This generalization bound for linear hypotheses is remarkable, since it does not depend directly on the dimension of the feature space, but only on the margin. It suggests that a small generalization error can be achieved when $\rho/(r\Lambda)$ is large (small second term) while the empirical margin loss is relatively small (first term). The latter occurs when few points are either classified incorrectly or correctly, but with margin less than ρ. When the training sample is linearly separable, for a linear hypothesis with geometric margin ρ_{geom} and the choice of the confidence margin

parameter $\rho = \rho_{\text{geom}}$, the empirical margin loss term is zero. Thus, if ρ_{geom} is relatively large, this provides a strong guarantee for the generalization error of the corresponding linear hypothesis.

The fact that the guarantee does not explicitly depend on the dimension of the feature space may seem surprising and appears to contradict the VC-dimension lower bounds of theorems 3.20 and 3.23. Those lower bounds show that for any learning algorithm $\mathcal{A}$ there exists a *bad* distribution for which the error of the hypothesis returned by the algorithm is $\Omega(\sqrt{d/m})$ with a non-zero probability. The bound of the corollary does not rule out such *bad* cases, however: for such bad distributions, the empirical margin loss would be large even for a relatively small margin ρ, and thus the bound of the corollary would be loose in that case.

Thus, in some sense, the learning guarantee of the corollary hinges upon the hope of a good margin value ρ: if there exists a relatively large margin value $\rho > 0$ for which the empirical margin loss is small, then a small generalization error is guaranteed by the corollary. This favorable margin situation depends on the distribution: while the learning bound is distribution-independent, the existence of a good margin is in fact distribution-dependent. A favorable margin seems to appear relatively often in applications.

The bound of the corollary gives a strong justification for margin-maximization algorithms such as SVMs. Choosing $\Lambda = 1$, by the generalization of corollary 5.11 to a uniform bound over $\rho \in (0, r]$, for any $\delta > 0$, with probability at least $1 - \delta$, the following holds for all $h \in \{\mathbf{x} \mapsto \mathbf{w} \cdot \mathbf{x} \colon \|\mathbf{w}\| \leq 1\}$ and $\rho \in (0, r]$:

$$R(h) \leq \widehat{R}_{S,\rho}(h) + 4\sqrt{\frac{r^2/\rho^2}{m}} + \sqrt{\frac{\log \log_2 \frac{2r}{\rho}}{m}} + \sqrt{\frac{\log \frac{2}{\delta}}{2m}}\,.$$

The inequality also trivially holds for ρ larger than r since in that case, by the Cauchy-Schwarz inequality, for any $\mathbf{w}$ with $\|\mathbf{w}\| \leq 1$, we have $y_i(\mathbf{w} \cdot \mathbf{x}_i) \leq r \leq \rho$ and $\widehat{R}_{S,\rho}(h)$ is equal to one for all h.

Now, for any $\rho > 0$, the ρ-margin loss function is upper bounded by the ρ-hinge loss:

$$\forall u \in \mathbb{R}, \Phi_\rho(u) = \min\left(1, \max\left(0, 1 - \frac{u}{\rho}\right)\right) \leq \max\left(0, 1 - \frac{u}{\rho}\right). \tag{5.45}$$

Thus, with probability at least $1 - \delta$, the following holds for all $h \in \{\mathbf{x} \mapsto \mathbf{w} \cdot \mathbf{x} \colon \|\mathbf{w}\| \leq 1\}$ and all $\rho > 0$:

$$R(h) \leq \frac{1}{m}\sum_{i=1}^{m} \max\left(0, 1 - \frac{y_i(\mathbf{w} \cdot \mathbf{x}_i)}{\rho}\right) + 4\sqrt{\frac{r^2/\rho^2}{m}} + \sqrt{\frac{\log \log_2 \frac{2r}{\rho}}{m}} + \sqrt{\frac{\log \frac{2}{\delta}}{2m}}\,.$$

Since for any $\rho > 0$, h/ρ admits the same generalization error as h, with probability at least $1-\delta$, the following holds for all $h \in \left\{\mathbf{x} \mapsto \mathbf{w}\cdot\mathbf{x}\colon \|\mathbf{w}\| \leq 1/\rho\right\}$ and all $\rho > 0$:

$$R(h) \leq \frac{1}{m}\sum_{i=1}^{m} \max\left(0, 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i)\right) + 4\sqrt{\frac{r^2/\rho^2}{m}} + \sqrt{\frac{\log\log_2 \frac{2r}{\rho}}{m}} + \sqrt{\frac{\log\frac{2}{\delta}}{2m}}\,. \quad (5.46)$$

This inequality can be used to derive an algorithm that selects $\mathbf{w}$ and $\rho > 0$ to minimize the right-hand side. The minimization with respect to ρ does not lead to a convex optimization and depends on theoretical constant factors affecting the second and third terms, which may not be optimal. Thus, instead, ρ is left as a free parameter of the algorithm, typically determined via cross-validation.

Now, since only the first term of the right-hand side depends on $\mathbf{w}$, for any $\rho > 0$, the bound suggests selecting $\mathbf{w}$ as the solution of the following optimization problem:

$$\min_{\|\mathbf{w}\|^2 \leq \frac{1}{\rho^2}} \frac{1}{m}\sum_{i=1}^{m} \max\left(0, 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i)\right). \quad (5.47)$$

Introducing a Lagrange variable $\lambda \geq 0$, the optimization problem can be written equivalently as

$$\min_{\mathbf{w}} \lambda\|\mathbf{w}\|^2 + \frac{1}{m}\sum_{i=1}^{m} \max\left(0, 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i)\right). \quad (5.48)$$

Since for any choice of ρ in the constraint of (5.47) there exists an equivalent dual variable λ in the formulation of (5.48) that achieves the same optimal $\mathbf{w}$, λ can be freely selected via cross-validation.[5] The resulting algorithm precisely coincides with SVMs. Note that an alternative objective function and thus algorithm would be based on the empirical margin loss instead of the hinge loss. However, the advantage of the hinge loss is that it is convex, while the margin loss is not.

As already pointed out, the bounds just discussed do not directly depend on the dimension of the feature space but guarantee good generalization when given a favorable margin. Thus, they suggest seeking large-margin separating hyperplanes in a very high-dimensional space. In view of the form of the dual optimization problems for SVMs, determining the solution of the optimization and using it for prediction both require computing many inner products in that space. For very high-dimensional spaces, the computation of these inner products could become very costly. The next chapter provides a solution to this problem which further provides a generalization of SVMs to non-vectorial input spaces.

[5] An equivalent analysis consists of choosing $\rho = 1/\|\mathbf{w}\|$ in (5.46).

5.5 Chapter notes

The maximum-margin or *optimal hyperplane* solution described in section 5.2 was introduced by Vapnik and Chervonenkis [1964]. The algorithm had limited applications since in most tasks in practice the data is not linearly separable. In contrast, the SVM algorithm of section 5.3 for the general non-separable case, introduced by Cortes and Vapnik [1995] under the name *support-vector networks*, has been widely adopted and been shown to be effective in practice. The algorithm and its theory have had a profound impact on theoretical and applied machine learning and inspired research on a variety of topics. Several specialized algorithms have been suggested for solving the specific QP that arises when solving the SVM problem, for example the SMO algorithm of Platt [1999] (see exercise 5.4) and a variety of other decomposition methods such as those used in the LibLinear software library [Hsieh et al., 2008], and [Allauzen et al., 2010] for solving the problem when using rational kernels (see chapter 6).

Much of the theory supporting the SVM algorithm ([Cortes and Vapnik, 1995, Vapnik, 1998]), in particular the margin theory presented in section 5.4, has been adopted in the learning theory and statistics communities and applied to a variety of other problems. The margin bound on the VC-dimension of canonical hyperplanes (exercise 5.7) is by Vapnik [1998], the proof is very similar to Novikoff's margin bound on the number of updates made by the Perceptron algorithm in the separable case. Our presentation of margin guarantees based on the Rademacher complexity follows the elegant analysis of Koltchinskii and Panchenko [2002] (see also Bartlett and Mendelson [2002], Shawe-Taylor et al. [1998]). Our proof of Talagrand's lemma 5.7 is a simpler and more concise version of a more general result given by Ledoux and Talagrand [1991, pp. 112–114]. See Höffgen et al. [1995] for hardness results related to the problem of finding a hyperplane with the minimal number of errors on a training sample.

5.6 Exercises

5.1 Soft margin hyperplanes. The function of the slack variables used in the optimization problem for soft margin hyperplanes has the form: $\xi \mapsto \sum_{i=1}^{m} \xi_i$. Instead, we could use $\xi \mapsto \sum_{i=1}^{m} \xi_i^p$, with $p > 1$.

(a) Give the dual formulation of the problem in this general case.

(b) How does this more general formulation ($p > 1$) compare to the standard setting ($p = 1$)? In the case $p = 2$ is the optimization still convex?

5.2 Tighter Rademacher Bound. Derive the following tighter version of the bound of theorem 5.9: for any $\delta > 0$, with probability at least $1 - \delta$, for all $h \in \mathcal{H}$ and $\rho \in (0, 1]$ the following holds:

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{2\gamma}{\rho}\mathfrak{R}_m(\mathcal{H}) + \sqrt{\frac{\log \log_\gamma \frac{\gamma}{\rho}}{m}} + \sqrt{\frac{\log \frac{2}{\delta}}{2m}} \tag{5.49}$$

for any $\gamma > 1$.

5.3 Importance weighted SVM. Suppose you wish to use SVMs to solve a learning problem where some training data points are more important than others. More formally, assume that each training point consists of a triplet (x_i, y_i, p_i), where $0 \leq p_i \leq 1$ is the importance of the ith point. Rewrite the primal SVM constrained optimization problem so that the penalty for mis-labeling a point x_i is scaled by the priority p_i. Then carry this modification through the derivation of the dual solution.

5.4 Sequential minimal optimization (SMO). The SMO algorithm is an optimization algorithm introduced to speed up the training of SVMs. SMO reduces a (potentially) large quadratic programming (QP) optimization problem into a series of small optimizations involving only two Lagrange multipliers. SMO reduces memory requirements, bypasses the need for numerical QP optimization and is easy to implement. In this question, we will derive the update rule for the SMO algorithm in the context of the dual formulation of the SVM problem.

(a) Assume that we want to optimize equation (5.33) only over α_1 and α_2. Show that the optimization problem reduces to

$$\max_{\alpha_1,\alpha_2} \underbrace{\alpha_1 + \alpha_2 - \frac{1}{2}K_{11}\alpha_1^2 - \frac{1}{2}K_{22}\alpha_2^2 - sK_{12}\alpha_1\alpha_2 - y_1\alpha_1 v_1 - y_2\alpha_2 v_2}_{\Psi_1(\alpha_1,\alpha_2)}$$
$$\text{subject to: } 0 \leq \alpha_1, \alpha_2 \leq C \wedge \alpha_1 + s\alpha_2 = \gamma\,,$$

where $\gamma = y_1 \sum_{i=3}^m y_i\alpha_i$, $s = y_1 y_2 \in \{-1, +1\}$, $K_{ij} = (\mathbf{x}_i \cdot \mathbf{x}_j)$ and $v_i = \sum_{j=3}^m \alpha_j y_j K_{ij}$ for $i = 1, 2$.

(b) Substitute the linear constraint $\alpha_1 = \gamma - s\alpha_2$ into Ψ_1 to obtain a new objective function Ψ_2 that depends only on α_2. Show that the α_2 that minimizes Ψ_2 (without the constraints $0 \leq \alpha_1, \alpha_2 \leq C$) can be expressed as

$$\alpha_2 = \frac{s(K_{11} - K_{12})\gamma + y_2(v_1 - v_2) - s + 1}{\eta}\,,$$

where $\eta = K_{11} + K_{22} - 2K_{12}$.

(c) Show that

$$v_1 - v_2 = f(\mathbf{x}_1) - f(\mathbf{x}_2) + \alpha_2^* y_2 \eta - s y_2 \gamma (K_{11} - K_{12})$$

where $f(\mathbf{x}) = \sum_{i=1}^{m} \alpha_i^* y_i (\mathbf{x}_i \cdot \mathbf{x}) + b^*$ and α_i^* are values for the Lagrange multipliers prior to optimization over α_1 and α_2 (similarly, b^* is the previous value for the offset).

(d) Show that

$$\alpha_2 = \alpha_2^* + y_2 \frac{(y_2 - f(\mathbf{x}_2)) - (y_1 - f(\mathbf{x}_1))}{\eta}.$$

(e) For $s = +1$, define $L = \max\{0, \gamma - C\}$ and $H = \min\{C, \gamma\}$ as the lower and upper bounds on α_2. Similarly, for $s = -1$, define $L = \max\{0, -\gamma\}$ and $H = \min\{C, C - \gamma\}$. The update rule for SMO involves "clipping" the value of α_2, i.e.,

$$\alpha_2^{clip} = \begin{cases} \alpha_2 & \text{if } L < \alpha_2 < H \\ L & \text{if } \alpha_2 \leq L \\ H & \text{if } \alpha_2 \geq H \end{cases}.$$

We subsequently solve for α_1 such that we satisfy the equality constraint, resulting in $\alpha_1 = \alpha_1^* + s(\alpha_2^* - \alpha_2^{clip})$. Why is "clipping" is required? How are L and H derived for the case $s = +1$?

5.5 SVMs hands-on.

(a) Download and install the `libsvm` software library from:

`http://www.csie.ntu.edu.tw/~cjlin/libsvm/`.

(b) Download the `satimage` data set found at:

`http://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/`.

Merge the training and validation sets into one. We will refer to the resulting set as the training set from now on. Normalize both the training and test vectors.

(c) Consider the binary classification that consists of distinguishing class 6 from the rest of the data points. Use SVMs combined with polynomial kernels (see chapter 6) to solve this classification problem. To do so, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let the other parameters of polynomial kernels in `libsvm`, γ and c, be equal to their default values 1).

Report the best value of the trade-off constant C measured on the validation set.

(d) Let (C^*, d^*) be the best pair found previously. Fix C to be C^*. Plot the ten-fold cross-validation training and test errors for the hypotheses obtained as a function of d. Plot the average number of support vectors obtained as a function of d.

(e) How many of the support vectors lie on the margin hyperplanes?

(f) In the standard two-group classification, errors on positive or negative points are treated in the same manner. Suppose, however, that we wish to penalize an error on a negative point (false positive error) $k > 0$ times more than an error on a positive point. Give the dual optimization problem corresponding to SVMs modified in this way.

(g) Assume that k is an integer. Show how you can use `libsvm` without writing any additional code to find the solution of the modified SVMs just described.

(h) Apply the modified SVMs to the classification task previously examined and compare with your previous SVMs results for $k = 2, 4, 8, 16$.

5.6 Sparse SVM. One can give two types of arguments in favor of the SVM algorithm: one based on the sparsity of the support vectors, another based on the notion of margin. Suppose that instead of maximizing the margin, we choose instead to maximize sparsity by minimizing the L_p norm of the vector $\boldsymbol{\alpha}$ that defines the weight vector $\mathbf{w}$, for some $p \geq 1$. First, consider the case $p = 2$. This gives the following optimization problem:

$$\begin{aligned} \min_{\boldsymbol{\alpha}, b} \ & \frac{1}{2} \sum_{i=1}^{m} \alpha_i^2 + C \sum_{i=1}^{m} \xi_i \\ \text{subject to } & y_i \Big(\sum_{j=1}^{m} \alpha_j y_j \mathbf{x}_i \cdot \mathbf{x}_j + b \Big) \geq 1 - \xi_i, i \in [m] \\ & \xi_i, \alpha_i \geq 0, i \in [m]. \end{aligned} \tag{5.50}$$

(a) Show that modulo the non-negativity constraint on $\boldsymbol{\alpha}$, the problem coincides with an instance of the primal optimization problem of SVM.

(b) Derive the dual optimization of problem of (5.50).

(c) Setting $p = 1$ will induce a more sparse $\boldsymbol{\alpha}$. Derive the dual optimization in this case.

5.7 VC-dimension of canonical hyperplanes. The objective of this problem is derive a bound on the VC-dimension of canonical hyperplanes that does not depend on

the dimension of feature space. Let $S \subseteq \{\mathbf{x}\colon \|\mathbf{x}\| \leq r\}$. We will show that the VC-dimension d of the set of canonical hyperplanes $\{x \mapsto \operatorname{sgn}(\mathbf{w}\cdot\mathbf{x})\colon \min_{x\in S} |\mathbf{w}\cdot\mathbf{x}| = 1 \wedge \|\mathbf{w}\| \leq \Lambda\}$ verifies

$$d \leq r^2\Lambda^2\,. \tag{5.51}$$

(a) Let $\{\mathbf{x}_1, \ldots, \mathbf{x}_d\}$ be a set that can be shattered. Show that for all $\mathbf{y} = (y_1, \ldots, y_d) \in \{-1, +1\}^d$, $d \leq \Lambda\|\sum_{i=1}^d y_i\mathbf{x}_i\|$.

(b) Use randomization over the labels $\mathbf{y}$ and Jensen's inequality to show that $d \leq \Lambda\sqrt{\sum_{i=1}^d \|\mathbf{x}_i\|^2}$.

(c) Conclude that $d \leq r^2\Lambda^2$.

6 Kernel Methods

Kernel methods are widely used in machine learning. They are flexible techniques that can be used to extend algorithms such as SVMs to define non-linear decision boundaries. Other algorithms that only depend on inner products between sample points can be extended similarly, many of which will be studied in future chapters.

The main idea behind these methods is based on so-called *kernels* or *kernel functions*, which, under some technical conditions of symmetry and *positive-definiteness*, implicitly define an inner product in a high-dimensional space. Replacing the original inner product in the input space with positive definite kernels immediately extends algorithms such as SVMs to a linear separation in that high-dimensional space, or, equivalently, to a non-linear separation in the input space.

In this chapter, we present the main definitions and key properties of positive definite symmetric kernels, including the proof of the fact that they define an inner product in a Hilbert space, as well as their closure properties. We then extend the SVM algorithm using these kernels and present several theoretical results including general margin-based learning guarantees for hypothesis sets based on kernels. We also introduce *negative definite symmetric kernels* and point out their relevance to the construction of positive definite kernels, in particular from distances or metrics. Finally, we illustrate the design of kernels for non-vectorial discrete structures by introducing a general family of kernels for sequences, *rational kernels*. We describe an efficient algorithm for the computation of these kernels and illustrate them with several examples.

6.1 Introduction

In the previous chapter, we presented an algorithm for linear classification, SVMs, which is both effective in applications and benefits from a strong theoretical justification. In practice, linear separation is often not possible. Figure 6.1a shows an example where any hyperplane crosses both populations. However, one can use

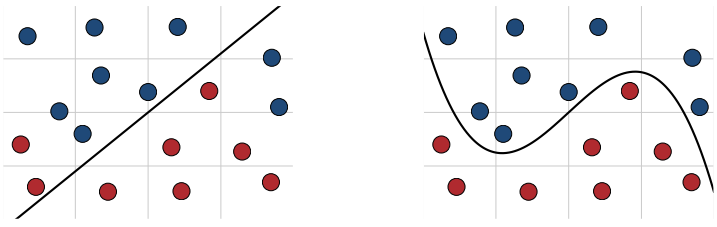


Figure 6.1
Non-linearly separable case. The classification task consists of discriminating between blue and red points. (a) No hyperplane can separate the two populations. (b) A non-linear mapping can be used instead.

more complex functions to separate the two sets as in figure 6.1b. One way to define such a non-linear decision boundary is to use a non-linear mapping Φ from the input space $\mathcal{X}$ to a higher-dimensional space $\mathbb{H}$, where linear separation is possible (see figure 6.2).

The dimension of $\mathbb{H}$ can truly be very large in practice. For example, in the case of document classification, one may wish to use as features sequences of three consecutive words, i.e., *trigrams*. Thus, with a vocabulary of just 100,000 words, the dimension of the feature space $\mathbb{H}$ reaches 10^{15}. On the positive side, the margin bounds presented in section 5.4 show that, remarkably, the generalization ability of large-margin classification algorithms such as SVMs do not depend on the dimension of the feature space, but only on the margin ρ and the number of training examples m. Thus, with a favorable margin ρ, such algorithms could succeed even in very high-dimensional space. However, determining the hyperplane solution requires multiple inner product computations in high-dimensional spaces, which can become be very costly.

A solution to this problem is to use *kernel methods*, which are based on *kernels* or *kernel functions*.

Definition 6.1 (Kernels) *A function* $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ *is called a* kernel over $\mathcal{X}$.

The idea is to define a kernel K such that for any two points $x, x' \in \mathcal{X}$, $K(x, x')$ be equal to an inner product of vectors $\Phi(x)$ and $\Phi(y)$:[6]

$$\forall x, x' \in \mathcal{X}, \quad K(x, x') = \langle \Phi(x), \Phi(x') \rangle \,, \tag{6.1}$$

[6] To differentiate that inner product from the one of the input space, we will typically denote it by $\langle \cdot, \cdot \rangle$.

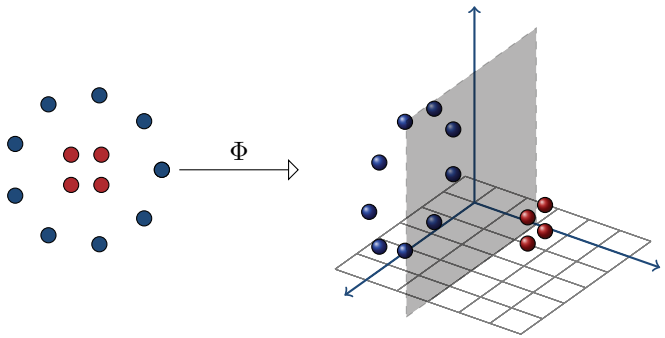


Figure 6.2
An example of a non-linear mapping from 2-dimensions to 3-dimensions, where the task becomes linearly seperable.

for some mapping $\Phi\colon \mathcal{X} \to \mathbb{H}$ to a Hilbert space $\mathbb{H}$ called a *feature space*. Since an inner product is a measure of the similarity of two vectors, K is often interpreted as a similarity measure between elements of the input space $\mathcal{X}$.

An important advantage of such a kernel K is efficiency: K is often significantly more efficient to compute than Φ and an inner product in $\mathbb{H}$. We will see several common examples where the computation of $K(x, x')$ can be achieved in $O(N)$ while that of $\langle \Phi(x), \Phi(x') \rangle$ typically requires $O(\dim(\mathbb{H}))$ work, with $\dim(\mathbb{H}) \gg N$. Furthermore, in some cases, the dimension of $\mathbb{H}$ is infinite.

Perhaps an even more crucial benefit of such a kernel function K is flexibility: there is no need to explicitly define or compute a mapping Φ. The kernel K can be arbitrarily chosen so long as the existence of Φ is guaranteed, i.e. K satisfies *Mercer's condition* (see theorem 6.2).

Theorem 6.2 (Mercer's condition) *Let $\mathcal{X} \subset \mathbb{R}^N$ be a compact set and let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a continuous and symmetric function. Then, K admits a uniformly convergent expansion of the form*

$$K(x, x') = \sum_{n=0}^{\infty} a_n \phi_n(x) \phi_n(x'),$$

with $a_n > 0$ iff for any square integrable function c ($c \in L_2(\mathcal{X})$), the following condition holds:

$$\int \int_{\mathcal{X} \times \mathcal{X}} c(x) c(x') K(x, x') dx dx' \geq 0.$$

This condition is important to guarantee the convexity of the optimization problem for algorithms such as SVMs, thereby ensuring convergence to a global minimum. A condition that is equivalent to Mercer's condition under the assumptions of the theorem is that the kernel K be *positive definite symmetric* (PDS). This property

is in fact more general since in particular it does not require any assumption about $\mathcal{X}$. In the next section, we give the definition of this property and present several commonly used examples of PDS kernels, then show that PDS kernels induce an inner product in a Hilbert space, and prove several general closure properties for PDS kernels.

6.2 Positive definite symmetric kernels

6.2.1 Definitions

Definition 6.3 (Positive definite symmetric kernels) *A kernel $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is said to be* positive definite symmetric *(PDS) if for any $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$, the matrix $\mathbf{K} = [K(x_i, x_j)]_{ij} \in \mathbb{R}^{m \times m}$ is symmetric positive semidefinite (SPSD).*

$\mathbf{K}$ is SPSD if it is symmetric and one of the following two equivalent conditions holds:

- the eigenvalues of $\mathbf{K}$ are non-negative;
- for any column vector $\mathbf{c} = (c_1, \ldots, c_m)^\top \in \mathbb{R}^{m \times 1}$,

$$\mathbf{c}^\top \mathbf{K} \mathbf{c} = \sum_{i,j=1}^{n} c_i c_j K(x_i, x_j) \geq 0. \tag{6.2}$$

For a sample $S = (x_1, \ldots, x_m)$, $\mathbf{K} = [K(x_i, x_j)]_{ij} \in \mathbb{R}^{m \times m}$ is called the *kernel matrix* or the *Gram matrix* associated to K and the sample S.

Let us insist on the terminology: the kernel matrix associated to a *positive definite kernel* is *positive semidefinite* . This is the correct mathematical terminology. Nevertheless, the reader should be aware that in the context of machine learning, some authors have chosen to use instead the term *positive definite kernel* to imply a *positive definite* kernel matrix or used new terms such as *positive semidefinite kernel*.

The following are some standard examples of PDS kernels commonly used in applications.

Example 6.4 (Polynomial kernels) For any constant $c > 0$, a *polynomial kernel of degree $d \in \mathbb{N}$* is the kernel K defined over $\mathbb{R}^N$ by:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \quad K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d. \tag{6.3}$$

Polynomial kernels map the input space to a higher-dimensional space of dimension $\binom{N+d}{d}$ (see exercise 6.12). As an example, for an input space of dimension $N = 2$, a second-degree polynomial ($d = 2$) corresponds to the following inner product in

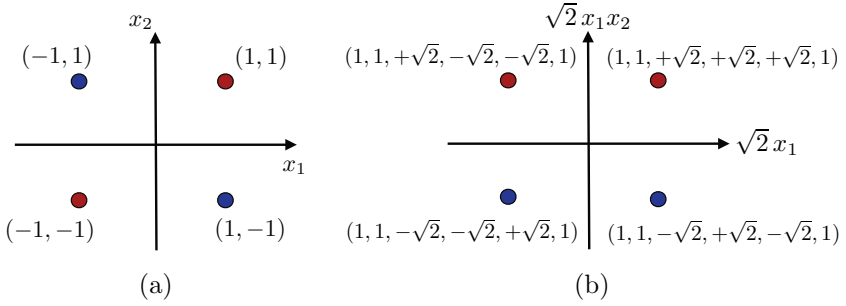


Figure 6.3
Illustration of the XOR classification problem and the use of polynomial kernels. (a) XOR problem linearly non-separable in the input space. (b) Linearly separable using second-degree polynomial kernel.

dimension 6:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^2, \quad K(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2' + c)^2 = \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}\, x_1 x_2 \\ \sqrt{2c}\, x_1 \\ \sqrt{2c}\, x_2 \\ c \end{bmatrix} \cdot \begin{bmatrix} {x'}_1^2 \\ {x'}_2^2 \\ \sqrt{2}\, x_1' x_2' \\ \sqrt{2c}\, x_1' \\ \sqrt{2c}\, x_2' \\ c \end{bmatrix}. \tag{6.4}$$

Thus, the features corresponding to a second-degree polynomial are the original features (x_1 and x_2), as well as products of these features, and the constant feature. More generally, the features associated to a polynomial kernel of degree d are all the monomials of degree at most d based on the original features. The explicit expression of polynomial kernels as inner products, as in (6.4), proves directly that they are PDS kernels.

To illustrate the application of polynomial kernels, consider the example of figure 6.3a which shows a simple data set in dimension two that is not linearly separable. This is known as the XOR problem due to its interpretation in terms of the exclusive OR (XOR) function: the label of a point is blue iff exactly one of its coordinates is 1. However, if we map these points to the six-dimensional space defined by a second-degree polynomial as described in (6.4), then the problem becomes separable by the hyperplane of equation $x_1 x_2 = 0$. Figure 6.3b illustrates that by showing the projection of these points on the two-dimensional space defined by their third and fourth coordinates.

Example 6.5 (Gaussian kernels) For any constant $\sigma > 0$, a *Gaussian kernel* or *radial basis function (RBF)* is the kernel K defined over $\mathbb{R}^N$ by:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \quad K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x}' - \mathbf{x}\|^2}{2\sigma^2}\right). \tag{6.5}$$

Gaussian kernels are among the most frequently used kernels in applications. We will prove in section 6.2.3 that they are PDS kernels and that they can be derived by *normalization* from the kernels $K' \colon (\mathbf{x}, \mathbf{x}') \mapsto \exp\left(\frac{\mathbf{x} \cdot \mathbf{x}'}{\sigma^2}\right)$. Using the power series expansion of the exponential function, we can rewrite the expression of K' as follows:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \quad K'(\mathbf{x}, \mathbf{x}') = \sum_{n=0}^{+\infty} \frac{(\mathbf{x} \cdot \mathbf{x}')^n}{\sigma^{2n}\, n!},$$

which shows that the kernels K', and thus Gaussian kernels, are positive linear combinations of polynomial kernels of all degrees $n \geq 0$.

Example 6.6 (Sigmoid kernels) For any real constants $a, b \geq 0$, a *sigmoid kernel* is the kernel K defined over $\mathbb{R}^N$ by:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \quad K(\mathbf{x}, \mathbf{x}') = \tanh\big(a(\mathbf{x} \cdot \mathbf{x}') + b\big). \tag{6.6}$$

Using sigmoid kernels with SVMs leads to an algorithm that is closely related to learning algorithms based on simple neural networks, which are also often defined via a sigmoid function. When $a < 0$ or $b < 0$, the kernel is not PDS and the corresponding neural network does not benefit from the convergence guarantees of convex optimization (see exercise 6.18).

6.2.2 Reproducing kernel Hilbert space

Here, we prove the crucial property of PDS kernels, which is to induce an inner product in a Hilbert space. The proof will make use of the following lemma.

Lemma 6.7 (Cauchy-Schwarz inequality for PDS kernels) *Let K be a PDS kernel. Then, for any $x, x' \in \mathcal{X}$,*

$$K(x, x')^2 \leq K(x, x) K(x', x'). \tag{6.7}$$

Proof: Consider the matrix $\mathbf{K} = \begin{pmatrix} K(x,x) & K(x,x') \\ K(x',x) & K(x',x') \end{pmatrix}$. By definition, if K is PDS, then $\mathbf{K}$ is SPSD for all $x, x' \in \mathcal{X}$. In particular, the product of the eigenvalues of $\mathbf{K}$, $\det(\mathbf{K})$, must be non-negative, thus, using $K(x', x) = K(x, x')$, we have

$$\det(\mathbf{K}) = K(x, x) K(x', x') - K(x, x')^2 \geq 0,$$

which concludes the proof. □

The following is the main result of this section.

Theorem 6.8 (Reproducing kernel Hilbert space (RKHS)) *Let $K \colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel. Then, there exists a Hilbert space $\mathbb{H}$ (see definition A.2) and a mapping Φ*

from $\mathcal{X}$ to $\mathbb{H}$ such that:

$$\forall x, x' \in \mathcal{X}, \quad K(x, x') = \langle \Phi(x), \Phi(x') \rangle . \tag{6.8}$$

Furthermore, $\mathbb{H}$ has the following property known as the reproducing property*:*

$$\forall h \in \mathbb{H}, \forall x \in \mathcal{X}, \quad h(x) = \langle h, K(x, \cdot) \rangle . \tag{6.9}$$

$\mathbb{H}$ is called a reproducing kernel Hilbert space *(RKHS) associated to K.*

Proof: For any $x \in \mathcal{X}$, define $\Phi(x) \colon \mathcal{X} \to \mathbb{R}^{\mathcal{X}}$ as follows:

$$\forall x' \in \mathcal{X}, \ \Phi(x)(x') = K(x, x').$$

We define $\mathbb{H}_0$ as the set of finite linear combinations of such functions $\Phi(x)$:

$$\mathbb{H}_0 = \left\{ \sum_{i \in I} a_i \Phi(x_i) \colon a_i \in \mathbb{R}, x_i \in \mathcal{X}, |I| < \infty \right\}.$$

Now, we introduce an operation $\langle \cdot, \cdot \rangle$ on $\mathbb{H}_0 \times \mathbb{H}_0$ defined for all $f, g \in \mathbb{H}_0$ with $f = \sum_{i \in I} a_i \Phi(x_i)$ and $g = \sum_{j \in J} b_j \Phi(x'_j)$ by

$$\langle f, g \rangle = \sum_{i \in I, j \in J} a_i b_j K(x_i, x'_j) = \sum_{j \in J} b_j f(x'_j) = \sum_{i \in I} a_i g(x_i).$$

By definition, $\langle \cdot, \cdot \rangle$ is symmetric. The last two equations show that $\langle f, g \rangle$ does not depend on the particular representations of f and g, and also show that $\langle \cdot, \cdot \rangle$ is bilinear. Further, for any $f = \sum_{i \in I} a_i \Phi(x_i) \in \mathbb{H}_0$, since K is PDS, we have

$$\langle f, f \rangle = \sum_{i,j \in I} a_i a_j K(x_i, x_j) \geq 0.$$

Thus, $\langle \cdot, \cdot \rangle$ is positive semidefinite bilinear form. This inequality implies more generally using the bilinearity of $\langle \cdot, \cdot \rangle$ that for any $f_1, \ldots, f_m$ and $c_1, \ldots, c_m \in \mathbb{R}$,

$$\sum_{i,j=1}^{m} c_i c_j \langle f_i, f_j \rangle = \Big\langle \sum_{i=1}^{m} c_i f_i, \sum_{j=1}^{m} c_j f_j \Big\rangle \geq 0.$$

Hence, $\langle \cdot, \cdot \rangle$ is a PDS kernel on $\mathbb{H}_0$. Thus, for any $f \in \mathbb{H}_0$ and any $x \in \mathcal{X}$, by lemma 6.7, we can write

$$\langle f, \Phi(x) \rangle^2 \leq \langle f, f \rangle \langle \Phi(x), \Phi(x) \rangle.$$

Further, we observe the reproducing property of $\langle \cdot, \cdot \rangle$: for any $f = \sum_{i \in I} a_i \Phi(x_i) \in \mathbb{H}_0$, by definition of $\langle \cdot, \cdot \rangle$,

$$\forall x \in \mathcal{X}, \quad f(x) = \sum_{i \in I} a_i K(x_i, x) = \langle f, \Phi(x) \rangle . \tag{6.10}$$

Thus, $[f(x)]^2 \leq \langle f, f \rangle K(x, x)$ for all $x \in \mathcal{X}$, which shows the definiteness of $\langle \cdot, \cdot \rangle$. This implies that $\langle \cdot, \cdot \rangle$ defines an inner product on $\mathbb{H}_0$, which thereby becomes a pre-Hilbert space. $\mathbb{H}_0$ can be completed to form a Hilbert space $\mathbb{H}$ in which it is dense, following a standard construction. By the Cauchy-Schwarz inequality, for any $x \in \mathcal{X}$, $f \mapsto \langle f, \Phi(x) \rangle$ is Lipschitz, therefore continuous. Thus, since $\mathbb{H}_0$ is dense in $\mathbb{H}$, the reproducing property (6.10) also holds over $\mathbb{H}$. □

The Hilbert space $\mathbb{H}$ defined in the proof of the theorem for a PDS kernel K is called *the reproducing kernel Hilbert space (RKHS) associated to* K. Any Hilbert space $\mathbb{H}$ such that there exists $\Phi \colon \mathcal{X} \to \mathbb{H}$ with $K(x, x') = \langle \Phi(x), \Phi(x') \rangle$ for all $x, x' \in \mathcal{X}$ is called a *feature space* associated to K and Φ is called a *feature mapping*. We will denote by $\| \cdot \|_{\mathbb{H}}$ the norm induced by the inner product in feature space $\mathbb{H}$: $\|\mathbf{w}\|_{\mathbb{H}} = \sqrt{\langle \mathbf{w}, \mathbf{w} \rangle}$ for all $\mathbf{w} \in \mathbb{H}$. Note that the feature spaces associated to K are in general not unique and may have different dimensions. In practice, when referring to the *dimension of the feature space* associated to K, we either refer to the dimension of the feature space based on a feature mapping described explicitly, or to that of the RKHS associated to K.

Theorem 6.8 implies that PDS kernels can be used to implicitly define a feature space or feature vectors. As already underlined in previous chapters, the role played by the features in the success of learning algorithms is crucial: with poor features, uncorrelated with the target labels, learning could become very challenging or even impossible; in contrast, good features could provide invaluable clues to the algorithm. Therefore, in the context of learning with PDS kernels and for a fixed input space, the problem of seeking useful features is replaced by that of finding useful PDS kernels. While features represented the user's prior knowledge about the task in the standard learning problems, here PDS kernels will play this role. Thus, in practice, an appropriate choice of PDS kernel for a task will be crucial.

6.2.3 Properties

This section highlights several important properties of PDS kernels. We first show that PDS kernels can be *normalized* and that the resulting normalized kernels are also PDS. We also introduce the definition of *empirical kernel maps* and describe their properties and extension. We then prove several important closure properties of PDS kernels, which can be used to construct complex PDS kernels from simpler ones.

To any kernel K, we can associate a *normalized kernel* K' defined by

$$\forall x, x' \in \mathcal{X}, \quad K'(x, x') = \begin{cases} 0 & \text{if } (K(x, x) = 0) \vee (K(x', x') = 0) \\ \frac{K(x, x')}{\sqrt{K(x, x) K(x', x')}} & \text{otherwise.} \end{cases} \tag{6.11}$$

By definition, for a normalized kernel K', $K'(x,x) = 1$ for all $x \in \mathcal{X}$ such that $K(x,x) \neq 0$. An example of normalized kernel is the Gaussian kernel with parameter $\sigma > 0$, which is the normalized kernel associated to $K' \colon (\mathbf{x}, \mathbf{x}') \mapsto \exp\left(\frac{\mathbf{x}\cdot\mathbf{x}'}{\sigma^2}\right)$:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^N, \frac{K'(\mathbf{x}, \mathbf{x}')}{\sqrt{K'(\mathbf{x}, \mathbf{x}) K'(\mathbf{x}', \mathbf{x}')}} = \frac{e^{\frac{\mathbf{x}\cdot\mathbf{x}'}{\sigma^2}}}{e^{\frac{\|\mathbf{x}\|^2}{2\sigma^2}} e^{\frac{\|\mathbf{x}'\|^2}{2\sigma^2}}} = \exp\left(-\frac{\|\mathbf{x}' - \mathbf{x}\|^2}{2\sigma^2}\right). \quad (6.12)$$

Lemma 6.9 (Normalized PDS kernels) *Let K be a PDS kernel. Then, the normalized kernel K' associated to K is PDS.*

Proof: Let $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$ and let $\mathbf{c}$ be an arbitrary vector in $\mathbb{R}^m$. We will show that the sum $\sum_{i,j=1}^m c_i c_j K'(x_i, x_j)$ is non-negative. By lemma 6.7, if $K(x_i, x_i) = 0$ then $K(x_i, x_j) = 0$ and thus $K'(x_i, x_j) = 0$ for all $j \in [m]$. Thus, we can assume that $K(x_i, x_i) > 0$ for all $i \in [m]$. Then, the sum can be rewritten as follows:

$$\sum_{i,j=1}^m \frac{c_i c_j K(x_i, x_j)}{\sqrt{K(x_i, x_i) K(x_j, x_j)}} = \sum_{i,j=1}^m \frac{c_i c_j \langle \Phi(x_i), \Phi(x_j) \rangle}{\|\Phi(x_i)\|_{\mathbb{H}} \|\Phi(x_j)\|_{\mathbb{H}}} = \left\| \sum_{i=1}^m \frac{c_i \Phi(x_i)}{\|\Phi(x_i)\|_{\mathbb{H}}} \right\|_{\mathbb{H}}^2 \geq 0,$$

where Φ is a feature mapping associated to K, which exists by theorem 6.8. □

As indicated earlier, PDS kernels can be interpreted as a similarity measure since they induce an inner product in some Hilbert space $\mathbb{H}$. This is more evident for a normalized kernel K since $K(x, x')$ is then exactly the cosine of the angle between the feature vectors $\Phi(x)$ and $\Phi(x')$, provided that none of them is zero: $\Phi(x)$ and $\Phi(x')$ are then unit vectors since $\|\Phi(x)\|_{\mathbb{H}} = \|\Phi(x')\|_{\mathbb{H}} = \sqrt{K(x,x)} = 1$.

While one of the advantages of PDS kernels is an implicit definition of a feature mapping, in some instances, it may be desirable to define an explicit feature mapping based on a PDS kernel. This may be to work in the primal for various optimization and computational reasons, to derive an approximation based on an explicit mapping, or as part of a theoretical analysis where an explicit mapping is more convenient. The *empirical kernel map* Φ associated to a PDS kernel K is a feature mapping that can be used precisely in such contexts. Given a training sample containing points $x_1, \ldots, x_m \in \mathcal{X}$, $\Phi \colon \mathcal{X} \to \mathbb{R}^m$ is defined for all $x \in \mathcal{X}$ by

$$\Phi(x) = \begin{bmatrix} K(x, x_1) \\ \vdots \\ K(x, x_m) \end{bmatrix}.$$

Thus, $\Phi(x)$ is the vector of the K-similarity measures of x with each of the training points. Let $\mathbf{K}$ be the kernel matrix associated to K and $\mathbf{e}_i$ the ith unit vector. Note that for any $i \in [m]$, $\Phi(x_i)$ is the ith column of $\mathbf{K}$, that is $\Phi(x_i) = \mathbf{K}\mathbf{e}_i$. In

particular, for all $i, j \in [m]$,

$$\langle \Phi(x_i), \Phi(x_j) \rangle = (\mathbf{K}\mathbf{e}_i)^\top (\mathbf{K}\mathbf{e}_j) = \mathbf{e}_i^\top \mathbf{K}^2 \mathbf{e}_j.$$

Thus, the kernel matrix $\mathbf{K}'$ associated to Φ is $\mathbf{K}^2$. It may desirable in some cases to define a feature mapping whose kernel matrix coincides with $\mathbf{K}$. Let $\mathbf{K}^{\dagger\frac{1}{2}}$ denote the SPSD matrix whose square is $\mathbf{K}^\dagger$, the pseudo-inverse of $\mathbf{K}$. $\mathbf{K}^{\dagger\frac{1}{2}}$ can be derived from $\mathbf{K}^\dagger$ via singular value decomposition and if the matrix $\mathbf{K}$ is invertible, $\mathbf{K}^{\dagger\frac{1}{2}}$ coincides with $\mathbf{K}^{-1/2}$ (see appendix A for properties of the pseudo-inverse). Then, Ψ can be defined as follows using the empirical kernel map Φ:

$$\forall x \in \mathcal{X}, \quad \Psi(x) = \mathbf{K}^{\dagger\frac{1}{2}} \Phi(x).$$

Using the identity $\mathbf{K}\mathbf{K}^\dagger\mathbf{K} = \mathbf{K}$ valid for any symmetric matrix $\mathbf{K}$, for all $i, j \in [m]$, the following holds:

$$\langle \Psi(x_i), \Psi(x_j) \rangle = (\mathbf{K}^{\dagger\frac{1}{2}} \mathbf{K}\mathbf{e}_i)^\top (\mathbf{K}^{\dagger\frac{1}{2}} \mathbf{K}\mathbf{e}_j) = \mathbf{e}_i^\top \mathbf{K}\mathbf{K}^\dagger \mathbf{K}\mathbf{e}_j = \mathbf{e}_i^\top \mathbf{K}\mathbf{e}_j.$$

Thus, the kernel matrix associated to Ψ is $\mathbf{K}$. Finally, note that for the feature mapping $\Omega \colon \mathcal{X} \to \mathbb{R}^m$ defined by

$$\forall x \in \mathcal{X}, \quad \Omega(x) = \mathbf{K}^\dagger \Phi(x),$$

for all $i, j \in [m]$, we have $\langle \Omega(x_i), \Omega(x_j) \rangle = \mathbf{e}_i^\top \mathbf{K}\mathbf{K}^\dagger\mathbf{K}^\dagger\mathbf{K}\mathbf{e}_j = \mathbf{e}_i^\top \mathbf{K}\mathbf{K}^\dagger \mathbf{e}_j$, using the identity $\mathbf{K}^\dagger\mathbf{K}^\dagger\mathbf{K} = \mathbf{K}^\dagger$ valid for any symmetric matrix $\mathbf{K}$. Thus, the kernel matrix associated to Ω is $\mathbf{K}\mathbf{K}^\dagger$, which reduces to the identity matrix $\mathbf{I} \in \mathbb{R}^{m \times m}$ when $\mathbf{K}$ is invertible, since $\mathbf{K}^\dagger = \mathbf{K}^{-1}$ in that case.

As pointed out in the previous section, kernels represent the user's prior knowledge about a task. In some cases, a user may come up with appropriate similarity measures or PDS kernels for some subtasks — for example, for different subcategories of proteins or text documents to classify. But how can the user combine these PDS kernels to form a PDS kernel for the entire class? Is the resulting combined kernel guaranteed to be PDS? In the following, we will show that PDS kernels are closed under several useful operations which can be used to design complex PDS kernels. These operations are the sum and the product of kernels, as well as the *tensor product* of two kernels K and K', denoted by $K \otimes K'$ and defined by

$$\forall x_1, x_2, x_1', x_2' \in \mathcal{X}, \quad (K \otimes K')(x_1, x_1', x_2, x_2') = K(x_1, x_2) K'(x_1', x_2').$$

They also include the pointwise limit: given a sequence of kernels $(K_n)_{n \in \mathbb{N}}$ such that for all $x, x' \in \mathcal{X}$ $(K_n(x, x'))_{n \in \mathbb{N}}$ admits a limit, the pointwise limit of $(K_n)_{n \in \mathbb{N}}$ is the kernel K defined for all $x, x' \in \mathcal{X}$ by $K(x, x') = \lim_{n \to +\infty} (K_n)(x, x')$. Similarly, if $\sum_{n=0}^{\infty} a_n x^n$ is a power series with radius of convergence $\rho > 0$ and K a kernel taking values in $(-\rho, +\rho)$, then $\sum_{n=0}^{\infty} a_n K^n$ is the kernel obtained by composition

of K with that power series. The following theorem provides closure guarantees for all of these operations.

Theorem 6.10 (PDS kernels — closure properties) *PDS kernels are closed under sum, product, tensor product, pointwise limit, and composition with a power series* $\sum_{n=0}^{\infty} a_n x^n$ *with* $a_n \geq 0$ *for all* $n \in \mathbb{N}$.

Proof: We start with two kernel matrices, $\mathbf{K}$ and $\mathbf{K}'$, generated from PDS kernels K and K' for an arbitrary set of m points. By assumption, these kernel matrices are SPSD. Observe that for any $\mathbf{c} \in \mathbb{R}^{m \times 1}$,

$$(\mathbf{c}^\top \mathbf{K}\mathbf{c} \geq 0) \wedge (\mathbf{c}^\top \mathbf{K}'\mathbf{c} \geq 0) \Rightarrow \mathbf{c}^\top (\mathbf{K} + \mathbf{K}')\mathbf{c} \geq 0.$$

By (6.2), this shows that $\mathbf{K} + \mathbf{K}'$ is SPSD and thus that $K + K'$ is PDS. To show closure under product, we will use the fact that for any SPSD matrix $\mathbf{K}$ there exists $\mathbf{M}$ such that $\mathbf{K} = \mathbf{M}\mathbf{M}^\top$. The existence of $\mathbf{M}$ is guaranteed as it can be generated via, for instance, singular value decomposition of $\mathbf{K}$, or by Cholesky decomposition. The kernel matrix associated to KK' is $(\mathbf{K}_{ij}\mathbf{K}'_{ij})_{ij}$. For any $\mathbf{c} \in \mathbb{R}^{m \times 1}$, expressing $\mathbf{K}_{ij}$ in terms of the entries of $\mathbf{M}$, we can write

$$\begin{aligned}
\sum_{i,j=1}^{m} c_i c_j (\mathbf{K}_{ij}\mathbf{K}'_{ij}) &= \sum_{i,j=1}^{m} c_i c_j \left(\Big[\sum_{k=1}^{m} \mathbf{M}_{ik}\mathbf{M}_{jk} \Big] \mathbf{K}'_{ij} \right) \\
&= \sum_{k=1}^{m} \Big[\sum_{i,j=1}^{m} c_i c_j \mathbf{M}_{ik}\mathbf{M}_{jk}\mathbf{K}'_{ij} \Big] \\
&= \sum_{k=1}^{m} \mathbf{z}_k^\top \mathbf{K}' \mathbf{z}_k \geq 0,
\end{aligned}$$

with $\mathbf{z}_k = \begin{bmatrix} c_1\mathbf{M}_{1k} \\ \vdots \\ c_m\mathbf{M}_{mk} \end{bmatrix}$. This shows that PDS kernels are closed under product. The tensor product of K and K' is PDS as the product of the two PDS kernels $(x_1, x'_1, x_2, x'_2) \mapsto K(x_1, x_2)$ and $(x_1, x'_1, x_2, x'_2) \mapsto K'(x'_1, x'_2)$. Next, let $(K_n)_{n\in\mathbb{N}}$ be a sequence of PDS kernels with pointwise limit K. Let $\mathbf{K}$ be the kernel matrix associated to K and $\mathbf{K}_n$ the one associated to K_n for any $n \in \mathbb{N}$. Observe that

$$(\forall n, \mathbf{c}^\top \mathbf{K}_n \mathbf{c} \geq 0) \Rightarrow \lim_{n\to\infty} \mathbf{c}^\top \mathbf{K}_n \mathbf{c} = \mathbf{c}^\top \mathbf{K}\mathbf{c} \geq 0.$$

This shows the closure under pointwise limit. Finally, assume that K is a PDS kernel with $|K(x, x')| < \rho$ for all $x, x' \in \mathcal{X}$ and let $f\colon x \mapsto \sum_{n=0}^{\infty} a_n x^n, a_n \geq 0$ be a power series with radius of convergence ρ. Then, for any $n \in \mathbb{N}$, K^n and thus $a_n K^n$ are PDS by closure under product. For any $N \in \mathbb{N}$, $\sum_{n=0}^{N} a_n K^n$ is PDS by closure under sum of $a_n K^n$s and $f \circ K$ is PDS by closure under the limit of $\sum_{n=0}^{N} a_n K^n$ as N tends to infinity. □

The theorem implies in particular that for any PDS kernel matrix K, $\exp(K)$ is PDS, since the radius of convergence of exp is infinite. In particular, the kernel $K'\colon (\mathbf{x}, \mathbf{x}') \mapsto \exp\left(\frac{\mathbf{x}\cdot\mathbf{x}'}{\sigma^2}\right)$ is PDS since $(\mathbf{x}, \mathbf{x}') \mapsto \frac{\mathbf{x}\cdot\mathbf{x}'}{\sigma^2}$ is PDS. Thus, by lemma 6.9, this shows that a Gaussian kernel, which is the normalized kernel associated to K', is PDS.

6.3 Kernel-based algorithms

In this section we discuss how SVMs can be used with kernels and analyze the impact that kernels have on generalization.

6.3.1 SVMs with PDS kernels

In chapter 5, we noted that the dual optimization problem for SVMs as well as the form of the solution did not directly depend on the input vectors but only on inner products. Since a PDS kernel implicitly defines an inner product (theorem 6.8), we can extend SVMs and combine it with an arbitrary PDS kernel K by replacing each instance of an inner product $x \cdot x'$ with $K(x, x')$. This leads to the following general form of the SVM optimization problem and solution with PDS kernels extending (5.33):

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \tag{6.13}$$
$$\text{subject to: } 0 \leq \alpha_i \leq C \wedge \sum_{i=1}^{m} \alpha_i y_i = 0, i \in [m].$$

In view of (5.34), the hypothesis h solution can be written as:

$$h(x) = \operatorname{sgn}\Big(\sum_{i=1}^{m} \alpha_i y_i K(x_i, x) + b\Big), \tag{6.14}$$

with $b = y_i - \sum_{j=1}^{m} \alpha_j y_j K(x_j, x_i)$ for any x_i with $0 < \alpha_i < C$. We can rewrite the optimization problem (6.13) in a vector form, by using the kernel matrix $\mathbf{K}$ associated to K for the training sample $(x_1, \ldots, x_m)$ as follows:

$$\max_{\boldsymbol{\alpha}} 2\, \mathbf{1}^\top \boldsymbol{\alpha} - (\boldsymbol{\alpha} \circ \mathbf{y})^\top \mathbf{K} (\boldsymbol{\alpha} \circ \mathbf{y}) \tag{6.15}$$
$$\text{subject to: } \mathbf{0} \leq \boldsymbol{\alpha} \leq \mathbf{C} \wedge \boldsymbol{\alpha}^\top \mathbf{y} = 0.$$

In this formulation, $\boldsymbol{\alpha} \circ \mathbf{y}$ is the Hadamard product or entry-wise product of the vectors $\boldsymbol{\alpha}$ and $\mathbf{y}$. Thus, it is the column vector in $\mathbb{R}^{m \times 1}$ whose ith component equals $\alpha_i y_i$. The solution in vector form is the same as in (6.14), but with $b = y_i - (\boldsymbol{\alpha} \circ \mathbf{y})^\top \mathbf{K} \mathbf{e}_i$ for any x_i with $0 < \alpha_i < C$.

This version of SVMs used with PDS kernels is the general form of SVMs we will consider in all that follows. The extension is important, since it enables an implicit non-linear mapping of the input points to a high-dimensional space where large-margin separation is sought.

Many other algorithms in areas including regression, ranking, dimensionality reduction or clustering can be extended using PDS kernels following the same scheme (see in particular chapters 9, 10, 11, 15).

6.3.2 Representer theorem

Observe that modulo the offset b, the hypothesis solution of SVMs can be written as a linear combination of the functions $K(x_i, \cdot)$, where x_i is a sample point. The following theorem known as the *representer theorem* shows that this is in fact a general property that holds for a broad class of optimization problems, including that of SVMs with no offset.

Theorem 6.11 (Representer theorem) *Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel and $\mathbb{H}$ its corresponding RKHS. Then, for any non-decreasing function $G\colon \mathbb{R} \to \mathbb{R}$ and any loss function $L\colon \mathbb{R}^m \to \mathbb{R} \cup \{+\infty\}$, the optimization problem*

$$\operatorname*{argmin}_{h \in \mathbb{H}} F(h) = \operatorname*{argmin}_{h \in \mathbb{H}} G(\|h\|_{\mathbb{H}}) + L\big(h(x_1), \ldots, h(x_m)\big)$$

admits a solution of the form $h^* = \sum_{i=1}^m \alpha_i K(x_i, \cdot)$. If G is further assumed to be increasing, then any solution has this form.

Proof: Let $\mathbb{H}_1 = \operatorname{span}(\{K(x_i, \cdot)\colon i \in [m]\})$. Any $h \in \mathbb{H}$ admits the decomposition $h = h_1 + h^\perp$ according to $\mathbb{H} = \mathbb{H}_1 \oplus \mathbb{H}_1^\perp$, where $\oplus$ is the direct sum. Since G is non-decreasing, $G(\|h_1\|_{\mathbb{H}}) \leq G(\sqrt{\|h_1\|_{\mathbb{H}}^2 + \|h^\perp\|_{\mathbb{H}}^2}) = G(\|h\|_{\mathbb{H}})$. By the reproducing property, for all $i \in [m]$, $h(x_i) = \langle h, K(x_i, \cdot)\rangle = \langle h_1, K(x_i, \cdot)\rangle = h_1(x_i)$. Thus, $L\big(h(x_1), \ldots, h(x_m)\big) = L\big(h_1(x_1), \ldots, h_1(x_m)\big)$ and $F(h_1) \leq F(h)$. This proves the first part of the theorem. If G is further increasing, then $F(h_1) < F(h)$ when $\|h^\perp\|_{\mathbb{H}} > 0$ and any solution of the optimization problem must be in $\mathbb{H}_1$. □

6.3.3 Learning guarantees

Here, we present general learning guarantees for hypothesis sets based on PDS kernels, which hold in particular for SVMs combined with PDS kernels.

The following theorem gives a general bound on the empirical Rademacher complexity of kernel-based hypotheses with bounded norm, that is a hypothesis set of the form $\mathcal{H} = \{h \in \mathbb{H}\colon \|h\|_{\mathbb{H}} \leq \Lambda\}$, for some $\Lambda \geq 0$, where $\mathbb{H}$ is the RKHS associated to a kernel K. By the reproducing property, any $h \in \mathcal{H}$ is of the form $x \mapsto \langle h, K(x, \cdot)\rangle = \langle h, \Phi(x)\rangle$ with $\|h\|_{\mathbb{H}} \leq \Lambda$, where Φ is a feature mapping associated to K, that is of the form $x \mapsto \langle \mathbf{w}, \Phi(x)\rangle$ with $\|\mathbf{w}\|_{\mathbb{H}} \leq \Lambda$.

Theorem 6.12 (Rademacher complexity of kernel-based hypotheses) *Let $K\colon \mathcal{X}\times\mathcal{X}\to\mathbb{R}$ be a PDS kernel and let $\Phi\colon \mathcal{X}\to\mathbb{H}$ be a feature mapping associated to K. Let $S\subseteq\{x\colon K(x,x)\leq r^2\}$ be a sample of size m, and let $\mathcal{H}=\{x\mapsto\langle\mathbf{w},\Phi(x)\rangle : \|\mathbf{w}\|_{\mathbb{H}}\leq\Lambda\}$ for some $\Lambda\geq 0$. Then*

$$\widehat{\mathfrak{R}}_S(\mathcal{H})\leq\frac{\Lambda\sqrt{\mathrm{Tr}[\mathbf{K}]}}{m}\leq\sqrt{\frac{r^2\Lambda^2}{m}}. \tag{6.16}$$

Proof: The proof steps are as follows:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\mathcal{H}) &= \frac{1}{m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\sup_{\|\mathbf{w}\|\leq\Lambda}\left\langle\mathbf{w},\sum_{i=1}^m\sigma_i\Phi(x_i)\right\rangle\right]\\
&= \frac{\Lambda}{m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\left\|\sum_{i=1}^m\sigma_i\Phi(x_i)\right\|_{\mathbb{H}}\right] && \text{(Cauchy-Schwarz, eq. case)}\\
&\leq \frac{\Lambda}{m}\left[\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\left\|\sum_{i=1}^m\sigma_i\Phi(x_i)\right\|_{\mathbb{H}}^2\right]\right]^{1/2} && \text{(Jensen's ineq.)}\\
&= \frac{\Lambda}{m}\left[\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\sum_{i=1}^m\|\Phi(x_i)\|_{\mathbb{H}}^2\right]\right]^{1/2} && (i\neq j\Rightarrow\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}[\sigma_i\sigma_j]=0)\\
&= \frac{\Lambda}{m}\left[\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\left[\sum_{i=1}^m K(x_i,x_i)\right]\right]^{1/2}\\
&= \frac{\Lambda\sqrt{\mathrm{Tr}[\mathbf{K}]}}{m}\leq\sqrt{\frac{r^2\Lambda^2}{m}}.
\end{aligned}$$

The initial equality holds by definition of the empirical Rademacher complexity (definition 3.1). The first inequality is due to the Cauchy-Schwarz inequality and $\|\mathbf{w}\|_{\mathbb{H}}\leq\Lambda$. The following inequality results from Jensen's inequality (theorem B.20) applied to the concave function $\sqrt{\cdot}$. The subsequent equality is a consequence of $\mathbb{E}_{\boldsymbol{\sigma}}[\sigma_i\sigma_j]=\mathbb{E}_{\boldsymbol{\sigma}}[\sigma_i]\,\mathbb{E}_{\boldsymbol{\sigma}}[\sigma_j]=0$ for $i\neq j$, since the Rademacher variables σ_i and σ_j are independent. The statement of the theorem then follows by noting that $\mathrm{Tr}[\mathbf{K}]\leq mr^2$. □

The theorem indicates that the trace of the kernel matrix is an important quantity for controlling the complexity of hypothesis sets based on kernels. Observe that by the Khintchine-Kahane inequality (D.24), the empirical Rademacher complexity $\widehat{\mathfrak{R}}_S(\mathcal{H})=\frac{\Lambda}{m}\,\mathbb{E}_{\boldsymbol{\sigma}}[\|\sum_{i=1}^m\sigma_i\Phi(x_i)\|_{\mathbb{H}}]$ can also be lower bounded by $\frac{1}{\sqrt{2}}\frac{\Lambda\sqrt{\mathrm{Tr}[\mathbf{K}]}}{m}$, which only differs from the upper bound found by the constant $\frac{1}{\sqrt{2}}$. Also, note that if $K(x,x)\leq r^2$ for all $x\in\mathcal{X}$, then the inequalities 6.16 hold for all samples S.

The bound of theorem 6.12 or the inequalities 6.16 can be plugged into any of the Rademacher complexity generalization bounds presented in the previous chapters. In particular, in combination with theorem 5.8, they lead directly to the following margin bound similar to that of corollary 5.11.

Corollary 6.13 (Margin bounds for kernel-based hypotheses) *Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel with $r^2 = \sup_{x\in\mathcal{X}} K(x,x)$. Let $\Phi\colon \mathcal{X} \to \mathbb{H}$ be a feature mapping associated to K and let $\mathcal{H} = \{\mathbf{x} \mapsto \mathbf{w} \cdot \Phi(x)\colon \|\mathbf{w}\|_{\mathbb{H}} \leq \Lambda\}$ for some $\Lambda \geq 0$. Fix $\rho > 0$. Then, for any $\delta > 0$, each of the following statements holds with probability at least $1-\delta$ for any $h \in \mathcal{H}$:*

$$R(h) \leq \widehat{R}_{S,\rho}(h) + 2\sqrt{\frac{r^2\Lambda^2/\rho^2}{m}} + \sqrt{\frac{\log\frac{1}{\delta}}{2m}} \tag{6.17}$$

$$R(h) \leq \widehat{R}_{S,\rho}(h) + 2\frac{\sqrt{\mathrm{Tr}[\mathbf{K}]\Lambda^2/\rho^2}}{m} + 3\sqrt{\frac{\log\frac{2}{\delta}}{2m}}\,. \tag{6.18}$$

6.4 Negative definite symmetric kernels

Often in practice, a natural distance or metric is available for the learning task considered. This metric could be used to define a similarity measure. As an example, Gaussian kernels have the form $\exp(-d^2)$, where d is a metric for the input vector space. Several natural questions arise such as: what other PDS kernels can we construct from a metric in a Hilbert space? What technical condition should d satisfy to guarantee that $\exp(-d^2)$ is PDS? A natural mathematical definition that helps address these questions is that of *negative definite symmetric (NDS) kernels*.

Definition 6.14 (Negative definite symmetric (NDS) kernels) *A kernel $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is said to be* negative-definite symmetric (NDS) *if it is symmetric and if for all $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$ and $\mathbf{c} \in \mathbb{R}^{m\times 1}$ with $\mathbf{1}^\top\mathbf{c} = 0$, the following holds:*

$$\mathbf{c}^\top\mathbf{K}\mathbf{c} \leq 0.$$

Clearly, if K is PDS, then $-K$ is NDS, but the converse does not hold in general. The following gives a standard example of an NDS kernel.

Example 6.15 (Squared distance — NDS kernel) The squared distance $(x, x') \mapsto \|x' - x\|^2$ in $\mathbb{R}^N$ defines an NDS kernel. Indeed, let $\mathbf{c} \in \mathbb{R}^{m\times 1}$ with $\sum_{i=1}^m c_i = 0$. Then,

for any $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$, we can write

$$\begin{aligned}
\sum_{i,j=1}^{m} c_i c_j ||\mathbf{x}_i - \mathbf{x}_j||^2 &= \sum_{i,j=1}^{m} c_i c_j (\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2 - 2\mathbf{x}_i \cdot \mathbf{x}_j) \\
&= \sum_{i,j=1}^{m} c_i c_j (\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2) - 2 \sum_{i=1}^{m} c_i \mathbf{x}_i \cdot \sum_{j=1}^{m} c_j \mathbf{x}_j \\
&= \sum_{i,j=1}^{m} c_i c_j (\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2) - 2 \Big\| \sum_{i=1}^{m} c_i \mathbf{x}_i \Big\|^2 \\
&\leq \sum_{i,j=1}^{m} c_i c_j (\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2) \\
&= \Big(\sum_{j=1}^{m} c_j \Big) \Big(\sum_{i=1}^{m} c_i (\|\mathbf{x}_i\|^2) \Big) + \Big(\sum_{i=1}^{m} c_i \Big) \Big(\sum_{j=1}^{m} c_j \|\mathbf{x}_j\|^2 \Big) = 0.
\end{aligned}$$

The next theorems show connections between NDS and PDS kernels. These results provide another series of tools for designing PDS kernels.

Theorem 6.16 *Let K' be defined for any x_0 by*

$$K'(x, x') = K(x, x_0) + K(x', x_0) - K(x, x') - K(x_0, x_0)$$

for all $x, x' \in \mathcal{X}$. Then K is NDS iff K' is PDS.

Proof: Assume that K' is PDS and define K such that for any x_0 we have $K(x, x') = K(x, x_0) + K(x_0, x') - K(x_0, x_0) - K'(x, x')$. Then for any $\mathbf{c} \in \mathbb{R}^m$ such that $\mathbf{c}^\top \mathbf{1} = 0$ and any set of points $(x_1, \ldots, x_m) \in \mathcal{X}^m$ we have

$$\begin{aligned}
\sum_{i,j=1}^{m} c_i c_j K(x_i, x_j) &= \Big(\sum_{i=1}^{m} c_i K(x_i, x_0) \Big) \Big(\sum_{j=1}^{m} c_j \Big) + \Big(\sum_{i=1}^{m} c_i \Big) \Big(\sum_{j=1}^{m} c_j K(x_0, x_j) \Big) \\
&\quad - \Big(\sum_{i=1}^{m} c_i \Big)^2 K(x_0, x_0) - \sum_{i,j=1}^{m} c_i c_j K'(x_i, x_j) = - \sum_{i,j=1}^{m} c_i c_j K'(x_i, x_j) \leq 0 \,.
\end{aligned}$$

which proves K is NDS.

Now, assume K is NDS and define K' for any x_0 as above. Then, for any $\mathbf{c} \in \mathbb{R}^m$, we can define $c_0 = -\mathbf{c}^\top \mathbf{1}$ and the following holds by the NDS property for any points $(x_1, \ldots, x_m) \in \mathcal{X}^m$ as well as x_0 defined previously: $\sum_{i,j=0}^{m} c_i c_j K(x_i, x_j) \leq 0$. This implies that

$$\begin{aligned}
&\Big(\sum_{i=0}^{m} c_i K(x_i, x_0) \Big) \Big(\sum_{j=0}^{m} c_j \Big) + \Big(\sum_{i=0}^{m} c_i \Big) \Big(\sum_{j=0}^{m} c_j K(x_0, x_j) \Big) \\
&\quad - \Big(\sum_{i=0}^{m} c_i \Big)^2 K(x_0, x_0) - \sum_{i,j=0}^{m} c_i c_j K'(x_i, x_j) = - \sum_{i,j=0}^{m} c_i c_j K'(x_i, x_j) \leq 0 \,,
\end{aligned}$$

which implies $2\sum_{i,j=1}^{m} c_i c_j K'(x_i, x_j) \geq -2c_0 \sum_{i=0}^{m} c_i K'(x_i, x_0) + c_0^2 K'(x_0, x_0) = 0$. The equality holds since $\forall x \in \mathcal{X}, K'(x, x_0) = 0$. □

This theorem is useful in showing other connections, such the following theorems, which are left as exercises (see exercises 6.17 and 6.18).

Theorem 6.17 *Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a symmetric kernel. Then, K is NDS iff $\exp(-tK)$ is a PDS kernel for all $t > 0$.*

The theorem provides another proof that Gaussian kernels are PDS: as seen earlier (Example 6.15), the squared distance $(x, x') \mapsto \|x - x'\|^2$ in $\mathbb{R}^N$ is NDS, thus $(x, x') \mapsto \exp(-t\|x - x'\|^2)$ is PDS for all $t > 0$.

Theorem 6.18 *Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be an NDS kernel such that for all $x, x' \in \mathcal{X}, K(x, x') = 0$ iff $x = x'$. Then, there exists a Hilbert space $\mathbb{H}$ and a mapping $\Phi\colon \mathcal{X} \to \mathbb{H}$ such that for all $x, x' \in \mathcal{X}$,*

$$K(x, x') = \|\Phi(x) - \Phi(x')\|^2.$$

Thus, under the hypothesis of the theorem, $\sqrt{K}$ defines a metric.

This theorem can be used to show that the kernel $(x, x') \mapsto \exp(-|x - x'|^p)$ in $\mathbb{R}$ is not PDS for $p > 2$. Otherwise, for any $t > 0$, $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$ and $\mathbf{c} \in \mathbb{R}^{m \times 1}$, we would have:

$$\sum_{i,j=1}^{m} c_i c_j e^{-t|x_i - x_j|^p} = \sum_{i,j=1}^{m} c_i c_j e^{-|t^{1/p} x_i - t^{1/p} x_j|^p} \geq 0.$$

This would imply that $(x, x') \mapsto |x - x'|^p$ is NDS for $p > 2$, which can be proven (via theorem 6.18) not to be valid.

6.5 Sequence kernels

The examples given in the previous sections, including the commonly used polynomial or Gaussian kernels, were all for PDS kernels over vector spaces. In many learning tasks found in practice, the input space $\mathcal{X}$ is not a vector space. The examples to classify in practice could be protein sequences, images, graphs, parse trees, finite automata, or other discrete structures which may not be directly given as vectors. PDS kernels provide a method for extending algorithms such as SVMs originally designed for a vectorial space to the classification of such objects. But, how can we define PDS kernels for these structures?

This section will focus on the specific case of *sequence kernels*, that is, kernels for sequences or strings. PDS kernels can be defined for other discrete structures in somewhat similar ways. Sequence kernels are particularly relevant to learning algorithms applied to computational biology or natural language processing, which are both important applications.

How can we define PDS kernels for sequences, which are similarity measures for sequences? One idea consists of declaring two sequences, e.g., two documents or two biosequences, as similar when they share common substrings or subsequences. One example could be the kernel between two sequences defined by the sum of the product of the counts of their common substrings. But which substrings should be used in that definition? Most likely, we would need some flexibility in the definition of the matching substrings. For computational biology applications, for example, the match could be imperfect. Thus, we may need to consider some number of mismatches, possibly gaps, or wildcards. More generally, we might need to allow various substitutions and might wish to assign different weights to common substrings to emphasize some matching substrings and deemphasize others.

As can be seen from this discussion, there are many different possibilities and we need a general framework for defining such kernels. In the following, we will introduce a general framework for sequence kernels, *rational kernels*, which will include all the kernels considered in this discussion. We will also describe a general and efficient algorithm for their computation and will illustrate them with some examples.

The definition of these kernels relies on that of *weighted transducers*. Thus, we start with the definition of these devices as well as some relevant algorithms.

6.5.1 Weighted transducers

Sequence kernels can be effectively represented and computed using *weighted transducers*. In the following definition, let Σ denote a finite input alphabet, Δ a finite output alphabet, and ϵ the *empty string* or null label, whose concatenation with any string leaves it unchanged.

Definition 6.19 *A* weighted transducer T *is a 7-tuple* $T = (\Sigma, \Delta, Q, I, F, E, \rho)$ *where* Σ *is a finite input alphabet,* Δ *a finite output alphabet,* Q *is a finite set of states,* $I \subseteq Q$ *the set of initial states,* $F \subseteq Q$ *the set of final states,* E *a finite multiset of transitions elements of* $Q \times (\Sigma \cup \{\epsilon\}) \times (\Delta \cup \{\epsilon\}) \times \mathbb{R} \times Q$*, and* $\rho : F \to \mathbb{R}$ *a final weight function mapping* F *to* $\mathbb{R}$*. The* size *of transducer* T *is the sum of its number of states and transitions and is denoted by* $|T|$*.*[7]

Thus, weighted transducers are finite automata in which each transition is labeled with both an input and an output label and carries some real-valued weight. Figure 6.4 shows an example of a weighted finite-state transducer. In this figure, the input and output labels of a transition are separated by a colon delimiter, and the weight is indicated after the slash separator. The initial states are represented by

[7] A multiset in the definition of the transitions is used to allow for the presence of several transitions from a state p to a state q with the same input and output label, and even the same weight, which may occur as a result of various operations.

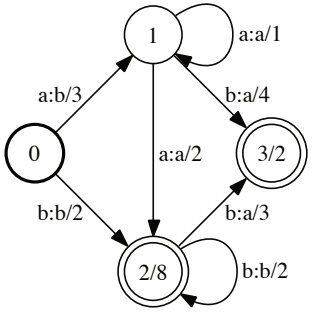


Figure 6.4
Example of weighted transducer.

a bold circle and final states by double circles. The final weight $\rho[q]$ at a final state q is displayed after the slash.

The input label of a path π is a string element of Σ^* obtained by concatenating input labels along π. Similarly, the output label of a path π is obtained by concatenating output labels along π. A path from an initial state to a final state is an *accepting path*. The weight of an accepting path is obtained by multiplying the weights of its constituent transitions and the weight of the final state of the path.

A weighted transducer defines a mapping from $\Sigma^* \times \Delta^*$ to $\mathbb{R}$. The weight associated by a weighted transducer T to a pair of strings $(x, y) \in \Sigma^* \times \Delta^*$ is denoted by $T(x, y)$ and is obtained by summing the weights of all accepting paths with input label x and output label y. For example, the transducer of figure 6.4 associates to the pair (aab, baa) the weight $3 \times 1 \times 4 \times 2 + 3 \times 2 \times 3 \times 2$, since there is a path with input label aab and output label baa and weight $3 \times 1 \times 4 \times 2$, and another one with weight $3 \times 2 \times 3 \times 2$.

The sum of the weights of all accepting paths of an acyclic transducer, that is a transducer T with no cycle, can be computed in linear time, that is $O(|T|)$, using a general *shortest-distance* or forward-backward algorithm. These are simple algorithms, but a detailed description would require too much of a digression from the main topic of this chapter.

Composition An important operation for weighted transducers is *composition*, which can be used to combine two or more weighted transducers to form more complex weighted transducers. As we shall see, this operation is useful for the creation and computation of sequence kernels. Its definition follows that of composition of relations. Given two weighted transducers $T_1 = (\Sigma, \Delta, Q_1, I_1, F_1, E_1, \rho_1)$ and $T_2 = (\Delta, \Omega, Q_2, I_2, F_2, E_2, \rho_2)$, the result of the composition of T_1 and T_2 is a

weighted transducer denoted by $T_1 \circ T_2$ and defined for all $x \in \Sigma^*$ and $y \in \Omega^*$ by

$$(T_1 \circ T_2)(x, y) = \sum_{z \in \Delta^*} T_1(x, z) \cdot T_2(z, y), \tag{6.19}$$

where the sum runs over all strings z over the alphabet Δ. Thus, composition is similar to matrix multiplication with infinite matrices.

There exists a general and efficient algorithm to compute the composition of two weighted transducers. In the absence of ϵs on the input side of T_1 or the output side of T_2, the states of $T_1 \circ T_2 = (\Sigma, \Delta, Q, I, F, E, \rho)$ can be identified with pairs made of a state of T_1 and a state of T_2, $Q \subseteq Q_1 \times Q_2$. Initial states are those obtained by pairing initial states of the original transducers, $I = I_1 \times I_2$, and similarly final states are defined by $F = Q \cap (F_1 \times F_2)$. The final weight at a state $(q_1, q_2) \in F_1 \times F_2$ is $\rho(q) = \rho_1(q_1)\rho_2(q_2)$, that is the product of the final weights at q_1 and q_2. Transitions are obtained by matching a transition of T_1 with one of T_2 from appropriate transitions of T_1 and T_2:

$$E = \biguplus_{\substack{(q_1, a, b, w_1, q_2) \in E_1 \\ (q_1', b, c, w_2, q_2') \in E_2}} \left\{ \Big((q_1, q_1'), a, c, w_1 \otimes w_2, (q_2, q_2') \Big) \right\}.$$

Here, $\uplus$ denotes the standard join operation of multisets as in $\{1, 2\} \uplus \{1, 3\} = \{1, 1, 2, 3\}$, to preserve the multiplicity of the transitions.

In the worst case, all transitions of T_1 leaving a state q_1 match all those of T_2 leaving state q_1', thus the space and time complexity of composition is quadratic: $O(|T_1||T_2|)$. In practice, such cases are rare and composition is very efficient. Figure 6.5 illustrates the algorithm in a particular case.

As illustrated by figure 6.6, when T_1 admits output ϵ labels or T_2 input ϵ labels, the algorithm just described may create redundant ϵ-paths, which would lead to an incorrect result. The weight of the matching paths of the original transducers would be counted p times, where p is the number of redundant paths in the result of composition. To avoid with this problem, all but one ϵ-path must be filtered out of the composite transducer. Figure 6.6 indicates in boldface one possible choice for that path, which in this case is the shortest. Remarkably, that filtering mechanism itself can be encoded as a finite-state transducer F (figure 6.6b).

To apply that filter, we need to first augment T_1 and T_2 with auxiliary symbols that make the semantics of ϵ explicit: let $\tilde{T}_1$ ($\tilde{T}_2$) be the weighted transducer obtained from T_1 (respectively T_2) by replacing the output (respectively input) ϵ labels with ϵ_2 (respectively ϵ_1) as illustrated by figure 6.6. Thus, matching with the symbol ϵ_1 corresponds to remaining at the same state of T_1 and taking a transition of T_2 with input ϵ. ϵ_2 can be described in a symmetric way. The filter transducer F disallows a matching (ϵ_2, ϵ_2) immediately after (ϵ_1, ϵ_1) since this can be done

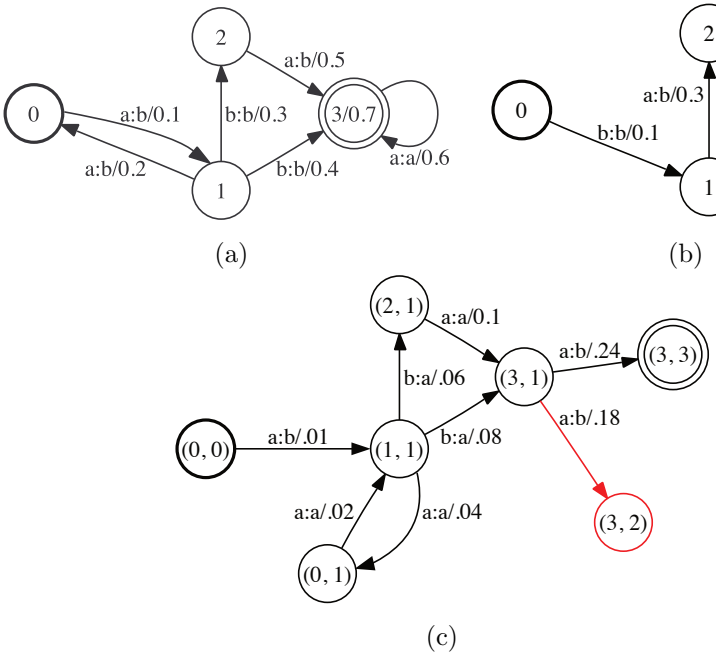


Figure 6.5
(a) Weighted transducer T_1. (b) Weighted transducer T_2. (c) Result of composition of T_1 and T_2, $T_1 \circ T_2$. Some states might be constructed during the execution of the algorithm that are not *co-accessible*, that is, they do not admit a path to a final state, e.g., $(3, 2)$. Such states and the related transitions (in red) can be removed by a trimming (or connection) algorithm in linear time.

instead via (ϵ_2, ϵ_1). By symmetry, it also disallows a matching (ϵ_1, ϵ_1) immediately after (ϵ_2, ϵ_2). In the same way, a matching (ϵ_1, ϵ_1) immediately followed by (ϵ_2, ϵ_1) is not permitted by the filter F since a path via the matchings $(\epsilon_2, \epsilon_1)(\epsilon_1, \epsilon_1)$ is possible. Similarly, $(\epsilon_2, \epsilon_2)(\epsilon_2, \epsilon_1)$ is ruled out. It is not hard to verify that the filter transducer F is precisely a finite automaton over pairs accepting the complement of the language

$$L = \sigma^*((\epsilon_1, \epsilon_1)(\epsilon_2, \epsilon_2) + (\epsilon_2, \epsilon_2)(\epsilon_1, \epsilon_1) + (\epsilon_1, \epsilon_1)(\epsilon_2, \epsilon_1) + (\epsilon_2, \epsilon_2)(\epsilon_2, \epsilon_1))\sigma^*,$$

where $\sigma = \{(\epsilon_1, \epsilon_1), (\epsilon_2, \epsilon_2), (\epsilon_2, \epsilon_1), x\}$. Thus, the filter F guarantees that exactly one ϵ-path is allowed in the composition of each ϵ sequences. To obtain the correct result of composition, it suffices then to use the ϵ-free composition algorithm already described and compute

$$\tilde{T}_1 \circ F \circ \tilde{T}_2. \tag{6.20}$$

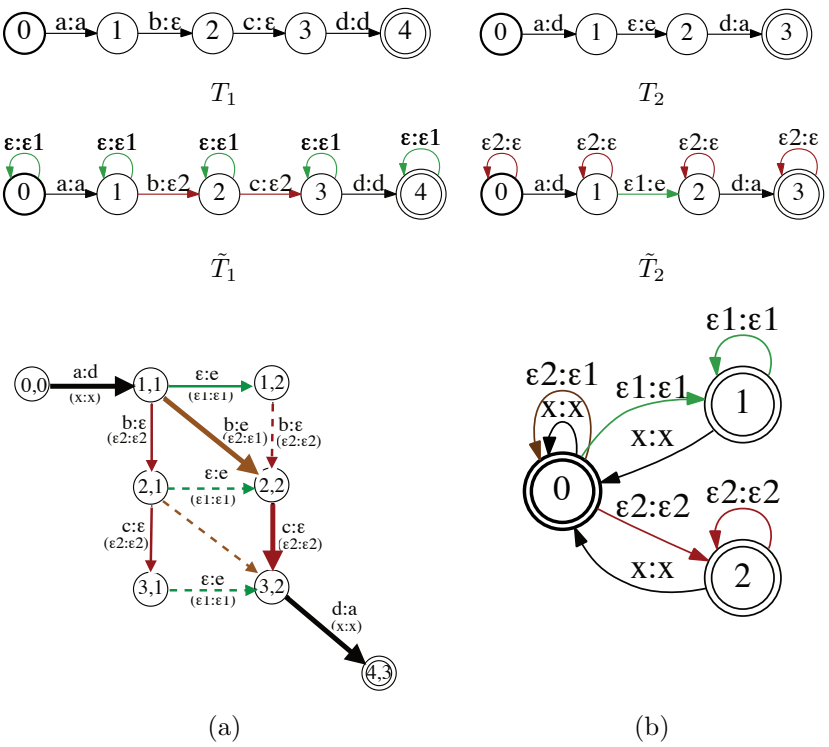


Figure 6.6
Redundant ϵ-paths in composition. All transition and final weights are equal to one. (a) A straightforward generalization of the ϵ-free case would generate all the paths from $(1, 1)$ to $(3, 2)$ when composing T_1 and T_2 and produce an incorrect results in non-idempotent semirings. (b) Filter transducer F. The shorthand x is used to represent an element of Σ.

Indeed, the two compositions in $\tilde{T}_1 \circ F \circ \tilde{T}_2$ no longer involve ϵs. Since the size of the filter transducer F is constant, the complexity of general composition is the same as that of ϵ-free composition, that is $O(|T_1||T_2|)$. In practice, the augmented transducers $\tilde{T}_1$ and $\tilde{T}_2$ are not explicitly constructed, instead the presence of the auxiliary symbols is simulated. Further filter optimizations help limit the number of non-coaccessible states created, for example, by examining more carefully the case of states with only outgoing non-ϵ-transitions or only outgoing ϵ-transitions.

6.5.2 Rational kernels

The following establishes a general framework for the definition of sequence kernels.

Definition 6.20 (Rational kernels) *A kernel* $K \colon \Sigma^* \times \Sigma^* \to \mathbb{R}$ *is said to be* rational *if it coincides with the mapping defined by some weighted transducer* U*:* $\forall x, y \in \Sigma^*, K(x, y) = U(x, y)$.

Note that we could have instead adopted a more general definition: instead of using weighted transducers, we could have used more powerful sequence mappings such as *algebraic transductions*, which are the functional counterparts of context-free languages, or even more powerful ones. However, an essential need for kernels is an efficient computation, and more complex definitions would lead to substantially more costly computational complexities for kernel computation. For rational kernels, there exists a general and efficient computation algorithm.

Computation We will assume that the transducer U defining a rational kernel K does not admit any ϵ-cycle with non-zero weight, otherwise the kernel value is infinite for all pairs. For any sequence x, let T_x denote a weighted transducer with just one accepting path whose input and output labels are both x and its weight equal to one. T_x can be straightforwardly constructed from x in linear time $O(|x|)$. Then, for any $x, y \in \Sigma^*$, $U(x, y)$ can be computed by the following two steps:

1. Compute $V = T_x \circ U \circ T_y$ using the composition algorithm in time $O(|U||T_x||T_y|)$.
2. Compute the sum of the weights of all accepting paths of V using a general shortest-distance algorithm in time $O(|V|)$.

By definition of composition, V is a weighted transducer whose accepting paths are precisely those accepting paths of U that have input label x and output label y. The second step computes the sum of the weights of these paths, that is, exactly $U(x, y)$. Since U admits no ϵ-cycle, V is acyclic, and this step can be performed in linear time. The overall complexity of the algorithm for computing $U(x, y)$ is then in $O(|U||T_x||T_y|)$. Since U is fixed for a rational kernel K and $|T_x| = O(|x|)$ for any x, this shows that the kernel values can be obtained in quadratic time $O(|x||y|)$. For some specific weighted transducers U, the computation can be more efficient, for example in $O(|x| + |y|)$ (see exercise 6.20).

PDS rational kernels For any transducer T, let T^{-1} denote the *inverse* of T, that is the transducer obtained from T by swapping the input and output labels of every transition. For all x, y, we have $T^{-1}(x, y) = T(y, x)$. The following theorem gives a general method for constructing a PDS rational kernel from an arbitrary weighted transducer.

Theorem 6.21 *For any weighted transducer $T = (\Sigma, \Delta, Q, I, F, E, \rho)$, the function $K = T \circ T^{-1}$ is a PDS rational kernel.*

Proof: By definition of composition and the inverse operation, for all $x, y \in \Sigma^*$,

$$K(x, y) = \sum_{z \in \Delta^*} T(x, z)\, T(y, z).$$

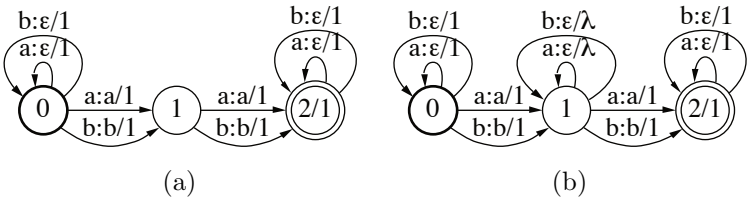


Figure 6.7
(a) Transducer T_{bigram} defining the bigram kernel $T_{\text{bigram}} \circ T_{\text{bigram}}^{-1}$ for $\Sigma = \{a, b\}$. (b) Transducer $T_{\text{gappy_bigram}}$ defining the gappy bigram kernel $T_{\text{gappy_bigram}} \circ T_{\text{gappy_bigram}}^{-1}$ with gap penalty $\lambda \in (0, 1)$.

K is the pointwise limit of the kernel sequence $(K_n)_{n \geq 0}$ defined by:

$$\forall n \in \mathbb{N}, \forall x, y \in \Sigma^*, \quad K_n(x, y) = \sum_{|z| \leq n} T(x, z)\, T(y, z),$$

where the sum runs over all sequences in Δ^* of length at most n. K_n is PDS since its corresponding kernel matrix $\mathbf{K}_n$ for any sample $(x_1, \ldots, x_m)$ is SPSD. This can be see form the fact that $\mathbf{K}_n$ can be written as $\mathbf{K}_n = \mathbf{A}\mathbf{A}^\top$ with $\mathbf{A} = (K_n(x_i, z_j))_{i \in [m], j \in [N]}$, where $z_1, \ldots, z_N$ is some arbitrary enumeration of the set of strings in Σ^* with length at most n. Thus, K is PDS as the pointwise limit of the sequence of PDS kernels $(K_n)_{n \in \mathbb{N}}$. □

The sequence kernels commonly used in computational biology, natural language processing, computer vision, and other applications are all special instances of rational kernels of the form $T \circ T^{-1}$. All of these kernels can be computed efficiently using the same general algorithm for the computational of rational kernels presented in the previous paragraph. Since the transducer $U = T \circ T^{-1}$ defining such PDS rational kernels has a specific form, there are different options for the computation of the composition $T_x \circ U \circ T_y$:

- compute $U = T \circ T^{-1}$ first, then $V = T_x \circ U \circ T_y$;
- compute $V_1 = T_x \circ T$ and $V_2 = T_y \circ T$ first, then $V = V_1 \circ V_2^{-1}$;
- compute first $V_1 = T_x \circ T$, then $V_2 = V_1 \circ T^{-1}$, then $V = V_2 \circ T_y$, or the similar series of operations with x and y permuted.

All of these methods lead to the same result after computation of the sum of the weights of all accepting paths, and they all have the same worst-case complexity. However, in practice, due to the sparsity of intermediate compositions, there may be substantial differences between their time and space computational costs. An alternative method based on an *n-way composition* can further lead to significantly more efficient computations.

Example 6.22 (Bigram and gappy bigram sequence kernels) Figure 6.7a shows a weighted transducer T_{bigram} defining a common sequence kernel, the *bigram sequence kernel*, for the specific case of an alphabet reduced to $\Sigma = \{a, b\}$. The bigram kernel associates to any two sequences x and y the sum of the product of the counts of all bigrams in x and y. For any sequence $x \in \Sigma^*$ and any bigram $z \in \{aa, ab, ba, bb\}$, $T_{\text{bigram}}(x, z)$ is exactly the number of occurrences of the bigram z in x. Thus, by definition of composition and the inverse operation, $T_{\text{bigram}} \circ T_{\text{bigram}}^{-1}$ computes exactly the bigram kernel.

Figure 6.7b shows a weighted transducer $T_{\text{gappy_bigram}}$ defining the so-called *gappy bigram kernel*. The gappy bigram kernel associates to any two sequences x and y the sum of the product of the counts of all gappy bigrams in x and y penalized by the length of their *gaps*. Gappy bigrams are sequences of the form *aua*, *aub*, *bua*, or *bub*, where $u \in \Sigma^*$ is called the gap. The count of a gappy bigram is multiplied by $\lambda^{|u|}$ for some fixed $\lambda \in (0, 1)$ so that gappy bigrams with longer gaps contribute less to the definition of the similarity measure. While this definition could appear to be somewhat complex, figure 6.7 shows that $T_{\text{gappy_bigram}}$ can be straightforwardly derived from T_{bigram}. The graphical representation of rational kernels helps understanding or modifying their definition.

Counting transducers The definition of most sequence kernels is based on the counts of some common patterns appearing in the sequences. In the examples just examined, these were bigrams or gappy bigrams. There exists a simple and general method for constructing a weighted transducer counting the number of occurrences of patterns and using them to define PDS rational kernels. Let $\mathcal{X}$ be a finite automaton representing the set of patterns to count. In the case of bigram kernels with $\Sigma = \{a, b\}$, $\mathcal{X}$ would be an automaton accepting exactly the set of strings $\{aa, ab, ba, bb\}$. Then, the weighted transducer of figure 6.8 can be used to compute exactly the number of occurrences of each pattern accepted by $\mathcal{X}$.

Theorem 6.23 *For any $x \in \Sigma^*$ and any sequence z accepted by $\mathcal{X}$, $T_{count}(x, z)$ is the number of occurrences of z in x.*

Proof: Let $x \in \Sigma^*$ be an arbitrary sequence and let z be a sequence accepted by $\mathcal{X}$. Since all accepting paths of T_{count} have weight one, $T_{\text{count}}(x, z)$ is equal to the number of accepting paths in T_{count} with input label x and output z.

Now, an accepting path π in T_{count} with input x and output z can be decomposed as $\pi = \pi_0\, \pi_{01}\, \pi_1$, where π_0 is a path through the loops of state 0 with input label some prefix x_0 of x and output label ϵ, π_{01} an accepting path from 0 to 1 with input and output labels equal to z, and π_1 a path through the self-loops of state 1 with input label a suffix x_1 of x and output ϵ. Thus, the number of such paths is exactly

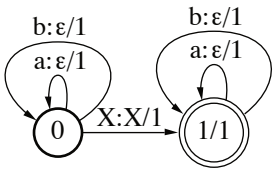


Figure 6.8
Counting transducer T_{count} for $\Sigma = \{a, b\}$. The "transition" $X : X/1$ stands for the weighted transducer created from the automaton $\mathfrak{X}$ by adding to each transition an output label identical to the existing label, and by making all transition and final weights equal to one.

the number of distinct ways in which we can write sequence x as $x = x_0 z x_1$, which is exactly the number of occurrences of z in x. □

The theorem provides a very general method for constructing PDS rational kernels $T_{\text{count}} \circ T_{\text{count}}^{-1}$ that are based on counts of some patterns that can be defined via a finite automaton, or equivalently a regular expression. Figure 6.8 shows the transducer for the case of an input alphabet reduced to $\Sigma = \{a, b\}$. The general case can be obtained straightforwardly by augmenting states 0 and 1 with other self-loops using other symbols than a and b. In practice, a lazy evaluation can be used to avoid the explicit creation of these transitions for all alphabet symbols and instead creating them on-demand based on the symbols found in the input sequence x. Finally, one can assign different weights to the patterns counted to emphasize or deemphasize some, as in the case of gappy bigrams. This can be done simply by changing the transitions weight or final weights of the automaton $\mathfrak{X}$ used in the definition of T_{count}.

6.6 Approximate kernel feature maps

In the previous sections, we have seen the benefits that kernel methods can provide by implicitly and efficiently mapping a learning problem from the input space $\mathfrak{X}$ to a richer feature space $\mathbb{H}$. One potential drawback when using kernel methods, is that the kernel function needs to be evaluated on all pairs of points in the training set. If this set contains a very large number of instances, then the $O(m^2)$ cost in memory and $O(m^2 C_K)$ cost in computation, where C_K is the cost of a single kernel function evaluation, may be prohibitive. Another consideration is the cost of making predictions with a trained model. Evaluating the kernelized function $h(x) = \sum_{i=1}^{m} \alpha_i K(x_i, x) + b$ requires $O(m)$ storage and $O(mC_K)$ computation cost (the exact amount of storage and number of operations depends on the number of support vectors).

Note that if we use explicit feature vectors $\mathbf{x} \in \mathbb{R}^N$, then the primal formulation of the SVM problem can be used for training. The primal formulation incurs only an $O(Nm)$ storage cost and evaluation requires only $O(N)$ storage and computation

Table 6.1
Examples of normalized shift-invariant kernels (defined over $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^N$) and their corresponding densities (defined over $\boldsymbol{\omega} \in \mathbb{R}^N$).

	$G(\mathbf{x} - \mathbf{x}')$	$p(\boldsymbol{\omega})$
Gaussian	$\exp\big(-\frac{\|\mathbf{x}-\mathbf{x}'\|^2}{2}\big)$	$(2\pi)^{\frac{-D}{2}} \exp\big(-\frac{\|\boldsymbol{\omega}\|^2}{2}\big)$
Laplacian	$\exp\big(-\|\mathbf{x} - \mathbf{x}'\|_1\big)$	$\prod_{i=1}^{N} \frac{1}{\pi(1+\omega_i^2)}$
Cauchy	$\prod_{i=1}^{N} \frac{2}{1+(x_i-x_i')^2}$	$\exp\big(-\|\boldsymbol{\omega}\|_1\big)$

time: $h(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$. However, these observations are only useful if $N < m$, which is likely not the case when considering the explicit feature maps $\Phi(x)$ induced by a kernel function. For example, given an input feature space of dimension N, the dimension of the kernel feature map for a polynomial kernel of degree d is $O(N^d)$. In the case of Gaussian kernels the explicit feature map dimension is infinite. So clearly using explicit kernel feature maps in general is not possible and again emphasizes that using kernel functions to compute inner products implicitly is crucial.

In this section we show that a compromise is possible by constructing *approximate kernel feature maps*. These are feature maps with a user-specified dimension D, $\Psi(x) \in \mathbb{R}^D$, which guarantee $\Psi(x) \cdot \Psi(x') \approx K(x, x')$ when using a sufficiently large dimension D. To begin, we state a classical result from the field of harmonic analysis.

Theorem 6.24 (Bochner's theorem) *A continuous kernel of the form $K(x, x') = G(x - x')$ defined over a locally compact set $\mathcal{X}$ is positive definite if and only if G is the Fourier transform of a non-negative measure. That is,*

$$G(x) = \int_{\mathcal{X}} p(\omega) e^{i\omega \cdot x} d\omega,$$

where p is a non-negative measure.

Kernels of the form $K(x, x') = G(x - x')$ are called *shift-invariant kernels*. Note that if the kernel is scaled such that $G(0) = 1$, then p is in fact a probability distribution. Several examples of such kernels and their corresponding distributions are displayed in table 6.1. The next proposition provides a simplified expression in the case of real-valued kernels.

Proposition 6.25 *Let K be a continuous real-valued shift-invariant kernel and let p denote its corresponding non-negative measure as in theorem 6.24. Furthermore, assume that for all $x \in \mathcal{X}$ we have $K(x, x) = 1$ so that p is a probability distribution. Then, the following identity holds:*

$$\mathop{\mathbb{E}}_{\omega \sim p} \Big[\big[\cos(\omega \cdot x), \sin(\omega \cdot x) \big]^\top \big[\cos(\omega \cdot x'), \sin(\omega \cdot x') \big] \Big] = K(x, x') \,.$$

Proof: First, since both K and p are real-valued, it suffices to consider only the real portion of e^{ix} when invoking theorem 6.24. Thus, using $\text{Re}[e^{ix}] = \text{Re}[\cos(x) + i\sin(x)] = \cos(x)$, we have

$$K(x, x') = \text{Re}[K(x, x')] = \int_{\mathcal{X}} p(\omega) \cos(\omega \cdot (x - x'))\, d\omega\,.$$

Next, by the standard trigonometric identity $\cos(a-b) = \cos(a)\cos(b)+\sin(a)\sin(b)$, we have

$$\begin{aligned}
&\int_{\mathcal{X}} p(\omega) \cos(\omega \cdot (x - x'))\, d\omega \\
&= \int_{\mathcal{X}} p(\omega) \big(\cos(\omega \cdot x)\cos(\omega \cdot x') + \sin(\omega \cdot x)\sin(\omega \cdot x') \big)\, d\omega \\
&= \mathop{\mathbb{E}}_{\omega \sim p} \Big[\big[\cos(\omega \cdot x), \sin(\omega \cdot x) \big]^\top \big[\cos(\omega \cdot x'), \sin(\omega \cdot x') \big] \Big]\,,
\end{aligned}$$

which completes the proof of the proposition. □

This proposition provides the motivation for a very simple method for generating for any $D \geq 1$, an approximate kernel map $\Psi \in \mathbb{R}^{2D}$, defined for all $x \in \mathcal{X}$ by

$$\Psi(x) = \sqrt{\frac{1}{D}} \Big[\cos(\omega_1 \cdot x), \sin(\omega_1 \cdot x), \ldots, \cos(\omega_D \cdot x), \sin(\omega_D \cdot x) \Big]^\top, \qquad (6.21)$$

where ω_is, $i = 1, \ldots, D$, are sampled i.i.d. according to the measure p over $\mathcal{X}$ corresponding to kernel K considered. Thus,

$$\Psi(x) \cdot \Psi(x') = \frac{1}{D} \sum_{i=1}^{D} \Big[\cos(\omega_i \cdot x), \sin(\omega_i \cdot x) \Big]^\top \Big[\cos(\omega_i \cdot x'), \sin(\omega_i \cdot x') \Big]$$

is the empirical analog of the expectation computed in proposition 6.25. The following theorem shows that this empirical estimate converges uniformly over all points in a compact domain $\mathcal{X}$ as D grows.

Lemma 6.26 *Let K be a continuously differentiable kernel function that satisfies the conditions of proposition 6.25 and has associated measure p. Furthermore, assume $\mathcal{X}$ is compact and let N denote its dimension, R denote the radius of the Euclidean ball containing $\mathcal{X}$, and $\sigma_p^2 = \mathbb{E}_{\omega \sim p}[\|\omega\|^2] < \infty$. Then, for $\Psi \in \mathbb{R}^D$ as defined in (6.21), the following holds for any $0 < r \leq 2R$ and $\epsilon > 0$:*

$$\mathbb{P}\left[\sup_{x, x' \in \mathcal{X}} \big| \Psi(x) \cdot \Psi(x') - K(x, x') \big| \geq \epsilon \right] \leq 2\mathcal{N}(2R, r) \exp\Big(-\frac{D\epsilon^2}{8} \Big) + \frac{4r\sigma_p}{\epsilon}\,.$$

Where the probability is with respect to the draws of $\omega \sim p$ and $\mathcal{N}(R, r)$ denotes the minimal number of balls of radius r needed to cover a ball of radius R.

Proof: Define $\mathcal{Z} = \{z \colon z = x - x',\ x, x' \in \mathcal{X}\}$ and note that $\mathcal{Z}$ is contained in a ball of radius at most $2R$. $\mathcal{Z}$ is a closed set since $\mathcal{X}$ is closed and thus $\mathcal{Z}$ is a compact set. For convenience, define $B = \mathcal{N}(2R, r)$ the number of balls of radius r needed

to cover $\mathcal{Z}$ and let z_j, for $j \in [B]$, denote the center of the covering balls. Thus, for any $z \in \mathcal{Z}$ there exists a j such that $z = z_j + \delta$ where $|\delta| < r$.

Next, define $S(z) = \Psi(x) \cdot \Psi(x') - K(x, x')$, where $z = x - x'$. Since S is continuously differentiable over the compact set $\mathcal{Z}$, it is L-Lipschitz with $L = \sup_{z \in \mathcal{Z}} \|\nabla S(z)\|$. Note that if $L < \frac{\epsilon}{2r}$ and for all $j \in [B]$ we have $|S(z_j)| < \frac{\epsilon}{2}$, then the following inequality holds for all $z = z_j + \delta \in \mathcal{Z}$:

$$|S(z)| = |S(z_j + \delta)| \leq L|z_j - (z_j + \delta)| + |S(z_j)| \leq rL + \frac{\epsilon}{2} < \epsilon\,. \tag{6.22}$$

The remainder of this proof bounds the probability of the events $L \geq \frac{\epsilon}{2r}$ and $|S(z_j)| \geq \frac{\epsilon}{2}$. Note, all following probabilities and expectations are with respect to the random variables $\omega_1, \ldots, \omega_D$.

To bound the probability of the first event, we use proposition 6.25 and the linearity of expectation, which implies the key fact $\mathbb{E}[\nabla(\Psi(x) \cdot \Psi(x'))] = \nabla K(x, x')$. We proceed with the following series of inequalities:

$$\begin{aligned}
\mathbb{E}[L^2] &= \mathbb{E}\left[\sup_{z \in \mathcal{Z}} \|\nabla S(z)\|^2\right] \\
&= \mathbb{E}\left[\sup_{x,x' \in \mathcal{X}} \|\nabla(\Psi(x) \cdot \Psi(x')) - \nabla K(x, x')\|^2\right] \\
&\leq 2\,\mathbb{E}\left[\sup_{x,x' \in \mathcal{X}} \|\nabla(\Psi(x) \cdot \Psi(x'))\|^2\right] + 2 \sup_{x,x' \in \mathcal{X}} \|\nabla K(x, x')\|^2 \\
&= 2\,\mathbb{E}\left[\sup_{x,x' \in \mathcal{X}} \|\nabla(\Psi(x) \cdot \Psi(x'))\|^2\right] + 2 \sup_{x,x' \in \mathcal{X}} \|\,\mathbb{E}[\nabla(\Psi(x) \cdot \Psi(x'))]\|^2 \\
&\leq 4\,\mathbb{E}\left[\sup_{x,x' \in \mathcal{X}} \|\nabla(\Psi(x) \cdot \Psi(x'))\|^2\right],
\end{aligned}$$

where the first inequality holds due to the the inequality $\|a + b\|^2 \leq 2\|a\|^2 + 2\|b\|^2$ (which follows from Jensen's inequality) and the subadditivity of the supremum function. The second inequality also holds by Jensen's inequality (applied twice) and again the subadditivity of supremum function. Furthermore, using a sum-difference trigonometric identity and computing the gradient with respect to $z = x - x'$, yield the following for any $x, x' \in \mathcal{X}$:

$$\begin{aligned}
\nabla(\Psi(x) \cdot \Psi(x')) &= \nabla\left(\frac{1}{D}\sum_{i=1}^{D} \cos(\omega_i \cdot x)\cos(\omega_i \cdot x') + \sin(\omega_i \cdot x)\sin(\omega_i \cdot x')\right) \\
&= \nabla\left(\frac{1}{D}\sum_{i=1}^{D} \cos(\omega_i \cdot (x - x'))\right) = \frac{1}{D}\sum_{i=1}^{D} \omega_i \sin(\omega_i \cdot (x - x'))\,.
\end{aligned}$$

Combining the two previous results gives

$$\begin{aligned}\mathbb{E}[L^2] &\leq 4\,\mathbb{E}\left[\sup_{x,x' \in \mathcal{X}} \left\| \frac{1}{D}\sum_{i=1}^{D} \omega_i \sin(\omega_i \cdot (x - x')) \right\|^2\right] \\ &\leq 4 \mathop{\mathbb{E}}_{\omega_1,\ldots,\omega_N}\left[\Big(\frac{1}{D}\sum_{i=1}^{D} \|\omega_i\|\Big)^2\right] \\ &\leq 4 \mathop{\mathbb{E}}_{\omega_1,\ldots,\omega_N}\left[\frac{1}{D}\sum_{i=1}^{D} \|\omega_i\|^2\right] = 4\,\mathop{\mathbb{E}}_{\omega}\left[\|\omega\|^2\right] = 4\sigma_p^2\,,\end{aligned}$$

which follows from the triangle inequality, $|\sin(\cdot)| \leq 1$, Jensen's inequality and the fact that the ω_is are drawn i.i.d. derive the final expression. Thus, we can bound the probability of the first event via Markov's inequality:

$$\mathbb{P}\left[L \geq \frac{\epsilon}{2r}\right] \leq \Big(\frac{4r\sigma_p}{\epsilon}\Big)^2. \tag{6.23}$$

To bound the probability of the second event, note that, by definition, $S(z)$ is a sum of D i.i.d. variables, each bounded in absolute value by $\frac{2}{D}$ (since, for all x and x', we have $|K(x,x')| \leq 1$ and $|\Psi(x) \cdot \Psi(x')| \leq 1$), and $\mathbb{E}[S(z)] = 0$. Thus, by Hoeffding's inequality and the union bound, we can write

$$\mathbb{P}\left[\exists j \in [B] \colon |S(z_j)| \geq \frac{\epsilon}{2}\right] \leq \sum_{i=1}^{B} \mathbb{P}\left[|S(z_j)| \geq \frac{\epsilon}{2}\right] \leq 2B \exp\Big(-\frac{D\epsilon^2}{8}\Big). \tag{6.24}$$

Finally, combining (6.22), (6.23), (6.24), and the definition of B we have

$$\mathbb{P}\left[\sup_{z \in \mathcal{Z}} |S(z)| \geq \epsilon\right] \leq 2\mathcal{N}(2R, r) \exp\Big(-\frac{D\epsilon^2}{8}\Big) + \Big(\frac{4r\sigma_p}{\epsilon}\Big)^2,$$

which completes the lemma. □

A key factor in the bound of the lemma is the covering number $\mathcal{N}(2R, r)$, which strongly depends on the dimension of the space N. In the following lemma, we make this dependency explicit for one especially simple case, although similar arguments hold for more general scenarios as well.

Lemma 6.27 *Let $\mathcal{X} \subset \mathbb{R}^N$ be a compact and let R denote the radius of the smallest enclosing ball. Then, the following inequality holds:*

$$\mathcal{N}(R, r) \leq \Big(\frac{3R}{r}\Big)^N.$$

Proof: First, by using the volume of balls in $\mathbb{R}^N$ we already see that $R^N/(r/3)^N = (3R/r)^N$ is a trivial upper bound on the number of balls of radius $r/3$ that can be packed into a ball of radius R without intersecting. Now, consider a maximal packing of at most $(3R/r)^N$ balls of radius $r/3$ into the ball of radius R. Every

point in the ball of radius R is at distance at most r from the center of at least one of the packing balls. If this were not true, we would be able to fit another ball into the packing, thereby contradicting the assumption that it is a maximal packing. Thus, if we grow the radius of the at most $(3R/r)^N$ balls to r, they will then provide a (not necessarily minimal) cover of the ball of radius R. □

Finally, by combining the two previous lemmas, we can present an explicit finite sample approximation bound.

Theorem 6.28 *Let K be a continuously differentiable kernel function that satisfies the conditions of proposition 6.25 and has associated measure p. Furthermore, assume $\sigma_p^2 = \mathbb{E}_{\omega\sim p}[\|\omega\|^2] < \infty$ and $\mathcal{X} \subset \mathbb{R}^N$. Let R denote the radius of the Euclidean ball containing $\mathcal{X}$. Then, for $\Psi \in \mathbb{R}^D$ as defined in* (6.21) *and any $0 < \epsilon \leq 32R\sigma_p$, the following holds*

$$\mathbb{P}\left[\sup_{x,x'\in\mathcal{X}} \left|\Psi(x)\cdot\Psi(x') - K(x,x')\right| \geq \epsilon\right] \leq \left(\frac{48R\sigma_p}{\epsilon}\right)^2 \exp\left(-\frac{D\epsilon^2}{4(N+2)}\right).$$

Proof: We use lemma 6.27 in conjunction with lemma 6.26 with the following choice of r:

$$r = \left\lceil \frac{2(6R)^N \exp(-\frac{D\epsilon^2}{8})}{\left(\frac{4\sigma_p}{\epsilon}\right)^2} \right\rceil^{\frac{2}{N+2}},$$

which results in the following expression

$$\mathbb{P}\left[\sup_{z\in\mathcal{Z}} |S(z)| \geq \epsilon\right] \leq 4\left(\frac{24R\sigma_p}{\epsilon}\right)^{\frac{2N}{N+2}} \exp\left(-\frac{D\epsilon^2}{4(N+2)}\right).$$

Since $32R\sigma_p/\epsilon \geq 1$, the exponent $\frac{2N}{N+2}$ can be replaced by 2, which completes the proof. □

The previous theorem provides the guarantee that a good estimate of the kernel function can be found, with high probability, by sampling a finite number of coordinates D. In particular, for an absolute error of at most ϵ it suffices to sample $D = O\left(\frac{N}{\epsilon^2}\log\left(\frac{R\sigma_p}{\epsilon}\right)\right)$ coordinates.

6.7 Chapter notes

The mathematical theory of PDS kernels in a general setting originated with the fundamental work of Mercer [1909] who also proved the equivalence of a condition similar to that of theorem 6.2 for continuous kernels with the PDS property. The connection between PDS and NDS kernels, in particular theorems 6.18 and 6.17, are due to Schoenberg [1938]. A systematic treatment of the theory of reproducing kernel Hilbert spaces was presented in a long and elegant paper by Aronszajn [1950]. For an excellent mathematical presentation of PDS kernels and positive definite

functions we refer the reader to Berg, Christensen, and Ressel [1984], which is also the source of several of the exercises given in this chapter.

The fact that SVMs could be extended by using PDS kernels was pointed out by Boser, Guyon, and Vapnik [1992]. The idea of kernel methods has been since then widely adopted in machine learning and applied in a variety of different tasks and settings. The following two books are in fact specifically devoted to the study of kernel methods: Schölkopf and Smola [2002] and Shawe-Taylor and Cristianini [2004]. The classical representer theorem is due to Kimeldorf and Wahba [1971]. A generalization to non-quadratic cost functions was stated by Wahba [1990]. The general form presented in this chapter was given by Schölkopf, Herbrich, Smola, and Williamson [2000].

Rational kernels were introduced by Cortes, Haffner, and Mohri [2004]. A general class of kernels, *convolution kernels*, was earlier introduced by Haussler [1999]. The convolution kernels for sequences described by Haussler [1999], as well as the pair-HMM string kernels described by Watkins [1999], are special instances of rational kernels. Rational kernels can be straightforwardly extended to define kernels for finite automata and even weighted automata [Cortes et al., 2004]. Cortes, Mohri, and Rostamizadeh [2008b] study the problem of *learning* rational kernels such as those based on counting transducers.

The composition of weighted transducers and the filter transducers in the presence of ϵ-paths are described in Pereira and Riley [1997], Mohri, Pereira, and Riley [2005], and Mohri [2009]. Composition can be further generalized to the *N-way composition* of weighted transducers [Allauzen and Mohri, 2009]. N-way composition of three or more transducers can substantially speed up computation, in particular for PDS rational kernels of the form $T \circ T^{-1}$. A generic *shortest-distance algorithm* which can be used with a large class of semirings and arbitrary queue disciplines is described by Mohri [2002]. A specific instance of that algorithm can be used to compute the sum of the weights of all paths as needed for the computation of rational kernels after composition. For a study of the class of languages linearly separable with rational kernels, see Cortes, Kontorovich, and Mohri [2007a].

The use of cosine-based approximate kernel feature maps was introduced by Rahimi and Recht [2007], as were the corresponding uniform convergence bounds, though their proofs were not complete. Sriperumbudur and Szabó [2015] gave an improved approximation bound that reduces the dependence on the radius of the data from $O(R^2)$ to only $O(\log(R))$. Bochner's theorem, which plays a central role in deriving an approximate map, is a classical result of harmonic analysis (for example, see Rudin [1990]). The general form of the theorem is due to Weil [1965], while Solomon Bochner recognized its importance to harmonic analysis.

6.8 Exercises

6.1 Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel, and let $\alpha\colon \mathcal{X} \to \mathbb{R}$ be a positive function. Show that the kernel K' defined for all $x, y \in \mathcal{X}$ by $K'(x,y) = \frac{K(x,y)}{\alpha(x)\alpha(y)}$ is a PDS kernel.

6.2 Show that the following kernels K are PDS:

(a) $K(x,y) = \cos(x-y)$ over $\mathbb{R} \times \mathbb{R}$.

(b) $K(x,y) = \cos(x^2 - y^2)$ over $\mathbb{R} \times \mathbb{R}$.

(c) For all integers $n > 0, K(\mathbf{x},\mathbf{y}) = \sum_{i=1}^{N} \cos^n(x_i^2 - y_i^2)$ over $\mathbb{R}^N \times \mathbb{R}^N$.

(d) $K(x,y) = (x+y)^{-1}$ over $(0,+\infty) \times (0,+\infty)$.

(e) $K(\mathbf{x},\mathbf{x}') = \cos \angle(\mathbf{x},\mathbf{x}')$ over $\mathbb{R}^n \times \mathbb{R}^n$, where $\angle(\mathbf{x},\mathbf{x}')$ is the angle between $\mathbf{x}$ and $\mathbf{x}'$.

(f) $\forall \lambda > 0,\ K(x,x') = \exp\big(-\lambda[\sin(x'-x)]^2\big)$ over $\mathbb{R} \times \mathbb{R}$.
(*Hint*: rewrite $[\sin(x'-x)]^2$ as the square of the norm of the difference of two vectors.)

(g) $\forall \sigma > 0, K(x,y) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|}{\sigma}}$ over $\mathbb{R}^N \times \mathbb{R}^N$.
(*Hint*: you could show that K is the normalized kernel of a kernel K' and show that K' is PDS using the following equality: $\|\mathbf{x}-\mathbf{y}\| = \frac{1}{2\Gamma(\frac{1}{2})} \int_0^{+\infty} \frac{1-e^{-t\|\mathbf{x}-\mathbf{y}\|^2}}{t^{\frac{3}{2}}}\, dt$ valid for all $\mathbf{x}, \mathbf{y}$.)

(h) $K(x,y) = \min(x,y) - xy$ over $[0,1] \times [0,1]$.
(*Hint*: you could consider the two integrals $\int_0^1 1_{t\in[0,x]} 1_{t\in[0,y]} dt$ and $\int_0^1 1_{t\in[x,1]} 1_{t\in[y,1]} dt$.)

(i) $K(x,x') = \frac{1}{\sqrt{1-(\mathbf{x}\cdot\mathbf{x}')}}$ over $\mathbf{x}, \mathbf{x}' \in \mathcal{X} = \{\mathbf{x} \in \mathbb{R}^N \colon \|\mathbf{x}\|_2 < 1\}$.
(*Hint*: one approach is to find an explicit expression of a feature mapping Φ by considering the Taylor expansion of the kernel function.)

(j) $\forall \sigma > 0, K(x,y) = \frac{1}{1+\frac{\|x-y\|^2}{\sigma^2}}$ over $\mathbb{R}^N \times \mathbb{R}^N$.
(*Hint*: the function $x \mapsto \int_0^{+\infty} e^{-sx} e^{-s} ds$ defined for all $x \geq 0$ could be useful for the proof.)

(k) $\forall \sigma > 0, K(x,y) = \exp\left(\frac{\sum_{i=1}^{N} \min(|x_i|,|y_i|)}{\sigma^2}\right)$ over $\mathbb{R}^N \times \mathbb{R}^N$.
(*Hint*: the function $(x_0,y_0) \mapsto \int_0^{+\infty} 1_{t\in[0,|x_0|]} 1_{t\in[0,|y_0|]} dt$ defined over $\mathbb{R} \times \mathbb{R}$ could be useful for the proof.)

6.3 Graph kernel. Let $G = (\mathcal{V}, \mathcal{E})$ be an undirected graph with vertex set $\mathcal{V}$ and edge set $\mathcal{E}$. $\mathcal{V}$ could represent a set of documents or biosequences and E the set of connections between them. Let $w[e] \in \mathbb{R}$ denote the weight assigned to edge $e \in \mathcal{E}$. The weight of a path is the product of the weights of its constituent edges. Show that the kernel K over $\mathcal{V} \times \mathcal{V}$ where $K(p, q)$ is the sum of the weights of all paths of length two between p and q is PDS (*Hint*: you could introduce the matrix $W = (W_{pq})$, where $W_{pq} = 0$ when there is no edge between p and q, W_{pq} equal to the weight of the edge between p and q otherwise).

6.4 Symmetric difference kernel. Let $\mathcal{X}$ be a finite set. Show that the kernel K defined over $2^{\mathcal{X}}$, the set of subsets of $\mathcal{X}$, by

$$\forall \mathcal{A}, \mathcal{B} \in 2^{\mathcal{X}}, K(\mathcal{A}, \mathcal{B}) = \exp\Big(-\frac{1}{2}|\mathcal{A}\Delta\mathcal{B}|\Big),$$

where $\mathcal{A}\Delta\mathcal{B}$ is the symmetric difference of $\mathcal{A}$ and $\mathcal{B}$ is PDS (*Hint*: you could use the fact that K is the result of the normalization of a kernel function K').

6.5 Set kernel. Let $\mathcal{X}$ be a finite set. Let K_0 be a PDS kernel over $\mathcal{X}$, show that K' defined by

$$\forall \mathcal{A}, \mathcal{B} \in 2^{\mathcal{X}}, K'(\mathcal{A}, \mathcal{B}) = \sum_{x \in \mathcal{A}, x' \in \mathcal{B}} K_0(x, x')$$

is a PDS kernel.

6.6 Show that the following kernels K are NDS:

(a) $K(x, y) = [\sin(x - y)]^2$ over $\mathbb{R} \times \mathbb{R}$.
(b) $K(x, y) = \log(x + y)$ over $(0, +\infty) \times (0, +\infty)$.

6.7 Define a *difference kernel* as $K(x, x') = |x - x'|$ for $x, x' \in \mathbb{R}$. Show that this kernel is not positive definite symmetric (PDS).

6.8 Is the kernel K defined over $\mathbb{R}^n \times \mathbb{R}^n$ by $K(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|^{3/2}$ PDS? Is it NDS?

6.9 Let $\mathcal{H}$ be a Hilbert space with the corresponding dot product $\langle \cdot, \cdot \rangle$. Show that the kernel K defined over $\mathcal{H} \times \mathcal{H}$ by $K(x, y) = 1 - \langle x, y \rangle$ is negative definite.

6.10 For any $p > 0$, let K_p be the kernel defined over $\mathbb{R}_+ \times \mathbb{R}_+$ by

$$K_p(x, y) = e^{-(x+y)^p}. \tag{6.25}$$

Show that K_p is positive definite symmetric (PDS) iff $p \leq 1$. (*Hint*: you can use the fact that if K is NDS, then for any $0 < \alpha \leq 1$, K^α is also NDS.)

6.11 Explicit mappings.

(a) Denote a data set $x_1, \ldots, x_m$ and a kernel $K(x_i, x_j)$ with a Gram matrix $\mathbf{K}$. Assuming $\mathbf{K}$ is positive semidefinite, then give a map $\Phi(\cdot)$ such that $K(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$.

(b) Show the converse of the previous statement, i.e., if there exists a mapping $\Phi(x)$ from input space to some Hilbert space, then the corresponding matrix $\mathbf{K}$ is positive semidefinite.

6.12 Explicit polynomial kernel mapping. Let K be a polynomial kernel of degree d, i.e., $K\colon \mathbb{R}^N \times \mathbb{R}^N \to \mathbb{R}$, $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$, with $c > 0$, Show that the dimension of the feature space associated to K is

$$\binom{N+d}{d}. \tag{6.26}$$

Write K in terms of kernels $k_i\colon (\mathbf{x}, \mathbf{x}') \mapsto (\mathbf{x} \cdot \mathbf{x}')^i$, $i \in \{0, \ldots, d\}$. What is the weight assigned to each k_i in that expression? How does it vary as a function of c?

6.13 High-dimensional mapping. Let $\Phi\colon \mathcal{X} \to \mathcal{H}$ be a feature mapping such that the dimension N of $\mathcal{H}$ is very large and let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel defined by

$$K(x, x') = \mathop{\mathbb{E}}_{i \sim \mathcal{D}} \left[[\Phi(x)]_i [\Phi(x')]_i \right], \tag{6.27}$$

where $[\Phi(x)]_i$ is the ith component of $\Phi(x)$ (and similarly for $\Phi'(x)$) and where $\mathcal{D}$ is a distribution over the indices i. We shall assume that $|[\Phi(x)]_i| \leq R$ for all $x \in \mathcal{X}$ and $i \in [N]$. Suppose that the only method available to compute $K(x, x')$ involved direct computation of the inner product (6.27), which would require $O(N)$ time. Alternatively, an approximation can be computed based on random selection of a subset I of the N components of $\Phi(x)$ and $\Phi(x')$ according to $\mathcal{D}$, that is:

$$K'(x, x') = \frac{1}{n} \sum_{i \in I} \mathcal{D}(i) [\Phi(x)]_i [\Phi(x')]_i, \tag{6.28}$$

where $|I| = n$.

(a) Fix x and x' in $\mathcal{X}$. Prove that

$$\mathop{\mathbb{P}}_{I \sim \mathcal{D}^n} [|K(x, x') - K'(x, x')| > \epsilon] \leq 2e^{\frac{-n\epsilon^2}{2r^2}}. \tag{6.29}$$

(*Hint*: use McDiarmid's inequality).

(b) Let $\mathbf{K}$ and $\mathbf{K}'$ be the kernel matrices associated to K and K'. Show that for any $\epsilon, \delta > 0$, for $n > \frac{r^2}{\epsilon^2} \log \frac{m(m+1)}{\delta}$, with probability at least $1 - \delta$, $|\mathbf{K}'_{ij} - \mathbf{K}_{ij}| \leq \epsilon$ for all $i, j \in [m]$.

6.14 Classifier based kernel. Let S be a training sample of size m. Assume that S has been generated according to some probability distribution $\mathcal{D}(x, y)$, where $(x, y) \in \mathcal{X} \times \{-1, +1\}$.

(a) Define the Bayes classifier $h^* \colon \mathcal{X} \to \{-1, +1\}$. Show that the kernel K^* defined by $K^*(x, x') = h^*(x)h^*(x')$ for any $x, x' \in \mathcal{X}$ is positive definite symmetric. What is the dimension of the natural feature space associated to K^*?

(b) Give the expression of the solution obtained using SVMs with this kernel. What is the number of support vectors? What is the value of the margin? What is the generalization error of the solution obtained? Under what condition are the data linearly separable?

(c) Let $h : \mathcal{X} \to \mathbb{R}$ be an arbitrary real-valued function. Under what condition on h is the kernel K defined by $K(x, x') = h(x)h(x')$, $x, x' \in \mathcal{X}$, positive definite symmetric?

6.15 Image classification kernel. For $\alpha \geq 0$, the kernel

$$K_\alpha \colon (\mathbf{x}, \mathbf{x}') \mapsto \sum_{k=1}^{N} \min(|x_k|^\alpha, |x'_k|^\alpha) \tag{6.30}$$

over $\mathbb{R}^N \times \mathbb{R}^N$ is used in image classification. Show that K_α is PDS for all $\alpha \geq 0$. To do so, proceed as follows.

(a) Use the fact that $(f, g) \mapsto \int_{t=0}^{+\infty} f(t)g(t)dt$ is an inner product over the set of measurable functions over $[0, +\infty)$ to show that $(x, x') \mapsto \min(x, x')$ is a PDS kernel. (*Hint*: associate an indicator function to x and another one to x'.)

(b) Use the result from (a) to first show that K_1 is PDS and similarly that K_α with other values of α is also PDS.

6.16 Fraud detection. To prevent fraud, a credit-card company decides to contact Professor Villebanque and provides him with a random list of several thousand fraudulent and non-fraudulent *events*. There are many different types of events, e.g., transactions of various amounts, changes of address or card-holder

information, or requests for a new card. Professor Villebanque decides to use SVMs with an appropriate kernel to help predict fraudulent events accurately. It is difficult for Professor Villebanque to define relevant features for such a diverse set of events. However, the risk department of his company has created a complicated method to estimate a probability $\mathbb{P}[U]$ for any event U. Thus, Professor Villebanque decides to make use of that information and comes up with the following kernel defined over all pairs of events (U, V):

$$K(U, V) = \mathbb{P}[U \wedge V] - \mathbb{P}[U]\,\mathbb{P}[V]. \tag{6.31}$$

Help Professor Villebanque show that his kernel is positive definite symmetric.

6.17 Relationship between NDS and PDS kernels. Prove the statement of theorem 6.17. (*Hint*: Use the fact that if K is PDS then $\exp(K)$ is also PDS, along with theorem 6.16.)

6.18 Metrics and Kernels. Let $\mathcal{X}$ be a non-empty set and $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a negative definite symmetric kernel such that $K(x, x) = 0$ for all $x \in \mathcal{X}$.

(a) Show that there exists a Hilbert space $\mathbb{H}$ and a mapping $\Phi(x)$ from $\mathcal{X}$ to $\mathbb{H}$ such that:

$$K(x, y) = ||\Phi(x) - \Phi(x')||^2 .$$

Assume that $K(x, x') = 0 \Rightarrow x = x'$. Use theorem 6.16 to show that $\sqrt{K}$ defines a metric on $\mathcal{X}$.

(b) Use this result to prove that the kernel $K(x, y) = \exp(-|x - x'|^p)$, $x, x' \in \mathbb{R}$, is not positive definite for $p > 2$.

(c) The kernel $K(x, x') = \tanh(a(x \cdot x') + b)$ was shown to be equivalent to a two-layer neural network when combined with SVMs. Show that K is not positive definite if $a < 0$ or $b < 0$. What can you conclude about the corresponding neural network when $a < 0$ or $b < 0$?

6.19 Sequence kernels. Let $\mathcal{X} = \{a, c, g, t\}$. To classify DNA sequences using SVMs, we wish to define a kernel between sequences defined over $\mathcal{X}$. We are given a finite set $\mathcal{I} \subset \mathcal{X}^*$ of non-coding regions (introns). For $x \in \mathcal{X}^*$, denote by $|x|$ the length of x and by $F(x)$ the set of factors of x, i.e., the set of subsequences of x with contiguous symbols. For any two strings $x, y \in \mathcal{X}^*$ define $K(x, y)$ by

$$K(x, y) = \sum_{z \in (F(x) \cap F(y)) - \mathcal{I}} \rho^{|z|}, \tag{6.32}$$

where $\rho \geq 1$ is a real number.

(a) Show that K is a rational kernel and that it is positive definite symmetric.

(b) Give the time and space complexity of the computation of $K(x, y)$ with respect to the size s of a minimal automaton representing $\mathcal{X}^* - \mathcal{I}$.

(c) Long common factors between x and y of length greater than or equal to n are likely to be important coding regions (exons). Modify the kernel K to assign weight $\rho_2^{|z|}$ to z when $|z| \geq n$, $\rho_1^{|z|}$ otherwise, where $1 \leq \rho_1 \ll \rho_2$. Show that the resulting kernel is still positive definite symmetric.

6.20 n-gram kernel. Show that for all $n \geq 1$, and any n-gram kernel K_n, $K_n(x, y)$ can be computed in linear time $O(|x| + |y|)$, for all $x, y \in \Sigma^*$ assuming n and the alphabet size are constants.

6.21 Mercer's condition. Let $\mathcal{X} \subset \mathbb{R}^N$ be a compact set and $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ a continuous kernel function. Prove that if K verifies Mercer's condition (theorem 6.2), then it is PDS. (*Hint*: assume that K is not PDS and consider a set $\{x_1, \ldots, x_m\} \subseteq \mathcal{X}$ and a column-vector $c \in \mathbb{R}^{m \times 1}$ such that $\sum_{i,j=1}^m c_i c_j K(x_i, x_j) < 0$.)

6.22 Anomaly detection. For this problem, consider a Hilbert space $\mathbb{H}$ with associated feature map $\Phi\colon \mathcal{X} \to \mathbb{H}$ and kernel $K(x, x') = \Phi(x) \cdot \Phi(x')$.

(a) First, let us consider finding the smallest enclosing sphere for a given sample $S = (x_1, \ldots, x_m)$. Let $\mathbf{c} \in \mathbb{H}$ denote the center of the sphere and let $r > 0$ be its radius, then clearly the following optimization problem searches for the smallest enclosing sphere:

$$\min_{r>0, \mathbf{c} \in \mathbb{H}} r^2$$
$$\text{subject to: } \forall i \in [m], \|\Phi(x_i) - \mathbf{c}\|^2 \leq r^2.$$

Show how to derive the equivalent dual optimization

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^m \alpha_i K(x_i, x_i) - \sum_{i,j=1}^m \alpha_i \alpha_j K(x_i, x_j)$$
$$\text{subject to: } \boldsymbol{\alpha} \geq \mathbf{0} \ \wedge \ \sum_{i=1}^m \alpha_i = 1\,,$$

and prove that the optimal solution satisfies $\mathbf{c} = \sum_i \alpha_i \Phi(x_i)$. In other words the location of the sphere only depends on points x_i with non-zero coefficients α_i. These points are analogous to the support vectors of SVM.

(b) Consider the hypothesis class

$$\mathcal{H} = \{x \mapsto r^2 - \|\Phi(x) - \mathbf{c}\|^2 \colon \|\mathbf{c}\| \leq \Lambda,\ 0 < r \leq R\}\,.$$

A hypothesis $h \in \mathcal{H}$ can be used to detect anomalies in data, where $h(x) \geq 0$ indicates a non-anomalous point and $h(x) < 0$ indicates an anomaly.

Show that if $\sup_x \|\Phi(x)\| \leq M$, then the solution to the optimization problem in part (a) is found in the hypothesis set $\mathcal{H}$ with $\Lambda \leq M$ and $R \leq 2M$.

(c) Let $\mathcal{D}$ denote the distribution of non-outlier points define the associated expected loss $R(h) = \mathbb{E}_{x\sim\mathcal{D}}[1_{h(x)<0}]$ and empirical margin loss $\widehat{R}_{S,\rho}(h) = \sum_{i=1}^m \frac{1}{m}\Phi_\rho(h(x_i)) \leq \sum_{i=1}^m \frac{1}{m} 1_{h(x_i)<\rho}$. These losses measure errors caused by *false-positive* predictions, i.e. errors caused by incorrectly labeling a point anomalous.

i. Show that the empirical Rademacher complexity for the hypothesis class $\mathcal{H}$ from part (b) can be upper bound as follows:

$$\widehat{\mathfrak{R}}_S(\mathcal{H}) \leq \frac{R^2 + \Lambda^2}{\sqrt{m}} + \Lambda\sqrt{\mathrm{Tr}[\mathbf{K}]}\,,$$

where $\mathbf{K}$ is the kernel matrix constructed with the sample.

ii. Prove that with probability at least $1-\delta$, the following holds for all $h \in \mathcal{H}$ and $\rho \in (0, 1]$:

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{4}{\rho}\Big(\frac{R^2 + \Lambda^2}{\sqrt{m}} + \Lambda\sqrt{\mathrm{Tr}[\mathbf{K}]}\Big) + \sqrt{\frac{\log\log_2 \frac{2}{\rho}}{m}} + 3\sqrt{\frac{\log\frac{4}{\delta}}{2m}}\,.$$

(d) Just as in the case of soft-margin SVM, we can also define a soft-margin objective for the smallest enclosing sphere that allows us tune the sensitivity to outliers in the training set by adjusting a regularization parameter C:

$$\min_{r>0,\mathbf{c}\in\mathbb{H},\xi} r^2 + C\sum_{i=1}^m \xi_i$$
$$\text{subject to: } \forall i \in [m], \|\Phi(x_i) - \mathbf{c}\|^2 \leq r^2 + \xi_i \ \wedge\ \xi_i \geq 0.$$

Show that the equivalent dual formulation of this problem is

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i K(x_i, x_i) - \sum_{i,j=1}^{m} \alpha_i \alpha_j K(x_i, x_j)$$
$$\text{subject to: } \mathbf{0} \leq \boldsymbol{\alpha} \leq C\mathbf{1} \ \wedge \ \sum_{i=1}^{m} \alpha_i = 1\,,$$

and that at the optimum we have $\mathbf{c} = \sum_{i=1}^{m} \alpha_i \Phi(x_i)$.

7 Boosting

Ensemble methods are general techniques in machine learning for combining several predictors to create a more accurate one. This chapter studies an important family of ensemble methods known as *boosting*, and more specifically the *AdaBoost* algorithm. This algorithm has been shown to be very effective in practice in some scenarios and is based on a rich theoretical analysis. We first introduce AdaBoost, show how it can rapidly reduce the empirical error as a function of the number of rounds of boosting, and point out its relationship with some known algorithms. Next, we present a theoretical analysis of the generalization properties of AdaBoost based on the VC-dimension of its hypothesis set and then based on the notion of margin. The margin theory developed in this context can be applied to other similar ensemble algorithms. A game-theoretic interpretation of AdaBoost further helps analyzing its properties and revealing the equivalence between the weak learning assumption and a separability condition. We end with a discussion of AdaBoost's benefits and drawbacks.

7.1 Introduction

It is often difficult, for a non-trivial learning task, to directly devise an accurate algorithm satisfying the strong PAC-learning requirements of chapter 2. But, there can be more hope for finding simple predictors guaranteed only to perform slightly better than random. The following gives a formal definition of such *weak learners*. As in the PAC-learning chapter, we let n be a number such that the computational cost of representing any element $x \in \mathcal{X}$ is at most $O(n)$ and denote by size(c) the maximal cost of the computational representation of $c \in \mathcal{C}$.

Definition 7.1 (Weak learning) *A concept class $\mathcal{C}$ is said to be* weakly PAC-learnable *if there exists an algorithm $\mathcal{A}$, $\gamma > 0$, and a polynomial function poly$(\cdot, \cdot, \cdot)$ such that for any $\delta > 0$, for all distributions $\mathcal{D}$ on $\mathcal{X}$ and for any target concept $c \in \mathcal{C}$,*

AdaBoost($S = ((x_1, y_1), \ldots, (x_m, y_m))$)

for $i \leftarrow 1$ **to** m **do**
  $\mathcal{D}_1(i) \leftarrow \frac{1}{m}$
for $t \leftarrow 1$ **to** T **do**
  $h_t \leftarrow$ base classifier in $\mathcal{H}$ with small error $\epsilon_t = \mathbb{P}_{i \sim \mathcal{D}_t}\left[h_t(x_i) \neq y_i\right]$
  $\alpha_t \leftarrow \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$
  $Z_t \leftarrow 2\left[\epsilon_t(1-\epsilon_t)\right]^{\frac{1}{2}}$ ▷ normalization factor
  for $i \leftarrow 1$ **to** m **do**
    $\mathcal{D}_{t+1}(i) \leftarrow \frac{\mathcal{D}_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$
$f \leftarrow \sum_{t=1}^{T} \alpha_t h_t$
return f

Figure 7.1
AdaBoost algorithm for a base classifier set $\mathcal{H} \subseteq \{-1, +1\}^{\mathcal{X}}$.

the following holds for any sample size $m \geq \mathit{poly}(1/\delta, n, \mathit{size}(c))$*:*

$$\mathop{\mathbb{P}}_{S \sim \mathcal{D}^m}\left[R(h_S) \leq \frac{1}{2} - \gamma\right] \geq 1 - \delta\,, \tag{7.1}$$

where h_S *is the hypothesis returned by algorithm* $\mathcal{A}$ *when trained on sample* S*. When such an algorithm* $\mathcal{A}$ *exists, it is called a* weak learning algorithm *for* $\mathcal{C}$ *or a* weak learner*. The hypotheses returned by a weak learning algorithm are called* base classifiers*.*

The key idea behind boosting techniques is to use a weak learning algorithm to build a *strong learner*, that is, an accurate PAC-learning algorithm. To do so, boosting techniques use an ensemble method: they combine different base classifiers returned by a weak learner to create a more accurate predictor. But which base classifiers should be used and how should they be combined? The next section addresses these questions by describing in detail one of the most prevalent and successful boosting algorithms, AdaBoost.

7.2 AdaBoost

We denote by $\mathcal{H}$ the hypothesis set out of which the base classifiers are selected, which we will sometimes refer to as the *base classifier set*. Figure 7.1 gives the

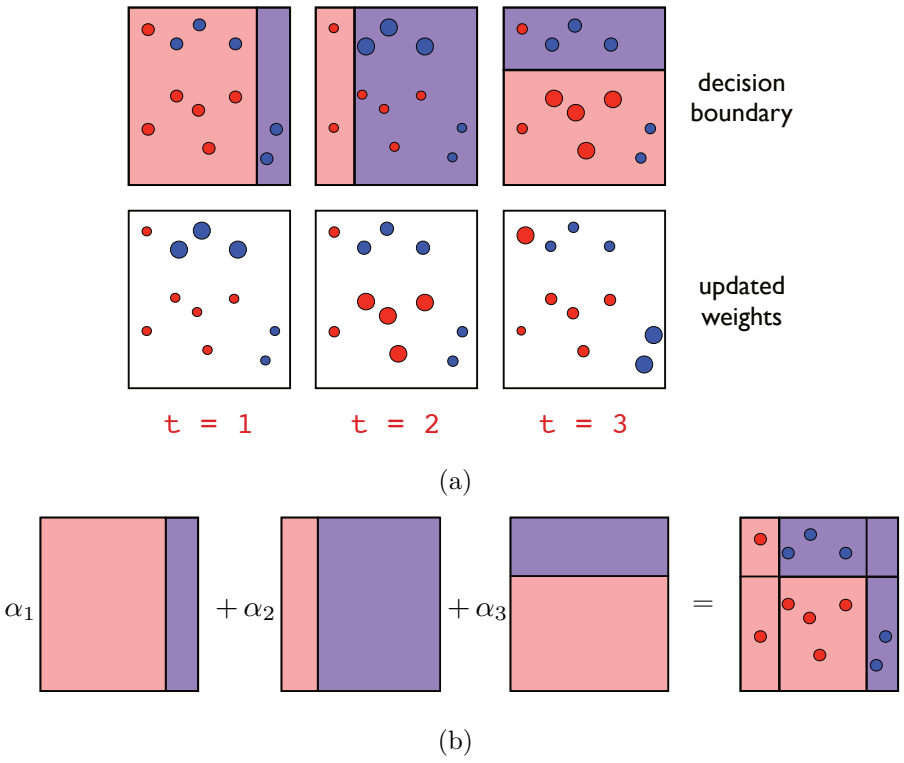


Figure 7.2
Example of AdaBoost with axis-aligned hyperplanes as base classifiers. (a) The top row shows decision boundaries at each boosting round. The bottom row shows how weights are updated at each round, with incorrectly (resp., correctly) points given increased (resp., decreased) weights. (b) Visualization of final classifier, constructed as a non-negative linear combination of base classifiers.

pseudocode of AdaBoost in the case where the base classifiers are functions mapping from $\mathcal{X}$ to $\{-1, +1\}$, thus $\mathcal{H} \subseteq \{-1, +1\}^{\mathcal{X}}$.

The algorithm takes as input a labeled sample $S = ((x_1, y_1), \ldots, (x_m, y_m))$, with $(x_i, y_i) \in \mathcal{X} \times \{-1, +1\}$ for all $i \in [m]$, and maintains a distribution over the indices $\{1, \ldots, m\}$. Initially (lines 1-2), the distribution is uniform ($\mathcal{D}_1$). At each *round of boosting*, that is each iteration $t \in [T]$ of the loop 3–8, a new base classifier $h_t \in \mathcal{H}$ is selected that minimizes the error on the training sample weighted by the distribution $\mathcal{D}_t$:

$$h_t \in \operatorname*{argmin}_{h \in \mathcal{H}} \mathop{\mathbb{P}}_{i \sim \mathcal{D}_t} \left[h(x_i) \neq y_i \right] = \operatorname*{argmin}_{h \in \mathcal{H}} \sum_{i=1}^{m} \mathcal{D}_t(i) 1_{h(x_i) \neq y_i}.$$

Z_t is simply a normalization factor to ensure that the weights $\mathcal{D}_{t+1}(i)$ sum to one. The precise reason for the definition of the coefficient α_t will become clear later. For now, observe that if ϵ_t, the error of the base classifier, is less than $\frac{1}{2}$, then $\frac{1-\epsilon_t}{\epsilon_t} > 1$ and α_t is positive ($\alpha_t > 0$). Thus, the new distribution $\mathcal{D}_{t+1}$ is defined from $\mathcal{D}_t$ by substantially increasing the weight on i if point x_i is incorrectly classified ($y_i h_t(x_i) < 0$), and, on the contrary, decreasing it if x_i is correctly classified. This has the effect of focusing more on the points incorrectly classified at the next round of boosting, less on those correctly classified by h_t.

After T rounds of boosting, the classifier returned by AdaBoost is based on the sign of function f, which is a non-negative linear combination of the base classifiers h_t. The weight α_t assigned to h_t in that sum is a logarithmic function of the ratio of the accuracy $1 - \epsilon_t$ and error ϵ_t of h_t. Thus, more accurate base classifiers are assigned a larger weight in that sum. Figure 7.2 illustrates the AdaBoost algorithm. The size of the points represents the distribution weight assigned to them at each boosting round.

For any $t \in [T]$, we will denote by f_t the linear combination of the base classifiers after t rounds of boosting: $f_t = \sum_{s=1}^{t} \alpha_s h_s$. In particular, we have $f_T = f$. The distribution $\mathcal{D}_{t+1}$ can be expressed in terms of f_t and the normalization factors Z_s, $s \in [t]$, as follows:

$$\forall i \in [m], \quad \mathcal{D}_{t+1}(i) = \frac{e^{-y_i f_t(x_i)}}{m \prod_{s=1}^{t} Z_s}. \tag{7.2}$$

We will make use of this identity several times in the proofs of the following sections. It can be shown straightforwardly by repeatedly expanding the definition of the distribution over the point x_i:

$$\begin{aligned}\mathcal{D}_{t+1}(i) = \frac{\mathcal{D}_t(i) e^{-\alpha_t y_i h_t(x_i)}}{Z_t} &= \frac{\mathcal{D}_{t-1}(i) e^{-\alpha_{t-1} y_i h_{t-1}(x_i)} e^{-\alpha_t y_i h_t(x_i)}}{Z_{t-1} Z_t} \\ &= \frac{e^{-y_i \sum_{s=1}^{t} \alpha_s h_s(x_i)}}{m \prod_{s=1}^{t} Z_s}.\end{aligned}$$

The AdaBoost algorithm can be generalized in several ways:

- Instead of a hypothesis with minimal weighted error, h_t can be more generally the base classifier returned by a weak learning algorithm trained on $\mathcal{D}_t$;
- The range of the base classifiers could be $[-1, +1]$, or more generally a bounded subset of $\mathbb{R}$. The coefficients α_t can then be different and may not even admit a closed form. In general, they are chosen to minimize an upper bound on the empirical error, as discussed in the next section. Of course, in that general case, the hypotheses h_t are not binary *classifiers*, but their sign could define the label, and their magnitude could be interpreted as a measure of confidence.

In rest of this chapter, the range of the base classifiers in $\mathcal{H}$ will be assumed to be included in $[-1, +1]$. We now further analyze the properties of AdaBoost and discuss its typical use in practice.

7.2.1 Bound on the empirical error

We first show that the empirical error of AdaBoost decreases exponentially fast as a function of the number of rounds of boosting.

Theorem 7.2 *The empirical error of the classifier returned by AdaBoost verifies:*

$$\widehat{R}_S(f) \leq \exp\left[-2\sum_{t=1}^{T}\left(\frac{1}{2}-\epsilon_t\right)^2\right]. \tag{7.3}$$

Furthermore, if for all $t \in [T]$, $\gamma \leq \left(\frac{1}{2}-\epsilon_t\right)$, then

$$\widehat{R}_S(f) \leq \exp(-2\gamma^2 T)\,. \tag{7.4}$$

Proof: Using the general inequality $1_{u\leq 0} \leq \exp(-u)$ valid for all $u \in \mathbb{R}$ and identity 7.2, we can write:

$$\widehat{R}_S(f) = \frac{1}{m}\sum_{i=1}^{m} 1_{y_i f(x_i)\leq 0} \leq \frac{1}{m}\sum_{i=1}^{m} e^{-y_i f(x_i)} = \frac{1}{m}\sum_{i=1}^{m}\left[m\prod_{t=1}^{T} Z_t\right]\mathcal{D}_{T+1}(i) = \prod_{t=1}^{T} Z_t.$$

Since for all $t \in [T]$, Z_t is a normalization factor, it can be expressed in terms of ϵ_t by:

$$\begin{aligned} Z_t = \sum_{i=1}^{m}\mathcal{D}_t(i)e^{-\alpha_t y_i h_t(x_i)} &= \sum_{i:y_i h_t(x_i)=+1}\mathcal{D}_t(i)e^{-\alpha_t} + \sum_{i:y_i h_t(x_i)=-1}\mathcal{D}_t(i)e^{\alpha_t} \\ &= (1-\epsilon_t)e^{-\alpha_t} + \epsilon_t e^{\alpha_t} \\ &= (1-\epsilon_t)\sqrt{\frac{\epsilon_t}{1-\epsilon_t}} + \epsilon_t\sqrt{\frac{1-\epsilon_t}{\epsilon_t}} = 2\sqrt{\epsilon_t(1-\epsilon_t)}\,. \end{aligned}$$

Thus, the product of the normalization factors can be expressed and upper bounded as follows:

$$\begin{aligned} \prod_{t=1}^{T} Z_t = \prod_{t=1}^{T} 2\sqrt{\epsilon_t(1-\epsilon_t)} = \prod_{t=1}^{T}\sqrt{1-4\left(\tfrac{1}{2}-\epsilon_t\right)^2} &\leq \prod_{t=1}^{T}\exp\left[-2\left(\tfrac{1}{2}-\epsilon_t\right)^2\right] \\ &= \exp\left[-2\sum_{t=1}^{T}\left(\tfrac{1}{2}-\epsilon_t\right)^2\right], \end{aligned}$$

where the inequality follows from the inequality $1 - x \leq e^{-x}$ valid for all $x \in \mathbb{R}$. □

Note that the value of γ, which is known as the *edge*, and the accuracy of the base classifiers do not need to be known to the algorithm. The algorithm adapts to their

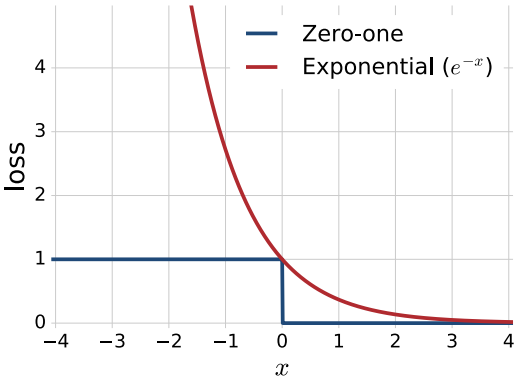


Figure 7.3
Visualization of the zero-one loss (blue) and the convex and differentiable upper bound on the zero-one loss (red) that is optimized by AdaBoost.

accuracy and defines a solution based on these values. This is the source of the extended name of AdaBoost: *adaptive boosting*.

The proof of theorem 7.2 reveals several other important properties. First, observe that α_t is the minimizer of the function $\varphi\colon \alpha \mapsto (1-\epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha}$. Indeed, φ is convex and differentiable, and setting its derivative to zero yields:

$$\varphi'(\alpha) = -(1-\epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha} = 0 \Leftrightarrow (1-\epsilon_t)e^{-\alpha} = \epsilon_t e^{\alpha} \Leftrightarrow \alpha = \frac{1}{2}\log\frac{1-\epsilon_t}{\epsilon_t}. \quad (7.5)$$

Thus, α_t is chosen to minimize $Z_t = \varphi(\alpha_t)$ and, in light of the bound $\widehat{R}_S(f) \leq \prod_{t=1}^{T} Z_t$ shown in the proof, these coefficients are selected to minimize an upper bound on the empirical error. In fact, for base classifiers whose range is $[-1,+1]$ or $\mathbb{R}$, α_t can be chosen in a similar fashion to minimize Z_t, and this is the way AdaBoost is extended to these more general cases.

Observe also that the equality $(1-\epsilon_t)e^{-\alpha_t} = \epsilon_t e^{\alpha_t}$ just shown in (7.5) implies that at each iteration, AdaBoost assigns equal distribution mass to correctly and incorrectly classified instances, since $(1-\epsilon_t)e^{-\alpha_t}$ is the total distribution assigned to correctly classified points and $\epsilon_t e^{\alpha_t}$ that of incorrectly classified ones. This may seem to contradict the fact that AdaBoost increases the weights of incorrectly classified points and decreases that of others, but there is in fact no inconsistency: the reason is that there are always fewer incorrectly classified points, since the base classifier's accuracy is better than random.

7.2.2 Relationship with coordinate descent

AdaBoost was originally designed to address the theoretical question of whether a weak learning algorithm could be used to derive a strong learning one. Here,

we will show that it coincides in fact with a very simple algorithm, which consists of applying a general coordinate descent technique to a convex and differentiable objective function.

For simplicity, in this section, we assume that the base classifier set $\mathcal{H}$ is finite, with cardinality N: $\mathcal{H} = \{h_1, \ldots, h_N\}$. An ensemble function f such as the one returned by AdaBoost can then be written as $f = \sum_{j=1}^{N} \bar{\alpha}_j h_j$, with $\bar{\alpha}_j \geq 0$. Given a labeled sample $S = ((x_1, y_1), \ldots, (x_m, y_m))$, let F be the objective function defined for all $\bar{\boldsymbol{\alpha}} = (\bar{\alpha}_1, \ldots, \bar{\alpha}_N) \in \mathbb{R}^N$ by

$$F(\bar{\boldsymbol{\alpha}}) = \frac{1}{m} \sum_{i=1}^{m} e^{-y_i f(x_i)} = \frac{1}{m} \sum_{i=1}^{m} e^{-y_i \sum_{j=1}^{N} \bar{\alpha}_j h_j(x_i)}. \tag{7.6}$$

Since the exponential loss $u \mapsto e^{-u}$ is an upper bound on the zero-one loss $u \mapsto 1_{u \leq 0}$ (see figure 7.3), F is an upper bound on the empirical error:

$$\widehat{R}_S(f) = \frac{1}{m} \sum_{i=1}^{m} 1_{y_i f(x_i) \leq 0} \leq \frac{1}{m} \sum_{i=1}^{m} e^{-y_i f(x_i)}. \tag{7.7}$$

F is a convex function of $\bar{\boldsymbol{\alpha}}$ since it is a sum of convex functions, each obtained by composition of the (convex) exponential function with an affine function of $\bar{\boldsymbol{\alpha}}$. F is also differentiable since the exponential function is differentiable. We will show that F is the objective function minimized by AdaBoost.

Different convex optimization techniques can be used to minimize F. Here, we will use a variant of the coordinate descent technique. Coordinate descent is applied over T rounds. Let $\bar{\boldsymbol{\alpha}}_0 = \mathbf{0}$ and let $\bar{\boldsymbol{\alpha}}_t$ denote the parameter vector at the end of iteration t. At each round $t \in [T]$, a direction $\mathbf{e}_k$ corresponding to the kth coordinate of $\bar{\boldsymbol{\alpha}}$ in $\mathbb{R}^N$ is selected, as well as a step size η along that direction. $\bar{\boldsymbol{\alpha}}_t$ is obtained from $\bar{\boldsymbol{\alpha}}_{t-1}$ according to the update $\bar{\boldsymbol{\alpha}}_t = \bar{\boldsymbol{\alpha}}_{t-1} + \eta \mathbf{e}_k$, where η is the step size chosen along the direction $\mathbf{e}_k$. Observe that if we denote by $\bar{g}_t$ the ensemble function defined by $\bar{\boldsymbol{\alpha}}_t$, that is $\bar{g}_t = \sum_{j=1}^{N} \bar{\alpha}_{t,j} h_j$, then the coordinate descent update coincides with the update $\bar{g}_t = \bar{g}_{t-1} + \eta h_k$, which is also the AdaBoost update. Thus, since both algorithms start with $\bar{g}_0 = 0$, to show that AdaBoost coincides with coordinate descent applied to F, it suffices to show at every iteration t, coordinate descent selects the same base hypothesis h_k and step η as AdaBoost. We will assume by induction that this holds up to iteration $t-1$, which implies the equality $\bar{g}_{t-1} = f_{t-1}$, and will show then that it also holds at iteration t.

The variant of coordinate descent we consider here consists of selecting, at each iteration, the maximum descent direction, that is the direction $\mathbf{e}_k$ along which the derivative of F is the largest in absolute value, and of selecting the best step along that direction, that is of choosing η to minimize $F(\bar{\boldsymbol{\alpha}}_{t-1} + \eta \mathbf{e}_k)$. To give the expressions of the direction and the step at each iteration, we first introduce similar

quantities to those appearing in the analysis of the boosting algorithm. For any $t \in [T]$, we define a distribution $\bar{\mathcal{D}}_t$ over the indices $\{1, \ldots, m\}$ as follows:

$$\bar{\mathcal{D}}_t(i) = \frac{e^{-y_i \sum_{j=1}^N \bar{\alpha}_{t-1,j} h_j(x_i)}}{\bar{Z}_t} = \frac{e^{-y_i \bar{g}_{t-1}(x_i)}}{\bar{Z}_t},$$

where $\bar{Z}_t$ is the normalization factor $\bar{Z}_t = \sum_{i=1}^m e^{-y_i \sum_{j=1}^N \bar{\alpha}_{t-1,j} h_j(x_i)}$. Observe that, since $\bar{g}_{t-1} = f_{t-1}$, $\bar{\mathcal{D}}_t$ coincides with $\mathcal{D}_t$. We also define for any base hypothesis h_j, $j \in [N]$, its expected error $\bar{\epsilon}_{t,j}$ with respect to the distribution $\bar{\mathcal{D}}_t$:

$$\bar{\epsilon}_{t,j} = \mathop{\mathbb{E}}_{i \sim \bar{\mathcal{D}}_t} \left[1_{y_i h_j(x_i) \leq 0}\right].$$

The directional derivative of F at $\bar{\boldsymbol{\alpha}}_{t-1}$ along $\mathbf{e}_k$ is denoted by $F'(\bar{\boldsymbol{\alpha}}_{t-1}, \mathbf{e}_k)$ and defined by

$$F'(\bar{\boldsymbol{\alpha}}_{t-1}, \mathbf{e}_k) = \lim_{\eta \to 0} \frac{F(\bar{\boldsymbol{\alpha}}_{t-1} + \eta \mathbf{e}_k) - F(\bar{\boldsymbol{\alpha}}_{t-1})}{\eta}.$$

Since $F(\bar{\boldsymbol{\alpha}}_{t-1} + \eta \mathbf{e}_k) = \sum_{i=1}^m e^{-y_i \sum_{j=1}^N \bar{\alpha}_{t-1,j} h_j(x_i) - \eta y_i h_k(x_i)}$, the directional derivative along $\mathbf{e}_k$ can be expressed as follows:

$$\begin{aligned}
F'(\bar{\boldsymbol{\alpha}}_{t-1}, \mathbf{e}_k) &= -\frac{1}{m} \sum_{i=1}^m y_i h_k(x_i) e^{-y_i \sum_{j=1}^N \bar{\alpha}_{t-1,j} h_j(x_i)} \\
&= -\frac{1}{m} \sum_{i=1}^m y_i h_k(x_i) \bar{\mathcal{D}}_t(i) \bar{Z}_t \\
&= -\left[\sum_{i=1}^m \bar{\mathcal{D}}_t(i) 1_{y_i h_k(x_i) = +1} - \sum_{i=1}^m \bar{\mathcal{D}}_t(i) 1_{y_i h_k(x_i) = -1}\right] \frac{\bar{Z}_t}{m} \\
&= -\left[(1 - \bar{\epsilon}_{t,k}) - \bar{\epsilon}_{t,k}\right] \frac{\bar{Z}_t}{m} = \left[2\bar{\epsilon}_{t,k} - 1\right] \frac{\bar{Z}_t}{m}.
\end{aligned}$$

Since $\frac{\bar{Z}_t}{m}$ does not depend on k, the maximum descent direction k is the one minimizing $\bar{\epsilon}_{t,k}$. Thus, the hypothesis h_k selected by coordinate descent at iteration t is the one with the smallest expected error on the sample S, where the expectation is taken with respect to $\bar{\mathcal{D}}_t = \mathcal{D}_t$. This matches exactly the choice made by AdaBoost at the tth round.

The step size η is selected to minimize the function along the direction $\mathbf{e}_k$ chosen: $\operatorname{argmin}_\eta F(\bar{\boldsymbol{\alpha}}_{t-1} + \eta\, \mathbf{e}_k)$. Since $F(\bar{\boldsymbol{\alpha}}_{t-1} + \eta\, \mathbf{e}_k)$ is a convex function of η, to find the

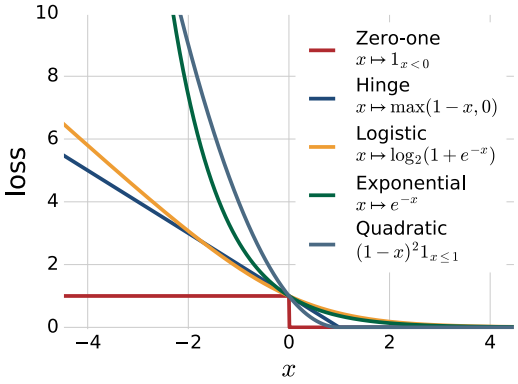


Figure 7.4
Examples of several convex upper bounds on the zero-one loss.

minimum, it suffices to set its derivative to zero:

$$\begin{aligned}
\frac{dF(\bar{\boldsymbol{\alpha}}_{t-1} + \eta \mathbf{e}_k)}{d\eta} = 0 &\Leftrightarrow -\sum_{i=1}^{m} y_i h_k(x_i) e^{-y_i \sum_{j=1}^{N} \bar{\alpha}_{t-1,j} h_j(x_i)} e^{-\eta y_i h_k(x_i)} = 0 \\
&\Leftrightarrow -\sum_{i=1}^{m} y_i h_k(x_i) \bar{\mathcal{D}}_t(i) \bar{Z}_t e^{-\eta y_i h_k(x_i)} = 0 \\
&\Leftrightarrow -\sum_{i=1}^{m} y_i h_k(x_i) \bar{\mathcal{D}}_t(i) e^{-\eta y_i h_k(x_i)} = 0 \\
&\Leftrightarrow -\left[(1-\bar{\epsilon}_{t,k}) e^{-\eta} - \bar{\epsilon}_{t,k} e^{\eta}\right] = 0 \\
&\Leftrightarrow \eta = \frac{1}{2} \log \frac{1-\bar{\epsilon}_{t,k}}{\bar{\epsilon}_{t,k}}.
\end{aligned}$$

This proves that the step size chosen by coordinate descent coincides with the weight α_t assigned by AdaBoost to the classifier chosen in the tth round. Thus, coordinate descent applied to exponential objective F precisely coincides with AdaBoost and F can be viewed as the objective function that AdaBoost seeks to minimize.

In light of this relationship, one may wish to consider similar applications of coordinate descent to other convex and differentiable functions of $\bar{\boldsymbol{\alpha}}$ upper-bounding the zero-one loss. In particular, the *logistic loss* $x \mapsto \log_2(1 + e^{-x})$ is convex and differentiable and upper bounds the zero-one loss. Figure 7.4 shows other examples of alternative convex loss functions upper-bounding the zero-one loss. Using the logistic loss, instead of the exponential loss used by AdaBoost, leads to an objective that coincides with *logistic regression*.

7.2.3 Practical use

Here, we briefly describe the standard practical use of AdaBoost. An important requirement for the algorithm is the choice of the base classifiers or that of the weak learner. The family of base classifiers typically used with AdaBoost in practice is that of *decision trees*, which are equivalent to hierarchical partitions of the space (see chapter 9, section 9.3.3). Among decision trees, those of depth one, also known as *stumps*, are by far the most frequently used base classifiers.

Boosting stumps are threshold functions associated to a single feature. Thus, a stump corresponds to a single axis-aligned partition of the space, as illustrated in figure 7.2. If the data is in $\mathbb{R}^N$, we can associate a stump to each of the N components. Thus, to determine the stump with the minimal weighted error at each round of boosting, the best component and the best threshold for each component must be computed.

To do so, we can first presort each component in $O(m \log m)$ time with a total computational cost of $O(mN \log m)$. For a given component, there are only $m + 1$ possible distinct thresholds, since two thresholds between the same consecutive component values are equivalent. To find the best threshold at each round of boosting, all of these possible $m + 1$ values can be compared, which can be done in $O(m)$ time. Thus, the total computational complexity of the algorithm for T rounds of boosting is $O(mN \log m + mNT)$.

Observe, however, that while boosting stumps are widely used in combination with AdaBoost and can perform well in practice, the algorithm that returns the stump with the minimal (weighted) empirical error is *not a weak learner* (see definition 7.1)! Consider, for example, the simple XOR example with four data points lying in $\mathbb{R}^2$ (see figure 6.3a), where points in the second and fourth quadrants are labeled positively and those in the first and third quadrants negatively. Then, no decision stump can achieve an accuracy better than $\frac{1}{2}$.

7.3 Theoretical results

In this section we present a theoretical analysis of the generalization properties of AdaBoost.

7.3.1 VC-dimension-based analysis

We start with an analysis of AdaBoost based on the VC-dimension of its hypothesis set. The family of functions $\mathcal{F}_T$ out of which AdaBoost selects its output after T rounds of boosting is

$$\mathcal{F}_T = \left\{ \operatorname{sgn}\left(\sum_{t=1}^{T} \alpha_t h_t \right) : \alpha_t \geq 0, h_t \in \mathcal{H}, t \in [T] \right\}. \tag{7.8}$$

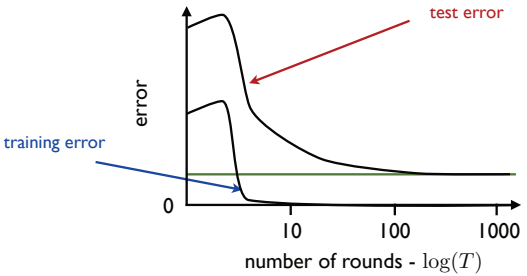


Figure 7.5
An empirical result using AdaBoost with C4.5 decision trees as base learners. In this example, the training error goes to zero after about 5 rounds of boosting ($T \approx 5$), yet the test error continues to decrease for larger values of T.

The VC-dimension of $\mathcal{F}_T$ can be bounded as follows in terms of the VC-dimension d of the family of base hypothesis $\mathcal{H}$ (exercise 7.1):

$$\text{VCdim}(\mathcal{F}_T) \leq 2(d+1)(T+1)\log_2((T+1)e)\,. \tag{7.9}$$

The upper bound grows as $O(dT \log T)$, thus, the bound suggests that AdaBoost could overfit for large values of T, and indeed this can occur. However, in many cases, it has been observed empirically that the generalization error of AdaBoost decreases as a function of the number of rounds of boosting T, as illustrated in figure 7.5! How can these empirical results be explained? The following sections present a margin-based analysis in support of AdaBoost that can serve as a theoretical explanation for these empirical observations.

7.3.2 L_1-geometric margin

In chapter 5, we introduced the definition of confidence margin and presented a series of general learning bounds based on that notion which, in particular, provided strong learning guarantees for SVMs. Here, we will similarly derive general learning bounds based on that same notion of confidence margin for ensemble methods, which we will use, in particular, to derive learning guarantees for AdaBoost.

Recall that the confidence margin of a real-valued function f at a point x labeled with y is the quantity $yf(x)$. For SVMs, we also defined the notion of geometric margin which, in the separable case, is a lower bound on the confidence margin of a linear hypothesis with a normalized weighted vector $\mathbf{w}$, $\|\mathbf{w}\|_2 = 1$. Here, we will also define a notion of geometric margin for linear hypotheses with a norm-1 constraint, such as the ensemble hypotheses returned by AdaBoost, and similarly relate that notion to that of confidence margin. This will also serve as an opportunity for us to point out the connection between several concepts and terminology used in the context of SVMs and those used in the context of boosting.

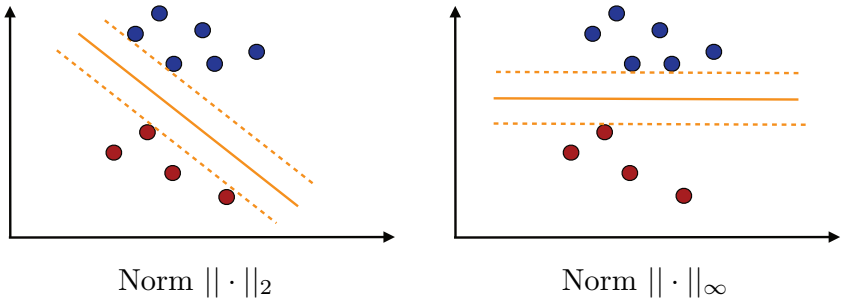


Figure 7.6
Maximum margin hyperplanes for norm-2 and norm-∞.

First note that a function $f = \sum_{t=1}^{T} \alpha_t h_t$ that is a linear combination of base hypotheses $h_1, \ldots, h_T$ can be equivalently expressed as an inner product $f = \boldsymbol{\alpha} \cdot \mathbf{h}$, where $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_T)^\top$ and $\mathbf{h} = [h_1, \ldots, h_T]^\top$. This makes the similarity between the linear hypotheses considered in this chapter and those of chapter 5 and chapter 6 evident: the vector of base hypothesis values $\mathbf{h}(x)$ can be viewed as a feature vector associated to x, which was denoted by $\boldsymbol{\Phi}(x)$ in previous chapters, and $\boldsymbol{\alpha}$ is the weight vector that was denoted by $\mathbf{w}$. For ensemble linear combinations such as those returned by AdaBoost, additionally, the weight vector is non-negative: $\boldsymbol{\alpha} \geq 0$. Next, we introduce a notion of geometric margin for such ensemble functions which differs from the one introduced for SVMs only by the norm-1 used instead of norm-2, using the notation just introduced.

Definition 7.3 (L_1-geometric margin) *The* L_1-geometric margin $\rho_f(x)$ *of a linear function* $f = \sum_{t=1}^{T} \alpha_t h_t$ *with* $\boldsymbol{\alpha} \neq 0$ *at a point* $x \in \mathcal{X}$ *is defined by*

$$\rho_f(x) = \frac{|f(x)|}{\|\boldsymbol{\alpha}\|_1} = \frac{|\sum_{t=1}^{T} \alpha_t h_t(x)|}{\|\boldsymbol{\alpha}\|_1} = \frac{|\boldsymbol{\alpha} \cdot \mathbf{h}(x)|}{\|\boldsymbol{\alpha}\|_1}. \tag{7.10}$$

The L_1-margin *of* f *over a sample* $S = (x_1, \ldots, x_m)$ *is its minimum margin at the points in that sample:*

$$\rho_f = \min_{i \in [m]} \rho_f(x_i) = \min_{i \in [m]} \frac{|\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)|}{\|\boldsymbol{\alpha}\|_1}. \tag{7.11}$$

This definition of geometric margin differs from definition 5.1 given in the context of the SVM algorithm only by the norm used for the weight vector: L_1-norm here, L_2-norm in definition 5.1. To distinguish them in the discussion that follows, let $\rho_1(x)$ denote the L_1-margin and $\rho_2(x)$ the L_2-margin at point x (definition 5.1):

$$\rho_1(x) = \frac{|\boldsymbol{\alpha} \cdot \mathbf{h}(x)|}{\|\boldsymbol{\alpha}\|_1} \quad \text{and} \quad \rho_2(x) = \frac{|\boldsymbol{\alpha} \cdot \mathbf{h}(x)|}{\|\boldsymbol{\alpha}\|_2}.$$

$\rho_2(x)$ is then the norm-2 distance of the vector $\mathbf{h}(x)$ to the hyperplane of equation $\boldsymbol{\alpha}\cdot\mathbf{x} = 0$ in $\mathbb{R}^T$. Similarly, $\rho_1(x)$ is the norm-∞ distance of $\mathbf{h}(x)$ to that hyperplane. This geometric difference is illustrated by figure 7.6.[8]

We will denote by

$$\bar{f} = \frac{f}{\sum_{t=1}^{T}\alpha_t} = \frac{f}{\|\boldsymbol{\alpha}\|_1}$$

the normalized version of the function f returned by AdaBoost. Note that if a point x with label y is correctly classified by f (or $\bar{f}$), then the confidence margin of $\bar{f}$ at x coincides with the L_1-geometric margin of f: $y\bar{f}(x) = \frac{yf(x)}{\|\boldsymbol{\alpha}\|_1} = \rho_f(x)$. Observe that, since the coefficients α_t are non-negative, $\rho_f(x)$ is then a convex combination of the base hypothesis values $h_t(x)$. In particular, if the base hypotheses h_t take values in $[-1, +1]$, then $\rho_f(x)$ is in $[-1, +1]$.

7.3.3 Margin-based analysis

To analyze the generalization properties of AdaBoost, we start by examining the Rademacher complexity of convex linear ensembles. For any hypothesis set $\mathcal{H}$ of real-valued functions, we denote by $\mathrm{conv}(\mathcal{H})$ its convex hull defined by

$$\mathrm{conv}(\mathcal{H}) = \left\{ \sum_{k=1}^{p} \mu_k h_k \colon p \geq 1, \forall k \in [p], \mu_k \geq 0, h_k \in \mathcal{H}, \sum_{k=1}^{p} \mu_k \leq 1 \right\}. \tag{7.12}$$

The following lemma shows that, remarkably, the empirical Rademacher complexity of $\mathrm{conv}(\mathcal{H})$, which in general is a strictly larger set including $\mathcal{H}$, coincides with that of $\mathcal{H}$.

Lemma 7.4 *Let $\mathcal{H}$ be a set of functions mapping from $\mathcal{X}$ to $\mathbb{R}$. Then, for any sample S, we have*

$$\widehat{\mathfrak{R}}_S\big(\mathrm{conv}(\mathcal{H})\big) = \widehat{\mathfrak{R}}_S(\mathcal{H})\,.$$

[8] More generally, for $p, q \geq 1$, p and q *conjugate*, that is $\frac{1}{p} + \frac{1}{q} = 1$, $\frac{|\boldsymbol{\alpha}\cdot\mathbf{h}(x)|}{\|\boldsymbol{\alpha}\|_p}$ is the norm-q distance of $\mathbf{h}(x)$ to the hyperplane of equation $\boldsymbol{\alpha}\cdot\mathbf{h}(x) = 0$.

Proof: The proof follows from a straightforward series of equalities:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S\big(\operatorname{conv}(\mathcal{H})\big) &= \frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h_1,\ldots,h_p\in\mathcal{H},\boldsymbol{\mu}\geq 0,\|\boldsymbol{\mu}\|_1\leq 1}\sum_{i=1}^{m}\sigma_i\sum_{k=1}^{p}\mu_k h_k(x_i)\right]\\
&= \frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h_1,\ldots,h_p\in\mathcal{H}}\sup_{\boldsymbol{\mu}\geq 0,\|\boldsymbol{\mu}\|_1\leq 1}\sum_{k=1}^{p}\mu_k\sum_{i=1}^{m}\sigma_i h_k(x_i)\right]\\
&= \frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h_1,\ldots,h_p\in\mathcal{H}}\max_{k\in[p]}\sum_{i=1}^{m}\sigma_i h_k(x_i)\right]\\
&= \frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i)\right] = \widehat{\mathfrak{R}}_S(\mathcal{H}),
\end{aligned}$$

where the third equality follows the definition of the dual norm (see section A.1.2) or the observation that the maximizing vector $\boldsymbol{\mu}$ for a convex combination of p terms is the one placing all the weight on the largest term. □

This theorem can be used directly in combination with theorem 5.8 to derive the following Rademacher complexity generalization bound for convex combination ensembles of hypotheses. Recall that $\widehat{R}_{S,\rho}(h)$ denotes the empirical margin loss with margin ρ.

Corollary 7.5 (Ensemble Rademacher margin bound) *Let $\mathcal{H}$ denote a set of real-valued functions. Fix $\rho > 0$. Then, for any $\delta > 0$, with probability at least $1-\delta$, each of the following holds for all $h \in \operatorname{conv}(\mathcal{H})$:*

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{2}{\rho}\mathfrak{R}_m\big(\mathcal{H}\big) + \sqrt{\frac{\log\frac{1}{\delta}}{2m}} \tag{7.13}$$

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{2}{\rho}\widehat{\mathfrak{R}}_S\big(\mathcal{H}\big) + 3\sqrt{\frac{\log\frac{2}{\delta}}{2m}}\,. \tag{7.14}$$

Using corollary 3.8 and corollary 3.18 to bound the Rademacher complexity in terms of the VC-dimension yields immediately the following VC-dimension-based generalization bounds for convex combination ensembles of hypotheses.

Corollary 7.6 (Ensemble VC-Dimension margin bound) *Let $\mathcal{H}$ be a family of functions taking values in $\{+1,-1\}$ with VC-dimension d. Fix $\rho > 0$. Then, for any $\delta > 0$, with probability at least $1-\delta$, the following holds for all $h \in \operatorname{conv}(\mathcal{H})$:*

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{2}{\rho}\sqrt{\frac{2d\log\frac{em}{d}}{m}} + \sqrt{\frac{\log\frac{1}{\delta}}{2m}}\,. \tag{7.15}$$

These bounds can be generalized to hold uniformly for all $\rho \in (0,1]$, at the price of an additional term of the form $\sqrt{\frac{\log\log_2\frac{2}{\rho}}{m}}$ as in theorem 5.9. They cannot be

directly applied to the function f returned by AdaBoost, since it is not a convex combination of base hypotheses, but they can be applied to its normalized version, $\bar{f} = \frac{\sum_{t=1}^{T} \alpha_t h_t}{\|\boldsymbol{\alpha}\|_1} \in \operatorname{conv}(\mathcal{H})$. Notice that from the point of view of binary classification, f and $\bar{f}$ are equivalent since $\operatorname{sgn}(f) = \operatorname{sgn}\left(\frac{f}{\|\boldsymbol{\alpha}\|_1}\right)$, thus $R(f) = R\left(\frac{f}{\|\boldsymbol{\alpha}\|_1}\right)$, but their empirical margin losses are distinct.

Let $f = \sum_{t=1}^{T} \alpha_t h_t$ denote the function defining the classifier returned by AdaBoost after T rounds of boosting when trained on sample S. Then, in view of (7.13), for any $\delta > 0$, the following holds with probability at least $1 - \delta$:

$$R(f) \leq \widehat{R}_{S,\rho}\left(\bar{f}\right) + \frac{2}{\rho}\mathfrak{R}_m\big(\mathcal{H}\big) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}. \tag{7.16}$$

Similar bounds can be derived from (7.14) and (7.15). Remarkably, the number of rounds of boosting T does not appear in the generalization bound (7.16). The bound depends only on the confidence margin ρ, the sample size m, and the Rademacher complexity of the family of base classifiers $\mathcal{H}$. Thus, the bound guarantees an effective generalization if the margin loss $R_\rho\big(\bar{f}\big)$ is small for a relatively large ρ. Recall that the margin loss can be upper bounded by the fraction of the points x labeled with y in the training sample with confidence margin at most ρ, that is $\frac{yf(x)}{\|\boldsymbol{\alpha}\|_1} \leq \rho$ (see (5.38)). With our definition of L_1-margin, this can also be written as follows:

$$\widehat{R}_{S,\rho}\left(\bar{f}\right) \leq \frac{|\{i \in [m] \colon y_i\, \rho_f(x_i) \leq \rho\}|}{m}. \tag{7.17}$$

Additionally, the following theorem provides a bound on the empirical margin loss, which decreases with T under conditions discussed later.

Theorem 7.7 *Let $f = \sum_{t=1}^{T} \alpha_t h_t$ denote the function returned by AdaBoost after T rounds of boosting and assume for all $t \in [T]$ that $\epsilon_t < \frac{1}{2}$, which implies $\alpha_t > 0$. Then, for any $\rho > 0$, the following holds:*

$$\widehat{R}_{S,\rho}(\bar{f}) \leq 2^T \prod_{t=1}^{T} \sqrt{\epsilon_t^{1-\rho}(1-\epsilon_t)^{1+\rho}}\,.$$

Proof: Using the general inequality $1_{u \leq 0} \leq \exp(-u)$ valid for all $u \in \mathbb{R}$, identity 7.2, that is $\mathcal{D}_{t+1}(i) = \frac{e^{-y_i f(x_i)}}{m \prod_{t=1}^{T} Z_t}$, the equality $Z_t = 2\sqrt{\epsilon_t(1-\epsilon_t)}$ from the proof of

theorem 7.2, and the definition of $\alpha_t = \frac{1}{2}\log(\frac{1-\epsilon_t}{\epsilon_t})$ in AdaBoost, we can write:

$$\begin{aligned}\frac{1}{m}\sum_{i=1}^{m} 1_{y_i f(x_i)-\rho\|\boldsymbol{\alpha}\|_1\leq 0} &\leq \frac{1}{m}\sum_{i=1}^{m}\exp(-y_i f(x_i)+\rho\|\boldsymbol{\alpha}\|_1)\\ &= \frac{1}{m}\sum_{i=1}^{m} e^{\rho\|\boldsymbol{\alpha}\|_1}\left[m\prod_{t=1}^{T} Z_t\right]\mathcal{D}_{T+1}(i)\\ &= e^{\rho\|\boldsymbol{\alpha}\|_1}\prod_{t=1}^{T} Z_t = e^{\rho\sum_{t'}\alpha_{t'}}\prod_{t=1}^{T} Z_t\\ &= 2^T\prod_{t=1}^{T}\left[\sqrt{\tfrac{1-\epsilon_t}{\epsilon_t}}\right]^{\rho}\sqrt{\epsilon_t(1-\epsilon_t)}\,,\end{aligned}$$

which concludes the proof. □

Moreover, if for all $t \in [T]$ we have $\gamma \leq (\frac{1}{2}-\epsilon_t)$ and $\rho \leq 2\gamma$, then the expression $4\epsilon_t^{1-\rho}(1-\epsilon_t)^{1+\rho}$ is maximized at $\epsilon_t = \frac{1}{2}-\gamma$.[9] Thus, the upper bound on the empirical margin loss can then be bounded by

$$\widehat{R}_{S,\rho}(\bar{f}) \leq \left[(1-2\gamma)^{1-\rho}(1+2\gamma)^{1+\rho}\right]^{\frac{T}{2}}. \tag{7.18}$$

Observe that $(1-2\gamma)^{1-\rho}(1+2\gamma)^{1+\rho} = (1-4\gamma^2)\big(\frac{1+2\gamma}{1-2\gamma}\big)^{\rho}$. This is an increasing function of ρ since we have $\big(\frac{1+2\gamma}{1-2\gamma}\big) > 1$ as a consequence of $\gamma > 0$. Thus, if $\rho < \gamma$, it can be strictly upper bounded as follows

$$(1-2\gamma)^{1-\rho}(1+2\gamma)^{1+\rho} < (1-2\gamma)^{1-\gamma}(1+2\gamma)^{1+\gamma}.$$

The function $\gamma \mapsto (1-2\gamma)^{1-\gamma}(1+2\gamma)^{1+\gamma}$ is strictly upper bounded by 1 over the interval $(0, 1/2)$, thus, if $\rho < \gamma$, then $(1-2\gamma)^{1-\rho}(1+2\gamma)^{1+\rho} < 1$ and the right-hand side of (7.18) decreases exponentially with T. Since the condition $\rho \gg O(1/\sqrt{m})$ is necessary in order for the given margin bounds to converge, this places a condition of $\gamma \gg O(1/\sqrt{m})$ on the edge value. In practice, the error ϵ_t of the base classifier at round t may increase as a function of t. Informally, this is because boosting presses the weak learner to concentrate on instances that are harder and harder to classify, for which even the best base classifier could not achieve an error significantly better than random. If ϵ_t becomes close to $\frac{1}{2}$ relatively fast as a function of t, then the bound of theorem 7.7 becomes uninformative.

[9] The differential of $f\colon \epsilon \mapsto \log[\epsilon^{1-\rho}(1-\epsilon)^{1+\rho}] = (1-\rho)\log\epsilon + (1+\rho)\log(1-\epsilon)$ over the interval $(0,1)$ is given by $f'(\epsilon) = \frac{1-\rho}{\epsilon} - \frac{1+\rho}{1-\epsilon} = 2\frac{(\frac{1}{2}-\frac{\rho}{2})-\epsilon}{\epsilon(1-\epsilon)}$. Thus, f is an increasing function over $(0, \frac{1}{2}-\frac{\rho}{2})$, which implies that it is increasing over $(0, \frac{1}{2}-\gamma)$ when $\gamma \geq \frac{\rho}{2}$.

The analysis and discussion that precede show that if AdaBoost admits a positive edge ($\gamma > 0$), then, for $\rho < \gamma$, the empirical margin loss $\widehat{R}_{S,\rho}(\bar{f})$ becomes zero for T sufficiently large (it decreases exponentially fast). Thus, AdaBoost achieves an L_1-geometric margin of γ over the training sample. In section 7.3.5, we will see that the edge γ is positive if and only if the training sample is separable. In that case, the edge can be chosen to be as large as half the maximum L_1-geometric margin $\rho_{\max}$ that can be achieved on the sample: $\gamma = \frac{\rho_{\max}}{2}$. Thus, for a separable data set, AdaBoost can asymptotically achieve a geometric margin that is at least half the maximum geometric margin, $\frac{\rho_{\max}}{2}$.

This analysis can serve as a theoretical explanation of the empirical observation that, in some tasks, the generalization error decreases as a function of T even after the error on the training sample is zero: the geometric margin continues to increase when the training sample is separable. In (7.16), for the ensemble function f determined by AdaBoost after T rounds, as T increases, ρ can be chosen as a larger quantity for which the first term on the right-hand side vanishes ($\widehat{R}_{S,\rho}(\bar{f}) = 0$) while the second term becomes more favorable since it decreases as $\frac{1}{\rho}$.

But, does AdaBoost achieve the maximum L_1-geometric margin $\rho_{\max}$? No. It has been shown that AdaBoost may converge, for a linearly separable sample, to a geometric margin that is significantly smaller than the maximum margin (e.g., $\frac{1}{3}$ instead of $\frac{3}{8}$).

7.3.4 Margin maximization

In view of these results, several algorithms have been devised with the explicit goal of maximizing the L_1-geometric margin. These algorithms correspond to different methods for solving a linear program (LP).

By definition of the L_1-margin, the maximum margin for a linearly separable sample $S = ((x_1, y_1), \ldots, (x_m, y_m))$ is given by

$$\rho = \max_{\boldsymbol{\alpha}} \min_{i \in [m]} \frac{y_i\big(\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)\big)}{\|\boldsymbol{\alpha}\|_1}\,. \tag{7.19}$$

By definition of the maximization, the optimization problem can be written as:

$$\max_{\boldsymbol{\alpha}} \rho$$
$$\text{subject to: } \frac{y_i\big(\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)\big)}{\|\boldsymbol{\alpha}\|_1} \geq \rho,\ \forall i \in [m].$$

Since $\frac{\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)}{\|\boldsymbol{\alpha}\|_1}$ is invariant to the scaling of $\boldsymbol{\alpha}$, we can restrict ourselves to $\|\boldsymbol{\alpha}\|_1 = 1$. Further seeking a non-negative $\boldsymbol{\alpha}$ as in the case of AdaBoost leads to the following

optimization:

$$\begin{aligned} &\max_{\boldsymbol{\alpha}} \rho \\ &\text{subject to: } y_i\big(\boldsymbol{\alpha} \cdot \mathbf{h}(x_i)\big) \geq \rho, \ \forall i \in [m]; \\ &\qquad \left(\sum_{t=1}^{T} \alpha_t = 1\right) \wedge \Big(\alpha_t \geq 0, \forall t \in [T]\Big). \end{aligned}$$

This is a linear program (LP), that is, a convex optimization problem with a linear objective function and linear constraints. There are several different methods for solving relative large LPs in practice, using the simplex method, interior-point methods, or a variety of special-purpose solutions.

Note that the solution of this algorithm differs from the margin-maximization defining SVMs in the separable case only by the definition of the geometric margin used (L_1 versus L_2) and the non-negativity constraint on the weight vector. Figure 7.6 illustrates the margin-maximizing hyperplanes found using these two distinct margin definitions in a simple case. The left figure shows the SVM solution, where the distance to the closest points to the hyperplane is measured with respect to the norm $\|\cdot\|_2$. The right figure shows the solution for the L_1-margin, where the distance to the closest points to the hyperplane is measured with respect to the norm $\|\cdot\|_\infty$.

By definition, the solution of the LP just described admits an L_1-margin that is larger or equal to that of the AdaBoost solution. However, empirical results do not show a systematic benefit for the solution of the LP. In fact, it appears that in many cases, AdaBoost outperforms that algorithm. The margin theory described does not seem sufficient to explain that performance.

7.3.5 Game-theoretic interpretation

In this section, we show that AdaBoost admits a natural game-theoretic interpretation. The application of von Neumann's theorem then helps us relate the maximum margin and the optimal edge and clarify the connection of AdaBoost's weak-learning assumption with the notion of L_1-margin. We first introduce the definition of the edge of a base classifier for a particular distribution.

Definition 7.8 *The* edge of a base classifier h_t for a distribution $\mathcal{D}$ *over the training sample* $S = ((x_1, y_1), \ldots, (x_m, y_m))$ *is defined by*

$$\gamma_t(\mathcal{D}) = \frac{1}{2} - \epsilon_t = \frac{1}{2} \sum_{i=1}^{m} y_i h_t(x_i) \mathcal{D}(i). \tag{7.20}$$

AdaBoost's weak learning condition can now be formulated as follows: there exists $\gamma > 0$ such that for any distribution $\mathcal{D}$ over the training sample and any base

Table 7.1
The loss matrix for the standard rock-paper-scissors game.

	rock	paper	scissors
rock	0	+1	-1
paper	-1	0	+1
scissors	+1	-1	0

classifier h_t, the following holds:

$$\gamma_t(\mathcal{D}) \geq \gamma. \tag{7.21}$$

This condition is required for the analysis of theorem 7.2 and the non-negativity of the coefficients α_t. We will frame boosting as a two-person zero-sum game.

Definition 7.9 (Zero-sum game) *A* finite two-person zero-sum game *consists of a* loss matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$*, where* m *is the number of possible* actions *(or* pure strategies*) for the row player and* n *the number of possible actions for the column player. The entry* M_{ij} *is the loss for the row player (or equivalently the payoff for the column payer) when the row player takes action* i *and the column player takes action* j*.*[10]

An example of a loss matrix for the familiar "rock-paper-scissors" game is shown in table 7.1.

Definition 7.10 (Mixed strategy) *A* mixed strategy *for the row player is a distribution* p *over the* m *possible row actions; a* mixed strategy *for the column player is a distribution* q *over the* n *possible column actions. The* expected loss *for the row player (expected payoff for the column player) with respect to the mixed strategies* p *and* q *is*

$$\mathop{\mathbb{E}}_{\substack{i \sim p \\ j \sim q}} [M_{ij}] = \sum_{i=1}^{m} \sum_{j=1}^{n} p_i M_{ij} q_j = \mathbf{p}^\top \mathbf{M} \mathbf{q}.$$

The following is a fundamental result in game theory proven in chapter 8.

Theorem 7.11 (Von Neumann's minimax theorem) *For any finite two-person zero-sum game defined by the matrix* $\mathbf{M}$*, the following equality holds:*

$$\min_{\mathbf{p}} \max_{\mathbf{q}} \mathbf{p}^\top \mathbf{M} \mathbf{q} = \max_{\mathbf{q}} \min_{\mathbf{p}} \mathbf{p}^\top \mathbf{M} \mathbf{q}\,. \tag{7.22}$$

The common value in (7.22) is called the *value of the game*. The theorem states that for any two-person zero-sum game, there exists a mixed strategy for each player

[10] To be consistent with the results discussed in other chapters, we consider the loss matrix as opposed to the payoff matrix (its opposite).

such that the expected loss for one is the same as the expected payoff for the other, both of which are equal to the value of the game.

Note that, given the row player's strategy, the column player can choose a pure strategy optimizes their payoff. That is, the column player can choose the single strategy corresponding the largest coordinate of the vector $\mathbf{p}^\top \mathbf{M}$. A similar comment applies to the reverse. Thus, an alternative and equivalent form of the minimax theorem is

$$\min_{\mathbf{p}} \max_{j \in [n]} \mathbf{p}^\top \mathbf{M} \mathbf{e}_j = \max_{\mathbf{q}} \min_{i \in [m]} \mathbf{e}_i^\top \mathbf{M} \mathbf{q}, \tag{7.23}$$

where $\mathbf{e}_i$ denotes the ith unit vector.

We can now view AdaBoost as a zero-sum game, where an action of the row player is the selection of a training instance x_i, $i \in [m]$, and an action of the column player the selection of a base learner h_t, $t \in [T]$. A mixed strategy for the row player is thus a distribution $\mathcal{D}$ over the training points' indices $[m]$. A mixed strategy for the column player is a distribution over the based classifiers' indices $[T]$. This can be defined from a non-negative vector $\boldsymbol{\alpha} \geq 0$: the weight assigned to $t \in [T]$ is $\alpha_t / \|\boldsymbol{\alpha}\|_1$. The loss matrix $\mathbf{M} \in \{-1, +1\}^{m \times T}$ for AdaBoost is defined by $M_{it} = y_i h_t(x_i)$ for all $(i, t) \in [m] \times [T]$. By von Neumann's theorem (7.23), the following holds:

$$\min_{\mathcal{D} \in \mathcal{D}} \max_{t \in [T]} \sum_{i=1}^{m} \mathcal{D}(i) y_i h_t(x_i) = \max_{\boldsymbol{\alpha} \geq 0} \min_{i \in [m]} \sum_{t=1}^{T} \frac{\alpha_t}{\|\boldsymbol{\alpha}\|_1} y_i h_t(x_i), \tag{7.24}$$

where $\mathcal{D}$ denotes the set of all distributions over the training sample. Let $\rho_{\boldsymbol{\alpha}}(x)$ denote the margin of point x for the classifier defined by $f = \sum_{t=1}^{T} \alpha_t h_t$. The result can be rewritten as follows in terms of the margins and edges:

$$2\gamma^* = 2 \min_{\mathcal{D}} \max_{t \in [T]} \gamma_t(\mathcal{D}) = \max_{\boldsymbol{\alpha}} \min_{i \in [m]} \rho_{\boldsymbol{\alpha}}(x_i) = \rho^*, \tag{7.25}$$

where ρ^* is the maximum margin of a classifier and γ^* the best possible edge. This result has several implications. First, it shows that the weak learning condition ($\gamma^* > 0$) implies $\rho^* > 0$ and thus the existence of a classifier with positive margin, which motivates the search for a non-zero margin. AdaBoost can be viewed as an algorithm seeking to achieve such a non-zero margin, though, as discussed earlier, AdaBoost does not always achieve an optimal margin and is thus suboptimal in that respect. Furthermore, we see that the "weak learning" assumption, which originally appeared to be the weakest condition one could require for an algorithm (that of performing better than random), is in fact a strong condition: it implies that the training sample is linearly separable with margin $2\gamma^* > 0$. Linear separability often does not hold for the data sets found in practice.

7.4 L_1-regularization

In practice, the training sample may not be linearly separable and AdaBoost may not admit a positive edge, in which case the weak learning condition does not hold. It may also be that AdaBoost does admit a positive edge but with γ very small. In such cases, running AdaBoost may result in large total mixture weights for some base classifiers h_j. This can be because the algorithm increasingly concentrates on a few examples that are hard to classify and whose weights keep growing. Only a few base classifiers might achieve the best performance for those examples and the algorithm keeps selecting them, thereby increasing their total mixture weights. These base classifiers with relatively large total mixture weight end up dominating in an ensemble f and therefore solely dictating the classification decision. The performance of the resulting ensemble is typically poor since it almost entirely hinges on that of a few base classifiers.

There are several methods for avoiding such situations. One consists of limiting the number of rounds of boosting T, which is also known as *early-stopping*. Another one consists of controlling the magnitude of the mixture weights. This can be done by augmenting the objective function of AdaBoost with a regularization term based on a norm of the vector of mixture weights. Using a norm-1 regularization leads to an algorithm that we will refer to as *L_1-regularized AdaBoost*. Given a labeled sample $S = ((x_1, y_1), \ldots, (x_m, y_m))$, the objective function G minimized by L_1-regularized AdaBoost is defined for all $\bar{\boldsymbol{\alpha}} = (\bar{\alpha}_1, \ldots, \bar{\alpha}_N) \in \mathbb{R}^N$ by

$$G(\bar{\boldsymbol{\alpha}}) = \frac{1}{m}\sum_{i=1}^{m} e^{-y_i f(x_i)} + \lambda\|\bar{\boldsymbol{\alpha}}\|_1 = \frac{1}{m}\sum_{i=1}^{m} e^{-y_i \sum_{j=1}^{N} \bar{\alpha}_j h_j(x_i)} + \lambda\|\bar{\boldsymbol{\alpha}}\|_1, \tag{7.26}$$

where, as for AdaBoost, f is an ensemble function defined by $f = \sum_{j=1}^{N} \bar{\alpha}_j h_j$, with $\bar{\alpha}_j \geq 0$. The objective function G is a convex function of $\bar{\boldsymbol{\alpha}}$ as the sum of the convex objective of AdaBoost and the norm-1 of $\bar{\boldsymbol{\alpha}}$. L_1-regularized AdaBoost consists of applying coordinate-descent to the objective function G.

We now show that the algorithm can be directed derived from the margin-based guarantee for ensemble methods of Corollary 7.5 or Corollary 7.6. Thus, in that way, L_1-regularized AdaBoost benefits from a more favorable and natural theoretical guarantee than AdaBoost.

By the generalization of Corollary 7.5 to a uniform convergence bound over ρ, for any $\delta > 0$, with probability at least $1 - \delta$, the following holds for all ensemble functions $f = \sum_{j=1}^{N} \bar{\alpha}_j h_j$ with $\|\bar{\boldsymbol{\alpha}}\|_1 \leq 1$ and all $\rho \in (0, 1]$:

$$R(f) \leq \frac{1}{m}\sum_{i=1}^{m} 1_{f(x_i) \leq \rho} + \frac{2}{\rho}\mathfrak{R}_m\big(\mathcal{H}\big) + \sqrt{\frac{\log\log_2 \frac{2}{\rho}}{m}} + \sqrt{\frac{\log \frac{2}{\delta}}{2m}}. \tag{7.27}$$

The inequality also trivially holds for $\rho > 1$ since, in that case, the first term on the right-hand side of the bound is equal to one. Indeed, in that case, by Hölder's inequality, for any $x \in \mathcal{X}$, we have $f(x) = \sum_{j=1}^{N} \bar{\alpha}_i h_j(x) \leq \|\bar{\boldsymbol{\alpha}}\|_1 \max_{j\in[N]} |h_j(x)| \leq \|\bar{\boldsymbol{\alpha}}\|_1 \leq 1 < \rho$.

Now, in view of the general upper bound $1_{u\leq 0} \leq e^{-u}$ valid for all $u \in \mathbb{R}$, with probability at least $1-\delta$, the following holds for all $f = \sum_{j=1}^{N} \bar{\alpha}_j h_j$ with $\|\bar{\boldsymbol{\alpha}}\|_1 \leq 1$ and all $\rho > 0$:

$$R(f) \leq \frac{1}{m}\sum_{i=1}^{m} e^{1-\frac{f(x_i)}{\rho}} + \frac{2}{\rho}\mathfrak{R}_m\big(\mathcal{H}\big) + \sqrt{\frac{\log\log_2 \frac{2}{\rho}}{m}} + \sqrt{\frac{\log\frac{2}{\delta}}{2m}}. \tag{7.28}$$

Since for any $\rho > 0$, f/ρ admits the same generalization error as f, with probability at least $1-\rho$, the following inequality holds for all $f = \sum_{j=1}^{N} \bar{\alpha}_j h_j$ with $\|\bar{\boldsymbol{\alpha}}\|_1 \leq 1/\rho$ and all $\rho > 0$:

$$R(f) \leq \frac{1}{m}\sum_{i=1}^{m} e^{1-f(x_i)} + \frac{2}{\rho}\mathfrak{R}_m\big(\mathcal{H}\big) + \sqrt{\frac{\log\log_2 \frac{2}{\rho}}{m}} + \sqrt{\frac{\log\frac{2}{\delta}}{2m}}. \tag{7.29}$$

This inequality can be used to derive an algorithm that selects $\bar{\boldsymbol{\alpha}}$ and $\rho > 0$ to minimize the right-hand side. The minimization with respect to ρ does not lead to a convex optimization and depends on theoretical constant factors affecting the second and third terms, which may not be optimal. Thus, instead, ρ is left as a free parameter of the algorithm, typically determined via cross-validation.

Now, since only the first term of the right-hand side depends on $\bar{\boldsymbol{\alpha}}$, the bound suggests selecting $\bar{\boldsymbol{\alpha}}$ as the solution of the following optimization problem:

$$\min_{\|\bar{\boldsymbol{\alpha}}\|_1 \leq \frac{1}{\rho}} \frac{1}{m}\sum_{i=1}^{m} e^{-f(x_i)} = \frac{1}{m}\sum_{i=1}^{m} e^{-\sum_{j=1}^{N} \bar{\alpha}_j h_j(x_i)}. \tag{7.30}$$

Introducing a Lagrange variable $\lambda \geq 0$, the optimization problem can be written equivalently as

$$\min_{\|\bar{\boldsymbol{\alpha}}\|_1 \leq \frac{1}{\rho}} \frac{1}{m}\sum_{i=1}^{m} e^{-\sum_{j=1}^{N} \bar{\alpha}_j h_j(x_i)} + \lambda\|\bar{\boldsymbol{\alpha}}\|_1. \tag{7.31}$$

Since for any choice of ρ in the constraint of (7.30) there exists an equivalent dual variable λ in the formulation (7.31) that achieves the same optimal $\bar{\boldsymbol{\alpha}}$, $\lambda \geq 0$ can be freely selected via cross-validation. The resulting objective function therefore precisely coincides with that of L_1-regularized AdaBoost.

7.5 Discussion

AdaBoost offers several advantages: it is simple, its implementation is straightforward, and the time complexity of each round of boosting as a function of the sample size is rather favorable. As already discussed, when using decision stumps, the time complexity of each round of boosting is in $O(mN)$. Of course, if the dimension of the feature space N is very large, then the algorithm could become in fact quite slow.

AdaBoost additionally benefits from a rich theoretical analysis. Nevertheless, there are still many theoretical questions related to the algorithm. For example, as we saw, the algorithm in fact does not maximize the margin, and yet algorithms that do maximize the margin do not always outperform it. This suggests that perhaps a finer analysis based on a notion different from that of minimal margin could shed more light on the properties of the algorithm.

A minor drawback of the algorithm is the need to select the parameter T and the base classifier set. The choice of the number of rounds of boosting T (stopping criterion) is crucial to the performance of the algorithm. As suggested by the VC-dimension analysis, larger values of T can lead to overfitting. In practice, T is typically determined via cross-validation. The choice of the base classifiers is also crucial. The complexity of the family of base classifiers $\mathcal{H}$ appeared in all the bounds presented and it is important to control it in order to guarantee generalization. On the other hand, insufficiently complex hypothesis sets could lead to low margins.

Probably the most serious disadvantage of AdaBoost is its performance in the presence of noise, at least in some tasks. The distribution weight assigned to examples that are harder to classify substantially increases with the number of rounds of boosting, by the nature of the algorithm. These examples may end up dominating the selection of the base classifiers, which, with a large enough number of rounds, will play a detrimental role in the definition of the linear combination defined by AdaBoost. Several solutions have been proposed to address these issues. One consists of using a "less aggressive" objective function than the exponential function of AdaBoost, such as the logistic loss, to penalize less incorrectly classified points. Another solution is based on a regularization, e.g., the L_1-regularized AdaBoost described in the previous section.

An empirical study of AdaBoost has shown that uniform noise severely damages its accuracy. This has also been corroborated by recent theoretical results showing that boosting algorithms based on convex potentials do not tolerate even low levels of random noise. Moreover, these issues have been shown to persist even when using L_1-regularization or early stopping. However, the uniform noise model used in those experiments or analysis is rather unrealistic and seems unlikely to appear

in practice. The model assumes that a label corruption with some fixed probability affects all instances uniformly. Clearly, the performance of any algorithm should degrade in the presence of such noise. Empirical results suggest, however, that the performance of AdaBoost tends to degrade more than that of other algorithms for this uniform noise model.

Finally, notice that the behavior of AdaBoost in the presence of noise can be used, in fact, as a useful feature for detecting *outliers*, that is, examples that are incorrectly labeled or that are hard to classify. Examples with large weights after a certain number of rounds of boosting can be identified as outliers.

7.6 Chapter notes

The question of whether a weak learning algorithm could be *boosted* to derive a strong learning algorithm was first posed by Kearns and Valiant [1988, 1994], who also gave a negative proof of this result for a distribution-dependent setting. The first positive proof of this result in a distribution-independent setting was given by Schapire [1990], and later by Freund [1990].

These early boosting algorithms, boosting by filtering [Schapire, 1990] or boosting by majority [Freund, 1990, 1995] were not practical. The AdaBoost algorithm introduced by Freund and Schapire [1997] solved several of these practical issues. Freund and Schapire [1997] further gave a detailed presentation and analysis of the algorithm including the bound on its empirical error, a VC-dimension analysis, and its applications to multi-class classification and regression.

Early experiments with AdaBoost were carried out by Drucker, Schapire, and Simard [1993], who gave the first implementation in OCR with weak learners based on neural networks and Drucker and Cortes [1995], who reported the empirical performance of AdaBoost combined with decision trees, in particular decision stumps.

The fact that AdaBoost coincides with coordinate descent applied to an exponential objective function was later shown by Duffy and Helmbold [1999], Mason et al. [1999], and Friedman [2000]. Friedman, Hastie, and Tibshirani [2000] also gave an interpretation of boosting in terms of additive models. They also pointed out the close connections between AdaBoost and logistic regression, in particular the fact that their objective functions have a similar behavior near zero or the fact that their expectation admit the same minimizer, and derived an alternative boosting algorithm, LogitBoost, based on the logistic loss. Lafferty [1999] showed how an incremental family of algorithms, including LogitBoost, can be derived from Bregman divergences and designed to closely approximate AdaBoost when varying a parameter. Kivinen and Warmuth [1999] gave an equivalent view of AdaBoost as an entropy projection. They showed that the distribution over the sample found

by Adaboost at each round is approximately the solution to the problem of finding the closest distribution to the one at the previous round, subject to the constraint that it be orthogonal to the vector of errors of the current base hypotheses. Here, closeness is measured by a Bregman divergence, which, for AdaBoost is the unnormalized relative entropy. Collins, Schapire, and Singer [2002] later showed that boosting and logistic regression were special instances of a common framework based on Bregman divergences and used that to give the first convergence proof of AdaBoost. Another direct relationship between AdaBoost and logistic regression is given by Lebanon and Lafferty [2001] who showed that the two algorithms minimize the same extended relative entropy objective function subject to the same feature constraints, except from an additional normalization constraint for logistic regression.

A margin-based analysis of AdaBoost was first presented by Schapire, Freund, Bartlett, and Lee [1997], including theorem 7.7 which gives a bound on the empirical margin loss. Our presentation is based on the elegant derivation of margin bounds by Koltchinskii and Panchenko [2002] using the notion of Rademacher complexity. Rudin et al. [2004] gave an example showing that, in general, AdaBoost does not maximize the L_1-margin. Rätsch and Warmuth [2002] provided asymptotic lower bounds for the margin achieved by AdaBoost under some conditions. The L_1-margin maximization based on an LP is due to Grove and Schuurmans [1998]. Rätsch, Onoda, and Müller [2001] suggested a modification of that algorithm using a soft-margin instead and pointed out its connections with SVMs. The game-theoretic interpretation of boosting and the application of von Neumann's minimax theorem [von Neumann, 1928] in that context were pointed out by Freund and Schapire [1996, 1999b]; see also Grove and Schuurmans [1998] and Breiman [1999].

The L_1-regularized AdaBoost algorithm described in Section 7.4 is presented and analyzed by Rätsch, Mika, and Warmuth [2001]. Cortes, Mohri, and Syed [2014] introduced a new boosting algorithm, *DeepBoost*, which they proved to benefit from finer learning guarantees, including favorable ones even when using as base classifier set relatively rich families, for example a family of very deep decision trees, or other similarly complex families. In DeepBoost, the decisions in each iteration of which classifier to add to the ensemble and which weight to assign to that classifier, depend on the (data-dependent) complexity of the sub-family to which the classifier belongs. Cortes, Mohri, and Syed [2014] further showed that empirically DeepBoost achieves a better performance than AdaBoost, Logistic Regression, and their L_1-regularized variants. Both AdaBoost and L_1-regularized AdaBoost can be viewed as special instances of DeepBoost.

Dietterich [2000] provided extensive empirical evidence for the fact that uniform noise can severely damage the accuracy of AdaBoost. This has been reported by

a number of other authors since then. Long and Servedio [2010] further recently showed the failure of boosting algorithms based on convex potentials to tolerate random noise, even with L_1-regularization or early stopping.

There are several excellent surveys and tutorials related to boosting [Schapire, 2003, Meir and Rätsch, 2002, Meir and Rätsch, 2003], including the recent book of Schapire and Freund [2012] fully dedicated to this topic, with an extensive list of references and a detailed presentation.

7.7 Exercises

7.1 VC-dimension of the hypothesis set of AdaBoost. Prove the upper bound on the VC-dimension of the hypothesis set $\mathcal{F}_T$ of AdaBoost after T rounds of boosting, as stated in equation (7.9).

7.2 Alternative objective functions. This problem studies boosting-type algorithms defined with objective functions different from that of AdaBoost. We assume that the training data are given as m labeled examples $(x_1, y_1), \ldots, (x_m, y_m) \in \mathcal{X} \times \{-1, +1\}$. We further assume that Φ is a strictly increasing convex and differentiable function over $\mathbb{R}$ such that: $\forall x \geq 0, \Phi(x) \geq 1$ and $\forall x < 0, \Phi(x) > 0$.

(a) Consider the loss function $L(\alpha) = \sum_{i=1}^{m} \Phi(-y_i f(x_i))$ where f is a linear combination of base classifiers, i.e., $f = \sum_{t=1}^{T} \alpha_t h_t$ (as in AdaBoost). Derive a new boosting algorithm using the objective function L. In particular, characterize the best base classifier h_u to select at each round of boosting if we use coordinate descent.

(b) Consider the following functions: (1) zero-one loss $\Phi_1(-u) = 1_{u \leq 0}$; (2) least squared loss $\Phi_2(-u) = (1-u)^2$; (3) SVM loss $\Phi_3(-u) = \max\{0, 1-u\}$; and (4) logistic loss $\Phi_4(-u) = \log(1 + e^{-u})$. Which functions satisfy the assumptions on Φ stated earlier in this problem?

(c) For each loss function satisfying these assumptions, derive the corresponding boosting algorithm. How do the algorithm(s) differ from AdaBoost?

7.3 Update guarantee. Assume that the main weak learner assumption of AdaBoost holds. Let h_t be the base learner selected at round t. Show that the base learner h_{t+1} selected at round $t+1$ must be different from h_t.

7.4 Weighted instances. Let the training sample be $S = ((x_1, y_1), \ldots, (x_m, y_m))$. Suppose we wish to penalize differently errors made on x_i versus x_j. To do that, we associate some non-negative importance weight w_i to each point x_i and define

the objective function $F(\boldsymbol{\alpha}) = \sum_{i=1}^{m} w_i e^{-y_i f(x_i)}$, where $f = \sum_{t=1}^{T} \alpha_t h_t$. Show that this function is convex and differentiable and use it to derive a boosting-type algorithm.

7.5 Define the unnormalized correlation of two vectors $\mathbf{x}$ and $\mathbf{x}'$ as the inner product between these vectors. Prove that the distribution vector $(\mathcal{D}_{t+1}(1), \ldots, \mathcal{D}_{t+1}(m))$ defined by AdaBoost and the vector of components $y_i h_t(x_i)$ are uncorrelated.

7.6 Fix $\epsilon \in (0, 1/2)$. Let the training sample be defined by m points in the plane with $\frac{m}{4}$ negative points all at coordinate $(1, 1)$, another $\frac{m}{4}$ negative points all at coordinate $(-1, -1)$, $\frac{m(1-\epsilon)}{4}$ positive points all at coordinate $(1, -1)$, and $\frac{m(1+\epsilon)}{4}$ positive points all at coordinate $(-1, +1)$. Describe the behavior of AdaBoost when run on this sample using boosting stumps. What solution does the algorithm return after T rounds?

7.7 Noise-tolerant AdaBoost. AdaBoost may be significantly overfitting in the presence of noise, in part due to the high penalization of misclassified examples. To reduce this effect, one could use instead the following objective function:

$$F = \sum_{i=1}^{m} G(-y_i f(x_i)), \tag{7.32}$$

where G is the function defined on $\mathbb{R}$ by

$$G(x) = \begin{cases} e^x & \text{if } x \leq 0 \\ x + 1 & \text{otherwise.} \end{cases} \tag{7.33}$$

(a) Show that the function G is convex and differentiable.

(b) Use F and greedy coordinate descent to derive an algorithm similar to AdaBoost.

(c) Compare the reduction of the empirical error rate of this algorithm with that of AdaBoost.

7.8 Simplified AdaBoost. Suppose we simplify AdaBoost by setting the parameter α_t to a fixed value $\alpha_t = \alpha > 0$, independent of the boosting round t.

(a) Let γ be such that $(\frac{1}{2} - \epsilon_t) \geq \gamma > 0$. Find the best value of α as a function of γ by analyzing the empirical error.

(b) For this value of α, does the algorithm assign the same probability mass to correctly classified and misclassified examples at each round? If not, which set is assigned a higher probability mass?

AdaBoost($\mathbf{M}, t_{\max}$)

1 $\lambda_{1,j} \leftarrow 0$ for $i = 1, \ldots, m$
2 **for** $t \leftarrow 1$ **to** $t_{\max}$ **do**
3 $\quad d_{t,i} \leftarrow \frac{\exp(-(\mathbf{M}\boldsymbol{\lambda}_t)_i)}{\sum_{k=1}^{m} \exp(-(\mathbf{M}\boldsymbol{\lambda}_t)_k)}$ for $i = 1, \ldots, m$
4 $\quad j_t \leftarrow \text{argmax}_j(\mathbf{d}_t^\top \mathbf{M})_j$
5 $\quad r_t \leftarrow (\mathbf{d}_t^\top \mathbf{M})_{j_t}$
6 $\quad \alpha_t \leftarrow \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
7 $\quad \boldsymbol{\lambda}_{t+1} \leftarrow \boldsymbol{\lambda}_t + \alpha_t \mathbf{e}_{j_t}$, where $\mathbf{e}_{j_t}$ is 1 in position j_t and 0 elsewhere.
8 **return** $\frac{\boldsymbol{\lambda}_{t\max}}{\|\boldsymbol{\lambda}_{t\max}\|_1}$

Figure 7.7
AdaBoost defined with respect to a matrix $\mathbf{M}$, which encodes the accuracy of each weak classifier on each training point.

(c) Using the previous value of α, give a bound on the empirical error of the algorithm that depends only on γ and the number of rounds of boosting T.

(d) Using the previous bound, show that for $T > \frac{\log m}{2\gamma^2}$, the resulting hypothesis is consistent with the sample of size m.

(e) Let s be the VC-dimension of the base learners used. Give a bound on the generalization error of the consistent hypothesis obtained after $T = \left\lfloor \frac{\log m}{2\gamma^2} \right\rfloor + 1$ rounds of boosting. (*Hint*: Use the fact that the VC-dimension of the family of functions $\{\text{sgn}(\sum_{t=1}^{T} \alpha_t h_t) : \alpha_t \in \mathbb{R}\}$ is bounded by $2(s+1)T \log_2(eT)$). Suppose now that γ varies with m. Based on the bound derived, what can be said if $\gamma(m) = O(\sqrt{\frac{\log m}{m}})$?)

7.9 AdaBoost example.

In this exercise we consider a concrete example that consists of eight training points and eight weak classifiers.

(a) Define an $m \times n$ matrix $\mathbf{M}$ where $\mathbf{M}_{ij} = y_i h_j(\mathbf{x}_i)$, i.e., $\mathbf{M}_{ij} = +1$ if training example i is classified correctly by weak classifier h_j, and -1 otherwise. Let $\mathbf{d}_t, \boldsymbol{\lambda}_t \in \mathbb{R}^n$, $\|\mathbf{d}_t\|_1 = 1$ and $d_{t,i}$ (respectively $\lambda_{t,i}$) equal the ith component of $\mathbf{d}_t$ (respectively $\boldsymbol{\lambda}_t$). Now, consider AdaBoost as described in figure 7.7

and define $\mathbf{M}$ as below with eight training points and eight weak classifiers.

$$\mathbf{M} = \begin{pmatrix} -1 & 1 & 1 & 1 & 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 & 1 & -1 & 1 & 1 \\ 1 & -1 & 1 & 1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 & 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 & 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & 1 & -1 \end{pmatrix}$$

Assume that we start with the following initial distribution over the datapoints:

$$\mathbf{d}_1 = \left(\frac{3-\sqrt{5}}{8}, \frac{3-\sqrt{5}}{8}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{\sqrt{5}-1}{8}, \frac{\sqrt{5}-1}{8}, 0\right)^\top$$

Compute the first few steps of the AdaBoost algorithm using $\mathbf{M}$, $\mathbf{d}_1$, and $t_{max} = 7$. What weak classifier is picked at each round of boosting? Do you notice any pattern?

(b) What is the L_1 norm margin produced by AdaBoost for this example?

(c) Instead of using AdaBoost, imagine we combined our classifiers using the following coefficients: $[2, 3, 4, 1, 2, 2, 1, 1] \times \frac{1}{16}$. What is the margin in this case? Does AdaBoost maximize the margin?

7.10 Boosting in the presence of unknown labels. Consider the following variant of the classification problem where, in addition to the positive and negative labels $+1$ and -1, points may be labeled with 0. This can correspond to cases where the true label of a point is unknown, a situation that often arises in practice, or more generally to the fact that the learning algorithm incurs no loss for predicting -1 or $+1$ for such a point. Let $\mathcal{X}$ be the input space and let $\mathcal{Y} = \{-1, 0, +1\}$. As in standard binary classification, the loss of $f \colon \mathcal{X} \to \mathbb{R}$ on a pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$ is defined by $1_{yf(x)<0}$.

Consider a sample $S = ((x_1, y_1), \ldots, (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ and a hypothesis set H of base functions taking values in $\{-1, 0, +1\}$. For a base hypothesis $h_t \in \mathcal{H}$ and a distribution $\mathcal{D}_t$ over indices $i \in [m]$, define ϵ_t^s for $s \in \{-1, 0, +1\}$ by $\epsilon_t^s = \mathbb{E}_{i \sim \mathcal{D}_t}[1_{y_i h_t(x_i) = s}]$.

(a) Derive a boosting-style algorithm for this setting in terms of ϵ_t^ss, using the same objective function as that of AdaBoost. You should carefully justify the definition of the algorithm.

(b) What is the weak-learning assumption in this setting?

(c) Write the full pseudocode of the algorithm.

(d) Give an upper bound on the training error of the algorithm as a function of the number of rounds of boosting and ϵ_t^ss.

7.11 HingeBoost. As discussed in the chapter, AdaBoost can be viewed as coordinate descent applied to an exponential objective function. Here, we consider an alternative ensemble method algorithm, HingeBoost, that consists of applying coordinate descent to an objective function based on the hinge loss. Consider the function F defined for all $\boldsymbol{\alpha} \in \mathbb{R}^N$ by

$$F(\boldsymbol{\alpha}) = \sum_{i=1}^{m} \max\left(0, 1 - y_i \sum_{j=1}^{N} \alpha_j h_j(x_i)\right), \tag{7.34}$$

where the h_js are base classifiers belonging to a hypothesis set H of functions taking values -1 or $+1$.

(a) Show that F is convex and admits a right- and left-derivative along any direction.

(b) For any $j \in [N]$, let $\boldsymbol{e}_j$ denote the direction corresponding to the base hypothesis h_j. Let $\boldsymbol{\alpha}_t$ denote the vector of coefficients $\alpha_{t,j}$, $j \in [N]$ obtained after $t \geq 0$ iterations of coordinate descent and $f_t = \sum_{j=1}^{N} \alpha_{t,j} h_j$ the predictor obtained after t iterations.

Give the expression of the right-derivative $F'_+(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j)$ and the left-derivative $F'_-(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j)$ after $t-1$ iterations in terms of f_{t-1}.

(c) For any $j \in [N]$, define the maximum directional derivative $\delta F(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j)$ at $\boldsymbol{\alpha}_{t-1}$ as follows:

$$\delta F(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j) = \begin{cases} 0 & \text{if } F'_-(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j) \leq 0 \leq F'_+(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j) \\ F'_+(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j) & \text{if } F'_-(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j) \leq F'_+(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j) \leq 0 \\ F'_-(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j) & \text{if } 0 \leq F'_-(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j) \leq F'_+(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j). \end{cases}$$

The direction $\boldsymbol{e}_j$ considered by the coordinate descent considered here is the one maximizing $|\delta F(\boldsymbol{\alpha}_{t-1}, \boldsymbol{e}_j)|$. Once the best direction j is selected, the

step η can be determined by minimizing $F(\boldsymbol{\alpha}_{t-1} + \eta \boldsymbol{e}_j)$ using a grid search. Give the pseudocode of HingeBoost.

7.12 Empirical margin loss boosting. As discussed in the chapter, AdaBoost can be viewed as coordinate descent applied to a convex upper bound on the empirical error. Here, we consider an algorithm seeking to minimize the empirical margin loss. For any $0 \leq \rho < 1$ let $\widehat{R}_{S,\rho}(f) = \frac{1}{m} \sum_{i=1}^{m} 1_{y_i f(x_i) \leq \rho}$ denote the empirical margin loss of a function f of the form $f = \frac{\sum_{t=1}^{T} \alpha_t h_t}{\sum_{t=1}^{T} \alpha_t}$ for a labeled sample $S = ((x_1, y_1), \ldots, (x_m, y_m))$.

(a) Show that $\widehat{R}_{S,\rho}(f)$ can be upper bounded as follows:

$$\widehat{R}_{S,\rho}(f) \leq \frac{1}{m} \sum_{i=1}^{m} \exp\left(-y_i \sum_{t=1}^{T} \alpha_t h_t(x_i) + \rho \sum_{t=1}^{T} \alpha_t \right).$$

(b) For any $\rho > 0$, let G_ρ be the objective function defined for all $\boldsymbol{\alpha} \geq 0$ by

$$G_\rho(\boldsymbol{\alpha}) = \frac{1}{m} \sum_{i=1}^{m} \exp\left(-y_i \sum_{j=1}^{N} \alpha_j h_j(x_i) + \rho \sum_{j=1}^{N} \alpha_j \right),$$

with $h_j \in H$ for all $j \in [N]$, with the notation used in class in the boosting lecture. Show that G_ρ is convex and differentiable.

(c) Derive a boosting-style algorithm $\mathcal{A}_\rho$ by applying (maximum) coordinate descent to G_ρ. You should justify in detail the derivation of the algorithm, in particular the choice of the base classifier selected at each round and that of the step. Compare both to their counterparts in AdaBoost.

(d) What is the equivalent of the weak learning assumption for $\mathcal{A}_\rho$ (*Hint*: use non-negativity of the step value)?

(e) Give the full pseudocode of the algorithm $\mathcal{A}_\rho$. What can you say about the $\mathcal{A}_0$ algorithm?

(f) Provide a bound on $\widehat{R}_{S,\rho}(f)$.

i. Prove the upper bound $\widehat{R}_{S,\rho}(f) \leq \exp\left(\sum_{t=1}^{T} \alpha_t \rho\right) \prod_{t=1}^{T} Z_t$, where the normalization factors Z_t are defined as in the case of AdaBoost (with α_t the step chosen by $\mathcal{A}_\rho$ at round t).

ii. Give the expression of Z_t as a function of ρ and ϵ_t, where ϵ_t is the weighted error of the hypothesis found by $\mathcal{A}_\rho$ at round t (defined in the same way

as for AdaBoost in class). Use that to prove the following upper bound

$$\widehat{R}_{S,\rho}(f) \leq \left(u^{\frac{1+\rho}{2}} + u^{-\frac{1-\rho}{2}}\right)^T \prod_{t=1}^{T} \sqrt{\epsilon_t^{1-\rho}(1-\epsilon_t)^{1+\rho}},$$

where $u = \frac{1-\rho}{1+\rho}$.

iii. Assume that for all $t \in [T]$, $\frac{1-\rho}{2} - \epsilon_t > \gamma > 0$. Use the result of the previous question to show that

$$\widehat{R}_{S,\rho}(f) \leq \exp\left(-\frac{2\gamma^2 T}{1-\rho^2}\right).$$

(*Hint*: you can use without proof the following identity:

$$\left(u^{\frac{1+\rho}{2}} + u^{-\frac{1-\rho}{2}}\right)\sqrt{\epsilon_t^{1-\rho}(1-\epsilon_t)^{1+\rho}} \leq 1 - 2\frac{\left(\frac{1-\rho}{2} - \epsilon_t\right)^2}{1-\rho^2},$$

valid for $\frac{1-\rho}{2} - \epsilon_t > 0$.) Show that for $T \geq \frac{(\log m)(1-\rho^2)}{2\gamma^2}$, all points of the training data have margin at least ρ.

8 On-Line Learning

This chapter presents an introduction to on-line learning, an important area with a rich literature and multiple connections with game theory and optimization that is increasingly influencing the theoretical and algorithmic advances in machine learning. In addition to the intriguing novel learning theory questions that they raise, on-line learning algorithms are particularly attractive in modern applications since they provide an efficient solution for large-scale problems.

These algorithms process one sample at a time with an update per iteration that is often computationally cheap and easy to implement. As a result, they are typically significantly more efficient both in time and space and more practical than batch algorithms, when processing modern data sets of several million or billion points. They are also typically easy to implement. Moreover, on-line algorithms do not require any distributional assumption; their analysis assumes an adversarial scenario. This makes them applicable in a variety of scenarios where the sample points are not drawn i.i.d. or according to a fixed distribution.

We first introduce the general scenario of on-line learning, then present and analyze several key algorithms for on-line learning with expert advice, including the deterministic and randomized weighted majority algorithms for the zero-one loss and an extension of these algorithms for convex losses. We also describe and analyze two standard on-line algorithms for linear classification, the Perceptron and Winnow algorithms, as well as some extensions. While on-line learning algorithms are designed for an adversarial scenario, they can be used, under some assumptions, to derive accurate predictors for a distributional scenario. We derive learning guarantees for this on-line to batch conversion. Finally, we briefly point out the connection of on-line learning with game theory by describing its use to derive a simple proof of von Neumann's minimax theorem.

8.1 Introduction

The learning framework for on-line algorithms is in stark contrast to the PAC learning or stochastic models discussed up to this point. First, instead of learning from a training set and then testing on a test set, the on-line learning scenario mixes the training and test phases. Second, PAC learning follows the key assumption that the distribution over data points is fixed over time, both for training and test points, and that points are sampled in an i.i.d. fashion. Under this assumption, the natural goal is to learn a hypothesis with a small expected loss or generalization error. In contrast, with on-line learning, *no distributional assumption* is made, and thus there is no notion of generalization. Instead, the performance of on-line learning algorithms is measured using a *mistake model* and the notion of *regret*. To derive guarantees in this model, theoretical analyses are based on a worst-case or adversarial assumption.

The general on-line setting involves T rounds. At the tth round, the algorithm receives an instance $x_t \in \mathcal{X}$ and makes a prediction $\widehat{y}_t \in \mathcal{Y}$. It then receives the true label $y_t \in \mathcal{Y}$ and incurs a loss $L(\widehat{y}_t, y_t)$, where $L\colon \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_+$ is a loss function. More generally, the prediction domain for the algorithm may be $\mathcal{Y}' \neq \mathcal{Y}$ and the loss function defined over $\mathcal{Y}' \times \mathcal{Y}$. For classification problems, we often have $\mathcal{Y} = \{0, 1\}$ and $L(y, y') = |y' - y|$, while for regression $\mathcal{Y} \subseteq \mathbb{R}$ and typically $L(y, y') = (y' - y)^2$. The objective in the on-line setting is to minimize the cumulative loss: $\sum_{t=1}^{T} L(\widehat{y}_t, y_t)$ over T rounds.

8.2 Prediction with expert advice

We first discuss the setting of online learning with expert advice, and the associated notion of *regret*. In this setting, at the tth round, in addition to receiving $x_t \in \mathcal{X}$, the algorithm also receives *advice* $y_{t,i} \in \mathcal{Y}$, $i \in [N]$, from N experts. Following the general framework of on-line algorithms, it then makes a prediction, receives the true label, and incurs a loss. After T rounds, the algorithm has incurred a cumulative loss. The objective in this setting is to minimize the *regret* R_T, also called *external regret*, which compares the cumulative loss of the algorithm to that of the best expert in hindsight after T rounds:

$$R_T = \sum_{t=1}^{T} L(\widehat{y}_t, y_t) - \min_{i=1}^{N} \sum_{t=1}^{T} L(\widehat{y}_{t,i}, y_t). \tag{8.1}$$



Figure 8.1
Weather forecast: an example of a prediction problem based on expert advice.

This problem arises in a variety of different domains and applications. Figure 8.1 illustrates the problem of predicting the weather using several forecasting sources as experts.

8.2.1 Mistake bounds and Halving algorithm

Here, we assume that the loss function is the standard zero-one loss used in classification. To analyze the expert advice setting, we first consider the realizable case, that is the setting where at least one of the experts makes no errors. As such, we discuss the *mistake bound model*, which asks the simple question "How many mistakes before we learn a particular concept?" Since we are in the realizable case, after some number of rounds T, we will learn the concept and no longer make errors in subsequent rounds. For any fixed concept c, we define the maximum number of mistakes a learning algorithm $\mathcal{A}$ makes as

$$M_{\mathcal{A}}(c) = \max_{x_1,\ldots,x_T} |\text{mistakes}(\mathcal{A}, c)|. \tag{8.2}$$

Further, for any concept in a concept class $\mathcal{C}$, the maximum number of mistakes a learning algorithm makes is

$$M_{\mathcal{A}}(\mathcal{C}) = \max_{c \in \mathcal{C}} M_{\mathcal{A}}(c). \tag{8.3}$$

Our goal in this setting is to derive *mistake bounds*, that is, a bound M on $M_{\mathcal{A}}(\mathcal{C})$. We will first do this for the Halving algorithm, an elegant and simple algorithm for which we can guarantee surprisingly favorable mistake bounds. At each round, the Halving algorithm makes its prediction by taking the majority vote over all *active* experts. After any incorrect prediction, it deactivates all experts that gave faulty advice. Initially, all experts are active, and by the time the algorithm has converged to the correct concept, the active set contains only those experts that are consistent with the target concept. The pseudocode for this algorithm is shown in figure 8.2. We also present straightforward mistake bounds in theorems 8.1 and 8.2, where the former deals with finite hypothesis sets and the latter relates mistake bounds to VC-dimension. Note that the hypothesis complexity term in theorem 8.1 is identical to the corresponding complexity term in the PAC model bound of theorem 2.5.

HALVING($\mathcal{H}$)

1 $\mathcal{H}_1 \leftarrow \mathcal{H}$
2 **for** $t \leftarrow 1$ **to** T **do**
3 RECEIVE(x_t)
4 $\widehat{y}_t \leftarrow$ MAJORITYVOTE($\mathcal{H}_t, x_t$)
5 RECEIVE(y_t)
6 **if** $(\widehat{y}_t \neq y_t)$ **then**
7 $\mathcal{H}_{t+1} \leftarrow \{c \in \mathcal{H}_t : c(x_t) = y_t\}$
8 **else** $\mathcal{H}_{t+1} \leftarrow \mathcal{H}_t$
9 **return** $\mathcal{H}_{T+1}$

Figure 8.2
Halving algorithm.

Theorem 8.1 *Let $\mathcal{H}$ be a finite hypothesis set. Then*

$$M_{\text{HALVING}}(\mathcal{H}) \leq \log_2 |\mathcal{H}|. \tag{8.4}$$

Proof: Since at each round the algorithm makes predictions using majority vote from the active set, at each mistake, the active set is reduced by at least half. Hence, after $\log_2 |\mathcal{H}|$ mistakes, there can only remain one active hypothesis, and since we are in the realizable case, this hypothesis must coincide with the target concept. □

Theorem 8.2 *Let $opt(\mathcal{H})$ be the optimal mistake bound for $\mathcal{H}$. Then,*

$$\text{VCdim}(\mathcal{H}) \leq opt(\mathcal{H}) \leq M_{\text{HALVING}}(\mathcal{H}) \leq \log_2 |\mathcal{H}|. \tag{8.5}$$

Proof: The second inequality is true by definition and the third inequality holds based on theorem 8.1. To prove the first inequality, we let $d = \text{VCdim}(\mathcal{H})$. Then there exists a shattered set of d points, for which we can form a complete binary tree of the mistakes with height d, and we can choose labels at each round of learning to ensure that d mistakes are made. Note that this adversarial argument is valid since the on-line setting makes no statistical assumptions about the data. □

```
WEIGHTED-MAJORITY(N)
 1  for i ← 1 to N do
 2       w_{1,i} ← 1
 3  for t ← 1 to T do
 4       RECEIVE(x_t)
 5       if Σ_{i: y_{t,i}=1} w_{t,i} ≥ Σ_{i: y_{t,i}=0} w_{t,i} then
 6            ŷ_t ← 1
 7       else ŷ_t ← 0
 8       RECEIVE(y_t)
 9       if (ŷ_t ≠ y_t) then
10            for i ← 1 to N do
11                 if (y_{t,i} ≠ y_t) then
12                      w_{t+1,i} ← β w_{t,i}
13                 else w_{t+1,i} ← w_{t,i}
14  return w_{T+1}
```

Figure 8.3
Weighted majority algorithm, $y_t, y_{t,i} \in \{0, 1\}$.

8.2.2 Weighted majority algorithm

In the previous section, we focused on the realizable setting in which the Halving algorithm simply discarded experts after a single mistake. We now move to the non-realizable setting and use a more general and less extreme algorithm, the Weighted Majority (WM) algorithm, that *weights* the importance of experts as a function of their mistake rate. The WM algorithm begins with uniform weights over all N experts. At each round, it generates predictions using a weighted majority vote. After receiving the true label, the algorithm then reduces the weight of each incorrect expert by a factor of $\beta \in [0, 1)$. Note that this algorithm reduces to the Halving algorithm when $\beta = 0$. The pseudocode for the WM algorithm is shown in figure 8.3.

Since we are not in the realizable setting, the mistake bounds of theorem 8.1 cannot apply. However, the following theorem presents a bound on the number of mistakes m_T made by the WM algorithm after $T \geq 1$ rounds of on-line learning as a function of the number of mistakes made by the best expert, that is the expert

who achieves the smallest number of mistakes for the sequence $y_1, \ldots, y_T$. Let us emphasize that this is the best expert *in hindsight*.

Theorem 8.3 *Fix $\beta \in (0,1)$. Let m_T be the number of mistakes made by algorithm WM after $T \geq 1$ rounds, and m_T^* be the number of mistakes made by the best of the N experts. Then, the following inequality holds:*

$$m_T \leq \frac{\log N + m_T^* \log \frac{1}{\beta}}{\log \frac{2}{1+\beta}}. \tag{8.6}$$

Proof: To prove this theorem, we first introduce a potential function. We then derive upper and lower bounds for this function, and combine them to obtain our result. This potential function method is a general proof technique that we will use throughout this chapter.

For any $t \geq 1$, we define our potential function as $W_t = \sum_{i=1}^{N} w_{t,i}$. Since predictions are generated using weighted majority vote, if the algorithm makes an error at round t, this implies that

$$W_{t+1} \leq \big[1/2 + (1/2)\beta\big] W_t = \left[\frac{1+\beta}{2}\right] W_t. \tag{8.7}$$

Since $W_1 = N$ and m_T mistakes are made after T rounds, we thus have the following upper bound:

$$W_T \leq \left[\frac{1+\beta}{2}\right]^{m_T} N. \tag{8.8}$$

Next, since the weights are all non-negative, it is clear that for any expert i, $W_T \geq w_{T,i} = \beta^{m_{T,i}}$, where $m_{T,i}$ is the number of mistakes made by the ith expert after T rounds. Applying this lower bound to the best expert and combining it with the upper bound in (8.8) gives us:

$$\begin{aligned} \beta^{m_T^*} &\leq W_T \leq \left[\frac{1+\beta}{2}\right]^{m_T} N \\ \Rightarrow\ & m_T^* \log \beta \leq \log N + m_T \log \left[\frac{1+\beta}{2}\right] \\ \Rightarrow\ & m_T \log \left[\frac{2}{1+\beta}\right] \leq \log N + m_T^* \log \frac{1}{\beta}, \end{aligned}$$

which concludes the proof. □

Thus, the theorem guarantees a bound of the following form for algorithm WM:

$$m_T \leq O(\log N) + \text{constant} \times |\text{mistakes of best expert}|.$$

Since the first term varies only logarithmically as a function of N, the theorem guarantees that the number of mistakes is roughly a constant times that of the best expert in hindsight. This is a remarkable result, especially because it requires no

assumption about the sequence of points and labels generated. In particular, the sequence could be chosen adversarially. In the realizable case where $m_T^* = 0$, the bound reduces to $m_T \leq O(\log N)$ as for the Halving algorithm.

8.2.3 Randomized weighted majority algorithm

In spite of the guarantees just discussed, the WM algorithm admits a drawback that affects all deterministic algorithms in the case of the zero-one loss: no deterministic algorithm can achieve a regret $R_T = o(T)$ over all sequences. Clearly, for any deterministic algorithm $\mathcal{A}$ and any $t \in [T]$, we can adversarially select y_t to be 1 if the algorithm predicts 0, and choose it to be 0 otherwise. Thus, $\mathcal{A}$ errs at every point of such a sequence and its cumulative mistake is $m_T = T$. Assume for example that $N = 2$ and that one expert always predicts 0, the other one always 1. The error of the best expert over that sequence (and in fact any sequence of that length) is then at most $m_T^* \leq T/2$. Thus, for that sequence, we have

$$R_T = m_T - m_T^* \geq T/2,$$

which shows that $R_T = o(T)$ cannot be achieved in general. Note that this does not contradict the bound proven in the previous section, since for any $\beta \in (0, 1)$, $\frac{\log \frac{1}{\beta}}{\log \frac{2}{1+\beta}} \geq 2$. As we shall see in the next section, this negative result does not hold for any loss that is convex with respect to one of its arguments. But for the zero-one loss, this leads us to consider randomized algorithms instead.

In the randomized scenario of on-line learning, we assume that a set $\mathcal{A} = \{1, \ldots, N\}$ of N actions is available. At each round $t \in [T]$, an on-line algorithm $\mathcal{A}$ selects a distribution $\mathbf{p}_t$ over the set of actions, receives a loss vector $\mathbf{l}_t$, whose ith component $l_{t,i} \in [0, 1]$ is the loss associated with action i, and incurs the expected loss $L_t = \sum_{i=1}^N p_{t,i}\, l_{t,i}$. The total loss incurred by the algorithm over T rounds is $\mathcal{L}_T = \sum_{t=1}^T L_t$. The total loss associated to action i is $\mathcal{L}_{T,i} = \sum_{t=1}^T l_{t,i}$. The minimal loss of a single action is denoted by $\mathcal{L}_T^{\min} = \min_{i \in \mathcal{A}} \mathcal{L}_{T,i}$. The regret R_T of the algorithm after T rounds is then typically defined by the difference of the loss of the algorithm and that of the best single action:[11]

$$R_T = \mathcal{L}_T - \mathcal{L}_T^{\min}.$$

Here, we consider specifically the case of zero-one losses and assume that $l_{t,i} \in \{0, 1\}$ for all $t \in [T]$ and $i \in \mathcal{A}$.

[11] Alternative definitions of the regret with comparison classes different from the set of single actions can be considered.

```
RANDOMIZED-WEIGHTED-MAJORITY (N)
 1  for i ← 1 to N do
 2      w_{1,i} ← 1
 3      p_{1,i} ← 1/N
 4  for t ← 1 to T do
 5      RECEIVE(l_t)
 6      for i ← 1 to N do
 7          if (l_{t,i} = 1) then
 8              w_{t+1,i} ← β w_{t,i}
 9          else w_{t+1,i} ← w_{t,i}
10      W_{t+1} ← Σ_{i=1}^{N} w_{t+1,i}
11      for i ← 1 to N do
12          p_{t+1,i} ← w_{t+1,i} / W_{t+1}
13  return w_{T+1}
```

Figure 8.4
Randomized weighted majority algorithm.

The WM algorithm admits a straightforward randomized version, the randomized weighted majority (RWM) algorithm. The pseudocode of this algorithm is given in figure 8.4. The algorithm updates the weight $w_{t,i}$ of expert i as in the case of the WM algorithm by multiplying it by β. The following theorem gives a strong guarantee on the regret R_T of the RWM algorithm, showing that it is in $O(\sqrt{T \log N})$.

Theorem 8.4 *Fix $\beta \in [1/2, 1)$. Then, for any $T \geq 1$, the loss of algorithm RWM on any sequence can be bounded as follows:*

$$\mathcal{L}_T \leq \frac{\log N}{1-\beta} + (2-\beta)\mathcal{L}_T^{\min}. \tag{8.9}$$

In particular, for $\beta = \max\{1/2, 1 - \sqrt{(\log N)/T}\}$, the loss can be bounded as:

$$\mathcal{L}_T \leq \mathcal{L}_T^{\min} + 2\sqrt{T \log N}. \tag{8.10}$$

Proof: As in the proof of theorem 8.3, we derive upper and lower bounds for the potential function $W_t = \sum_{i=1}^{N} w_{t,i}$, $t \in [T]$, and combine these bounds to obtain

the result. By definition of the algorithm, for any $t \in [T]$, W_{t+1} can be expressed as follows in terms of W_t:

$$\begin{aligned} W_{t+1} = \sum_{i:\, l_{t,i}=0} w_{t,i} + \beta \sum_{i:\, l_{t.i}=1} w_{t,i} &= W_t + (\beta - 1) \sum_{i:\, l_{t,i}=1} w_{t,i} \\ &= W_t + (\beta - 1) W_t \sum_{i:\, l_{t,i}=1} p_{t,i} \\ &= W_t + (\beta - 1) W_t L_t \\ &= W_t (1 - (1 - \beta) L_t). \end{aligned}$$

Thus, since $W_1 = N$, it follows that $W_{T+1} = N \prod_{t=1}^{T} (1 - (1 - \beta) L_t)$. On the other hand, the following lower bound clearly holds: $W_{T+1} \geq \max_{i \in [N]} w_{T+1,i} = \beta^{\mathcal{L}_T^{\min}}$. This leads to the following inequality and series of derivations after taking the log and using the inequalities $\log(1 - x) \leq -x$ valid for all $x < 1$, and $-\log(1 - x) \leq x + x^2$ valid for all $x \in [0, 1/2]$:

$$\begin{aligned} \beta^{\mathcal{L}_T^{\min}} \leq N \prod_{t=1}^{T} (1 - (1 - \beta) L_t) &\implies \mathcal{L}_T^{\min} \log \beta \leq \log N + \sum_{t=1}^{T} \log(1 - (1 - \beta) L_t) \\ &\implies \mathcal{L}_T^{\min} \log \beta \leq \log N - (1 - \beta) \sum_{t=1}^{T} L_t \\ &\implies \mathcal{L}_T^{\min} \log \beta \leq \log N - (1 - \beta) \mathcal{L}_T \\ &\implies \mathcal{L}_T \leq \frac{\log N}{1 - \beta} - \frac{\log \beta}{1 - \beta} \mathcal{L}_T^{\min} \\ &\implies \mathcal{L}_T \leq \frac{\log N}{1 - \beta} - \frac{\log(1 - (1 - \beta))}{1 - \beta} \mathcal{L}_T^{\min} \\ &\implies \mathcal{L}_T \leq \frac{\log N}{1 - \beta} + (2 - \beta) \mathcal{L}_T^{\min}. \end{aligned}$$

This shows the first statement. Since $\mathcal{L}_T^{\min} \leq T$, this also implies

$$\mathcal{L}_T \leq \frac{\log N}{1 - \beta} + (1 - \beta) T + \mathcal{L}_T^{\min}. \tag{8.11}$$

Differentiating the upper bound with respect to β and setting it to zero gives $\frac{\log N}{(1-\beta)^2} - T = 0$, that is $\beta = 1 - \sqrt{(\log N)/T} < 1$. Thus, if $1 - \sqrt{(\log N)/T} \geq 1/2$, $\beta_0 = 1 - \sqrt{(\log N)/T}$ is the minimizing value of β, otherwise the boundary value $\beta_0 = 1/2$ is the optimal value. The second statement follows by replacing β with β_0 in (8.11). □

The bound (8.10) assumes that the algorithm additionally receives as a parameter the number of rounds T. As we shall see in the next section, however, there exists a general *doubling trick* that can be used to relax this requirement at the price of a

small constant factor increase. Inequality 8.10 can be written directly in terms of the regret R_T of the RWM algorithm:

$$R_T \leq 2\sqrt{T \log N}. \tag{8.12}$$

Thus, for N constant, the regret verifies $R_T = O(\sqrt{T})$ and the *average regret* or *regret per round* R_T/T decreases as $O(1/\sqrt{T})$. These results are optimal, as shown by the following theorem.

Theorem 8.5 *Let $N = 2$. There exists a stochastic sequence of losses for which the regret of any on-line learning algorithm verifies $\mathbb{E}[R_T] \geq \sqrt{T/8}$.*

Proof: For any $t \in [T]$, let the vector of losses $\mathbf{l}_t$ take the values $\mathbf{l}_{01} = (0,1)^\top$ and $\mathbf{l}_{10} = (1,0)^\top$ with equal probability. Then, the expected loss of any randomized algorithm $\mathcal{A}$ is

$$\mathbb{E}[\mathcal{L}_T] = \mathbb{E}\Big[\sum_{t=1}^{T} \mathbf{p}_t \cdot \mathbf{l}_t\Big] = \sum_{t=1}^{T} \mathbf{p}_t \cdot \mathbb{E}[\mathbf{l}_t] = \sum_{t=1}^{T} \frac{1}{2} p_{t,1} + \frac{1}{2}(1 - p_{t,1}) = T/2,$$

where we denoted by $\mathbf{p}_t$ the distribution selected by $\mathcal{A}$ at round t. By definition, $\mathcal{L}_T^{\min}$ can be written as follows:

$$\mathcal{L}_T^{\min} = \min\{\mathcal{L}_{T,1}, \mathcal{L}_{T,2}\} = \frac{1}{2}(\mathcal{L}_{T,1} + \mathcal{L}_{T,2} - |\mathcal{L}_{T,1} - \mathcal{L}_{T,2}|) = T/2 - |\mathcal{L}_{T,1} - T/2|,$$

using the fact that $\mathcal{L}_{T,1} + \mathcal{L}_{T,2} = T$. Thus, the expected regret of $\mathcal{A}$ is

$$\mathbb{E}[R_T] = \mathbb{E}[\mathcal{L}_T] - \mathbb{E}[\mathcal{L}_T^{\min}] = \mathbb{E}[|\mathcal{L}_{T,1} - T/2|].$$

Let σ_t, $t \in [T]$, denote Rademacher variables taking values in $\{-1, +1\}$, then $\mathcal{L}_{T,1}$ can be rewritten as $\mathcal{L}_{T,1} = \sum_{t=1}^{T} \frac{1+\sigma_t}{2} = T/2 + \frac{1}{2}\sum_{t=1}^{T} \sigma_t$. Thus, introducing scalars $x_t = 1/2$, $t \in [T]$, by the Khintchine-Kahane inequality, (D.24) we have:

$$\mathbb{E}[R_T] = \mathbb{E}\Big[|\sum_{t=1}^{T} \sigma_t x_t|\Big] \geq \sqrt{\frac{1}{2}\sum_{t=1}^{T} x_t^2} = \sqrt{T/8},$$

which concludes the proof. □

More generally, for $T \geq N$, a lower bound of $R_T = \Omega(\sqrt{T \log N})$ can be proven for the regret of any algorithm.

8.2.4 Exponential weighted average algorithm

The WM algorithm can be extended to other loss functions L taking values in $[0,1]$. The Exponential Weighted Average algorithm presented here can be viewed as that extension for the case where L is convex in its first argument. Note that this algorithm is deterministic and yet, as we shall see, admits a very favorable regret

EXPONENTIAL-WEIGHTED-AVERAGE (N)

for $i \leftarrow 1$ **to** N **do**
    $w_{1,i} \leftarrow 1$
for $t \leftarrow 1$ **to** T **do**
    RECEIVE(x_t)
    $\widehat{y}_t \leftarrow \frac{\sum_{i=1}^N w_{t,i} y_{t,i}}{\sum_{i=1}^N w_{t,i}}$
    RECEIVE(y_t)
    for $i \leftarrow 1$ **to** N **do**
        $w_{t+1,i} \leftarrow w_{t,i}\, e^{-\eta L(\widehat{y}_{t,i}, y_t)}$
return $\mathbf{w}_{T+1}$

Figure 8.5
Exponential weighted average, $L(\widehat{y}_{t,i}, y_t) \in [0,1]$.

guarantee. Figure 8.5 gives its pseudocode. At round $t \in [T]$, the algorithm's prediction is

$$\widehat{y}_t = \frac{\sum_{i=1}^N w_{t,i} y_{t,i}}{\sum_{i=1}^N w_{t,i}}, \tag{8.13}$$

where $y_{t,i}$ is the prediction by expert i and $w_{t,i}$ the weight assigned by the algorithm to that expert. Initially, all weights are set to one. The algorithm then updates the weights at the end of round t according to the following rule:

$$w_{t+1,i} \leftarrow w_{t,i}\, e^{-\eta L(\widehat{y}_{t,i}, y_t)} = e^{-\eta L_{t,i}}, \tag{8.14}$$

where $L_{t,i}$ is the total loss incurred by expert i after t rounds. Note that this algorithm, as well as the others presented in this chapter, are simple, since they do not require keeping track of the losses incurred by each expert at all previous rounds but only of their cumulative performance. Furthermore, this property is also computationally advantageous. The following theorem presents a regret bound for this algorithm.

Theorem 8.6 *Assume that the loss function L is convex in its first argument and takes values in $[0,1]$. Then, for any $\eta > 0$ and any sequence $y_1, \ldots, y_T \in \mathcal{Y}$, the regret of the Exponential Weighted Average algorithm after T rounds satisfies*

$$R_T \leq \frac{\log N}{\eta} + \frac{\eta T}{8}. \tag{8.15}$$

In particular, for $\eta = \sqrt{8 \log N / T}$*, the regret is bounded as*

$$R_T \leq \sqrt{(T/2) \log N}. \tag{8.16}$$

Proof: We apply the same potential function analysis as in previous proofs but using as potential $\Phi_t = \log \sum_{i=1}^N w_{t,i}$, $t \in [T]$. Let $\mathbf{p}_t$ denote the distribution over $\{1, \ldots, N\}$ with $p_{t,i} = \frac{w_{t,i}}{\sum_{i=1}^N w_{t,i}}$. To derive an upper bound on Φ_t, we first examine the difference of two consecutive potential values:

$$\Phi_{t+1} - \Phi_t = \log \frac{\sum_{i=1}^N w_{t,i}\, e^{-\eta L(\widehat{y}_{t,i}, y_t)}}{\sum_{i=1}^N w_{t,i}} = \log \big(\mathop{\mathbb{E}}_{\mathbf{p}_t} [e^{\eta X}] \big),$$

with $X = -L(\widehat{y}_{t,i}, y_t) \in [-1, 0]$. To upper bound the expression appearing in the right-hand side, we apply Hoeffding's lemma (lemma D.1) to the centered random variable $X - \mathbb{E}_{\mathbf{p}_t}[X]$, then Jensen's inequality (theorem B.20) using the convexity of L with respect to its first argument:

$$\begin{aligned}
\Phi_{t+1} - \Phi_t &= \log \big(\mathop{\mathbb{E}}_{\mathbf{p}_t} \big[e^{\eta(X - \mathbb{E}[X]) + \eta\, \mathbb{E}[X]} \big] \big) \\
&\leq \frac{\eta^2}{8} + \eta \mathop{\mathbb{E}}_{\mathbf{p}_t}[X] = \frac{\eta^2}{8} - \eta \mathop{\mathbb{E}}_{\mathbf{p}_t}[L(\widehat{y}_{t,i}, y_t)] && \text{(Hoeffding's lemma)} \\
&\leq -\eta L\big(\mathop{\mathbb{E}}_{\mathbf{p}_t}[\widehat{y}_{t,i}], y_t \big) + \frac{\eta^2}{8} && \text{(convexity of first arg. of } L\text{)} \\
&= -\eta L(\widehat{y}_t, y_t) + \frac{\eta^2}{8}.
\end{aligned}$$

Summing up these inequalities yields the following upper bound:

$$\Phi_{T+1} - \Phi_1 \leq -\eta \sum_{t=1}^T L(\widehat{y}_t, y_t) + \frac{\eta^2 T}{8}. \tag{8.17}$$

We obtain a lower bound for the same quantity as follows:

$$\Phi_{T+1} - \Phi_1 = \log \sum_{i=1}^N e^{-\eta L_{T,i}} - \log N \geq \log \max_{i=1}^N e^{-\eta L_{T,i}} - \log N = -\eta \min_{i=1}^N L_{T,i} - \log N.$$

Combining the upper and lower bounds yields:

$$\begin{aligned}
&-\eta \min_{i=1}^N L_{T,i} - \log N \leq -\eta \sum_{t=1}^T L(\widehat{y}_t, y_t) + \frac{\eta^2 T}{8} \\
\implies &\sum_{t=1}^T L(\widehat{y}_t, y_t) - \min_{i=1}^N L_{T,i} \leq \frac{\log N}{\eta} + \frac{\eta T}{8},
\end{aligned}$$

and concludes the proof. □

The optimal choice of η in theorem 8.6 requires knowledge of the horizon T, which is an apparent disadvantage of this analysis. However, we can use a standard *doubling trick* to eliminate this requirement, at the price of a small constant factor. This consists of dividing time into periods $[2^k, 2^{k+1}-1]$ of length 2^k with $k = 0, \ldots, n$ and $T \geq 2^n - 1$, and then choosing $\eta_k = \sqrt{\frac{8 \log N}{2^k}}$ in each period. The following theorem presents a regret bound when using the doubling trick to select η. A more general method consists of interpreting η as a function of time, i.e., $\eta_t = \sqrt{(8 \log N)/t}$, which can lead to a further constant factor improvement over the regret bound of the following theorem.

Theorem 8.7 *Assume that the loss function L is convex in its first argument and takes values in $[0,1]$. Then, for any $T \geq 1$ and any sequence $y_1, \ldots, y_T \in \mathcal{Y}$, the regret of the Exponential Weighted Average algorithm after T rounds is bounded as follows:*

$$R_T \leq \frac{\sqrt{2}}{\sqrt{2}-1}\sqrt{(T/2)\log N} + \sqrt{\log N/2}. \tag{8.18}$$

Proof: Let $T \geq 1$ and let $\mathcal{I}_k = [2^k, 2^{k+1}-1]$, for $k \in [0, n]$, with $n = \lfloor \log(T+1) \rfloor$. Let $L_{\mathcal{I}_k}$ denote the loss incurred in the interval $\mathcal{I}_k$. By theorem 8.6 (8.16), for any $k \in \{0, \ldots, n\}$, we have

$$L_{\mathcal{I}_k} - \min_{i=1}^{N} L_{\mathcal{I}_k, i} \leq \sqrt{2^k/2 \, \log N}. \tag{8.19}$$

Thus, we can bound the total loss incurred by the algorithm after T rounds as:

$$\begin{aligned} L_T = \sum_{k=0}^{n} L_{\mathcal{I}_k} &\leq \sum_{k=0}^{n} \min_{i=1}^{N} L_{\mathcal{I}_k, i} + \sum_{k=0}^{n} \sqrt{2^k \, (\log N)/2} \\ &\leq \min_{i=1}^{N} L_{T,i} + \sqrt{(\log N)/2} \cdot \sum_{k=0}^{n} 2^{\frac{k}{2}}, \end{aligned} \tag{8.20}$$

where the second inequality follows from the super-additivity of min, that is $\min_i X_i + \min_i Y_i \leq \min_i (X_i + Y_i)$ for any sequences $(X_i)_i$ and $(Y_i)_i$, which implies $\sum_{k=0}^{n} \min_{i=1}^{N} L_{\mathcal{I}_k, i} \leq \min_{i=1}^{N} \sum_{k=0}^{n} L_{\mathcal{I}_k, i}$. The geometric sum appearing in the right-hand side of (8.20) can be expressed as follows:

$$\sum_{k=0}^{n} 2^{\frac{k}{2}} = \frac{2^{(n+1)/2} - 1}{\sqrt{2}-1} \leq \frac{\sqrt{2}\sqrt{T+1} - 1}{\sqrt{2}-1} \leq \frac{\sqrt{2}(\sqrt{T}+1) - 1}{\sqrt{2}-1} = \frac{\sqrt{2}\sqrt{T}}{\sqrt{2}-1} + 1.$$

Plugging back into (8.20) and rearranging terms yields (8.18). □

The $O(\sqrt{T})$ dependency on T presented in this bound cannot be improved for general loss functions.

```
PERCEPTRON(w_0)
1  w_1 ← w_0        ▷ typically w_0 = 0
2  for t ← 1 to T do
3      RECEIVE(x_t)
4      ŷ_t ← sgn(w_t · x_t)
5      RECEIVE(y_t)
6      if (ŷ_t ≠ y_t) then
7          w_{t+1} ← w_t + y_t x_t    ▷ more generally η y_t x_t, η > 0.
8      else w_{t+1} ← w_t
9  return w_{T+1}
```

Figure 8.6
Perceptron algorithm.

8.3 Linear classification

This section presents two well-known on-line learning algorithms for linear classification: the Perceptron and Winnow algorithms.

8.3.1 Perceptron algorithm

The Perceptron algorithm is one of the earliest machine learning algorithms. It is an on-line linear classification algorithm. Thus, it learns a decision function based on a hyperplane by processing training points one at a time. Figure 8.6 gives its pseudocode.

The algorithm maintains a weight vector $\mathbf{w}_t \in \mathbb{R}^N$ defining the hyperplane learned, starting with an arbitrary vector $\mathbf{w}_0$. At each round $t \in [T]$, it predicts the label of the point $\mathbf{x}_t \in \mathbb{R}^N$ received, using the current vector $\mathbf{w}_t$ (line 4). When the prediction made does not match the correct label (lines 6-7), it updates $\mathbf{w}_t$ by adding $y_t\mathbf{x}_t$. More generally, when a learning rate $\eta > 0$ is used, the vector added is $\eta y_t\mathbf{x}_t$. This update can be partially motivated by examining the inner product of the current weight vector with $y_t\mathbf{x}_t$, whose sign determines the classification of $\mathbf{x}_t$. Just before an update, $\mathbf{x}_t$ is misclassified and thus $y_t\mathbf{w}_t \cdot \mathbf{x}_t$ is negative; afterward, $y_t\mathbf{w}_{t+1} \cdot \mathbf{x}_t = y_t\mathbf{w}_t \cdot \mathbf{x}_t + \eta\|\mathbf{x}_t\|^2$, thus, the update corrects the weight vector in the direction of making the inner product $y_t\mathbf{w}_t \cdot \mathbf{x}_t$ positive by augmenting it with the quantity $\eta\|\mathbf{x}_t\|^2 > 0$.

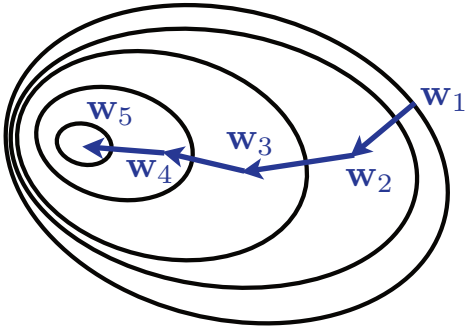


Figure 8.7
An example path followed by the iterative stochastic gradient descent technique. Each inner contour indicates a region of lower elevation.

The Perceptron algorithm can be shown in fact to seek a weight vector $\mathbf{w}$ minimizing an objective function F precisely based on the quantities $(-y_t\mathbf{w}\cdot\mathbf{x}_t)$, $t\in[T]$. Since $(-y_t\mathbf{w}\cdot\mathbf{x}_t)$ is positive when $\mathbf{x}_t$ is misclassified by $\mathbf{w}$, F is defined for all $\mathbf{w}\in\mathbb{R}^N$ by

$$F(\mathbf{w})=\frac{1}{T}\sum_{t=1}^{T}\max\Big(0,-y_t(\mathbf{w}\cdot\mathbf{x}_t)\Big)=\mathop{\mathbb{E}}_{\mathbf{x}\sim\widehat{\mathcal{D}}}[\widetilde{F}(\mathbf{w},\mathbf{x})],\tag{8.21}$$

where $\widetilde{F}(\mathbf{w},\mathbf{x})=\max\big(0,-f(\mathbf{x})(\mathbf{w}\cdot\mathbf{x})\big)$ with $f(\mathbf{x})$ denoting the label of $\mathbf{x}$, and $\widehat{\mathcal{D}}$ is the empirical distribution associated with the sample $(\mathbf{x}_1,\ldots,\mathbf{x}_T)$. For any $t\in[T]$, $\mathbf{w}\mapsto-y_t(\mathbf{w}\cdot\mathbf{x}_t)$ is linear and thus convex. Since the max operator preserves convexity, this shows that F is convex. However, F is not differentiable. Nevertheless, the Perceptron algorithm coincides with the application of the *stochastic subgradient descent* technique to F.

The stochastic (or on-line) subgradient descent technique examines one point $\mathbf{x}_t$ at a time. Note, the function $\widetilde{F}(\cdot,\mathbf{x}_t)$ is non-differentiable for any $\mathbf{w}_t$ where $\mathbf{w}_t\cdot\mathbf{x}_t=0$. In such a case any subgradient of $\widetilde{F}$, i.e. any vector in the convex hull of $\mathbf{0}$ and $-y_t\mathbf{x}_t$, may be used for the update step (see B.4.1). Choosing the subgradient $-y_t\mathbf{x}_t$, we arrive at the following general update for each point $\mathbf{x}_t$:

$$\mathbf{w}_{t+1}\leftarrow\begin{cases}\mathbf{w}_t-\eta\nabla_{\mathbf{w}}\widetilde{F}(\mathbf{w}_t,\mathbf{x}_t) & \text{if } \mathbf{w}_t\cdot\mathbf{x}_t\neq 0\\ \mathbf{w}_t+\eta y_t\mathbf{x}_t & \text{otherwise,}\end{cases}\tag{8.22}$$

where $\eta>0$ is a learning rate parameter. Figure 8.7 illustrates an example path the gradient descent follows. In the specific case we are considering, $\mathbf{w}\mapsto\widetilde{F}(\mathbf{w},\mathbf{x}_t)$ is differentiable at any $\mathbf{w}$ such that $y_t(\mathbf{w}\cdot\mathbf{x}_t)\neq 0$ with $\nabla_{\mathbf{w}}\widetilde{F}(\mathbf{w},\mathbf{x}_t)=-y\mathbf{x}_t$ if $y_t(\mathbf{w}\cdot\mathbf{x}_t)<0$ and $\nabla_{\mathbf{w}}\widetilde{F}(\mathbf{w},\mathbf{x}_t)=0$ if $y_t(\mathbf{w}\cdot\mathbf{x}_t)>0$. Thus, the stochastic gradient

descent update becomes

$$\mathbf{w}_{t+1} \leftarrow \begin{cases} \mathbf{w}_t + \eta y_t \mathbf{x}_t & \text{if } y_t(\mathbf{w}_t \cdot \mathbf{x}_t) \leq 0; \\ \mathbf{w}_t & \text{if } y_t(\mathbf{w}_t \cdot \mathbf{x}_t) > 0, \end{cases} \tag{8.23}$$

which coincides exactly with the update of the Perceptron algorithm.

The following theorem gives a margin-based upper bound on the number of mistakes or updates made by the Perceptron algorithm when processing a sequence of T points that can be linearly separated by a hyperplane with margin $\rho > 0$.

Theorem 8.8 *Let $\mathbf{x}_1, \ldots, \mathbf{x}_T \in \mathbb{R}^N$ be a sequence of T points with $\|\mathbf{x}_t\| \leq r$ for all $t \in [T]$, for some $r > 0$. Assume that there exist $\rho > 0$ and $\mathbf{v} \in \mathbb{R}^N$ such that for all $t \in [T]$, $\rho \leq \frac{y_t(\mathbf{v} \cdot \mathbf{x}_t)}{\|\mathbf{v}\|}$. Then, the number of updates made by the Perceptron algorithm when processing $\mathbf{x}_1, \ldots, \mathbf{x}_T$ is bounded by r^2/ρ^2.*

Proof: Let $\mathcal{I}$ be the subset of the T rounds at which there is an update, and let M be the total number of updates, i.e., $|\mathcal{I}| = M$. Summing up the assumption inequalities yields:

$$\begin{aligned} M\rho \leq \frac{\mathbf{v} \cdot \sum_{t \in \mathcal{I}} y_t \mathbf{x}_t}{\|\mathbf{v}\|} &\leq \Big\| \sum_{t \in \mathcal{I}} y_t \mathbf{x}_t \Big\| && \text{(Cauchy-Schwarz inequality)} \\ &= \Big\| \sum_{t \in \mathcal{I}} (\mathbf{w}_{t+1} - \mathbf{w}_t) \Big\| && \text{(definition of updates)} \\ &= \|\mathbf{w}_{T+1}\| && \text{(telescoping sum, } \mathbf{w}_0 = 0) \\ &= \sqrt{\sum_{t \in \mathcal{I}} \|\mathbf{w}_{t+1}\|^2 - \|\mathbf{w}_t\|^2} && \text{(telescoping sum, } \mathbf{w}_0 = 0) \\ &= \sqrt{\sum_{t \in \mathcal{I}} \|\mathbf{w}_t + y_t \mathbf{x}_t\|^2 - \|\mathbf{w}_t\|^2} && \text{(definition of updates)} \\ &= \sqrt{\sum_{t \in \mathcal{I}} 2 \underbrace{y_t \mathbf{w}_t \cdot \mathbf{x}_t}_{\leq 0} + \|\mathbf{x}_t\|^2} \\ &\leq \sqrt{\sum_{t \in \mathcal{I}} \|\mathbf{x}_t\|^2} \leq \sqrt{M r^2}. \end{aligned}$$

Comparing the left- and right-hand sides gives $\sqrt{M} \leq r/\rho$, that is, $M \leq r^2/\rho^2$. □

By definition of the algorithm, the weight vector $\mathbf{w}_T$ after processing T points is a linear combination of the vectors $\mathbf{x}_t$ at which an update was made: $\mathbf{w}_T = \sum_{t \in \mathcal{I}} y_t \mathbf{x}_t$. Thus, as in the case of SVMs, these vectors can be referred to as *support vectors* for the Perceptron algorithm.

The bound of theorem 8.8 is remarkable, since it depends only on the normalized margin ρ/r and not on the dimension N of the space. This bound can be shown to be tight, that is the number of updates can be equal to r^2/ρ^2 in some instances (see exercise 8.3 to show the upper bound is tight).

The theorem required no assumption about the sequence of points $\mathbf{x}_1, \ldots, \mathbf{x}_T$. A standard setting for the application of the Perceptron algorithm is one where a finite sample S of size $m < T$ is available and where the algorithm makes multiple passes over these m points. The result of the theorem implies that when S is linearly separable, the Perceptron algorithm converges after a finite number of updates and thus passes. For a small margin ρ, the convergence of the algorithm can be quite slow, however. In fact, for some samples, regardless of the order in which the points in S are processed, the number of updates made by the algorithm is in $\Omega(2^N)$ (see exercise 8.1). Of course, if S is not linearly separable, the Perceptron algorithm does not converge. In practice, it is stopped after some number of passes over S.

There are many variants of the standard Perceptron algorithm which are used in practice and have been theoretically analyzed. One notable example is the *voted Perceptron algorithm*, which predicts according to the rule $\operatorname{sgn}\left((\sum_{t \in \mathcal{I}} c_t \mathbf{w}_t) \cdot \mathbf{x}\right)$, where c_t is a weight proportional to the number of iterations that $\mathbf{w}_t$ survives, i.e., the number of iterations between $\mathbf{w}_t$ and $\mathbf{w}_{t+1}$.

For the following theorem, we consider the case where the Perceptron algorithm is trained via multiple passes till convergence over a finite sample that is linearly separable. In view of theorem 8.8, convergence occurs after a finite number of updates.

For a linearly separable sample S, we denote by r_S the radius of the smallest origin-centered sphere containing all points in S and by ρ_S the largest margin of a separating hyperplane for S. We also denote by $M(S)$ the number of updates made by the algorithm after training over S.

Theorem 8.9 *Assume that the data is linearly separable. Let h_S be the hypothesis returned by the Perceptron algorithm after training over a sample S of size m drawn according to some distribution $\mathcal{D}$. Then, the expected error of h_S is bounded as follows:*

$$\mathop{\mathbb{E}}_{S \sim \mathcal{D}^m}[R(h_S)] \leq \mathop{\mathbb{E}}_{S \sim \mathcal{D}^{m+1}}\left[\frac{\min\left(M(S), r_S^2/\rho_S^2\right)}{m+1}\right].$$

Proof: Let S be a linearly separable sample of size $m+1$ drawn i.i.d. according to $\mathcal{D}$ and let $\mathbf{x}$ be a point in S. If $h_{S-\{\mathbf{x}\}}$ misclassifies $\mathbf{x}$, then $\mathbf{x}$ must be a support vector for h_S. Thus, the leave-one-out error of the Perceptron algorithm on sample

S is at most $\frac{M(S)}{m+1}$. The result then follows lemma 5.3, which relates the expected leave-one-out error to the expected error, along with the upper bound on $M(S)$ given by theorem 8.8. □

This result can be compared with a similar one given for the SVM algorithm (with no offset) in the following theorem, which is an extension of theorem 5.4. We denote by $N_{\mathrm{SV}}(S)$ the number of support vectors that define the hypothesis h_S returned by SVMs when trained on a sample S.

Theorem 8.10 *Assume that the data is linearly separable. Let h_S be the hypothesis returned by SVMs used with no offset ($b = 0$) after training over a sample S of size m drawn according to some distribution $\mathcal{D}$. Then, the expected error of h_S is bounded as follows:*

$$\mathop{\mathbb{E}}_{S\sim\mathcal{D}^m}[R(h_S)] \leq \mathop{\mathbb{E}}_{S\sim\mathcal{D}^{m+1}}\left[\frac{\min\left(N_{SV}(S), r_S^2/\rho_S^2\right)}{m+1}\right].$$

Proof: The fact that the expected error can be upper bounded by the average fraction of support vectors ($N_{\mathrm{SV}}(S)/(m+1)$) was already shown by theorem 5.4. Thus, it suffices to show that it is also upper bounded by the expected value of $(r_S^2/\rho_S^2)/(m+1)$. To do so, we will bound the leave-one-out error of the SVM algorithm for a sample S of size $m+1$ by $(r_S^2/\rho_S^2)/(m+1)$. The result will then follow by lemma 5.3, which relates the expected leave-one-out error to the expected error.

Let $S = (\mathbf{x}_1, \ldots, \mathbf{x}_{m+1})$ be a linearly separable sample drawn i.i.d. according to $\mathcal{D}$ and let $\mathbf{x}$ be a point in S that is misclassified by $h_{S-\{\mathbf{x}\}}$. We will analyze the case where $\mathbf{x} = \mathbf{x}_{m+1}$, the analysis of other cases is similar. We denote by S' the sample $(\mathbf{x}_1, \ldots, \mathbf{x}_m)$.

For any $q \in [m+1]$, let G_q denote the function defined over $\mathbb{R}^q$ by $G_q \colon \boldsymbol{\alpha} \mapsto \sum_{i=1}^q \alpha_i - \frac{1}{2}\sum_{i,j=1}^q \alpha_i\alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$. Then, G_{m+1} is the objective function of the dual optimization problem for SVMs associated to the sample S and G_m the one for the sample S'. Let $\boldsymbol{\alpha} \in \mathbb{R}^{m+1}$ denote a solution of the dual SVM problem $\max_{\boldsymbol{\alpha}\geq 0} G_{m+1}(\boldsymbol{\alpha})$ and $\boldsymbol{\alpha}' \in \mathbb{R}^{m+1}$ the vector such that $(\alpha'_1, \ldots, \alpha'_m)^\top \in \mathbb{R}^m$ is a solution of $\max_{\boldsymbol{\alpha}\geq 0} G_m(\boldsymbol{\alpha})$ and $\alpha'_{m+1} = 0$. Let $\mathbf{e}_{m+1}$ denote the $(m+1)$th unit vector in $\mathbb{R}^{m+1}$. By definition of $\boldsymbol{\alpha}$ and $\boldsymbol{\alpha}'$ as maximizers, $\max_{\beta\geq 0} G_{m+1}(\boldsymbol{\alpha}' + \beta\mathbf{e}_{m+1}) \leq G_{m+1}(\boldsymbol{\alpha})$ and $G_{m+1}(\boldsymbol{\alpha} - \alpha_{m+1}\mathbf{e}_{m+1}) \leq G_m(\boldsymbol{\alpha}')$. Thus, the quantity $A = G_{m+1}(\boldsymbol{\alpha}) - G_m(\boldsymbol{\alpha}')$ admits the following lower and upper bounds:

$$\max_{\beta\geq 0} G_{m+1}(\boldsymbol{\alpha}' + \beta\mathbf{e}_{m+1}) - G_m(\boldsymbol{\alpha}') \leq A \leq G_{m+1}(\boldsymbol{\alpha}) - G_{m+1}(\boldsymbol{\alpha} - \alpha_{m+1}\mathbf{e}_{m+1}).$$

Let $\mathbf{w} = \sum_{i=1}^{m+1} y_i\alpha_i\mathbf{x}_i$ denote the weight vector returned by SVMs for the sample S. Since $h_{S'}$ misclassifies $\mathbf{x}_{m+1}$, $\mathbf{x}_{m+1}$ must be a support vector for h_S, thus

$y_{m+1}\mathbf{w}\cdot\mathbf{x}_{m+1}=1$. In view of that, the upper bound can be rewritten as follows:

$$\begin{aligned}
&G_{m+1}(\boldsymbol{\alpha})-G_{m+1}(\boldsymbol{\alpha}-\alpha_{m+1}\mathbf{e}_{m+1})\\
&\qquad=\alpha_{m+1}-\sum_{i=1}^{m+1}(y_i\alpha_i\mathbf{x}_i)\cdot(y_{m+1}\alpha_{m+1}\mathbf{x}_{m+1})+\frac{1}{2}\alpha_{m+1}^2\|\mathbf{x}_{m+1}\|^2\\
&\qquad=\alpha_{m+1}(1-y_{m+1}\mathbf{w}\cdot\mathbf{x}_{m+1})+\frac{1}{2}\alpha_{m+1}^2\|\mathbf{x}_{m+1}\|^2\\
&\qquad=\frac{1}{2}\alpha_{m+1}^2\|\mathbf{x}_{m+1}\|^2.
\end{aligned}$$

Similarly, let $\mathbf{w}'=\sum_{i=1}^{m}y_i\alpha_i'\mathbf{x}_i$. Then, for any $\beta\geq 0$, the quantity maximized in the lower bound can be written as

$$\begin{aligned}
&G_{m+1}(\boldsymbol{\alpha}'+\beta\mathbf{e}_{m+1})-G_m(\boldsymbol{\alpha}')\\
&\qquad=\beta\big(1-y_{m+1}(\mathbf{w}'+\beta y_{m+1}\mathbf{x}_{m+1})\cdot\mathbf{x}_{m+1}\big)+\frac{1}{2}\beta^2\|\mathbf{x}_{m+1}\|^2\\
&\qquad=\beta(1-y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1})-\frac{1}{2}\beta^2\|\mathbf{x}_{m+1}\|^2.
\end{aligned}$$

The right-hand side is maximized for the following value of β: $\frac{1-y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1}}{\|\mathbf{x}_{m+1}\|^2}$. Plugging in this value in the right-hand side gives $\frac{1}{2}\frac{(1-y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1})^2}{\|\mathbf{x}_{m+1}\|^2}$. Thus,

$$A\geq\frac{1}{2}\frac{(1-y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1})^2}{\|\mathbf{x}_{m+1}\|^2}\geq\frac{1}{2\|\mathbf{x}_{m+1}\|^2},$$

using the fact that $y_{m+1}\mathbf{w}'\cdot\mathbf{x}_{m+1}<0$, since $\mathbf{x}_{m+1}$ is misclassified by $\mathbf{w}'$. Comparing this lower bound on A with the upper bound previously derived leads to $\frac{1}{2\|\mathbf{x}_{m+1}\|^2}\leq\frac{1}{2}\alpha_{m+1}^2\|\mathbf{x}_{m+1}\|^2$, that is

$$\alpha_{m+1}\geq\frac{1}{\|\mathbf{x}_{m+1}\|^2}\geq\frac{1}{r_S^2}.$$

The analysis carried out in the case $\mathbf{x}=\mathbf{x}_{m+1}$ holds similarly for any $\mathbf{x}_i$ in S that is misclassified by $h_{S-\{\mathbf{x}_i\}}$. Let $\mathcal{I}$ denote the set of such indices i. Then, we can write:

$$\sum_{i\in\mathcal{I}}\alpha_i\geq\frac{|\mathcal{I}|}{r_S^2}.$$

By (5.19), the following simple expression holds for the margin: $\sum_{i=1}^{m+1}\alpha_i=1/\rho_S^2$. Using this identity leads to

$$|\mathcal{I}|\leq r_S^2\sum_{i\in\mathcal{I}}\alpha_i\leq r_S^2\sum_{i=1}^{m+1}\alpha_i=\frac{r_S^2}{\rho_S^2}.$$

Since by definition $|\mathcal{I}|$ is the total number of leave-one-out errors, this concludes the proof. □

Thus, the guarantees given by theorem 8.9 and theorem 8.10 in the separable case have a similar form. These bounds do not seem sufficient to distinguish the effectiveness of the SVM and Perceptron algorithms. Note, however, that while the same margin quantity ρ_S appears in both bounds, the radius r_S can be replaced by a finer quantity that is different for the two algorithms: in both cases, instead of the radius of the sphere containing all sample points, r_S can be replaced by the radius of the sphere containing the support vectors, as can be seen straightforwardly from the proof of the theorems. Thus, the position of the support vectors in the case of SVMs can provide a more favorable guarantee than that of the support vectors (update vectors) for the Perceptron algorithm. Finally, the guarantees given by these theorems are somewhat weak. These are not high probability bounds, they hold only for the expected error of the hypotheses returned by the algorithms and in particular provide no information about the variance of their error.

The following two theorems give bounds on the number of updates or mistakes made by the Perceptron algorithm in the more general scenario of a non-linearly separable sample in terms of the ρ-Hinge losses of an arbitrary weight vector $\mathbf{v}$.

Theorem 8.11 *Let $\mathcal{I}$ denote the set of indices $t \in [T]$ at which the Perceptron algorithm makes an update when processing a sequence $\mathbf{x}_1, \ldots, \mathbf{x}_T$ with $\|\mathbf{x}_t\| \leq r$ for some $r > 0$. Then, the number of updates $M = |\mathcal{I}|$ made by the algorithm can be bounded as follows:*

$$M \leq \inf_{\rho>0, \|\mathbf{v}\|_2 \leq 1} \left[\frac{\frac{r}{\rho} + \sqrt{\frac{r^2}{\rho^2} + 4\|\mathbf{l}_\rho\|_1}}{2} \right]^2 \leq \inf_{\rho>0, \|v\|_2 \leq 1} \left(\frac{r}{\rho} + \sqrt{\|\mathbf{l}_\rho\|_1} \right)^2 ,$$

where $\mathbf{l}_\rho = (l_t)_{t \in \mathcal{I}}$ with $l_t = \max\left\{0, 1 - \frac{y_t(\mathbf{v} \cdot \mathbf{x}_t)}{\rho}\right\}$.

Proof: Fix $\rho > 0$ and $\mathbf{v}$ with $\|\mathbf{v}\|_2 = 1$. By definition of l_t, for any t, we have $1 - \frac{y_t(\mathbf{v} \cdot \mathbf{x}_t)}{\rho} \leq l_t$. Summing up these inequalities over all $t \in \mathcal{I}$ yields

$$\begin{aligned} M &\leq \sum_{t \in \mathcal{I}} l_t + \sum_{t \in \mathcal{I}} \frac{y_t(\mathbf{v} \cdot \mathbf{x}_t)}{\rho} \\ &= \|\mathbf{l}_\rho\|_1 + \sum_{t \in \mathcal{I}} \frac{y_t(\mathbf{v} \cdot \mathbf{x}_t)}{\rho} \leq \|\mathbf{l}_\rho\|_1 + \frac{\sqrt{Mr^2}}{\rho} , \end{aligned} \tag{8.24}$$

where the last inequality holds by the bound shown in the proof of the separable case (theorem 8.8): $\frac{\mathbf{v} \cdot \sum_{t \in \mathcal{I}} y_t \mathbf{x}_t}{\|\mathbf{v}\|} \leq \sqrt{Mr^2}$. Now, solving the resulting second-degree inequality $M \leq \|\mathbf{l}_\rho\|_1 + \frac{\sqrt{Mr^2}}{\rho}$ gives $\sqrt{M} \leq \frac{1}{2}\left(\frac{r}{\rho} + \sqrt{\frac{r^2}{\rho^2} + 4\|\mathbf{l}_\rho\|_1}\right)$, which proves the first inequality. The second inequality follows from the sub-additivity of the square-root function. □

Theorem 8.12 *Let $\mathcal{I}$ denote the set of indices $t \in [T]$ at which the Perceptron algorithm makes an update when processing a sequence $\mathbf{x}_1, \ldots, \mathbf{x}_T$ with $\|\mathbf{x}_t\| \leq r$ for some $r > 0$. Then, the number of updates $M = |\mathcal{I}|$ made by the algorithm can be bounded as follows:*

$$M \leq \inf_{\rho > 0, \|v\|_2 \leq 1} \left(\frac{r}{\rho} + \sqrt{\|\mathbf{l}_\rho\|_2} \right)^2,$$

where $\mathbf{l}_\rho = (l_t)_{t \in \mathcal{I}}$ with $l_t = \max\left\{0, 1 - \frac{y_t(\mathbf{v} \cdot \mathbf{x}_t)}{\rho}\right\}$.

Proof: Fix $\rho > 0$ and $\mathbf{v}$ with $\|\mathbf{v}\|_2 = 1$. Starting with line (8.24) of theorem 8.11 and using $\|\mathbf{l}_\rho\|_1 \leq \sqrt{M}\|\mathbf{l}_\rho\|_2$, which holds by the Cauchy-Schwarz inequality, give

$$M \leq \|\mathbf{l}_\rho\|_1 + \frac{\sqrt{Mr^2}}{\rho} \leq \sqrt{M}\|\mathbf{l}_\rho\|_2 + \frac{\sqrt{Mr^2}}{\rho}.$$

This implies $\sqrt{M} \leq \|\mathbf{l}_\rho\|_2 + \frac{\sqrt{r^2}}{\rho}$ and proves the statement. □

These bounds strictly generalize the bounds given in the separable case (theorem 8.8) since in that case the vector $\mathbf{v}$ can be chosen to be that of a maximum-margin hyperplane with no Hinge loss at any point. The main difference between the two bounds is the L_1-norm of the vector of Hinge losses in Theorem 8.11 versus the L_2-norm in Theorem 8.12. Note that, since the L_2-norm bound follows from upper bounding inequality (8.24), which is equivalent to the first inequality of Theorem 8.11, the first L_1-norm bound of Theorem 8.11 is always tighter than the L_2-norm bound of Theorem 8.12.

The Perceptron algorithm can be generalized, as in the case of SVMs, to define a linear separation in a high-dimensional space. It admits an equivalent dual form, the dual Perceptron algorithm, which is presented in figure 8.8. The dual Perceptron algorithm maintains a vector $\boldsymbol{\alpha} \in \mathbb{R}^T$ of coefficients assigned to each point $\mathbf{x}_t$, $t \in [T]$. The label of a point $\mathbf{x}_t$ is predicted according to the rule $\operatorname{sgn}(\mathbf{w} \cdot \mathbf{x}_t)$, where $\mathbf{w} = \sum_{s=1}^T \alpha_s y_s \mathbf{x}_s$. The coefficient α_t is incremented by one when this prediction does not match the correct label. Thus, an update for $\mathbf{x}_t$ is equivalent to augmenting the weight vector $\mathbf{w}$ with $y_t \mathbf{x}_t$, which shows that the dual algorithm matches exactly the standard Perceptron algorithm. The dual Perceptron algorithm can be written solely in terms of inner products between training instances. Thus, as in the case of SVMs, instead of the inner product between points in the input space, an arbitrary PDS kernel can be used, which leads to the kernel Perceptron algorithm detailed in figure 8.9. The kernel Perceptron algorithm and its average variant, i.e., voted Perceptron with uniform weights c_t, are commonly used algorithms in a variety of applications.

```
DUALPERCEPTRON(α_0)
1  α ← α_0        ▷ typically α_0 = 0
2  for t ← 1 to T do
3      RECEIVE(x_t)
4      ŷ_t ← sgn(Σ_{s=1}^{T} α_s y_s (x_s · x_t))
5      RECEIVE(y_t)
6      if (ŷ_t ≠ y_t) then
7          α_t ← α_t + 1
8      else α_t ← α_t
9  return α
```

Figure 8.8
Dual Perceptron algorithm.

8.3.2 Winnow algorithm

This section presents an alternative on-line linear classification algorithm, the *Winnow algorithm*. Thus, it learns a weight vector defining a separating hyperplane by sequentially processing the training points. As suggested by the name, the algorithm is particularly well suited to cases where a relatively small number of dimensions or experts can be used to define an accurate weight vector. Many of the other dimensions may then be irrelevant.

The Winnow algorithm is similar to the Perceptron algorithm, but, instead of the additive update of the weight vector in the Perceptron case, Winnow's update is multiplicative. The pseudocode of the algorithm is given in figure 8.10. The algorithm takes as input a learning parameter $\eta > 0$. It maintains a non-negative weight vector $\mathbf{w}_t$ with components summing to one ($\|\mathbf{w}_t\|_1 = 1$) starting with the uniform weight vector (line 1). At each round $t \in [T]$, if the prediction does not match the correct label (line 6), each component $w_{t,i}$, $i \in [N]$, is updated by multiplying it by $\exp(\eta y_t x_{t,i})$ and dividing by the normalization factor Z_t to ensure that the weights sum to one (lines 7–9). Thus, if the label y_t and $x_{t,i}$ share the same sign, then $w_{t,i}$ is increased, while, in the opposite case, it is significantly decreased.

The Winnow algorithm is closely related to the WM algorithm: when $\mathbf{x}_{t,i} \in \{-1, +1\}$, $\text{sgn}(\mathbf{w}_t \cdot \mathbf{x}_t)$ coincides with the majority vote, since multiplying the weight of correct or incorrect experts by e^{η} or $e^{-\eta}$ is equivalent to multiplying the weight

KernelPerceptron($\boldsymbol{\alpha}_0$)

$\boldsymbol{\alpha} \leftarrow \boldsymbol{\alpha}_0$ ▷ typically $\boldsymbol{\alpha}_0 = \mathbf{0}$
for $t \leftarrow 1$ **to** T **do**
  Receive(x_t)
  $\widehat{y}_t \leftarrow \operatorname{sgn}(\sum_{s=1}^{T} \alpha_s y_s K(x_s, x_t))$
  Receive(y_t)
  if $(\widehat{y}_t \neq y_t)$ **then**
    $\alpha_t \leftarrow \alpha_t + 1$
  else $\alpha_t \leftarrow \alpha_t$
return $\boldsymbol{\alpha}$

Figure 8.9
Kernel Perceptron algorithm for PDS kernel K.

of incorrect ones by $\beta = e^{-2\eta}$. The multiplicative update rule of Winnow is of course also similar to that of AdaBoost.

The following theorem gives a mistake bound for the Winnow algorithm in the separable case, which is similar in form to the bound of theorem 8.8 for the Perceptron algorithm.

Theorem 8.13 *Let $\mathbf{x}_1, \ldots, \mathbf{x}_T \in \mathbb{R}^N$ be a sequence of T points with $\|x_t\|_\infty \leq r_\infty$ for all $t \in [T]$, for some $r_\infty > 0$. Assume that there exist $\mathbf{v} \in \mathbb{R}^N$, $\mathbf{v} \geq 0$, and $\rho_\infty > 0$ such that for all $t \in [T]$, $\rho_\infty \leq \frac{y_t(\mathbf{v} \cdot \mathbf{x}_t)}{\|\mathbf{v}\|_1}$. Then, for $\eta = \frac{\rho_\infty}{r_\infty^2}$, the number of updates made by the Winnow algorithm when processing $\mathbf{x}_1, \ldots, \mathbf{x}_T$ is upper bounded by $2\,(r_\infty^2/\rho_\infty^2) \log N$.*

Proof: Let $\mathcal{I} \subseteq [T]$ be the set of iterations at which there is an update, and let M be the total number of updates, i.e., $|\mathcal{I}| = M$. The potential function Φ_t, $t \in [T]$, used for this proof is the relative entropy of the distribution defined by the normalized weights $v_i/\|\mathbf{v}\|_1 \geq 0$, $i \in [N]$, and the one defined by the components of the weight vector $w_{t,i}$, $i \in [N]$:

$$\Phi_t = \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{v_i/\|\mathbf{v}\|_1}{w_{t,i}}\,.$$

To derive an upper bound on Φ_t, we analyze the difference of the potential functions at two consecutive rounds. For all $t \in \mathcal{I}$, this difference can be expressed and

WINNOW(η)

$w_1 \leftarrow \mathbf{1}/N$
for $t \leftarrow 1$ **to** T **do**
  RECEIVE($\mathbf{x}_t$)
  $\widehat{y}_t \leftarrow \text{sgn}(\mathbf{w}_t \cdot \mathbf{x}_t)$
  RECEIVE(y_t)
  if $(\widehat{y}_t \neq y_t)$ **then**
    $Z_t \leftarrow \sum_{i=1}^{N} w_{t,i} \exp(\eta y_t x_{t,i})$
    for $i \leftarrow 1$ **to** N **do**
      $w_{t+1,i} \leftarrow \frac{w_{t,i} \exp(\eta y_t x_{t,i})}{Z_t}$
  else $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t$
return $\mathbf{w}_{T+1}$

Figure 8.10
Winnow algorithm, with $y_t \in \{-1, +1\}$ for all $t \in [T]$.

bounded as follows:

$$\begin{aligned}
\Phi_{t+1} - \Phi_t &= \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{w_{t,i}}{w_{t+1,i}} \\
&= \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{Z_t}{\exp(\eta y_t x_{t,i})} \\
&= \log Z_t - \eta \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} y_t x_{t,i} \\
&\leq \log \Big[\sum_{i=1}^{N} w_{t,i} \exp(\eta y_t x_{t,i}) \Big] - \eta \rho_\infty \\
&= \log \mathop{\mathbb{E}}_{i \sim \mathbf{w}_t} \big[\exp(\eta y_t x_{t,i}) \big] - \eta \rho_\infty \\
&= \log \mathop{\mathbb{E}}_{i \sim \mathbf{w}_t} \big[\exp(\eta y_t x_{t,i} - \eta y_t \mathbf{w}_t \cdot x_t + \eta y_t \mathbf{w}_t \cdot x_t) \big] - \eta \rho_\infty \\
&\leq \log \big[\exp(\eta^2 (2r_\infty)^2 / 8) \big] + \underbrace{\eta y_t (\mathbf{w}_t \cdot x_t)}_{\leq 0} - \eta \rho_\infty \\
&\leq \eta^2 r_\infty^2 / 2 - \eta \rho_\infty.
\end{aligned}$$

The first inequality follows the definition of ρ_∞. The subsequent equality rewrites the summation as an expectation over the distribution defined by $\mathbf{w}_t$. The next inequality uses Hoeffding's lemma (lemma D.1) and the last one the fact that there has been an update at t, which implies $y_t(\mathbf{w}_t \cdot x_t) \leq 0$. Summing up these inequalities over all $t \in \mathcal{I}$ yields:

$$\Phi_{T+1} - \Phi_1 \leq M(\eta^2 r_\infty^2/2 - \eta\rho_\infty).$$

Next, we derive a lower bound by noting that

$$\Phi_1 = \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{v_i/\|\mathbf{v}\|_1}{1/N} = \log N + \sum_{i=1}^{N} \frac{v_i}{\|\mathbf{v}\|_1} \log \frac{v_i}{\|\mathbf{v}\|_1} \leq \log N\,.$$

Additionally, since the relative entropy is always non-negative, we have $\Phi_{T+1} \geq 0$. This yields the following lower bound:

$$\Phi_{T+1} - \Phi_1 \geq 0 - \log N = -\log N\,.$$

Combining the upper and lower bounds we see that $-\log N \leq M(\eta^2 r_\infty^2/2 - \eta\rho_\infty)$. Setting $\eta = \frac{\rho_\infty}{r_\infty^2}$ yields the statement of the theorem. □

The margin-based mistake bounds of theorem 8.8 and theorem 8.13 for the Perceptron and Winnow algorithms have a similar form, but they are based on different norms. For both algorithms, the norm $\|\cdot\|_p$ used for the input vectors $\mathbf{x}_t$, $t \in [T]$, is the dual of the norm $\|\cdot\|_q$ used for the margin vector $\mathbf{v}$, that is p and q are conjugate: $1/p + 1/q = 1$: in the case of the Perceptron algorithm $p = q = 2$, while for Winnow $p = \infty$ and $q = 1$.

These bounds imply different types of guarantees. The bound for Winnow is favorable when a sparse set of the experts $i \in [N]$ can predict well. For example, if $\mathbf{v} = \mathbf{e}_1$ where $\mathbf{e}_1$ is the unit vector along the first axis in $\mathbb{R}^N$ and if $\mathbf{x}_t \in \{-1, +1\}^N$ for all t, then the upper bound on the number of mistakes given for Winnow by theorem 8.13 is only $2 \log N$, while the upper bound of theorem 8.8 for the Perceptron algorithm is N. The guarantee for the Perceptron algorithm is more favorable in the opposite situation, where sparse solutions are not effective.

8.4 On-line to batch conversion

The previous sections presented several algorithms for the scenario of on-line learning, including the Perceptron and Winnow algorithms, and analyzed their behavior within the mistake model, where no assumption is made about the way the training sequence is generated. Can these algorithms be used to derive hypotheses with small generalization error in the standard stochastic setting? How can the interme-

diate hypotheses they generate be combined to form an accurate predictor? These are the questions addressed in this section.

Let $\mathcal{H}$ be a hypothesis of functions mapping $\mathcal{X}$ to $\mathcal{Y}'$, and let $L\colon \mathcal{Y}' \times \mathcal{Y} \to \mathbb{R}_+$ be a bounded loss function, that is $L \leq M$ for some $M \geq 0$. We assume a standard supervised learning setting where a labeled sample $S = ((x_1, y_1), \ldots, (x_T, y_T)) \in (\mathcal{X} \times \mathcal{Y})^T$ is drawn i.i.d. according to some fixed but unknown distribution $\mathcal{D}$. The sample is sequentially processed by an on-line learning algorithm $\mathcal{A}$. The algorithm starts with an initial hypothesis $h_1 \in \mathcal{H}$ and generates a new hypothesis $h_{t+1} \in \mathcal{H}$, after processing pair (x_t, y_t), $t \in [m]$. The regret of the algorithm is defined as before by

$$R_T = \sum_{t=1}^{T} L(h_t(x_t), y_t) - \min_{h \in \mathcal{H}} \sum_{t=1}^{T} L(h(x_t), y_t). \tag{8.25}$$

The generalization error of a hypothesis $h \in \mathcal{H}$ is its expected loss $R(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[L(h(x), y)]$.

The following lemma gives a bound on the average of the generalization errors of the hypotheses generated by $\mathcal{A}$ in terms of its average loss $\frac{1}{T}\sum_{t=1}^{T} L(h_t(x_t), y_t)$.

Lemma 8.14 *Let $S = ((x_1, y_1), \ldots, (x_T, y_T)) \in (\mathcal{X} \times \mathcal{Y})^T$ be a labeled sample drawn i.i.d. according to $\mathcal{D}$, L a loss bounded by M and $h_1, \ldots, h_T$ the sequence of hypotheses generated by an on-line algorithm $\mathcal{A}$ sequentially processing S. Then, for any $\delta > 0$, with probability at least $1 - \delta$, the following holds:*

$$\frac{1}{T}\sum_{t=1}^{T} R(h_t) \leq \frac{1}{T}\sum_{t=1}^{T} L(h_t(x_t), y_t) + M\sqrt{\frac{2\log\frac{1}{\delta}}{T}}. \tag{8.26}$$

Proof: For any $t \in [T]$, let V_t be the random variable defined by $V_t = R(h_t) - L(h_t(x_t), y_t)$. Observe that for any $t \in [T]$,

$$\mathbb{E}[V_t | x_1, \ldots, x_{t-1}] = R(h_t) - \mathbb{E}[L(h_t(x_t), y_t) | h_t] = R(h_t) - R(h_t) = 0.$$

Since the loss is bounded by M, V_t takes values in the interval $[-M, +M]$ for all $t \in [T]$. Thus, by Azuma's inequality (theorem D.7), $\mathbb{P}[\frac{1}{T}\sum_{t=1}^{T} V_t \geq \epsilon] \leq \exp(-2T\epsilon^2/(2M)^2))$. Setting the right-hand side to be equal to $\delta > 0$ yields the statement of the lemma. □

When the loss function is convex with respect to its first argument, the lemma can be used to derive a bound on the generalization error of the average of the hypotheses generated by $\mathcal{A}$, $\frac{1}{T}\sum_{t=1}^{T} h_t$, in terms of the average loss of $\mathcal{A}$ on S, or in terms of the regret R_T and the infimum error of hypotheses in $\mathcal{H}$.

Theorem 8.15 *Let $S = ((x_1, y_1), \ldots, (x_T, y_T)) \in (\mathcal{X} \times \mathcal{Y})^T$ be a labeled sample drawn i.i.d. according to $\mathcal{D}$, L a loss bounded by M and convex with respect to its first argument, and $h_1, \ldots, h_T$ the sequence of hypotheses generated by an on-line algorithm $\mathcal{A}$ sequentially processing S. Then, for any $\delta > 0$, with probability at least*

$1-\delta$, *each of the following holds:*

$$R\left(\frac{1}{T}\sum_{t=1}^{T} h_t\right) \le \frac{1}{T}\sum_{t=1}^{T} L(h_t(x_t), y_t) + M\sqrt{\frac{2\log\frac{1}{\delta}}{T}} \tag{8.27}$$

$$R\left(\frac{1}{T}\sum_{t=1}^{T} h_t\right) \le \inf_{h\in\mathcal{H}} R(h) + \frac{R_T}{T} + 2M\sqrt{\frac{2\log\frac{2}{\delta}}{T}}. \tag{8.28}$$

Proof: By the convexity of L with respect to its first argument, for any $(x, y) \in \mathcal{X}\times\mathcal{Y}$, we have $L(\frac{1}{T}\sum_{t=1}^{T} h_t(x), y) \le \frac{1}{T}\sum_{t=1}^{T} L(h_t(x), y)$. Taking the expectation gives $R(\frac{1}{T}\sum_{t=1}^{T} h_t) \le \frac{1}{T}\sum_{t=1}^{T} R(h_t)$. The first inequality then follows by lemma 8.14. Thus, by definition of the regret R_T, for any $\delta > 0$, the following holds with probability at least $1-\delta/2$:

$$\begin{aligned} R\left(\frac{1}{T}\sum_{t=1}^{T} h_t\right) &\le \frac{1}{T}\sum_{t=1}^{T} L(h_t(x_t), y_t) + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} \\ &\le \min_{h\in\mathcal{H}} \frac{1}{T}\sum_{t=1}^{T} L(h(x_t), y_t) + \frac{R_T}{T} + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}}. \end{aligned}$$

By definition of $\inf_{h\in\mathcal{H}} R(h)$, for any $\epsilon > 0$, there exists $h^* \in \mathcal{H}$ with $R(h^*) \le \inf_{h\in\mathcal{H}} R(h) + \epsilon$. By Hoeffding's inequality, for any $\delta > 0$, with probability at least $1-\delta/2$, $\frac{1}{T}\sum_{t=1}^{T} L(h^*(x_t), y_t) \le R(h^*) + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}}$. Thus, for any $\epsilon > 0$, by the union bound, the following holds with probability at least $1-\delta$:

$$\begin{aligned} R\left(\frac{1}{T}\sum_{t=1}^{T} h_t\right) &\le \frac{1}{T}\sum_{t=1}^{T} L(h^*(x_t), y_t) + \frac{R_T}{T} + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} \\ &\le R(h^*) + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} + \frac{R_T}{T} + M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} \\ &= R(h^*) + \frac{R_T}{T} + 2M\sqrt{\frac{2\log\frac{2}{\delta}}{T}} \\ &\le \inf_{h\in\mathcal{H}} R(h) + \epsilon + \frac{R_T}{T} + 2M\sqrt{\frac{2\log\frac{2}{\delta}}{T}}. \end{aligned}$$

Since this inequality holds for all $\epsilon > 0$, it implies the second statement of the theorem. □

The theorem can be applied to a variety of on-line regret minimization algorithms, for example when $R_T/T = O(1/\sqrt{T})$. In particular, we can apply the theorem to the exponential weighted average algorithm. Assuming that the loss L is bounded

by $M = 1$ and that the number of rounds T is known to the algorithm, we can use the regret bound of theorem 8.6. The doubling trick (used in theorem 8.7) can be used to derive a similar bound if T is not known in advance. Thus, for any $\delta > 0$, with probability at least $1 - \delta$, the following holds for the generalization error of the average of the hypotheses generated by exponential weighted average:

$$R\left(\frac{1}{T}\sum_{t=1}^{T} h_t\right) \leq \inf_{h \in \mathcal{H}} R(h) + \sqrt{\frac{\log N}{2T}} + 2\sqrt{\frac{2\log \frac{2}{\delta}}{T}},$$

where N is the number of experts, or the dimension of the weight vectors.

8.5 Game-theoretic connection

The existence of regret minimization algorithms can be used to give a simple proof of von Neumann's theorem. For any $m \geq 1$, we will denote by Δ_m the set of all distributions over $\{1, \ldots, m\}$, that is $\Delta_m = \{\mathbf{p} \in \mathbb{R}^m \colon \mathbf{p} \geq 0 \wedge \|\mathbf{p}\|_1 = 1\}$.

Theorem 8.16 (Von Neumann's minimax theorem) *Let $m, n \geq 1$. Then, for any two-person zero-sum game defined by matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$,*

$$\min_{\mathbf{p} \in \Delta_m} \max_{\mathbf{q} \in \Delta_n} \mathbf{p}^\top \mathbf{M} \mathbf{q} = \max_{\mathbf{q} \in \Delta_n} \min_{\mathbf{p} \in \Delta_m} \mathbf{p}^\top \mathbf{M} \mathbf{q}. \tag{8.29}$$

Proof: The inequality $\max_{\mathbf{q}} \min_{\mathbf{p}} \mathbf{p}^\top \mathbf{M} \mathbf{q} \leq \min_{\mathbf{p}} \max_{\mathbf{q}} \mathbf{p}^\top \mathbf{M} \mathbf{q}$ is straightforward, since by definition of min, for all $\mathbf{p} \in \Delta_m, \mathbf{q} \in \Delta_n$, we have $\min_{\mathbf{p}} \mathbf{p}^\top \mathbf{M} \mathbf{q} \leq \mathbf{p}^\top \mathbf{M} \mathbf{q}$. Taking the maximum over $\mathbf{q}$ of both sides gives: $\max_{\mathbf{q}} \min_{\mathbf{p}} \mathbf{p}^\top \mathbf{M} \mathbf{q} \leq \max_{\mathbf{q}} \mathbf{p}^\top \mathbf{M} \mathbf{q}$ for all $\mathbf{p}$, subsequently taking the minimum over $\mathbf{p}$ proves the inequality.[12]

To show the reverse inequality, consider an on-line learning setting where at each round $t \in [T]$, algorithm $\mathcal{A}$ returns $\mathbf{p}_t$ and incurs loss $\mathbf{M}\mathbf{q}_t$. We can assume that $\mathbf{q}_t$ is selected in the optimal adversarial way, that is $\mathbf{q}_t \in \operatorname{argmax}_{q \in \Delta_m} \mathbf{p}_t^\top \mathbf{M} \mathbf{q}$, and that $\mathcal{A}$ is a regret minimization algorithm, that is $R_T/T \to 0$, where $R_T = \sum_{t=1}^{T} \mathbf{p}_t^\top \mathbf{M} \mathbf{q}_t - \min_{\mathbf{p} \in \Delta_m} \sum_{t=1}^{T} \mathbf{p}^\top \mathbf{M} \mathbf{q}_t$. Then, the following holds:

$$\min_{\mathbf{p} \in \Delta_m} \max_{\mathbf{q} \in \Delta_n} \mathbf{p}^\top \mathbf{M} \mathbf{q} \leq \max_{\mathbf{q}} \left(\frac{1}{T}\sum_{t=1}^{T} \mathbf{p}_t\right)^\top \mathbf{M} \mathbf{q} \leq \frac{1}{T}\sum_{t=1}^{T} \max_{\mathbf{q}} \mathbf{p}_t^\top \mathbf{M} \mathbf{q} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{p}_t^\top \mathbf{M} \mathbf{q}_t.$$

[12] More generally, the maxmin is always upper bounded by the minmax for any function or two arguments and any constraint sets, following the same proof.

By definition of regret, the right-hand side can be expressed and bounded as follows:

$$\frac{1}{T}\sum_{t=1}^{T}\mathbf{p}_t^\top\mathbf{M}\mathbf{q}_t = \min_{\mathbf{p}\in\Delta_m}\frac{1}{T}\sum_{t=1}^{T}\mathbf{p}^\top\mathbf{M}\mathbf{q}_t + \frac{R_T}{T} = \min_{\mathbf{p}\in\Delta_m}\mathbf{p}^\top\mathbf{M}\Big(\frac{1}{T}\sum_{t=1}^{T}\mathbf{q}_t\Big) + \frac{R_T}{T}$$
$$\leq \max_{\mathbf{q}\in\Delta_n}\min_{\mathbf{p}\in\Delta_m}\mathbf{p}^\top\mathbf{M}\mathbf{q} + \frac{R_T}{T}.$$

This implies that the following bound holds for the minmax for all $T \geq 1$:

$$\min_{\mathbf{p}\in\Delta_m}\max_{\mathbf{q}\in\Delta_n}\mathbf{p}^\top\mathbf{M}\mathbf{q} \leq \max_{\mathbf{q}\in\Delta_n}\min_{\mathbf{p}\in\Delta_m}\mathbf{p}^\top\mathbf{M}\mathbf{q} + \frac{R_T}{T}$$

Since $\lim_{T\to+\infty}\frac{R_T}{T} = 0$, this shows that $\min_{\mathbf{p}}\max_{\mathbf{q}}\mathbf{p}^\top\mathbf{M}\mathbf{q} \leq \max_{\mathbf{q}}\min_{\mathbf{p}}\mathbf{p}^\top\mathbf{M}\mathbf{q}$.□

8.6 Chapter notes

Algorithms for regret minimization were initiated with the pioneering work of Hannan [1957] who gave an algorithm whose regret decreases as $O(\sqrt{T})$ as a function of T but whose dependency on N is linear. The weighted majority algorithm and the randomized weighted majority algorithm, whose regret is only logarithmic in N, are due to Littlestone and Warmuth [1989]. The exponential weighted average algorithm and its analysis, which can be viewed as an extension of the WM algorithm to convex non-zero-one losses is due to the same authors [Littlestone and Warmuth, 1989, 1994]. The analysis we presented follows Cesa-Bianchi [1999] and Cesa-Bianchi and Lugosi [2006]. The doubling trick technique appears in Vovk [1990] and Cesa-Bianchi et al. [1997]. The algorithm of exercise 8.7 and the analysis leading to a second-order bound on the regret are due to Cesa-Bianchi et al. [2005]. The lower bound presented in theorem 8.5 is from Blum and Mansour [2007].

While the regret bounds presented are logarithmic in the number of the experts N, when N is exponential in the size of the input problem, the computational complexity of an expert algorithm could be exponential. For example, in the online shortest paths problem, N is the number of paths between two vertices of a directed graph. However, several computationally efficient algorithms have been presented for broad classes of such problems by exploiting their structure [Takimoto and Warmuth, 2002, Kalai and Vempala, 2003, Zinkevich, 2003].

The notion of regret (or *external regret*) presented in this chapter can be generalized to that of *internal regret* or even *swap regret*, by comparing the loss of the algorithm not just to that of the best expert in retrospect, but to that of any modification of the actions taken by the algorithm by replacing each occurrence of some specific action with another one (internal regret), or even replacing actions via an arbitrary mapping (swap regret) [Foster and Vohra, 1997, Hart and Mas-Colell, 2000, Lehrer, 2003]. Several algorithms for low internal regret have been given

[Foster and Vohra, 1997, 1998, 1999, Hart and Mas-Colell, 2000, Cesa-Bianchi and Lugosi, 2001, Stoltz and Lugosi, 2003], including a conversion of low external regret to low swap regret by Blum and Mansour [2005].

The Perceptron algorithm was introduced by Rosenblatt [1958]. The algorithm raised a number of reactions, in particular by Minsky and Papert [1969], who objected that the algorithm could not be used to recognize the XOR function. Of course, the kernel Perceptron algorithm already given by Aizerman et al. [1964] could straightforwardly succeed to do so using second-degree polynomial kernels. The margin bound for the Perceptron algorithm was proven by Novikoff [1962] and is one of the first results in learning theory. We presented two extensions of Novikoff's result which hold in the more general non-separable case: Theorem 8.12 due to Freund and Schapire [1999a] and Theorem 8.11 due to Mohri and Rostamizadeh [2013]. Our proof of Theorem 8.12 is significantly more concise that the original proof given by Freund and Schapire [1999a] and shows that the bound of Theorem 8.11 is always tighter than that of Theorem 8.12. See [Mohri and Rostamizadeh, 2013] for other more general data-dependent upper bounds on the number of updates made by the Perceptron algorithm in the non-separable case. The leave-one-out analysis for SVMs is described by Vapnik [1998]. The Winnow algorithm was introduced by Littlestone [1987].

The analysis of the on-line to batch conversion and exercises 8.10 and 8.11 are from Cesa-Bianchi et al. [2001, 2004] (see also Littlestone [1989]). Von Neumann's minimax theorem admits a number of different generalizations. See Sion [1958] for a generalization to quasi-concave-convex functions semi-continuous in each argument and the references therein. The simple proof of von Neumann's theorem presented here is entirely based on learning-related techniques. A proof of a more general version using multiplicative updates was presented by Freund and Schapire [1999b].

On-line learning is a very broad and fast-growing research area in machine learning. The material presented in this chapter should be viewed only as an introduction to the topic, but the proofs and techniques presented should indicate the flavor of most results in this area. For a more comprehensive presentation of on-line learning and related game theory algorithms and techniques, the reader could consult the book of Cesa-Bianchi and Lugosi [2006].

8.7 Exercises

8.1 Perceptron lower bound. Let S be a labeled sample of m points in $\mathbb{R}^N$ with

$$x_i = (\underbrace{(-1)^i, \ldots, (-1)^i, (-1)^{i+1}}_{i \text{ first components}}, 0, \ldots, 0) \quad \text{and} \quad y_i = (-1)^{i+1}. \tag{8.30}$$

```
ON-LINE-SVM(w_0)
1  w_1 ← w_0        ▷ typically w_0 = 0
2  for t ← 1 to T do
3       RECEIVE(x_t, y_t)
4       if y_t(w_t · x_t) < 1 then
5            w_{t+1} ← w_t − η(w_t − C y_t x_t)
6       elseif y_t(w_t · x_t) > 1 then
7            w_{t+1} ← w_t − η w_t
8       else w_{t+1} ← w_t
9  return w_{T+1}
```

Figure 8.11
On-line SVM algorithm.

Show that the Perceptron algorithm makes $\Omega(2^N)$ updates before finding a separating hyperplane, regardless of the order in which it receives the points.

8.2 Generalized mistake bound. Theorem 8.8 presents a margin bound on the maximum number of updates for the Perceptron algorithm for the special case $\eta = 1$. Consider now the general Perceptron update $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta y_t \mathbf{x}_t$, where $\eta > 0$. Prove a bound on the maximum number of mistakes. How does η affect the bound?

8.3 Sparse instances. Suppose each input vector $\mathbf{x}_t$, $t \in [T]$, coincides with the tth unit vector of $\mathbb{R}^T$. How many updates are required for the Perceptron algorithm to converge? Show that the number of updates matches the margin bound of theorem 8.8.

8.4 Tightness of lower bound. Is the lower bound of theorem 8.5 tight? Explain why or show a counter-example.

8.5 On-line SVM algorithm. Consider the algorithm described in figure 8.11. Show that this algorithm corresponds to the stochastic gradient descent technique applied to the SVM problem (5.24) with hinge loss and no offset (i.e., fix $p = 1$ and $b = 0$).

```
MarginPerceptron()
1  w_1 ← 0
2  for t ← 1 to T do
3       Receive(x_t)
4       Receive(y_t)
5       if ((w_t = 0) or (y_t w_t · x_t / ||w_t|| < ρ/2)) then
6            w_{t+1} ← w_t + y_t x_t
7       else w_{t+1} ← w_t
8  return w_{T+1}
```

Figure 8.12
Margin Perceptron algorithm.

8.6 Margin Perceptron. Given a training sample S that is linearly separable with a maximum margin $\rho > 0$, theorem 8.8 states that the Perceptron algorithm run cyclically over S is guaranteed to converge after at most R^2/ρ^2 updates, where R is the radius of the sphere containing the sample points. However, this theorem does not guarantee that the hyperplane solution of the Perceptron algorithm achieves a margin close to ρ. Suppose we modify the Perceptron algorithm to ensure that the margin of the hyperplane solution is at least $\rho/2$. In particular, consider the algorithm described in figure 8.12. In this problem we show that this algorithm converges after at most $16R^2/\rho^2$ updates. Let $\mathcal{I}$ denote the set of times $t \in [T]$ at which the algorithm makes an update and let $M = |\mathcal{I}|$ be the total number of updates.

(a) Using an analysis similar to the one given for the Perceptron algorithm, show that $M\rho \leq \|\mathbf{w}_{T+1}\|$. Conclude that if $\|\mathbf{w}_{T+1}\| < \frac{4R^2}{\rho}$, then $M < 4R^2/\rho^2$. (For the remainder of this problem, we will assume that $\|\mathbf{w}_{T+1}\| \geq \frac{4R^2}{\rho}$.)

(b) Show that for any $t \in \mathcal{I}$ (including $t = 0$), the following holds:

$$\|\mathbf{w}_{t+1}\|^2 \leq (\|\mathbf{w}_t\| + \rho/2)^2 + R^2.$$

(c) From (b), infer that for any $t \in \mathcal{I}$ we have

$$\|\mathbf{w}_{t+1}\| \leq \|\mathbf{w}_t\| + \rho/2 + \frac{R^2}{\|\mathbf{w}_t\| + \|\mathbf{w}_{t+1}\| + \rho/2}.$$

(d) Using the inequality from (c), show that for any $t \in \mathcal{I}$ such that either $\|\mathbf{w}_t\| \geq \frac{4R^2}{\rho}$ or $\|\mathbf{w}_{t+1}\| \geq \frac{4R^2}{\rho}$, we have

$$\|\mathbf{w}_{t+1}\| \leq \|\mathbf{w}_t\| + \frac{3}{4}\rho.$$

(e) Show that $\|\mathbf{w}_1\| \leq R \leq 4R^2/\rho$. Since by assumption we have $\|\mathbf{w}_{T+1}\| \geq \frac{4R^2}{\rho}$, conclude that there must exist a largest time $t_0 \in \mathcal{I}$ such that $\|\mathbf{w}_{t_0}\| \leq \frac{4R^2}{\rho}$ and $\|\mathbf{w}_{t_0+1}\| \geq \frac{4R^2}{\rho}$.

(f) Show that $\|\mathbf{w}_{T+1}\| \leq \|\mathbf{w}_{t_0}\| + \frac{3}{4}M\rho$. Conclude that $M \leq 16R^2/\rho^2$.

8.7 Second-order regret bound. Consider the randomized algorithm that differs from the RWM algorithm only by the weight update, i.e., $w_{t+1,i} \leftarrow (1-(1-\beta)l_{t,i})w_{t,i}$, $t \in [T]$, which is applied to all $i \in [N]$ with $1/2 \leq \beta < 1$. This algorithm can be used in a more general setting than RWM since the losses $l_{t,i}$ are only assumed to be in $[0,1]$. The objective of this problem is to show that a similar upper bound can be shown for the regret.

(a) Use the same potential W_t as for the RWM algorithm and derive a simple upper bound for $\log W_{T+1}$:

$$\log W_{T+1} \leq \log N - (1-\beta)\mathcal{L}_T\,.$$

(*Hint*: Use the inequality $\log(1-x) \leq -x$ for $x \in [0, 1/2]$.)

(b) Prove the following lower bound for the potential for all $i \in [N]$:

$$\log W_{T+1} \geq -(1-\beta)\mathcal{L}_{T,i} - (1-\beta)^2 \sum_{t=1}^{T} l_{t,i}^2\,.$$

(*Hint*: Use the inequality $\log(1-x) \geq -x - x^2$, which is valid for all $x \in [0, 1/2]$.)

(c) Use upper and lower bounds to derive the following regret bound for the algorithm: $R_T \leq 2\sqrt{T \log N}$.

8.8 Polynomial weighted algorithm. The objective of this problem is to show how another regret minimization algorithm can be defined and studied. Let L be a loss function convex in its first argument and taking values in $[0, M]$.

We will assume $N > e^2$ and then for any expert $i \in [N]$, we denote by $r_{t,i}$ the instantaneous regret of that expert at time $t \in [T]$, $r_{t,i} = L(\widehat{y}_t, y_t) - L(y_{t,i}, y_t)$,

and by $R_{t,i}$ its cumulative regret up to time t: $R_{t,i} = \sum_{s=1}^{t} r_{t,i}$. For convenience, we also define $R_{0,i} = 0$ for all $i \in [N]$. For any $x \in \mathbb{R}$, $(x)_+$ denotes $\max(x, 0)$, that is the positive part of x, and for $\mathbf{x} = (x_1, \ldots, x_N)^\top \in \mathbb{R}^N$, $(\mathbf{x})_+ = ((x_1)_+, \ldots, (x_N)_+)^\top$.

Let $\alpha > 2$ and consider the algorithm that predicts at round $t \in [T]$ according to $\widehat{y}_t = \frac{\sum_{i=1}^{n} w_{t,i} y_{t,i}}{\sum_{i=1}^{n} w_{t,i}}$, with the weight $w_{t,i}$ defined based on the αth power of the regret up to time $(t-1)$: $w_{t,i} = (R_{t-1,i})_+^{\alpha-1}$. The potential function we use to analyze the algorithm is based on the function Φ defined over $\mathbb{R}^N$ by $\Phi \colon \mathbf{x} \mapsto \|(\mathbf{x})_+\|_\alpha^2 = \left[\sum_{i=1}^{N} (x_i)_+^\alpha\right]^{\frac{2}{\alpha}}$.

(a) Show that Φ is twice differentiable over $\mathbb{R}^N - B$, where B is defined as follows:

$$B = \{\mathbf{u} \in \mathbb{R}^N \colon (\mathbf{u})_+ = 0\}.$$

(b) For any $t \in [T]$, let $\mathbf{r}_t$ denote the vector of instantaneous regrets, $\mathbf{r}_t = (r_{t,1}, \ldots, r_{t,N})^\top$, and similarly $\mathbf{R}_t = (R_{t,1}, \ldots, R_{t,N})^\top$. We define the potential function as $\Phi(\mathbf{R}_t) = \|(\mathbf{R}_t)_+\|_\alpha^2$. Compute $\nabla\Phi(\mathbf{R}_{t-1})$ for $\mathbf{R}_{t-1} \notin B$ and show that $\nabla\Phi(\mathbf{R}_{t-1}) \cdot \mathbf{r}_t \leq 0$ (*Hint*: use the convexity of the loss with respect to the first argument).

(c) Prove the inequality $\mathbf{r}^\top[\nabla^2\Phi(\mathbf{u})]\mathbf{r} \leq 2(\alpha-1)\|\mathbf{r}\|_\alpha^2$ valid for all $\mathbf{r} \in \mathbb{R}^N$ and $\mathbf{u} \in \mathbb{R}^N - B$ (*Hint*: write the Hessian $\nabla^2\Phi(\mathbf{u})$ as a sum of a diagonal matrix and a positive semidefinite matrix multiplied by $(2-\alpha)$. Also, use Hölder's inequality generalizing Cauchy-Schwarz: for any $p > 1$ and $q > 1$ with $\frac{1}{p} + \frac{1}{q} = 1$ and $\mathbf{u}, \mathbf{v} \in \mathbb{R}^N$, $|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\|_p \|\mathbf{v}\|_q$).

(d) Using the answers to the two previous questions and Taylor's formula, show that for all $t \geq 1$, $\Phi(\mathbf{R}_t) - \Phi(\mathbf{R}_{t-1}) \leq (\alpha-1)\|\mathbf{r}_t\|_\alpha^2$, if $\gamma\mathbf{R}_{t-1} + (1-\gamma)\mathbf{R}_t \notin B$ for all $\gamma \in [0, 1]$.

(e) Suppose there exists $\gamma \in [0, 1]$ such that $(1-\gamma)\mathbf{R}_{t-1} + \gamma\mathbf{R}_t \in B$. Show that $\Phi(\mathbf{R}_t) \leq (\alpha-1)\|\mathbf{r}_t\|_\alpha^2$.

(f) Using the two previous questions, derive an upper bound on $\Phi(\mathbf{R}_T)$ expressed in terms of T, N, and M.

(g) Show that $\Phi(\mathbf{R}_T)$ admits as a lower bound the square of the regret R_T of the algorithm.

(h) Using the two previous questions give an upper bound on the regret R_T. For what value of α is the bound the most favorable? Give a simple expression of the upper bound on the regret for a suitable approximation of that optimal value.

8.9 General inequality. In this exercise we generalize the result of exercise 8.7 by using a more general inequality: $\log(1-x) \geq -x - \frac{x^2}{\alpha}$ for some $0 < \alpha < 2$.

(a) First prove that the inequality is true for $x \in [0, 1 - \frac{\alpha}{2}]$. What does this imply about the valid range of β?

(b) Give a generalized version of the regret bound derived in exercise 8.7 in terms of α, which shows:

$$R_T \leq \frac{\log N}{1-\beta} + \frac{1-\beta}{\alpha} T\,.$$

What is the optimal choice of β and the resulting bound in this case?

(c) Explain how α may act as a regularization parameter. What is the optimal choice of α?

8.10 On-line to batch — non-convex loss.

The on-line to batch result of theorem 8.15 heavily relies on the fact that the loss is convex in order to provide a generalization guarantee for the uniformly averaged hypothesis $\frac{1}{T}\sum_{i=1}^{T} h_i$. For general losses, instead of using the averaged hypothesis we will use a different strategy and try to estimate the best single base hypothesis and show the expected loss of this hypothesis is bounded.

Let m_i denote the cumulative loss of hypothesis h_i on the points $(x_i, \ldots, x_T)$, that is $m_i = \sum_{t=i}^{T} L(h_i(x_t), y_t)$. Then we define the *penalized risk estimate* of hypothesis h_i as,

$$\frac{m_i}{T-i+1} + c_\delta(T-i+1) \quad \text{where} \quad c_\delta(x) = \sqrt{\frac{1}{2x}\log\frac{T(T+1)}{\delta}}\,.$$

The term c_δ penalizes the empirical error when the test sample is small. Define $\widehat{h} = h_{i^*}$ where $i^* = \operatorname{argmin}_i m_i/(T-i+1) + c_\delta(T-i+1)$. We will then show under the same conditions of theorem 8.15 (with $M = 1$ for simplicity), but without requiring the convexity of L, that the following holds with probability at least $1-\delta$:

$$R(\widehat{h}) \leq \frac{1}{T}\sum_{i=1}^{T} L(h_i(x_i), y_i) + 6\sqrt{\frac{1}{T}\log\frac{2(T+1)}{\delta}}\,. \tag{8.31}$$

(a) Prove the following inequality:

$$\min_{i\in[T]}(R(h_i) + 2c_\delta(T-i+1)) \leq \frac{1}{T}\sum_{i=1}^{T} R(h_i) + 4\sqrt{\frac{1}{T}\log\frac{T+1}{\delta}}\,.$$

(b) Use part (a) to show that with probability at least $1-\delta$,

$$\min_{i\in[T]}(R(h_i)+2c_\delta(T-i+1)) < \sum_{i=1}^{T} L(h_i(x_i),y_i)+\sqrt{\frac{2}{T}\log\frac{1}{\delta}}+4\sqrt{\frac{1}{T}\log\frac{T+1}{\delta}}\,.$$

(c) By design, the definition of c_δ ensures that with probability at least $1-\delta$

$$R(\widehat{h}) \leq \min_{i\in[T]}(R(h_i)+2c_\delta(T-i+1))\,.$$

Use this property to complete the proof of (8.31).

8.11 On-line to batch — kernel Perceptron margin bound. In this problem, we give a margin-based generalization guarantee for the kernel Perceptron algorithm. Let $h_1,\ldots,h_T$ be the sequence of hypotheses generated by the kernel Perceptron algorithm and let $\widehat{h}$ be defined as in exercise 8.10. Finally, let L denote the zero-one loss. We now wish to more precisely bound the generalization error of $\widehat{h}$ in this setting.

(a) First, show that

$$\sum_{i=1}^{T} L(h_i(x_i),y_i) \leq \inf_{h\in\mathbb{H}:\|h\|\leq 1}\sum_{i=1}^{T}\max\left(0,1-\frac{y_i h(x_i)}{\rho}\right)+\frac{1}{\rho}\sqrt{\sum_{i\in I}K(x_i,x_i)},$$

where I is the set of indices where the kernel Perceptron makes an update and where δ and ρ are defined as in theorem 8.12.

(b) Now, use the result of exercise 8.10 to derive a generalization guarantee for $\widehat{h}$ in the case of kernel Perceptron, which states that for any $0<\delta\leq 1$, the following holds with probability at least $1-\delta$:

$$R(\widehat{h}) \leq \inf_{h\in\mathbb{H}:\|h\|\leq 1}\widehat{R}_{S,\rho}(h)+\frac{1}{\rho T}\sqrt{\sum_{i\in I}K(x_i,x_i)}+6\sqrt{\frac{1}{T}\log\frac{2(T+1)}{\delta}},$$

where $\widehat{R}_{S,\rho}(h)=\frac{1}{T}\sum_{i=1}^{T}\max\left(0,1-\frac{y_i h(x_i)}{\rho}\right)$. Compare this result with the margin bounds for kernel-based hypotheses given by corollary 6.13.

9 Multi-Class Classification

The classification problems we examined in the previous chapters were all binary. However, in most real-world classification problems the number of classes is greater than two. The problem may consist of assigning a topic to a text document, a category to a speech utterance or a function to a biological sequence. In all of these tasks, the number of classes may be on the order of several hundred or more.

In this chapter, we analyze the problem of multi-class classification. We first introduce the multi-class classification learning problem and discuss its multiple settings, and then derive generalization bounds for it using the notion of Rademacher complexity. Next, we describe and analyze a series of algorithms for tackling the multi-class classification problem. We will distinguish between two broad classes of algorithms: *uncombined algorithms* that are specifically designed for the multi-class setting such as multi-class SVMs, decision trees, or multi-class boosting, and *aggregated algorithms* that are based on a reduction to binary classification and require training multiple binary classifiers. We will also briefly discuss the problem of structured prediction, which is a related problem arising in a variety of applications.

9.1 Multi-class classification problem

Let $\mathcal{X}$ denote the input space and $\mathcal{Y}$ denote the output space, and let $\mathcal{D}$ be an unknown distribution over $\mathcal{X}$ according to which input points are drawn. We will distinguish between two cases: the *mono-label case*, where $\mathcal{Y}$ is a finite set of classes that we mark with numbers for convenience, $\mathcal{Y} = \{1, \ldots, k\}$, and the *multi-label case* where $\mathcal{Y} = \{-1, +1\}^k$. In the mono-label case, each example is labeled with a single class, while in the multi-label case it can be labeled with several. The latter can be illustrated by the case of text documents, which can be labeled with several different relevant topics, e.g., *sports*, *business*, and *society*. The positive components of a vector in $\{-1, +1\}^k$ indicate the classes associated with an example.

In either case, the learner receives a labeled sample $S = \big((x_1, y_1), \ldots, (x_m, y_m)\big) \in (\mathcal{X} \times \mathcal{Y})^m$ with $x_1, \ldots, x_m$ drawn i.i.d. according to $\mathcal{D}$, and $y_i = f(x_i)$ for all $i \in [m]$, where $f \colon \mathcal{X} \to \mathcal{Y}$ is the target labeling function. Thus, we consider a deterministic scenario, which, as discussed in section 2.4.1, can be straightforwardly extended to a stochastic one that admits a distribution over $\mathcal{X} \times \mathcal{Y}$.

Given a hypothesis set $\mathcal{H}$ of functions mapping $\mathcal{X}$ to $\mathcal{Y}$, the multi-class classification problem consists of using the labeled sample S to find a hypothesis $h \in \mathcal{H}$ with small generalization error $R(h)$ with respect to the target f:

$$R(h) = \mathop{\mathbb{E}}_{x \sim \mathcal{D}}[1_{h(x) \neq f(x)}] \qquad \text{mono-label case} \tag{9.1}$$

$$R(h) = \mathop{\mathbb{E}}_{x \sim \mathcal{D}}\Big[\sum_{l=1}^{k} 1_{[h(x)]_l \neq [f(x)]_l}\Big] \qquad \text{multi-label case.} \tag{9.2}$$

The notion of *Hamming distance* d_H, that is, the number of corresponding components in two vectors that differ, can be used to give a common formulation for both errors:

$$R(h) = \mathop{\mathbb{E}}_{x \sim \mathcal{D}}\Big[d_H(h(x), f(x))\Big]. \tag{9.3}$$

The empirical error of $h \in \mathcal{H}$ is denoted by $\widehat{R}_S(h)$ and defined by

$$\widehat{R}_S(h) = \frac{1}{m} \sum_{i=1}^{m} d_H(h(x_i), y_i)\,. \tag{9.4}$$

Several issues, both computational and learning-related, often arise in the multi-class setting. Computationally, dealing with a large number of classes can be problematic. The number of classes k directly enters the time complexity of the algorithms we will present. Even for a relatively small number of classes such as $k = 100$ or $k = 1{,}000$, some techniques may become prohibitive to use in practice. This dependency is even more critical in the case where k is very large or even infinite as in the case of some structured prediction problems.

A learning-related issue that commonly appears in the multi-class setting is the existence of unbalanced classes. Some classes may be represented by less than 5 percent of the labeled sample, while others may dominate a very large fraction of the data. When separate binary classifiers are used to define the multi-class solution, we may need to train a classifier distinguishing between two classes with only a small representation in the training sample. This implies training on a small sample, with poor performance guarantees. Alternatively, when a large fraction of the training instances belong to one class, it may be tempting to propose a hypothesis always returning that class, since its generalization error as defined earlier is likely to be relatively low. However, this trivial solution is typically not the

one intended. Instead, the loss function may need to be reformulated by assigning different misclassification weights to each pair of classes.

Another learning-related issue is the relationship between classes, which can be hierarchical. For example, in the case of document classification, the error of misclassifying a document dealing with *world politics* as one dealing with *real estate* should naturally be penalized more than the error of labeling a document with *sports* instead of the more specific label *baseball*. Thus, a more complex and more useful multi-class classification formulation would take into consideration the hierarchical relationships between classes and define the loss function in accordance with this hierarchy. More generally, there may be a graph relationship between classes as in the case of gene ontology in computational biology. The use of hierarchical relationships between classes leads to a richer and more complex multi-class classification problem.

9.2 Generalization bounds

In this section, we present margin-based generalization bounds for multi-class classification in the mono-label case. In the binary setting, classifiers are often defined based on the sign of a scoring function. In the multi-class setting, a hypothesis is defined based on a scoring function $h\colon \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$. The label associated to point x is the one resulting in the largest score $h(x, y)$, which defines the following mapping from $\mathcal{X}$ to $\mathcal{Y}$:

$$x \mapsto \operatorname*{argmax}_{y \in \mathcal{Y}} h(x, y).$$

This naturally leads to the following definition of the *margin* $\rho_h(x, y)$ *of the function* h *at a labeled example* (x, y):

$$\rho_h(x, y) = h(x, y) - \max_{y' \neq y} h(x, y').$$

Thus, h misclassifies (x, y) iff $\rho_h(x, y) \leq 0$. For any $\rho > 0$, we can define the *empirical margin loss of a hypothesis* h for multi-class classification as

$$\widehat{R}_{S,\rho}(h) = \frac{1}{m} \sum_{i=1}^{m} \Phi_\rho(\rho_h(x_i, y_i)), \tag{9.5}$$

where Φ_ρ is the margin loss function (definition 5.5). Thus, the empirical margin loss for multi-class classification is upper bounded by the fraction of the training points misclassified by h or correctly classified but with confidence less than or

equal to ρ:

$$\widehat{R}_{S,\rho}(h) \leq \frac{1}{m}\sum_{i=1}^{m} 1_{\rho_h(x_i,y_i)\leq\rho}. \tag{9.6}$$

The following lemma will be used in the proof of the main result of this section.

Lemma 9.1 *Let $\mathcal{F}_1,\ldots,\mathcal{F}_l$ be l hypothesis sets in $\mathbb{R}^{\mathcal{X}}$, $l\geq 1$, and let $\mathcal{G}=\{\max\{h_1,\ldots,h_l\}\colon h_i\in\mathcal{F}_i, i\in[l]\}$. Then, for any sample S of size m, the empirical Rademacher complexity of $\mathcal{G}$ can be upper bounded as follows:*

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) \leq \sum_{j=1}^{l}\widehat{\mathfrak{R}}_S(\mathcal{F}_j). \tag{9.7}$$

Proof: Let $S=(x_1,\ldots,x_m)$ be a sample of size m. We first prove the result in the case $l=2$. By definition of the max operator, for any $h_1\in\mathcal{F}_1$ and $h_2\in\mathcal{F}_2$,

$$\max\{h_1,h_2\} = \frac{1}{2}[h_1+h_2+|h_1-h_2|].$$

Thus, we can write:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\mathcal{G}) &= \frac{1}{m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{\substack{h_1\in\mathcal{F}_1\\ h_2\in\mathcal{F}_2}}\sum_{i=1}^{m}\sigma_i\max\{h_1(x_i),h_2(x_i)\}\Big]\\
&= \frac{1}{2m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{\substack{h_1\in\mathcal{F}_1\\ h_2\in\mathcal{F}_2}}\sum_{i=1}^{m}\sigma_i\big(h_1(x_i)+h_2(x_i)+|(h_1-h_2)(x_i)|\big)\Big]\\
&\leq \frac{1}{2}\widehat{\mathfrak{R}}_S(\mathcal{F}_1)+\frac{1}{2}\widehat{\mathfrak{R}}_S(\mathcal{F}_2)+\frac{1}{2m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{\substack{h_1\in\mathcal{F}_1\\ h_2\in\mathcal{F}_2}}\sum_{i=1}^{m}\sigma_i|(h_1-h_2)(x_i)|\Big],
\end{aligned} \tag{9.8}$$

using the sub-additivity of sup. Since $x\mapsto|x|$ is 1-Lipschitz, by Talagrand's lemma (lemma 5.7), the last term can be bounded as follows

$$\begin{aligned}
\frac{1}{2m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{\substack{h_1\in\mathcal{F}_1\\ h_2\in\mathcal{F}_2}}\sum_{i=1}^{m}\sigma_i|(h_1-h_2)(x_i)|\Big] &\leq \frac{1}{2m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{\substack{h_1\in\mathcal{F}_1\\ h_2\in\mathcal{F}_2}}\sum_{i=1}^{m}\sigma_i(h_1-h_2)(x_i)\Big]\\
&\leq \frac{1}{2}\widehat{\mathfrak{R}}_S(\mathcal{F}_1)+\frac{1}{2m}\mathop{\mathbb{E}}_{\boldsymbol{\sigma}}\Big[\sup_{h_2\in\mathcal{F}_2}\sum_{i=1}^{m}-\sigma_i h_2(x_i)\Big]\\
&= \frac{1}{2}\widehat{\mathfrak{R}}_S(\mathcal{F}_1)+\frac{1}{2}\widehat{\mathfrak{R}}_S(\mathcal{F}_2),
\end{aligned} \tag{9.9}$$

where we again use the sub-additivity of sup for the second inequality and the fact that σ_i and $-\sigma_i$ have the same distribution for any $i\in[m]$ for the last equality. Combining (9.8) and (9.9) yields $\widehat{\mathfrak{R}}_S(\mathcal{G})\leq\widehat{\mathfrak{R}}_S(\mathcal{F}_1)+\widehat{\mathfrak{R}}_S(\mathcal{F}_2)$. The general case can be derived from the case $l=2$ using $\max\{h_1,\ldots,h_l\}=\max\{h_1,\max\{h_2,\ldots,h_l\}\}$ and an immediate recurrence. □

For any family of hypotheses mapping $\mathcal{X} \times \mathcal{Y}$ to $\mathbb{R}$, we define $\Pi_1(\mathcal{H})$ by

$$\Pi_1(\mathcal{H}) = \{x \mapsto h(x, y) \colon y \in \mathcal{Y}, h \in \mathcal{H}\}.$$

The following theorem gives a general margin bound for multi-class classification.

Theorem 9.2 (Margin bound for multi-class classification) *Let $\mathcal{H} \subseteq \mathbb{R}^{\mathcal{X} \times \mathcal{Y}}$ be a hypothesis set with $\mathcal{Y} = \{1, \ldots, k\}$. Fix $\rho > 0$. Then, for any $\delta > 0$, with probability at least $1 - \delta$, the following multi-class classification generalization bound holds for all $h \in \mathcal{H}$:*

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{4k}{\rho}\mathfrak{R}_m(\Pi_1(\mathcal{H})) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}. \tag{9.10}$$

Proof: We will need the following definition for this proof:

$$\rho_{\theta,h}(x, y) = \min_{y'}(h(x, y) - h(x, y') + \theta 1_{y'=y}),$$

where $\theta > 0$ is an arbitrary constant. Observe that $\mathbb{E}[1_{\rho_h(x,y)\leq 0}] \leq \mathbb{E}[1_{\rho_{\theta,h}(x,y)\leq 0}]$ since the inequality $\rho_{\theta,h}(x, y) \leq \rho_h(x, y)$ holds for all $(x, y) \in \mathcal{X} \times \mathcal{Y}$:

$$\begin{aligned}
\rho_{\theta,h}(x, y) &= \min_{y'} \big(h(x, y) - h(x, y') + \theta 1_{y'=y}\big) \\
&\leq \min_{y' \neq y} \big(h(x, y) - h(x, y') + \theta 1_{y'=y}\big) \\
&= \min_{y' \neq y} \big(h(x, y) - h(x, y')\big) = \rho_h(x, y),
\end{aligned}$$

where the inequality follows from taking the minimum over a smaller set.

Now, similar to the proof of theorem 5.8, let $\widetilde{\mathcal{H}} = \{(x, y) \mapsto \rho_{\theta,h}(x, y) \colon h \in \mathcal{H}\}$ and $\widetilde{\mathcal{H}} = \{\Phi_\rho \circ \widetilde{h} \colon \widetilde{h} \in \widetilde{\mathcal{H}}\}$. By theorem 3.3, with probability at least $1 - \delta$, for all $h \in \mathcal{H}$,

$$\mathbb{E}\left[\Phi_\rho(\rho_{\theta,h}(x, y))\right] \leq \frac{1}{m}\sum_{i=1}^{m} \Phi_\rho(\rho_{\theta,h}(x_i, y_i)) + 2\mathfrak{R}_m(\widetilde{\mathcal{H}}) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}.$$

Since $1_{u\leq 0} \leq \Phi_\rho(u)$ for all $u \in \mathbb{R}$, the generalization error $R(h)$ is a lower bound on the left-hand side, $R(h) = \mathbb{E}[1_{\rho_h(x,y)\leq 0}] \leq \mathbb{E}[1_{\rho_{\theta,h}(x,y)\leq 0}] \leq \mathbb{E}\left[\Phi_\rho(\rho_{\theta,h}(x, y))\right]$, and we can write:

$$R(h) \leq \frac{1}{m}\sum_{i=1}^{m} \Phi_\rho(\rho_{\theta,h}(x_i, y_i)) + 2\mathfrak{R}_m(\widetilde{\mathcal{H}}) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}.$$

Fixing $\theta = 2\rho$, we observe that $\Phi_\rho(\rho_{\theta,h}(x_i, y_i)) = \Phi_\rho(\rho_h(x_i, y_i))$. Indeed, either $\rho_{\theta,h}(x_i, y_i) = \rho_h(x_i, y_i)$ or $\rho_{\theta,h}(x_i, y_i) = 2\rho \leq \rho_h(x_i, y_i)$, which implies the desired result. Furthermore, Talagrand's lemma (lemma 5.7) yields $\mathfrak{R}_m(\widetilde{\mathcal{H}}) \leq \frac{1}{\rho}\mathfrak{R}_m(\widetilde{\mathcal{H}})$ since Φ_ρ is a $\frac{1}{\rho}$-Lipschitz function. Therefore, for any $\delta > 0$, with probability at

least $1-\delta$, for all $h \in \mathcal{H}$:

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{2}{\rho}\mathfrak{R}_m(\widetilde{\mathcal{H}}) + \sqrt{\frac{\log\frac{1}{\delta}}{2m}}.$$

and to complete the proof it suffices to show that $\mathfrak{R}_m(\widetilde{\mathcal{H}}) \leq 2k\mathfrak{R}_m(\Pi_1(\mathcal{H}))$.

Here $\mathfrak{R}_m(\widetilde{\mathcal{H}})$ can be upper-bounded as follows:

$$\begin{aligned}\mathfrak{R}_m(\widetilde{\mathcal{H}}) &= \frac{1}{m}\mathop{\mathbb{E}}_{S,\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i(h(x_i,y_i) - \max_y(h(x_i,y) - 2\rho 1_{y=y_i}))\right]\\ &\leq \frac{1}{m}\mathop{\mathbb{E}}_{S,\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i,y_i)\right] + \frac{1}{m}\mathop{\mathbb{E}}_{S,\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i \max_y(h(x_i,y) - 2\rho 1_{y=y_i})\right].\end{aligned}$$

Now we bound the first term above. Observe that

$$\begin{aligned}\frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i,y_i)\right] &= \frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sum_{y\in\mathcal{Y}}\sigma_i h(x_i,y)1_{y_i=y}\right]\\ &\leq \frac{1}{m}\sum_{y\in\mathcal{Y}}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i,y)1_{y_i=y}\right]\\ &= \sum_{y\in\mathcal{Y}}\frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i,y)\left(\frac{\epsilon_i}{2}+\frac{1}{2}\right)\right],\end{aligned}$$

where $\epsilon_i = 2\cdot 1_{y_i=y} - 1$. Since $\epsilon_i \in \{-1,+1\}$, we have that σ_i and $\sigma_i\epsilon_i$ admit the same distribution and, for any $y \in \mathcal{Y}$, each of the terms of the right-hand side can be bounded as follows:

$$\begin{aligned}&\frac{1}{m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i,y)\Big(\frac{\epsilon_i}{2}+\frac{1}{2}\Big)\right]\\ &\quad\leq \frac{1}{2m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i\epsilon_i h(x_i,y)\right] + \frac{1}{2m}\mathop{\mathbb{E}}_{\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i,y)\right]\\ &\quad\leq \widehat{\mathfrak{R}}_m(\Pi_1(\mathcal{H})).\end{aligned}$$

Thus, we can write $\frac{1}{m}\mathbb{E}_{S,\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i,y_i)\right] \leq k\,\mathfrak{R}_m(\Pi_1(\mathcal{H}))$. To bound the second term, we first apply lemma 9.1 which immediately yields that

$$\begin{aligned}&\frac{1}{m}\mathop{\mathbb{E}}_{S,\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i\max_y(h(x_i,y) - 2\rho 1_{y=y_i})\right]\\ &\qquad\leq \sum_{y\in\mathcal{Y}}\frac{1}{m}\mathop{\mathbb{E}}_{S,\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i(h(x_i,y) - 2\rho 1_{y=y_i})\right]\end{aligned}$$

and since Rademacher variables are mean zero, we observe that

$$\begin{aligned}\mathop{\mathbb{E}}_{S,\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i(h(x_i,y)-2\rho 1_{y=y_i})\right] &= \mathop{\mathbb{E}}_{S,\sigma}\left[\sup_{h\in\mathcal{H}}\left(\sum_{i=1}^{m}\sigma_i h(x_i,y)\right)-2\rho\sum_{i=1}^{m}\sigma_i 1_{y=y_i}\right]\\ &= \mathop{\mathbb{E}}_{S,\sigma}\left[\sup_{h\in\mathcal{H}}\sum_{i=1}^{m}\sigma_i h(x_i,y)\right]\leq \mathfrak{R}_m(\Pi_1(\mathcal{H}))\end{aligned}$$

which completes the proof. □

These bounds can be generalized to hold uniformly for all $\rho > 0$ at the cost of an additional term $\sqrt{(\log\log_2(2/\rho))/m}$, as in theorem 5.9 and exercise 5.2. As for other margin bounds presented in previous sections, they show the conflict between two terms: the larger the desired pairwise ranking margin ρ, the smaller the middle term, at the price of a larger empirical multi-class classification margin loss $\widehat{R}_{S,\rho}$. Note, however, that here there is additionally a dependency on the number of classes k. This suggests either weaker guarantees when learning with a large number of classes or the need for even larger margins ρ for which the empirical margin loss would be small.

For some hypothesis sets, a simple upper bound can be derived for the Rademacher complexity of $\Pi_1(\mathcal{H})$, thereby making theorem 9.2 more explicit. We will show this for kernel-based hypotheses. Let $K\colon \mathcal{X}\times\mathcal{X}\to\mathbb{R}$ be a PDS kernel and let $\mathbf{\Phi}\colon \mathcal{X}\to\mathbb{H}$ be a feature mapping associated to K. In multi-class classification, a kernel-based hypothesis is based on k weight vectors $\mathbf{w}_1,\ldots,\mathbf{w}_k\in\mathbb{H}$. Each weight vector $\mathbf{w}_l$, $l\in[k]$, defines a scoring function $x\mapsto \mathbf{w}_l\cdot\mathbf{\Phi}(x)$ and the class associated to point $x\in\mathcal{X}$ is given by

$$\operatorname*{argmax}_{y\in\mathcal{Y}}\mathbf{w}_y\cdot\mathbf{\Phi}(x).$$

We denote by $\mathbf{W}$ the matrix formed by these weight vectors: $\mathbf{W}=(\mathbf{w}_1,\ldots,\mathbf{w}_k)^\top$ and for any $p\geq 1$ denote by $\|\mathbf{W}\|_{\mathbb{H},p}$ the $L_{\mathbb{H},p}$ group norm of $\mathbf{W}$ defined by

$$\|\mathbf{W}\|_{\mathbb{H},p}=\Big(\sum_{l=1}^{k}\|\mathbf{w}_l\|_{\mathbb{H}}^p\Big)^{1/p}.$$

For any $p\geq 1$, the family of kernel-based hypotheses we will consider is[13]

$$\mathcal{H}_{K,p}=\{(x,y)\in\mathcal{X}\times\{1,\ldots,k\}\mapsto \mathbf{w}_y\cdot\mathbf{\Phi}(x)\colon \mathbf{W}=(\mathbf{w}_1,\ldots,\mathbf{w}_k)^\top, \|\mathbf{W}\|_{\mathbb{H},p}\leq\Lambda\}.$$

Proposition 9.3 (Rademacher complexity of multi-class kernel-based hypotheses) *Let $K\colon \mathcal{X}\times\mathcal{X}\to\mathbb{R}$ be a PDS kernel and let $\mathbf{\Phi}\colon \mathcal{X}\to\mathbb{H}$ be a feature mapping associated to*

[13] The hypothesis set $\mathcal{H}$ can also be defined via $\mathcal{H}=\{h\in\mathbb{R}^{\mathcal{X}\times\mathcal{Y}}\colon h(\cdot,y)\in\mathbb{H}\wedge\|h\|_{K,p}\leq\Lambda\}$, where $\|h\|_{K,p}=\big(\sum_{y=1}^{k}\|h(\cdot,y)\|_{\mathbb{H}}^p\big)^{1/p}$, without referring to a feature mapping for K.

K. Assume that there exists $r > 0$ such that $K(x,x) \leq r^2$ for all $x \in \mathcal{X}$. Then, for any $m \geq 1$, $\mathfrak{R}_m(\Pi_1(\mathcal{H}_{K,p}))$ can be bounded as follows:

$$\mathfrak{R}_m(\Pi_1(\mathcal{H}_{K,p})) \leq \sqrt{\frac{r^2\Lambda^2}{m}}.$$

Proof: Let $S = (x_1, \ldots, x_m)$ denote a sample of size m. Observe that for all $l \in [k]$, the inequality $\|\mathbf{w}_l\|_{\mathbb{H}} \leq \left(\sum_{l=1}^k \|\mathbf{w}_l\|_{\mathbb{H}}^p\right)^{1/p} = \|\mathbf{W}\|_{\mathbb{H},p}$ holds. Thus, the condition $\|\mathbf{W}\|_{\mathbb{H},p} \leq \Lambda$ implies that $\|\mathbf{w}_l\|_{\mathbb{H}} \leq \Lambda$ for all $l \in [k]$. In view of that, the Rademacher complexity of the hypothesis set $\Pi_1(\mathcal{H}_{K,p})$ can be expressed and bounded as follows:

$$\begin{aligned}
\mathfrak{R}_m(\Pi_1(\mathcal{H}_{K,p})) &= \frac{1}{m} \mathop{\mathbb{E}}_{S,\boldsymbol{\sigma}}\left[\sup_{\substack{y \in \mathcal{Y} \\ \|\mathbf{W}\| \leq \Lambda}} \left\langle \mathbf{w}_y, \sum_{i=1}^m \sigma_i \boldsymbol{\Phi}(x_i)\right\rangle\right] \\
&\leq \frac{1}{m} \mathop{\mathbb{E}}_{S,\boldsymbol{\sigma}}\left[\sup_{\substack{y \in \mathcal{Y} \\ \|\mathbf{W}\| \leq \Lambda}} \|\mathbf{w}_y\|_{\mathbb{H}} \left\| \sum_{i=1}^m \sigma_i \Phi(x_i)\right\|_{\mathbb{H}}\right] \quad \text{(Cauchy-Schwarz ineq.)} \\
&\leq \frac{\Lambda}{m} \mathop{\mathbb{E}}_{S,\boldsymbol{\sigma}}\left[\left\| \sum_{i=1}^m \sigma_i \Phi(x_i)\right\|_{\mathbb{H}}\right] \\
&\leq \frac{\Lambda}{m}\left[\mathop{\mathbb{E}}_{S,\boldsymbol{\sigma}}\left[\left\| \sum_{i=1}^m \sigma_i \Phi(x_i)\right\|_{\mathbb{H}}^2\right]\right]^{1/2} \quad \text{(Jensen's inequality)} \\
&= \frac{\Lambda}{m}\left[\mathop{\mathbb{E}}_{S,\boldsymbol{\sigma}}\left[\sum_{i=1}^m \|\Phi(x_i)\|_{\mathbb{H}}^2\right]\right]^{1/2} \quad (i \neq j \Rightarrow \mathop{\mathbb{E}}_{\boldsymbol{\sigma}}[\sigma_i\sigma_j] = 0) \\
&= \frac{\Lambda}{m}\left[\mathop{\mathbb{E}}_{S,\boldsymbol{\sigma}}\left[\sum_{i=1}^m K(x_i,x_i)\right]\right]^{1/2} \\
&\leq \frac{\Lambda\sqrt{mr^2}}{m} = \sqrt{\frac{r^2\Lambda^2}{m}},
\end{aligned}$$

which concludes the proof. □

Combining theorem 9.2 and proposition 9.3 yields directly the following result.

Corollary 9.4 (Margin bound for multi-class classification with kernel-based hypotheses) *Let $K\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a PDS kernel and let $\boldsymbol{\Phi}\colon \mathcal{X} \to \mathbb{H}$ be a feature mapping associated to K. Assume that there exists $r > 0$ such that $K(x,x) \leq r^2$ for all $x \in \mathcal{X}$. Fix $\rho > 0$. Then, for any $\delta > 0$, with probability at least $1 - \delta$, the following multi-class classification generalization bound holds for all $h \in \mathcal{H}_{K,p}$:*

$$R(h) \leq \widehat{R}_{S,\rho}(h) + 4k\sqrt{\frac{r^2\Lambda^2/\rho^2}{m}} + \sqrt{\frac{\log\frac{1}{\delta}}{2m}}. \tag{9.11}$$

In the next two sections, we describe multi-class classification algorithms that belong to two distinct families: *uncombined algorithms*, which are defined by a single optimization problem, and *aggregated algorithms*, which are obtained by training multiple binary classifications and by combining their outputs.

9.3 Uncombined multi-class algorithms

In this section, we describe three algorithms designed specifically for multi-class classification. We start with a multi-class version of SVMs, then describe a boosting-type multi-class algorithm, and conclude with *decision trees*, which are often used as base learners in boosting.

9.3.1 Multi-class SVMs

We describe an algorithm that can be derived directly from the theoretical guarantees presented in the previous section. Proceeding as in section 5.4 for classification, the guarantee of corollary 9.4 can be expressed as follows: for any $\delta > 0$, with probability at least $1 - \delta$, for all $h \in \mathcal{H}_{K,2} = \{(x, y) \to \mathbf{w}_y \cdot \mathbf{\Phi}(x) \colon \mathbf{W} = (\mathbf{w}_1, \ldots, \mathbf{w}_k)^\top, \sum_{l=1}^k \|\mathbf{w}_l\|^2 \leq \Lambda^2\}$,

$$R(h) \leq \frac{1}{m} \sum_{i=1}^m \xi_i + 4k \sqrt{\frac{r^2 \Lambda^2}{m}} + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}, \tag{9.12}$$

where $\xi_i = \max\left(1 - [\mathbf{w}_{y_i} \cdot \mathbf{\Phi}(x_i) - \max_{y' \neq y_i} \mathbf{w}_{y'} \cdot \mathbf{\Phi}(x_i)], 0\right)$ for all $i \in [m]$.

An algorithm based on this theoretical guarantee consists of minimizing the right-hand side of (9.12), that is, minimizing an objective function with a term corresponding to the sum of the slack variables ξ_i, and another one minimizing $\|\mathbf{W}\|_{\mathbb{H},2}$ or equivalently $\sum_{l=1}^k \|\mathbf{w}_l\|^2$. This is precisely the optimization problem defining the *multi-class SVM* algorithm:

$$\begin{aligned} \min_{\mathbf{W}, \boldsymbol{\xi}} \; & \frac{1}{2} \sum_{l=1}^k \|\mathbf{w}_l\|^2 + C \sum_{i=1}^m \xi_i \\ \text{subject to: } & \forall i \in [m], \forall l \in \mathcal{Y} - \{y_i\}, \\ & \mathbf{w}_{y_i} \cdot \mathbf{\Phi}(x_i) \geq \mathbf{w}_l \cdot \mathbf{\Phi}(x_i) + 1 - \xi_i, \\ & \xi_i \geq 0. \end{aligned}$$

The decision function learned is of the form $x \mapsto \operatorname{argmax}_{l \in \mathcal{Y}} \mathbf{w}_l \cdot \mathbf{\Phi}(x)$. As with the primal problem of SVMs, this is a convex optimization problem: the objective function is convex, since it is a sum of convex functions, and the constraints are affine and thus qualified. The objective and constraint functions are differentiable, and the KKT conditions hold at the optimum. Defining the Lagrangian and applying

these conditions leads to the equivalent dual optimization problem, which can be expressed in terms of the kernel function K alone:

$$\max_{\boldsymbol{\alpha}\in\mathbb{R}^{m\times k}} \sum_{i=1}^{m} \boldsymbol{\alpha}_i \cdot \mathbf{e}_{y_i} - \frac{1}{2}\sum_{i=1}^{m}(\boldsymbol{\alpha}_i \cdot \boldsymbol{\alpha}_j)K(x_i, x_j)$$
$$\text{subject to: } \forall i \in [m], (0 \leq \alpha_{iy_i} \leq C) \wedge (\forall j \neq y_i, \alpha_{ij} \leq 0) \wedge (\boldsymbol{\alpha}_i \cdot \mathbf{1} = 0).$$

Here, $\boldsymbol{\alpha} \in \mathbb{R}^{m\times k}$ is a matrix, $\boldsymbol{\alpha}_i$ denotes the ith row of $\boldsymbol{\alpha}$, and $\mathbf{e}_l$ the lth unit vector in $\mathbb{R}^k$, $l \in [k]$. Both the primal and dual problems are simple QPs generalizing those of the standard SVM algorithm. However, the size of the solution and the number of constraints for both problems is in $\Omega(mk)$, which, for a large number of classes k, can make it difficult to solve. However, there exist specific optimization solutions designed for this problem based on a decomposition of the problem into m disjoint sets of constraints.

9.3.2 Multi-class boosting algorithms

We describe a boosting algorithm for multi-class classification called *AdaBoost.MH*, which in fact coincides with a special instance of AdaBoost. An alternative multi-class classification algorithm based on similar boosting ideas, AdaBoost.MR, is described and analyzed in exercise 9.4. AdaBoost.MH applies to the multi-label setting where $\mathcal{Y} = \{-1, +1\}^k$. As in the binary case, it returns a convex combination of base classifiers selected from a hypothesis set $\mathcal{H} = \{h_1, \ldots, h_N\}$. Let F be the following objective function defined for all samples $S = ((x_1, y_1), \ldots, (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ and $\bar{\boldsymbol{\alpha}} = (\bar{\alpha}_1, \ldots, \bar{\alpha}_N) \in \mathbb{R}^N$, $N \geq 1$, by

$$F(\bar{\boldsymbol{\alpha}}) = \sum_{i=1}^{m}\sum_{l=1}^{k} e^{-y_i[l] f_N(x_i,l)} = \sum_{i=1}^{m}\sum_{l=1}^{k} e^{-y_i[l]\sum_{j=1}^{N}\bar{\alpha}_j h_j(x_i,l)}, \tag{9.13}$$

where $f_N = \sum_{j=1}^{N}\bar{\alpha}_j h_j$ and where $y_i[l]$ denotes the lth coordinate of y_i for any $i \in [m]$ and $l \in [k]$. F is a convex and differentiable upper bound on the multi-class multi-label loss:

$$\sum_{i=1}^{m}\sum_{l=1}^{k} 1_{y_i[l]\neq f_N(x_i,l)} \leq \sum_{i=1}^{m}\sum_{l=1}^{k} e^{-y_i[l] f_N(x_i,l)}, \tag{9.14}$$

since for any $x \in \mathcal{X}$ with label $y = f(x)$ and any $l \in [k]$, the inequality $1_{y[l]\neq f_N(x,l)} \leq e^{-y[l] f_N(x,l)}$ holds. Using the same arguments as in section 7.2.2, we see that AdaBoost.MH coincides exactly with the application of coordinate descent to the objective function F. Figure 9.1 gives the pseudocode of the algorithm in the case where the base classifiers are functions mapping from $\mathcal{X} \times \mathcal{Y}$ to $\{-1, +1\}$. The algorithm takes as input a labeled sample $S = ((x_1, y_1), \ldots, (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ and maintains a distribution $\mathcal{D}_t$ over $\{1, \ldots, m\} \times \mathcal{Y}$. The remaining details of the

AdaBoost.MH$(S = ((x_1, y_1), \ldots, (x_m, y_m)))$

for $i \leftarrow 1$ **to** m **do**
  for $l \leftarrow 1$ **to** k **do**
    $\mathcal{D}_1(i,l) \leftarrow \frac{1}{mk}$
for $j \leftarrow 1$ **to** N **do**
  $h_j \leftarrow$ base classifier in $\mathcal{H}$ with small error $\epsilon_j = \mathbb{P}_{(i,l)\sim\mathcal{D}_j}[h_j(x_i,l) \neq y_i[l]]$
  $\bar{\alpha}_j \leftarrow \frac{1}{2}\log\frac{1-\epsilon_j}{\epsilon_j}$
  $Z_t \leftarrow 2[\epsilon_j(1-\epsilon_j)]^{\frac{1}{2}}$ ▷ normalization factor
  for $i \leftarrow 1$ **to** m **do**
    for $l \leftarrow 1$ **to** k **do**
      $\mathcal{D}_{j+1}(i,l) \leftarrow \frac{\mathcal{D}_j(i,l)\exp(-\bar{\alpha}_j y_i[l] h_j(x_i,l))}{Z_j}$
$f_N \leftarrow \sum_{j=1}^N \bar{\alpha}_j h_j$
return $h = \operatorname{sgn}(f_N)$

Figure 9.1
AdaBoost.MH algorithm, for $\mathcal{H} \subseteq (\{-1,+1\}^k)^{\mathcal{X}\times\mathcal{Y}}$.

algorithm are similar to AdaBoost. In fact, AdaBoost.MH exactly coincides with AdaBoost applied to the training sample derived from S by splitting each labeled point (x_i, y_i) into k labeled examples $((x_i, l), y_i[l])$, with each example (x_i, l) in $\mathcal{X} \times \mathcal{Y}$ and its label in $\{-1, +1\}$:

$$(x_i, y_i) \to ((x_i, 1), y_i[1]), \ldots, ((x_i, k), y_i[k]), i \in [m].$$

Let S' denote the resulting sample, then $S' = ((x_1, 1), y_1[1]), \ldots, (x_m, k), y_m[k]))$. S' contains mk examples and the expression of the objective function F in (9.13) coincides exactly with that of the objective function of AdaBoost for the sample S'. In view of this connection, the theoretical analysis along with the other observations we presented for AdaBoost in chapter 7 also apply here. Hence, we will focus on aspects related to the computational efficiency and to the weak learning condition that are specific to the multi-class scenario.

The complexity of the algorithm is that of AdaBoost applied to a sample of size mk. For $\mathcal{X} \subseteq \mathbb{R}^d$, using boosting stumps as base classifiers, the complexity of the algorithm is therefore in $O((mk)\log(mk) + mkdN)$. Thus, for a large number of classes k, the algorithm may become impractical using a single processor. The

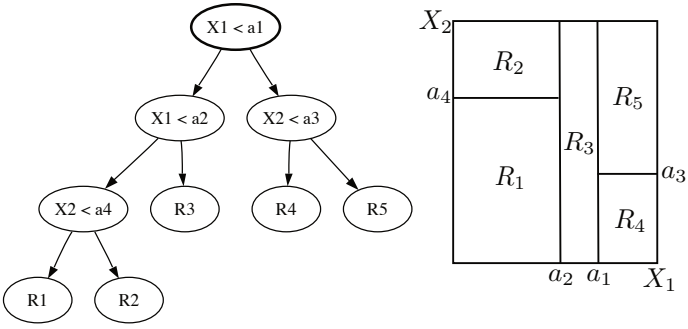


Figure 9.2
Left: example of a decision tree with numerical questions based on two variables X_1 and X_2. Here, each leaf is marked with the region it defines. The class labeling for a leaf is obtained via majority vote based on the training points falling in the region it defines. Right: Partition of the two-dimensional space induced by that decision tree.

weak learning condition for the application of AdaBoost in this scenario requires that at each round there exists a base classifier $h_j \colon \mathcal{X} \times \mathcal{Y} \to \{-1, +1\}$ such that $\mathbb{P}_{(i,l)\sim\mathcal{D}_j}[h_j(x_i, l) \neq y_i[l]] < 1/2$. This may be hard to achieve if some classes difficult to distinguish between. It is also more difficult in this context to come up with "rules of thumb" h_j defined over $\mathcal{X} \times \mathcal{Y}$.

9.3.3 Decision trees

We present and discuss the general learning method of *decision trees* that can be used in multi-class classification, but also in other learning problems such as regression (chapter 11) and clustering. Although the empirical performance of decision trees often is not state-of-the-art, decision trees can be used as weak learners with boosting to define effective learning algorithms. Decision trees are also typically fast to train and evaluate and relatively easy to interpret.

Definition 9.5 (Binary decision tree) *A* binary decision tree *is a tree representation of a partition of the feature space. Figure 9.2 shows a simple example in the case of a two-dimensional space based on two features X_1 and X_2, as well as the partition it represents. Each interior node of a decision tree corresponds to a question related to the features. It can be a* numerical question *of the form $X_i \leq a$ for a feature variable X_i, $i \in [N]$, and some threshold $a \in \mathbb{R}$, as in the example of figure 9.2, or a* categorical question *such as $X_i \in \{blue, white, red\}$, when feature X_i takes a categorical value such as a color. Each leaf is labeled with a label $l \in \mathcal{Y}$.*

Decision trees can be defined using more complex node questions, resulting in partitions based on more complex decision surfaces. For example, *binary space*

GreedyDecisionTrees($S = ((x_1, y_1), \ldots, (x_m, y_m))$)

1 $\mathsf{tree} \leftarrow \{\mathsf{n}_0\}$ ▷ root node.
2 **for** $t \leftarrow 1$ **to** T **do**
3 $\quad (\mathsf{n}_t, \mathsf{q}_t) \leftarrow \operatorname{argmax}_{(\mathsf{n},\mathsf{q})} \widetilde{F}(\mathsf{n}, \mathsf{q})$
4 $\quad$ Split($\mathsf{tree}, \mathsf{n}_t, \mathsf{q}_t$)
5 **return** tree

Figure 9.3
Greedy algorithm for building a decision tree from a labeled sample S. The procedure Split($\mathsf{tree}, \mathsf{n}_t, \mathsf{q}_t$) splits node n_t by making it an internal node with question q_t and leaf children $\mathsf{n}_-(\mathsf{n}, \mathsf{q})$ and $\mathsf{n}_+(\mathsf{n}, \mathsf{q})$, each labeled with the dominating class of the region it defines, with ties broken arbitrarily.

partition (BSP) trees partition the space with convex polyhedral regions, based on questions of the form $\sum_{i=1}^{n} \alpha_i X_i \leq a$, and *sphere trees* partition with pieces of spheres based on questions of the form $\|X - a_0\| \leq a$, where X is a feature vector, a_0 a fixed vector, and a is a fixed positive real number. More complex tree questions lead to richer partitions and thus hypothesis sets, which can cause overfitting in the absence of a sufficiently large training sample. They also increase the computational complexity of prediction and training. Decision trees can also be generalized to branching factors greater than two, but binary trees are most commonly used due their more limited computational cost.

Prediction/partitioning: To predict the label of any point $x \in \mathcal{X}$ we start at the root node of the decision tree and go down the tree until a leaf is found, by moving to the right child of a node when the response to the node question is positive, and to the left child otherwise. When we reach a leaf, we associate x with the label of this leaf.

Thus, each leaf defines a *region* of $\mathcal{X}$ formed by the set of points corresponding exactly to the same node responses and thus the same traversal of the tree. By definition, no two regions intersect and all points belong to exactly one region. Thus, leaf regions define a partition of $\mathcal{X}$, as shown in the example of figure 9.2. In multi-class classification, the label of a leaf is determined using the training sample: the class with the majority representation among the training points falling in a leaf region defines the label of that leaf, with ties broken arbitrarily.

Learning: We will discuss two different methods for learning a decision tree using a labeled sample. The first method is a greedy technique. This is motivated by the fact that the general problem of finding a decision tree with the smallest

error is NP-hard. The method consists of starting with a tree reduced to a single (root) node, which is a leaf whose label is the class that has majority over the entire sample. Next, at each round, a node n_t is split based on some question q_t. The pair $(\mathsf{n}_t, \mathsf{q}_t)$ is chosen so that the *node impurity* is maximally decreased according to some measure of impurity F. We denote by $F(\mathsf{n})$ the impurity of n. The decrease in node impurity after a split of node n based on question q is defined as follows. Let $\mathsf{n}_+(\mathsf{n}, \mathsf{q})$ denote the right child of n after the split, $\mathsf{n}_-(\mathsf{n}, \mathsf{q})$ the left child, and $\eta(\mathsf{n}, \mathsf{q})$ the fraction of the points in the region defined by n that are moved to $\mathsf{n}_-(\mathsf{n}, \mathsf{q})$. The total impurity of the leaves $\mathsf{n}_-(\mathsf{n}, \mathsf{q})$ and $\mathsf{n}_+(\mathsf{n}, \mathsf{q})$ is therefore $\eta(\mathsf{n}, \mathsf{q})F(\mathsf{n}_-(\mathsf{n}, \mathsf{q})) + (1 - \eta(\mathsf{n}, \mathsf{q}))F(\mathsf{n}_+(\mathsf{n}, \mathsf{q}))$. Thus, the decrease in impurity $\widetilde{F}(\mathsf{n}, \mathsf{q})$ by that split is given by

$$\widetilde{F}(\mathsf{n}, \mathsf{q}) = F(\mathsf{n}) - [\eta(\mathsf{n}, \mathsf{q})F(\mathsf{n}_-(\mathsf{n}, \mathsf{q})) + (1 - \eta(\mathsf{n}, \mathsf{q}))F(\mathsf{n}_+(\mathsf{n}, \mathsf{q}))].$$

Figure 9.3 shows the pseudocode of this greedy construction based on $\widetilde{F}$. In practice, the algorithm is stopped once all nodes have reached a sufficient level of purity, when the number of points per leaf has become too small for further splitting or based on some other similar heuristic.

For any node n and class $l \in [k]$, let $p_l(\mathsf{n})$ denote the fraction of points at n that belong to class l. Then, the three most commonly used measures of node impurity F are defined as follows:

$$F(\mathsf{n}) = \begin{cases} 1 - \max_{l \in [k]} p_l(\mathsf{n}) & \textit{misclassification}; \\ -\sum_{l=1}^{k} p_l(\mathsf{n}) \log_2 p_l(\mathsf{n}) & \textit{entropy}; \\ \sum_{l=1}^{k} p_l(\mathsf{n})(1 - p_l(\mathsf{n})) & \textit{Gini index}. \end{cases}$$

Figure 9.4 illustrates these definitions in the special cases of two classes ($k = 2$). The entropy and Gini index impurity functions are upper bounds on the misclassification impurity function. All three functions are concave, which ensures that

$$F(\mathsf{n}) - [\eta(\mathsf{n}, \mathsf{q})F(\mathsf{n}_-(\mathsf{n}, \mathsf{q})) + (1 - \eta(\mathsf{n}, \mathsf{q}))F(\mathsf{n}_+(\mathsf{n}, \mathsf{q}))] \geq 0.$$

However, the misclassification function is piecewise linear, so $\widetilde{F}(\mathsf{n}, \mathsf{q})$ is zero if the fraction of positive points remains less than (or more than) half after a split. In some cases, the impurity cannot be decreased by any split using that criterion. In contrast, the entropy and Gini functions are strictly concave, which guarantees a strict decrease in impurity. Furthermore, they are differentiable which is a useful feature for numerical optimization. Thus, the Gini index and the entropy criteria are typically preferred in practice.

The greedy method just described faces some issues. One issue relates to the greedy nature of the algorithm: a seemingly bad split may dominate subsequent useful splits, which could lead to trees with less impurity overall. This can be

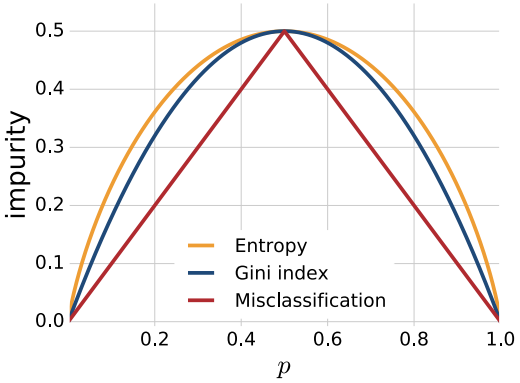


Figure 9.4
Three node impurity definitions plotted as a function of the fraction of positive examples in the binary case: misclassification, entropy (scaled by 0.5 to set the maximum to the same value for all three functions), and the Gini index.

addressed to a certain extent by using a look-ahead of some depth d to determine the splitting decisions, but such look-aheads can be computationally very costly. Another issue relates to the size of the resulting tree. To achieve some desired level of impurity, trees of relatively large sizes may be needed. However, larger trees define overly complex hypotheses with high VC-dimensions (see exercise 9.5) and thus could overfit.

An alternative method for learning decision trees using a labeled training sample is based on the so-called *grow-then-prune strategy*. First a very large tree is grown until it fully fits the training sample or until no more than a very small number of points are left at each leaf. Then, the resulting tree, denoted as tree, is pruned back to minimize an objective function defined (based on generalization bounds) as the sum of an empirical error and a complexity term. The complexity can be expressed in terms of the size of $\widetilde{\mathsf{tree}}$, the set of leaves of tree. The resulting objective is

$$G_\lambda(\mathsf{tree}) = \sum_{\mathsf{n}\in\widetilde{\mathsf{tree}}} |\mathsf{n}|F(\mathsf{n}) + \lambda|\widetilde{\mathsf{tree}}|\,, \tag{9.15}$$

where $\lambda \geq 0$ is a regularization parameter determining the trade-off between misclassification, or more generally impurity, versus tree complexity. For any tree tree', we denote by $\widehat{R}(\mathsf{tree}')$ the total empirical error $\sum_{\mathsf{n}\in\widetilde{\mathsf{tree}}'} |\mathsf{n}|F(\mathsf{n})$. We seek a sub-tree tree_λ of tree that minimizes G_λ and that has the smallest size. tree_λ can be shown to be unique. To determine tree_λ, the following pruning method is used, which defines a finite sequence of nested sub-trees $\mathsf{tree}^{(0)}, \ldots, \mathsf{tree}^{(n)}$. We start with the full tree $\mathsf{tree}^{(0)} = \mathsf{tree}$ and for any $i \in \{0, \ldots, n-1\}$, define $\mathsf{tree}^{(i+1)}$ from $\mathsf{tree}^{(i)}$ by

collapsing an internal node n' of $\mathsf{tree}^{(i)}$, that is by replacing the sub-tree rooted at n' with a leaf, or equivalently by combining the regions of all the leaves dominated by n'. n' is chosen so that collapsing it causes the smallest per node increase in $\widehat{R}(\mathsf{tree}^{(i)})$, that is the smallest $r(\mathsf{tree}^{(i)}, \mathsf{n}')$ defined by

$$r(\mathsf{tree}^{(i)}, \mathsf{n}') = \frac{|\mathsf{n}'|F(\mathsf{n}') - \widehat{R}(\mathsf{tree}')}{|\widetilde{\mathsf{tree}}'| - 1},$$

where n' is an internal node of $\mathsf{tree}^{(i)}$. If several nodes n' in $\mathsf{tree}^{(i)}$ cause the same smallest increase per node $r(\mathsf{tree}^{(i)}, \mathsf{n}')$, then all of them are pruned to define $\mathsf{tree}^{(i+1)}$ from $\mathsf{tree}^{(i)}$. This procedure continues until the tree $\mathsf{tree}^{(n)}$ obtained has a single node. The sub-tree tree_λ can be shown to be among the elements of the sequence $\mathsf{tree}^{(0)}, \ldots, \mathsf{tree}^{(n)}$. The parameter λ is determined via n-fold cross-validation.

Decision trees seem relatively easy to interpret, and this is often underlined as one of their most useful features. However, such interpretations should be carried out with care since decision trees are *unstable*: small changes in the training data may lead to very different splits and thus entirely different trees, as a result of their hierarchical nature. Decision trees can also be used in a natural manner to deal with the problem of *missing features*, which often appears in learning applications; in practice, some features values may be missing because the proper measurements were not taken or because of some noise source causing their systematic absence. In such cases, only those variables available at a node can be used in prediction. Finally, decision trees can be used and learned from data in a similar way in the *regression* setting (see chapter 11).[14]

9.4 Aggregated multi-class algorithms

In this section, we discuss a different approach to multi-class classification that reduces the problem to that of multiple binary classification tasks. A binary classification algorithm is then trained for each of these tasks independently, and the multi-class predictor is defined as a combination of the hypotheses returned by each of these algorithms. We first discuss two standard techniques for the reduction of multi-class classification to binary classification, and then show that they are both special instances of a more general framework.

[14] The only changes to the description for classification are the following. For prediction, the label of a leaf is defined as the mean squared average of the labels of the points falling in that region. For learning, the impurity function is the mean squared error.

9.4.1 One-versus-all

Let $S = ((x_1, y_1), \ldots, x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ be a labeled training sample. A straightforward reduction of the multi-class classification to binary classification is based on the so-called *one-versus-all (OVA) or one-versus-the-rest technique*. This technique consists of learning k binary classifiers $h_l \colon \mathcal{X} \to \{-1, +1\}$, $l \in \mathcal{Y}$, each seeking to discriminate one class $l \in \mathcal{Y}$ from all the others. For any $l \in \mathcal{Y}$, h_l is obtained by training a binary classification algorithm on the full sample S after relabeling points in class l with 1 and all others with -1. For $l \in \mathcal{Y}$, assume that h_l is derived from the sign of a scoring function $f_l \colon \mathcal{X} \to \mathbb{R}$, that is $h_l = \operatorname{sgn}(f_l)$, as in the case of many of the binary classification algorithms discussed in the previous chapters. Then, the multi-class hypothesis $h \colon \mathcal{X} \to \mathcal{Y}$ defined by the OVA technique is given by:

$$\forall x \in \mathcal{X}, \quad h(x) = \operatorname*{argmax}_{l \in \mathcal{Y}} f_l(x). \tag{9.16}$$

This formula may seem similar to those defining a multi-class classification hypothesis in the case of uncombined algorithms. Note, however, that for uncombined algorithms the functions f_l are learned together, while here they are learned independently. Formula (9.16) is well-founded when the scores given by functions f_l can be interpreted as confidence scores, that is when $f_l(x)$ is learned as an estimate of the probability of x conditioned on class l. However, in general, the scores given by functions f_l, $l \in \mathcal{Y}$, are not comparable and the OVA technique based on (9.16) admits *no principled justification*. This is sometimes referred to as a *calibration problem*. Clearly, this problem cannot be corrected by simply normalizing the scores of each function to make their magnitudes uniform, or by applying other similar heuristics. When it is justifiable, the OVA technique is simple and its computational cost is k times that of training a binary classification algorithm, which is similar to the computation costs for many uncombined algorithms.

9.4.2 One-versus-one

An alternative technique, known as the *one-versus-one (OVO) technique*, consists of using the training data to learn (independently), for each pair of distinct classes $(l, l') \in \mathcal{Y}^2$, $l \neq l'$, a binary classifier $h_{ll'} \colon \mathcal{X} \to \{-1, 1\}$ discriminating between classes l and l'. For any $(l, l') \in \mathcal{Y}^2$, $h_{ll'}$ is obtained by training a binary classification algorithm on the sub-sample containing exactly the points labeled with l or l', with the value $+1$ returned for class l' and -1 for class l. This requires training $\binom{k}{2} = k(k-1)/2$ classifiers, which are combined to define a multi-class classification hypothesis h via majority vote:

$$\forall x \in \mathcal{X}, \quad h(x) = \operatorname*{argmax}_{l' \in \mathcal{Y}} \big|\{l \colon h_{ll'}(x) = 1\}\big|. \tag{9.17}$$

Table 9.1
Comparison of the time complexity the OVA and OVO techniques for both training and testing. The table assumes a full training sample of size m with each class represented by m/k points. The time for training a binary classification algorithm on a sample of size n is assumed to be in $O(n^\alpha)$. Thus, the training time for the OVO technique is in $O(k^2(m/k)^\alpha) = O(k^{2-\alpha}m^\alpha)$. c_t denotes the cost of testing a single classifier.

	Training	Testing
OVA	$O(km^\alpha)$	$O(kc_t)$
OVO	$O(k^{2-\alpha}m^\alpha)$	$O(k^2c_t)$

Thus, for a fixed point $x \in \mathcal{X}$, if we describe the prediction values $h_{ll'}(x)$ as the results of the matches in a tournament between two players l and l', with $h_{ll'}(x) = 1$ indicating l' winning over l, then the class predicted by h can be interpreted as the one with the largest number of wins in that tournament.

Let $x \in \mathcal{X}$ be a point belonging to class l'. By definition of the OVO technique, if $h_{ll'}(x) = 1$ for all $l \neq l'$, then the class associated to x by OVO is the correct class l' since $\left|\{l\colon h_{ll'}(x) = 1\}\right| = k - 1$ and no other class can reach $(k-1)$ wins. By contraposition, if the OVO hypothesis misclassifies x, then at least one of the $(k-1)$ binary classifiers $h_{ll'}$, $l \neq l'$, incorrectly classifies x. Assume that the generalization error of all binary classifiers $h_{ll'}$ used by OVO is at most r, then, in view of this discussion, the generalization error of the hypothesis returned by OVO is at most $(k-1)r$.

The OVO technique is not subject to the calibration problem pointed out in the case of the OVA technique. However, when the size of the sub-sample containing members of the classes l and l' is relatively small, $h_{ll'}$ may be learned without sufficient data or with increased risk of overfitting. Another concern often raised for the use of this technique is the computational cost of training $k(k-1)/2$ binary classifiers versus that of the OVA technique.

Taking a closer look at the computational requirements of these two methods reveals, however, that the disparity may not be so great and that in fact under some assumptions the time complexity of training for OVO could be less than that of OVA. Table 9.1 compares the computational complexity of these methods both for training and testing assuming that the complexity of training a binary classifier on a sample of size m is in $O(m^\alpha)$ and that each class is equally represented in the training set, that is by m/k points. Under these assumptions, if $\alpha \in [2, 3)$ as in the case of some algorithms solving a QP problem, such as SVMs, then the time complexity of training for the OVO technique is in fact more favorable than that of OVA. For $\alpha = 1$, the two are comparable and it is only for sub-linear algorithms that the OVA technique would benefit from a better complexity. In all cases, at test time, OVO requires $k(k-1)/2$ classifier evaluations, which is $(k-1)$

times more than OVA. However, for some algorithms the evaluation time for each classifier could be much smaller for OVO. For example, in the case of SVMs, the average number of support vectors may be significantly smaller for OVO, since each classifier is trained on a significantly smaller sample. If the number of support vectors is k times smaller and if sparse feature representations are used, then the time complexities of both techniques for testing are comparable.

9.4.3 Error-correcting output codes

A more general method for the reduction of multi-class to binary classification is based on the idea of *error-correcting output codes (ECOC)*. This technique consists of assigning to each class $l \in \mathcal{Y}$ a *code word* of length $c \geq 1$, which in the simplest case is a binary vector $\mathbf{M}_l \in \{-1, +1\}^c$. $\mathbf{M}_l$ serves as a signature for class l, and together these vectors define a matrix $\mathbf{M} \in \{-1, +1\}^{k \times c}$ whose lth row is $\mathbf{M}_l$, as illustrated by figure 9.5. Next, for each column $j \in [c]$, a binary classifier $h_j \colon \mathcal{X} \to \{-1, +1\}$ is learned using the full training sample S, after all points that belong to a class represented by $+1$ in column j are labeled with $+1$, while all other points are labeled with -1. For any $x \in \mathcal{X}$, let $\mathbf{h}(x)$ denote the vector $\mathbf{h}(x) = (h_1(x), \ldots, h_c(x))^\top$. Then, the multi-class hypothesis $h \colon \mathcal{X} \to \mathcal{Y}$ is defined by

$$\forall x \in \mathcal{X}, \quad h(x) = \operatorname*{argmin}_{l \in \mathcal{Y}} d_H\big(\mathbf{M}_l, \mathbf{h}(x)\big). \tag{9.18}$$

Thus, the class predicted is the one whose signatures is the closest to $\mathbf{h}(x)$ in Hamming distance. Figure 9.5 illustrates this definition: no row of matrix $\mathbf{M}$ matches the vector of predictions $\mathbf{h}(x)$ in that case, but the third row shares the largest number of components with $\mathbf{h}(x)$.

The success of the ECOC technique depends on the minimal Hamming distance between the class code words. Let d denote that distance, then up to $r_0 = \left\lfloor \frac{d-1}{2} \right\rfloor$ binary classification errors can be corrected by this technique: by definition of d, even if $r < r_0$ binary classifiers h_l misclassify $x \in \mathcal{X}$, $\mathbf{h}(x)$ is closest to the code word of the correct class of x. For a fixed c, the design of error-correction matrix $\mathbf{M}$ is subject to a trade-off, since larger d values may imply substantially more difficult binary classification tasks. In practice, each column may correspond to a class feature determined based on domain knowledge.

The ECOC technique just described can be extended in two ways. First, instead of using only the label predicted by each classifier h_j the magnitude of the scores defining h_j is used. Thus, if $h_j = \operatorname{sgn}(f_j)$ for some function f_j whose values can be interpreted as confidence scores, then the multi-class hypothesis $h \colon \mathcal{X} \to \mathcal{Y}$ is

codes

classes	1	2	3	4	5	6
1	-1	-1	-1	+1	-1	-1
2	+1	-1	-1	-1	-1	-1
3	-1	+1	+1	-1	+1	-1
4	+1	+1	-1	-1	-1	-1
5	+1	+1	-1	-1	+1	-1
6	-1	-1	+1	+1	-1	+1
7	-1	-1	+1	-1	-1	-1
8	-1	+1	-1	+1	-1	-1

$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$	$f_5(x)$	$f_6(x)$
-1	+1	+1	-1	+1	+1

new example x

Figure 9.5
Illustration of error-correcting output codes for multi-class classification. Left: binary code matrix $\mathbf{M}$, with each row representing the code word of length $c = 6$ of a class $l \in [8]$. Right: vector of predictions $\mathbf{h}(x)$ for a test point x. The ECOC classifier assigns label 3 to x, since the binary code for the third class yields the minimal Hamming distance with $\mathbf{h}(x)$ (distance of 1).

defined by

$$\forall x \in \mathcal{X}, \quad h(x) = \operatorname*{argmin}_{l \in \mathcal{Y}} \sum_{j=1}^{c} L(m_{lj} f_j(x)), \tag{9.19}$$

where (m_{lj}) are the entries of $\mathbf{M}$ and where $L \colon \mathbb{R} \to \mathbb{R}_+$ is a loss function. When L is defined by $L(x) = \frac{1-\operatorname{sgn}(x)}{2}$ for all $x \in \mathcal{X}$ and $h_l = f_l$, we can write:

$$\sum_{j=1}^{c} L(m_{lj} f_j(x)) = \sum_{j=1}^{c} \frac{1 - \operatorname{sgn}(m_{lj} h_j(x))}{2} = d_h(\mathbf{M}_l, \mathbf{h}(x)),$$

and (9.19) coincides with (9.18). Furthermore, ternary codes can be used with matrix entries in $\{-1, 0, +1\}$ so that examples in classes labeled with 0 are disregarded when training a binary classifier for each column. With these extensions, both OVA and OVO become special instances of the ECOC technique. The matrix $\mathbf{M}$ for OVA is a square matrix, that is $c = k$, with all terms equal to -1 except from the diagonal ones which are all equal to $+1$. The matrix $\mathbf{M}$ for OVO has $c = k(k-1)/2$ columns. Each column corresponds to a pair of distinct classes (l, l'), $l \neq l'$, with all entries equal to 0 except from the one with row l, which is -1, and the one with row l', which is $+1$.

Since the values of the scoring functions are assumed to be confidence scores, $m_{lj} f_j(x)$ can be interpreted as the margin of classifier j on point x and (9.19) is thus based on some loss L defined with respect to the binary classifier's margin.

A further extension of ECOC consists of extending discrete codes to continuous ones by letting the matrix entries take arbitrary real values and by using the training sample to *learn* matrix $\mathbf{M}$. Starting with a discrete version of $\mathbf{M}$, c binary classifiers

with scoring functions f_l, $l \in [c]$, are first learned as described previously. We will denote by $\mathbf{F}(x)$ the vector $(f_1(x), \ldots, f_c(x))^\top$ for any $x \in \mathcal{X}$. Next, the entries of $\mathbf{M}$ are relaxed to take real values and learned from the training sample with the objective of making the row of $\mathbf{M}$ corresponding to the class of any point $x \in \mathcal{X}$ more similar to $\mathbf{F}(x)$ than other rows. The similarity can be measured using any PDS kernel K. An example of an algorithm for learning $\mathbf{M}$ using a PDS kernel K and the idea just discussed is in fact multi-class SVMs, which, in this context, can be formulated as follows:

$$\begin{aligned} &\min_{\mathbf{M}, \boldsymbol{\xi}} \|\mathbf{M}\|_F^2 + C \sum_{i=1}^m \xi_i \\ &\text{subject to: } \forall (i, l) \in [m] \times \mathcal{Y}, \\ &\qquad K(\mathbf{f}(x_i), \mathbf{M}_{y_i}) \geq K(\mathbf{f}(x_i), \mathbf{M}_l) + 1 - \xi_i. \end{aligned}$$

Similar algorithms can be defined using other matrix norms. The resulting multi-class classification decision function has the following form:

$$h \colon x \mapsto \operatorname*{argmax}_{l \in \{1, \ldots, k\}} K(\mathbf{f}(x), \mathbf{M}_l).$$

9.5 Structured prediction algorithms

In this section, we briefly discuss an important class of problems related to multi-class classification that frequently arises in computer vision, computational biology, and natural language processing. These include all sequence labeling problems and complex problems such as parsing, machine translation, and speech recognition.

In these applications, the output labels have a rich internal structure. For example, in *part-of-speech tagging* the problem consists of assigning a part-of-speech tag such as N (noun), V (verb), or A (adjective), to every word of a sentence. Thus, the label of the sentence $\omega_1 \ldots \omega_n$ made of the words ω_i is a sequence of part-of-speech tags $t_1 \ldots t_n$. This can be viewed as a multi-class classification problem where each sequence of tags is a possible label. However, several critical aspects common to such *structured output* problems make them distinct from the standard multi-class classification.

First, the label set is exponentially large as a function of the size of the output. For example, if Σ denotes the alphabet of part-of-speech tags, for a sentence of length n there are $|\Sigma|^n$ possible tag sequences. Second, there are dependencies between the substructures of a label that are important to take into account for an accurate prediction. For example, in part-of-speech tagging, some tag sequences may be ungrammatical or unlikely. Finally, the loss function used is typically not a zero-one loss, but one that depends on the substructures. Let $L \colon \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ denote

a loss function such that $L(y', y)$ measures the penalty of predicting the label $y' \in \mathcal{Y}$ instead of the correct label $y \in \mathcal{Y}$.[15] In part-of-speech tagging, $L(y', y)$ could be for example the Hamming distance between y' and y.

The relevant features in structured output problems often depend on both the input and the output. Thus, we will denote by $\mathbf{\Phi}(x, y) \in \mathbb{R}^N$ the feature vector associated to a pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$.

To model the label structures and their dependency, the label set $\mathcal{Y}$ is typically assumed to be endowed with a *graphical model* structure, that is, a graph giving a probabilistic model of the conditional dependence between the substructures. It is also assumed that both the feature vector $\mathbf{\Phi}(x, y)$ associated to an input $x \in \mathcal{X}$ and output $y \in \mathcal{Y}$ and the loss $L(y', y)$ factorize according to the cliques of that graphical model.[16] A detailed treatment of this topic would require a further background in graphical models, and is thus beyond the scope of this section.

The hypothesis set used by most structured prediction algorithms is then defined as the set of functions $h \colon \mathcal{X} \to \mathcal{Y}$ such that

$$\forall x \in \mathcal{X}, \quad h(x) = \operatorname*{argmax}_{y \in \mathcal{Y}} \mathbf{w} \cdot \mathbf{\Phi}(x, y), \tag{9.20}$$

for some vector $\mathbf{w} \in \mathbb{R}^N$. Let $S = ((x_1, y_1), \ldots, x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ be an i.i.d. labeled sample. Since the hypothesis set is linear, we can seek to define an algorithm similar to multi-class SVMs. The optimization problem for multi-class SVMs can be rewritten equivalently as follows:

$$\min_{\mathbf{w}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{m} \max_{y \neq y_i} \max \Big(0, 1 - \mathbf{w} \cdot [\mathbf{\Phi}(x_i, y_i) - \mathbf{\Phi}(x_i, y)]\Big), \tag{9.21}$$

However, here we need to take into account the loss function L, that is $L(y, y_i)$ for each $i \in [m]$ and $y \in \mathcal{Y}$, and there are multiple ways to proceed. One possible way is to let the margin violation be penalized additively with $L(y, y_i)$. Thus, in that case $L(y, y_i)$ is added to the margin violation. Another natural method consists of penalizing the margin violation by multiplying it with $L(y, y_i)$. A margin violation with a larger loss is then penalized more than one with a smaller loss.

---

[15] More generally, in some applications, the loss function could also depend on the input. Thus, L is then a function mapping $L \colon \mathcal{X} \times \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$, with $L(x, y', y)$ measuring the penalty of predicting the label y' instead of y given the input x.

[16] In an undirected graph, a *clique* is a set of fully connected vertices.

The additive penalization leads to the following algorithm known as *Maximum Margin Markov Networks (M^3N)*:

$$\min_{\mathbf{w}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m} \max_{y \neq y_i} \max\Big(0, L(y_i, y) - \mathbf{w} \cdot [\mathbf{\Phi}(x_i, y_i) - \mathbf{\Phi}(x_i, y)]\Big). \tag{9.22}$$

An advantage of this algorithm is that, as in the case of SVMs, it admits a natural use of PDS kernels. As already indicated, the label set $\mathcal{Y}$ is assumed to be endowed with a graph structure with a Markov property, typically a chain or a tree, and the loss function is assumed to be decomposable in the same way. Under these assumptions, by exploiting the graphical model structure of the labels, a polynomial-time algorithm can be given to determine its solution.

A multiplicative combination of the loss with the margin leads to the following algorithm known as *SVMStruct*:

$$\min_{\mathbf{w}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m} \max_{y \neq y_i} L(y_i, y) \max\Big(0, 1 - \mathbf{w} \cdot [\mathbf{\Phi}(x_i, y_i) - \mathbf{\Phi}(x_i, y)]\Big). \tag{9.23}$$

This problem can be equivalently written as a QP with an infinite number of constraints. In practice, it is solved iteratively by augmenting at each round the finite set of constraints of the previous round with the most violating constraint. This method can be applied in fact under very general assumptions and for arbitrary loss definitions. As in the case of M^3N, SVMStruct naturally admits the use of PDS kernels and thus an extension to non-linear models for the solution.

Another standard algorithm for structured prediction problems is *Conditional Random Fields (CRFs)*. We will not describe this algorithm in detail, but point out its similarity with the algorithms just described, in particular M^3N. The optimization problem for CRFs can be written as

$$\min_{\mathbf{w}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m} \log \sum_{y \in \mathcal{Y}} \exp\Big(L(y_i, y) - \mathbf{w} \cdot [\mathbf{\Phi}(x_i, y_i) - \mathbf{\Phi}(x_i, y)]\Big). \tag{9.24}$$

Assume for simplicity that $\mathcal{Y}$ is finite and has cardinality k and let f denote the function $(x_1, \ldots, x_k) \mapsto \log(\sum_{j=1}^{k} e^{x_j})$. f is a convex function known as the *soft-max*, since it provides a smooth approximation of $(x_1, \ldots, x_k) \mapsto \max(x_1, \ldots, x_k)$. Then, problem (9.24) is similar to (9.22) modulo the replacement of the max operator with the soft-max function just described.

9.6 Chapter notes

The margin-based generalization bound for multi-class classification presented in theorem 9.2 is due to Kuznetsov, Mohri, and Syed [2014]. It admits only a lin-